React Handbook — Vol 2

TypeScript, testing, server rendering, patterns, interview and machine coding, cheat sheet, question bank (Ch 11–17)

7 parts · one place to study: DesignGurus courses + classic books + the web, combined, beginner to intermediate

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 11 — TypeScript with React**
   - 11.1 Setup and how type-checking fits in
   - 11.2 Typing props
   - 11.3 Typing children and extending native elements
   - 11.4 Typing events
   - 11.5 Typing state and reducers
   - 11.6 Typing refs
   - 11.7 Typing custom Hooks and Context
   - 11.8 Generic components
   - 11.9 Discriminated unions for props
   - 11.10 Reading type errors, and common mistakes
   - Chapter 11 TL;DR
2. **Chapter 12 — Testing React**
   - 12.1 What to test, and the Testing Library philosophy
   - 12.2 Your first component test
   - 12.3 Queries and their priority
   - 12.4 `user-event` — interacting like a user
   - 12.5 Async tests
   - 12.6 Mocking `fetch` (and when to use MSW)
   - 12.7 Testing custom Hooks
   - 12.8 Providers, routers and custom render
   - 12.9 Setting up Vitest or Jest in a real project
   - 12.10 Good tests, bad tests, and the bigger picture
   - Chapter 12 TL;DR
3. **Chapter 13 — Server Rendering and Frameworks**
   - 13.1 Rendering strategies: CSR, SSR, SSG, ISR, streaming, PPR
   - 13.2 The server rendering APIs
   - 13.3 Streaming SSR with `<Suspense>`
   - 13.4 Hydration end to end
   - 13.5 Hydration mismatches
   - 13.6 Static generation and partial prerendering
   - 13.7 React Server Components
   - 13.8 Directives: `'use client'` and `'use server'`
   - 13.9 Next.js App Router overview
   - 13.10 React Router framework mode
   - 13.11 Choosing a framework
   - Chapter 13 TL;DR
4. **Chapter 14 — Patterns and Architecture**
   - 14.1 Composition first
   - 14.2 Container / presentational
   - 14.3 Extracting custom Hooks
   - 14.4 Compound components
   - 14.5 Render props
   - 14.6 Higher-order components (HOCs)
   - 14.7 Choosing a reuse pattern
   - 14.8 Portals
   - 14.9 Error boundaries
   - 14.10 Accessibility in React
   - 14.11 Folder structure
   - 14.12 Design systems
   - 14.13 Client architecture patterns: MVC, MVP, MVVM, VIPER — and where React fits
   - Chapter 14 TL;DR
5. **Chapter 15 — React Interview and Machine Coding**
   - 15.1 How React interviews are run
   - 15.2 Interview questions: rendering, reconciliation and keys
   - 15.3 Interview questions: state, events and forms
   - 15.4 Interview questions: effects and refs
   - 15.5 Interview questions: hooks, context and state management
   - 15.6 Interview questions: performance and the React Compiler
   - 15.7 Interview questions: Suspense, `use()` and transitions
   - 15.8 Interview questions: server rendering, hydration and Server Components
   - 15.9 Interview questions: error boundaries, portals, patterns, testing and accessibility
   - 15.10 Build 1 — Todo list with filters
   - 15.11 Build 2 — Autocomplete (debounce, abort, keyboard)
   - 15.12 Build 3 — Infinite scroll
   - 15.13 Build 4 — Modal dialog (portal, focus trap, Escape)
   - 15.14 Build 5 — Accessible tabs
   - 15.15 Build 6 — Star rating
   - 15.16 Build 7 — Pagination
   - 15.17 Build 8 — Multi-step form wizard with validation
   - 15.18 Other common prompts — which techniques they need
   - Chapter 15 TL;DR
6. **Chapter 16 — Cheat Sheets and Glossary**
   - 16.1 JSX rules
   - 16.2 Components and props
   - 16.3 Every Hook in one table
   - 16.4 State update rules
   - 16.5 The render and commit lifecycle
   - 16.6 Effects: decision guide
   - 16.7 Refs
   - 16.8 Forms and Actions (React 19)
   - 16.9 Context and choosing state management
   - 16.10 Routing (React Router v8) and TanStack Query
   - 16.11 Performance: checklist, memo and the React Compiler

- 16.12 Styling
- 16.13 TypeScript + React types
- 16.14 Testing: query priority and user-event
- 16.15 Rendering strategies and directives
- 16.16 Patterns
- 16.17 Accessibility checklist
- 16.18 Common errors and fixes
- 16.19 Interview question index
- 16.20 Where to go next
- 16.21 Glossary (A–Z)
- Chapter 16 TL;DR

7. **Chapter 17 — Interview Question Bank**
   - How to use this bank
   - 1. React fundamentals and JSX
   - 2. Components, props and composition
   - 3. State and events
   - 4. Rendering, reconciliation, keys and the virtual DOM
   - 5. `useEffect`, its pitfalls, and refs
   - 6. Other Hooks, Rules of Hooks and custom Hooks
   - 7. React 19 features
   - 8. State management: Context, Redux, Zustand and server state
   - 9. Forms, routing and data fetching
   - 10. Performance
   - 11. Error boundaries, portals, patterns and accessibility
   - 12. SSR, SSG, hydration and Next.js
   - 13. Testing and TypeScript
   - 14. "What will this code print?" — output and behaviour puzzles
   - 15. Machine coding and scenario questions
   - Sources of the question lists

Chapter 11 — TypeScript with React

TypeScript is JavaScript plus a type system (labels saying what kind of value each variable holds — string, number, an object of a certain shape) checked **before** the code runs. In React it turns a class of bugs — a missing prop, a misspelled variant, reading `user.name` while `user` can still be `null`, forgetting to handle one action in a reducer — into editor squiggles instead of broken screens in production. It also makes components self-documenting: the props type *is* the component's API.

This chapter assumes the TypeScript basics from the JavaScript Handbook (Chapter 16 — types, narrowing, generics) and applies them to props, children, events, state, reducers, refs, Hooks, context, generic components and discriminated-union props. "Wrong" examples show the **real compiler errors**.

> **Key term — narrowing and generics:** *Narrowing* means TypeScript shrinks a broad type after a check — after `if (user !== null)`, `user` is a `User`. *Generics* are types with a placeholder `<T>` filled in where used — `Array<string>` — so one piece of code works for many types and is still checked.

Code: `code/`. Each `.tsx` file **runs** (bundled with esbuild; React 19.3 in jsdom under Node.js) **and** is type-checked with `tsc` 7.0.2, `@types/react` 19.3, `strict` mode (all stricter checks on). The deliberately broken examples in `code/bad_types.tsx` are only type-checked.

11.1 Setup and how type-checking fits in

What it is

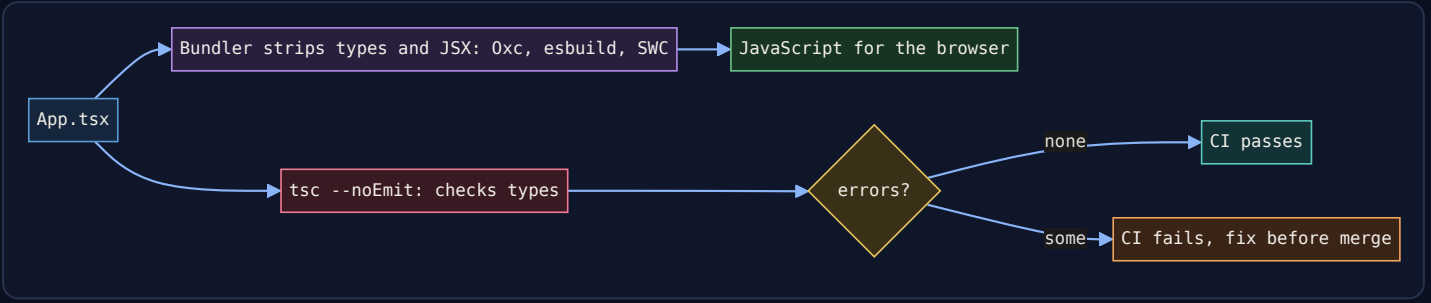
Piece	Role
`.tsx` files	TypeScript files that may contain JSX (`.ts` for files without JSX)
`@types/react`, `@types/react-dom`	Type definitions for React (React is written in JavaScript; these describe its API)
`tsconfig.json`	Compiler options: strictness, JSX mode, which files to check

Start with `npm create vite@latest my-app -- --template react-ts`, or a framework such as Next.js, which asks whether you want TypeScript and configures it (Chapter 1).

How it works — two separate jobs

1. **Removing types.** Browsers can't run TypeScript, so the bundler (Vite 8 uses Oxc, earlier Vite esbuild; Next.js uses SWC — Oxc and SWC are fast Rust compilers) strips types and turns JSX into function calls. Fast because it **does not check** anything.
2. **Checking types.** `tsc` reads all files and reports errors; `--noEmit` checks without writing files. Your editor runs the same checker continuously.

So **a project with type errors can still build and run**. Add `tsc --noEmit` to CI (the server that runs checks on every push/pull request) and often to the build (`"build": "tsc --noEmit && vite build"`) so errors block merges.



The tsconfig options that matter for React

`code/tsconfig.check.json` (the `paths` line exists only because this book keeps npm packages in a shared folder; a normal project names the file `tsconfig.json`):

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "bundler",     // resolve imports the way Vite/esbuild do
    "jsx": "react-jsx",                // the modern JSX transform: no `import React` needed
    "strict": true,                    // turns on strictNullChecks, noImplicitAny, ... - always use it
    "noEmit": true,                    // the bundler produces JS; tsc only checks
    "skipLibCheck": true,              // don't re-check .d.ts files in node_modules (faster)
    "allowImportingTsExtensions": true,
    "lib": ["ES2023", "DOM", "DOM.Iterable"],   // DOM types such as HTMLInputElement
    "types": [],
    "paths": { "*": ["../../../../.web-books/node_modules/@types/*", "../../../../.web-books/node_modules/*"] }
  },
  "include": ["*.tsx", "*.ts"],
  "exclude": ["bad_*.tsx"]
}
```

```
$ tsc --noEmit -p tsconfig.check.json && echo "0 type errors"
0 type errors
```

Common mistakes

- Turning `strict` off to "get it compiling" — you lose most of the value, especially null checks.
- Using `any` ("anything, don't check") to silence an error — it switches checking off for that value *and everything it flows into*. Prefer `unknown` ("type not known yet; check before use") and narrow, or fix the type.
- Missing `@types/react` / `@types/react-dom`, or versions not matching your React major version (the first number in `19.3.0`, bumped on breaking changes).

11.2 Typing props

What it is

Props are one object: describe it with a `type` (or `interface`) and annotate the destructured parameter.

`code/01_props_and_children.tsx` (part 1):

```
type BadgeProps = {
  label: string;                      // required
  count?: number;                     // optional -> number | undefined inside the component
  tone?: 'info' | 'warning' | 'error';  // only these three strings are allowed
};
function Badge({ label, count = 0, tone = 'info' }: BadgeProps) {
  return <span className={`badge ${tone}`}>{label}: {count}</span>;
}

type AvatarProps = {
  user: { name: string; imageUrl: string };  // an object shape
  size?: number;
  style?: CSSProperties;                     // a style object, checked key by key
  onSelect?: (name: string) => void;         // a callback prop
};
function Avatar({ user, size = 32, style, onSelect }: AvatarProps) {
  return <img src={user.imageUrl} alt={user.name} width={size} style={style} onClick={() => onSelect?.(user.name)}
/>;
}
```

How it works

1. Each `<Badge ...>` is checked against `BadgeProps` like an object literal.
2. `?` makes a prop optional; inside, its type includes `undefined` until a **default value** in the destructuring (`count = 0` → plain `number`).
3. A **union of string literals** (`'info' | 'warning' | 'error'`) means "one of these" — autocompleted, and typos are errors.
4. A callback prop is a **function type**: `(name: string) => void` = "called with a string, return ignored". Call optional callbacks with `onSelect?.(...)` (optional chaining).

`type` or `interface`?

Both work. `interface` can `extends` and merge by re-declaring; `type` can express unions (`A | B`), which props often need (§11.9). Many teams use `type` everywhere; pick one convention per codebase.

What about `React.FC`?

`const Badge: React.FC<BadgeProps> = ...` works. Since React 18's types `FC` no longer adds an implicit `children`, so the main objection is gone; still, annotating the props parameter is simpler, works with generics (§11.8), and is what most modern code does.

Real-world examples

Design-system components use literal unions for `variant` / `size` / `tone`, so design tokens (the agreed named values for colours, spacing, sizes) are enforced by the compiler.

11.3 Typing children and extending native elements

Which type for `children`?

Type	Accepts	Use for
`ReactNode`	anything renderable: JSX, strings, numbers, `null`, `undefined`, booleans, arrays of these	`children` and "slot" props — the default
`ReactElement`	only a JSX element (`<Foo />`), not strings or `null`	a prop that must be one element (rare)
`JSX.Element`	type of a JSX expression (basically `ReactElement<any, any>`)	component return type, if you annotate it
`PropsWithChildren<P>`	adds `children?: ReactNode` to `P`	short way to accept optional children

`code/01_props_and_children.tsx` (part 2):

```tsx
type CardProps = { title: string; children: ReactNode; footer?: ReactNode };
function Card({ title, children, footer }: CardProps) {
  return <section><h2>{title}</h2>{children}{footer && <footer>{footer}</footer>}</section>;
}
function Note({ children }: PropsWithChildren) { return <aside>{children}</aside>; }
```

Extending a native element's props

A reusable `<Button>` should accept **everything** a `<button>` does — `type`, `disabled`, `aria-*` (accessibility attributes), `onClick` — without listing them. `ComponentProps<'button'>` is the built-in element's full props type:

```tsx
type ButtonProps = ComponentProps<'button'> & { variant?: 'primary' | 'ghost' };
function Button({ variant = 'primary', className, ...rest }: ButtonProps) {
  return <button className={[variant, className].filter(Boolean).join(' ')} {...rest} />;
}
```

```tsx
<Card title="Inbox" footer={<small>updated now</small>}>
  <Badge label="unread" count={3} tone="warning" />
  <Note>plain text child</Note>
  <Button type="submit" disabled aria-label="send">Send</Button>
  <Avatar user={{ name: 'Ada', imageUrl: '/ada.png' }} style={{ borderRadius: '50%' }} />
</Card>
```

Output:

```tsx
<section>
<h2>Inbox</h2>
<span class="badge warning">unread: 3</span>
<aside>plain text child</aside>
<button class="primary" type="submit" disabled="" aria-label="send">Send</button>
<img alt="Ada" width="32" style="border-radius: 50%;" src="/ada.png">
<footer>
<small>updated now</small>
</footer>
</section>
```

Related: `ComponentProps<typeof MyComponent>` extracts **your** component's props (handy when a library doesn't export its props type); `ComponentPropsWithoutRef<'input'>` omits `ref`; `Omit<ComponentProps<'a'>, 'href'>` removes a prop so you can redefine it.

Common mistakes

- `children: JSX.Element` — then `<Card>hello</Card>` or two children fail. Use `ReactNode`.
- Spreading `{...rest}` *before* your own props, or carelessly, so the caller's `className` overwrites yours. Merge deliberately, as above.

11.4 Typing events

What it is

React's event objects (synthetic events, Chapter 3) are generic over the **element** the handler is on, which makes `e.target.value` or `e.currentTarget.checked` type-safe.

Event	Type	Typical element	Common props
`onChange` on input / select / textarea	`ChangeEvent<HTMLInputElement>` (etc.)	`HTMLInputElement`, `HTMLSelectElement`, `HTMLTextAreaElement`	`e.target.value`, `e.target.checked`
`onSubmit`	`SubmitEvent<HTMLFormElement>` (older code: `FormEvent`)	`HTMLFormElement`	`e.preventDefault()`, `e.submitter`
`onClick`	`MouseEvent<HTMLButtonElement>`	any element	`e.currentTarget`, `e.shiftKey`, `e.clientX`
`onKeyDown`	`KeyboardEvent<HTMLInputElement>`	any focusable element	`e.key`, `e.ctrlKey`
`onFocus` / `onBlur`	`FocusEvent<HTMLInputElement>`		`e.relatedTarget`

Import them from `react`, **not** the DOM (an unimported `MouseEvent` / `SubmitEvent` is the browser's native type, which differs).

Version note: recent `@types/react` 19 releases (here 19.3) type `onSubmit` with `SubmitEvent` and mark `FormEvent` **deprecated** (still compiles, flagged for removal; React never had a real "form event"). Most existing code uses `FormEvent<HTMLFormElement>`; prefer `SubmitEvent` in new code if your types have it.

How it works

1. **Inline handlers need no annotation** — `onChange={e => setEmail(e.target.value)}` infers `e` from the prop's type (*contextual typing*).
2. **Separately declared handlers need a type** — annotate the parameter (`e: ChangeEvent<HTMLInputElement>`) or the function (`const h: ChangeEventHandler<HTMLSelectElement> = e => ...`).
3. Unsure? Write it inline and hover `e` to copy the exact type.

`code/02_events.tsx` (abridged):

```
function handleEmail(e: ChangeEvent<HTMLInputElement>) {
  setEmail(e.target.value);                    // e.target is an HTMLInputElement -> .value is a string
}
const handlePlan: ChangeEventHandler<HTMLSelectElement> = e => setPlan(e.target.value); // type the whole function
function handleSubmit(e: SubmitEvent<HTMLFormElement>) {   // older code: FormEvent (now deprecated)
  e.preventDefault();
  console.log('submitted by the button:', e.submitter?.textContent);
  onSubmit({ email, plan });
}
function handleKey(e: KeyboardEvent<HTMLInputElement>) { if (e.key === 'Escape') setEmail(''); }
function handleHelp(e: MouseEvent<HTMLButtonElement>) {
  console.log(`help clicked at button "${e.currentTarget.textContent}" (shift: ${e.shiftKey})`);
}
// <form onSubmit={handleSubmit}> <input onChange={handleEmail} onKeyDown={handleKey} /> <select onChange=
{handlePlan}>
// <button type="button" onClick={handleHelp}>Help</button>
// <button type="submit" onClick={e => console.log('inline handler, e is', e.type)}>Sign up</button>
```

Output (type an email, choose Pro, shift-click Help, submit, press Escape):

```
help clicked at button "Help" (shift: true)
inline handler, e is click
submitted by the button: Sign up
submitted {"email":"ada@example.com","plan":"pro"}
after Escape, email = ""
```

`target` VS `currentTarget`

`e.currentTarget` is the element the handler is on, typed as you declared (`HTMLButtonElement`). `e.target` is whatever was actually clicked (maybe an `<svg>` inside), so usually only `EventTarget`. For **change** events React types `e.target` as the input itself — why `e.target.value` works there.

Form actions (React 19)

With `<form action={fn}>` (Chapter 4) there's no event: `fn(formData: FormData)` (the browser's field-name/value object). `formData.get('email')` returns `FormDataEntryValue | null` (a `string` or `File`), so narrow: `String(formData.get('email') ?? '')`, or validate the whole object with a schema library such as Zod (checks real data against a declared shape).

11.5 Typing state and reducers

`useState`

The type is **inferred** from the initial value; add a type argument when that value doesn't show the full picture:

Code	Type	Why
`useState(0)`	`number`	inferred
`useState<User | null>(null)`	`User | null`	`null` alone infers type `null` forever
`useState<string[]>([])`	`string[]`	`[]` alone infers `never[]` — nothing could be added
`useState<'idle' | 'saving' | 'saved'>('idle')`	the union	`'idle'` alone infers `string`
`useState(() => loadDraft())`	return type of `loadDraft`	lazy initializer (run only on first render) is inferred too

`code/03_state_and_reducer.tsx` (part 1, abridged):

```tsx
type User = { id: number; name: string };
function Profile() {
  const [count, setCount] = useState(0);                    // inferred: number
  const [user, setUser] = useState<User | null>(null);      // explicit: starts null, later a User
  const [tags, setTags] = useState<string[]>([]);           // [] alone would be never[]
  const [status, setStatus] = useState<'idle' | 'saving' | 'saved'>('idle');
  // <p>{user ? `${user.name} (#${user.id})` : 'signed out'} ...</p>
  // user.name without the check would not compile: 'user' is possibly 'null'
}
```

The setter's type is `Dispatch<SetStateAction<T>>` : it accepts a `T` or an updater `(prev: T) => T`. Use that name when passing a setter as a prop (`setUser: Dispatch<SetStateAction<User | null>>`), or simply `(u: User | null) => void` if the child never uses the updater form.

`useReducer` with a discriminated union of actions

A **discriminated union** is a union of object types sharing one literal-typed field (the *discriminant*, here `type`). Checking it **narrows** the object to one member, so TypeScript knows which other fields exist.

`code/03_state_and_reducer.tsx` (part 2):

```ts
type CartItem = { sku: string; qty: number };
type CartState = { items: CartItem[]; coupon: string | null };
type CartAction =
  | { type: 'add'; sku: string }
  | { type: 'remove'; sku: string }
  | { type: 'applyCoupon'; code: string }
  | { type: 'clear' };

function cartReducer(state: CartState, action: CartAction): CartState {
  switch (action.type) {
    case 'add': {
      const found = state.items.find(i => i.sku === action.sku);   // action.sku is known to exist here
      return {
        ...state,
        items: found
          ? state.items.map(i => (i.sku === action.sku ? { ...i, qty: i.qty + 1 } : i))
          : [...state.items, { sku: action.sku, qty: 1 }],
      };
    }
    case 'remove':      return { ...state, items: state.items.filter(i => i.sku !== action.sku) };
    case 'applyCoupon': return { ...state, coupon: action.code };  // action.code exists only here
    case 'clear':       return { items: [], coupon: null };
    default: {
      const unreachable: never = action;                          // compile error if a case is missing
      return unreachable;
    }
  }
}
// const [cart, dispatch] = useReducer(cartReducer, { items: [], coupon: null });
// dispatch accepts only CartAction objects: dispatch({ type: 'add', sku: 'MUG' })
```

Output (sign in, add mug twice, apply coupon, clear):

```
initial        signed out | 0 |  | idle  ||  empty coupon=none
after clicks   Ada (#7) | 1 | admin | saved  ||  MUGx2 coupon=SAVE10
after clear    Ada (#7) | 1 | admin | saved  ||  empty coupon=none
```

How the `never` check works

In `default`, every handled member has been removed from `action`'s type; if all are handled, it's `never` and the assignment is fine. Add an action to `CartAction` and forget its `case`, and the assignment **fails to compile** (§11.10, error 9) — an **exhaustiveness check**.

`useReducer` infers state from the reducer, and `dispatch` accepts only valid actions. (React 19's types also allow `useReducer<CartState, [CartAction]>(...)`, but annotating the reducer is simpler.)

11.6 Typing refs

DOM refs and value refs

Purpose	Declaration	`.current` type
Point at a DOM element	`useRef<HTMLInputElement>(null)`	`HTMLInputElement | null`
Hold a mutable value (timer ID, previous value)	`useRef<number | null>(null)`, `useRef(0)`	what you declared

In React 19's types `useRef` **always requires an argument** and returns a `RefObject<T>` with an assignable `current` (older types split read-only `RefObject` from `MutableRefObject`; `MutableRefObject` is now deprecated). DOM refs start `null` because the element doesn't exist until React attaches it after render — hence `ref.current?.focus()`.

`ref` as a prop (React 19)

Function components receive `ref` as an **ordinary prop**; `forwardRef` isn't needed in new code (Chapter 5). `ComponentProps<'input'>` already includes `ref?: Ref<HTMLInputElement>`, so a wrapper spreading its props forwards the ref. For a custom handle (an object of methods you expose instead of the DOM node), type the prop `Ref<YourHandle>` and fill it with `useImperativeHandle`.

`code/04_refs.tsx` (abridged):

```
// 1. A DOM ref
const inputRef = useRef<HTMLInputElement>(null);
useEffect(() => { inputRef.current?.focus(); }, []);   // ?. because current is null until React attaches it

// 2. A value ref (not attached to JSX), read/written only in handlers
const intervalId = useRef<ReturnType<typeof setInterval> | null>(null);

// 3. ref as an ordinary prop - ComponentProps<'input'> includes `ref`
function TextField({ label, ...inputProps }: { label: string } & ComponentProps<'input'>) {
  return <label>{label} <input {...inputProps} /></label>;   // ref travels inside inputProps
}

// 4. Exposing a custom handle instead of the DOM node
export type VideoHandle = { play: () => void; pause: () => void };
function Video({ ref, src }: { ref?: Ref<VideoHandle>; src: string }) {
  const [state, setState] = useState('paused');
  useImperativeHandle(ref, () => ({ play: () => setState('playing'), pause: () => setState('paused') }), []);
  return <p>video {src}: {state}</p>;
}
// App: <TextField label="Name" ref={nameRef} placeholder="Ada" />, <Video ref={videoRef} src="intro.mp4" />,
videoRef.current?.play()
```

Output:

```
focused after mount: search
stopwatch: stop timer
nameRef.current is INPUT with placeholder Ada
video intro.mp4: playing
```

`ReturnType<typeof setInterval>` gives the timer ID type of the current environment (a `number` in browsers, an object in Node).

Common mistakes

- `useRef<HTMLInputElement>()` with no argument — error with React 19 types (§11.10).
- Wrong element type: `useRef<HTMLDivElement>(null)` on an `<input>` errors at `ref={...}`.
- Reading/writing `ref.current` **during render** — refs are for handlers and Effects; React Compiler's lint rules flag render-time ref access (Chapter 10).

11.7 Typing custom Hooks and Context

Returning a tuple: `as const`

Returning `[on, toggle]` infers `(boolean | (() => void))[]` — each element could be either. `as const` makes a **readonly tuple** `readonly [boolean, () => void]` (fixed length, a type per position, unchangeable):

```
function useToggle(initial = false) {
  const [on, setOn] = useState(initial);
  const toggle = useCallback(() => setOn(v => !v), []);
  return [on, toggle] as const;
}
```

Returning an **object** (`{ on, toggle }`) avoids the problem and is easier to extend — prefer it beyond two values.

A generic Hook with a discriminated-union result

`code/05_hooks_and_context.tsx` :

```tsx
type FetchState<T> =
  | { status: 'loading' }
  | { status: 'error'; error: string }
  | { status: 'success'; data: T };

function useJson<T>(url: string): FetchState<T> {
  const [state, setState] = useState<FetchState<T>>({ status: 'loading' });
  useEffect(() => {
    let ignore = false;
    setState({ status: 'loading' });
    fetch(url)
      .then(r => (r.ok ? (r.json() as Promise<T>) : Promise.reject(new Error(`HTTP ${r.status}`))))
      .then(data => { if (!ignore) setState({ status: 'success', data }); })
      .catch((e: unknown) => { if (!ignore) setState({ status: 'error', error: e instanceof Error ? e.message :
String(e) }); });
    return () => { ignore = true; };
  }, [url]);
  return state;
}
```

The caller picks `T` and *must* check `status` before touching `data` :

```tsx
const result = useJson<Todo>('/api/todos/1');
if (result.status === 'loading') return <p>loading...</p>;
if (result.status === 'error') return <p>error: {result.error}</p>;
// Here TypeScript knows result.data is a Todo.
```

Better than three fields (`data?` , `error?` , `loading`), which allow nonsense like "loading **and** error". In strict mode caught values are `unknown` ; narrow with `instanceof Error` .

`as Promise<T>` is not validation — it's "trust me"; a different API shape goes unnoticed. At trust boundaries (network, `localStorage` , URL params) validate with a schema library such as Zod, which checks at runtime and produces the TypeScript type. TanStack Query is generic the same way (`useQuery<Todo>` , Chapter 9).

Context that is never `null` for consumers

With no sensible default, use `null` plus a custom Hook that throws when the provider is missing — callers get the non-nullable type:

```tsx
type Auth = { user: string; logout: () => void };
const AuthContext = createContext<Auth | null>(null);

function useAuth(): Auth {                       // callers get Auth, never null
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth must be used inside <AuthProvider>');
  return ctx;
}
function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState('ada');
  // (in a real app, memoize this object - see Chapter 10, section 10.10)
  return <AuthContext value={{ user, logout: () => setUser('guest') }}>{children}</AuthContext>;
}
```

Output (fetch faked offline; second part renders `TodoTitle` without a provider):

```
loading...
guest: loaded from /api/todos/1 (#1)
without provider -> useAuth must be used inside <AuthProvider>
```

`createContext<Auth>({} as Auth)` compiles but lies: outside the provider it crashes later with a confusing `undefined is not a function` .

11.8 Generic components

What it is

A **generic component** has a type parameter: `function List<T>(props: ListProps<T>)`. TypeScript **infers** `T` from the props and checks the rest against it — one `List` for books, users or strings, with `renderItem` always receiving the right type.

`code/06_generic_components.tsx` (abridged):

```tsx
type ListProps<T> = { items: T[]; getKey: (item: T) => Key; renderItem: (item: T) => ReactNode; emptyText?: string
};
function List<T>({ items, getKey, renderItem, emptyText = 'Nothing here' }: ListProps<T>) {
  if (items.length === 0) return <p>{emptyText}</p>;
  return <ul>{items.map(item => <li key={getKey(item)}>{renderItem(item)}</li>)}</ul>;
}
// In an arrow function inside a .tsx file write <T,> so the parser doesn't read <T> as JSX:
const Count = <T,>({ items }: { items: T[] }) => <small>{items.length} items</small>;

// A select whose value type is a union of string literals, inferred from `options`
type SelectProps<V extends string> = { label: string; value: V; options: readonly { value: V; label: string }[];
onChange: (value: V) => void };
function Select<V extends string>({ label, value, options, onChange }: SelectProps<V>) {
  return <label>{label}{' '}<select value={value} onChange={e => onChange(e.target.value as V)}>
    {options.map(o => <option key={o.value} value={o.value}>{o.label}</option>)}</select></label>;
}

// A table whose columns may only name real keys of the row type
type Column<Row> = { key: keyof Row & string; header: string };
function Table<Row extends { id: number }>({ rows, columns }: { rows: Row[]; columns: Column<Row>[] }) { /* ... */
}

const SORTS = [{ value: 'title', label: 'Title' }, { value: 'year', label: 'Year' }] as const;
// App: const [sort, setSort] = useState<'title' | 'year'>('title');
<Select label="Sort by" value={sort} options={SORTS} onChange={setSort} />
<List items={sorted} getKey={b => b.id} renderItem={b => `${b.title} (${b.year})`} />
<List items={[] as string[]} getKey={s => s} renderItem={s => s} emptyText="No tags" />
<Table rows={books} columns={[{ key: 'title', header: 'Title' }, { key: 'year', header: 'Year' }]} />
```

Output (then the select changes to "year"):

```
list : Beloved (1987) | Dune (1965) | Neuromancer (1984)
year : Dune (1965) | Neuromancer (1984) | Beloved (1987)
other: No tags / 3 items
table: TitleYear | Dune1965 | Neuromancer1984 | Beloved1987
```

How it works

1. `items={sorted}` is `Book[]`, so `T = Book`; `renderItem={b => ...}` is checked as `(item: Book) => ReactNode` — `b.title` autocompletes, `b.author` errors.
2. **Constraints** (`V extends string`, `Row extends { id: number }`) restrict `T` and let the component rely on them (`r.id` as key).
3. `keyof Row & string` is the union of `Row`'s property names (`'id' | 'title' | 'year'`), so a column for a missing field is a compile error (§11.10, error 8).
4. `as const` on `SORTS` keeps literals `'title'`/`'year'`, so `V = 'title' | 'year'` and `onChange={setSort}` type-checks.

When not to use generics

A component that only handles one type is clearer with a plain props type. Generics pay off in reusable blocks: lists, tables, selects, comboboxes (text box with suggestion dropdown), form fields.

11.9 Discriminated unions for props

The problem it solves

`{ status: string; error?: string; count?: number; onRetry?: () => void }` allows nonsense ("success" with an error, "error" without retry); consumers guess, components check defensively. A **union of props types**, one per case, makes invalid combinations **impossible to write**.

`code/07_union_props.tsx` :

```tsx
// 1. Each `status` value comes with exactly the props it needs
type StatusProps =
  | { status: 'loading' }
  | { status: 'error'; error: string; onRetry: () => void }
  | { status: 'success'; count: number };

function Status(props: StatusProps) {
  switch (props.status) {                    // narrow on the discriminant `status`
    case 'loading': return <p>Loading…</p>;
    case 'error':   return <p role="alert">{props.error} <button onClick={props.onRetry}>Retry</button></p>;
    case 'success': return <p>{props.count} results</p>;
  }
}

// 2. Either a link or a button
type LinkLike = { as: 'a'; href: string } & Omit<ComponentProps<'a'>, 'href'>;
type ButtonLike = { as?: 'button' } & ComponentProps<'button'>;
type ActionProps = (LinkLike | ButtonLike) & { children: ReactNode };

function Action(props: ActionProps) {
  if (props.as === 'a') {
    const { as: _as, ...rest } = props;       // here props is LinkLike
    return <a className="action" {...rest} />;
  }
  const { as: _as, ...rest } = props;         // here props is ButtonLike
  return <button className="action" type="button" {...rest} />;
}

// 3. "One or the other, not both": the `never` trick forbids the other prop
type ImageSource =
  | { src: string; icon?: never }
  | { icon: 'user' | 'star'; src?: never };
function Thumb(props: ImageSource & { alt: string }) {
  return props.src ? <img src={props.src} alt={props.alt} /> : <span aria-label={props.alt}>[{props.icon}]</span>;
}
```

```tsx
<Status status="loading" />
<Status status="error" error="Network down" onRetry={() => {}} />
<Status status="success" count={42} />
<Action as="a" href="/docs">Docs</Action>
<Action onClick={() => {}}>Save</Action>
<Thumb src="/a.png" alt="avatar" />
<Thumb icon="star" alt="favourite" />
```

Output:

```
<p>Loading…</p>
<p role="alert">Network down <button>Retry</button>
</p>
<p>42 results</p>
<a class="action" href="/docs">Docs</a>
<button class="action" type="button">Save</button>
<img alt="avatar" src="/a.png">
<span aria-label="favourite">[star]</span>
```

How it works

1. The shared literal field (`status`, `as`) is the **discriminant**.
2. At the call site TypeScript picks the member whose discriminant matches and checks the rest: `<Status status="success" />` without `count` fails (§11.10, error 6).
3. Inside, checking the discriminant narrows `props`, so `props.error` exists only in the `'error'` branch.
4. Don't destructure in the parameter list (`function Status({ status, error })`) — destructuring before narrowing loses the link between fields. (Recent TypeScript narrows destructured `const`s in simple cases, but keeping `props` whole is the reliable pattern.)
5. `prop?: never` = "must not be passed" — the standard "exactly one of these".

Real-world examples

- Libraries use `as` / `asChild` or link-vs-button unions so clickable things render correct, accessible HTML.
- Alert/toast `variant` unions; form fields where `type: 'select'` requires `options`; modals where `dismissible: true` requires `onClose`.

In the interview

> **30-second answer:** "I type props with a type alias on the destructured parameter, `children` as `ReactNode`, and extend native elements with `ComponentProps<'button'>`. Handlers use React's event types such as `ChangeEvent<HTMLInputElement>`, though inline handlers are inferred. For state I let inference work and add generics for `null` or empty arrays. Variant props use discriminated unions so invalid combinations don't compile; reducers get action unions with a `never` exhaustiveness check. Reusable lists and tables are generic components."

- *`ReactNode` vs `ReactElement` vs `JSX.Element`?* — `ReactNode` is anything renderable (children); the others are only a JSX element.
- *How do you type `ref` in React 19?* — A normal prop `ref?: Ref<HTMLInputElement>`, or free via `ComponentProps<'input'>`; no `forwardRef`.
- *Why not `any`?* — It disables checking and spreads; use `unknown` and narrow, or a proper type.

11.10 Reading type errors, and common mistakes

Errors follow one pattern: *"Type **A** (what you gave) is not assignable to type **B** (what was expected)"*, often followed by indented lines zooming in on the differing property. Read the **last** indented line first — usually the real reason.

`code/bad_types.tsx` collects twelve common mistakes (excluded from the normal check and runner):

```tsx
const e1 = <Badge />;                                  // 1. required prop missing
const e2 = <Badge label="x" tone="danger" />;          // 2. value outside the union

function Profile() {
  const [user] = useState<{ name: string } | null>(null);
  const [tags, setTags] = useState([]);
  setTags(['a']);                                      // 4. useState([]) without a type -> never[]
  return <p>{user.name} {tags.length}</p>;             // 3. user may be null
}

function Plan() {
  function handle(e: ChangeEvent<HTMLInputElement>) { console.log(e.target.value); }
  return <select onChange={handle} />;                 // 5. input handler on a <select>
}

const e6 = <Status status="success" />;                // 6. 'success' requires count
const e7 = <Thumb src="/a.png" icon="star" />;         // 7. both src and icon
const e8 = <Table rows={[{ id: 1, title: 'Dune' }]} columns={[{ key: 'author', header: 'Author' }]} />;  // 8. no such key

type Action = { type: 'add'; sku: string } | { type: 'clear' };
function reducer(state: string[], action: Action): string[] {
  switch (action.type) {
    case 'add': return [...state, action.sku];
    default: { const unreachable: never = action; return unreachable; }   // 9. 'clear' not handled
  }
}
function Cart() { const [items, dispatch] = useReducer(reducer, []); dispatch({ type: 'remove' }); return <p>{items}</p>; }  // 10. unknown action

function Focus() {
  const ref = useRef<HTMLInputElement>();              // 11. React 19 types: useRef needs an argument
  return <input ref={ref} />;
}

const e12 = <Box>{{ title: 'not renderable' }}</Box>;   // 12. a plain object is not a ReactNode
```

Output of `tsc --noEmit -p tsconfig.bad.json` (real, unedited):

```
bad_types.tsx(7,13): error TS2741: Property 'label' is missing in type '{}' but required in type 'BadgeProps'.
bad_types.tsx(8,29): error TS2322: Type '"danger"' is not assignable to type '"error" | "info" | "warning" | undefined'.
bad_types.tsx(13,12): error TS2322: Type 'string' is not assignable to type 'never'.
bad_types.tsx(14,14): error TS18047: 'user' is possibly 'null'.
bad_types.tsx(19,18): error TS2322: Type '(e: ChangeEvent<HTMLInputElement, Element>) => void' is not assignable to type 'ChangeEventHandler<HTMLSelectElement, HTMLSelectElement>'.
  Types of parameters 'e' and 'event' are incompatible.
    Type 'ChangeEvent<HTMLSelectElement, HTMLSelectElement>' is not assignable to type 'ChangeEvent<HTMLInputElement, Element>'.
      Type 'HTMLSelectElement' is missing the following properties from type 'HTMLInputElement': accept, align, alt, capture, and 38 more.
bad_types.tsx(24,13): error TS2322: Type '{ status: "success"; }' is not assignable to type 'IntrinsicAttributes & StatusProps'.
  Property 'count' is missing in type '{ status: "success"; }' but required in type '{ status: "success"; count: number; }'.
bad_types.tsx(28,32): error TS2322: Type 'string' is not assignable to type '"star" | "user" | undefined'.
bad_types.tsx(32,65): error TS2322: Type '"author"' is not assignable to type '"id" | "title"'.
bad_types.tsx(38,22): error TS2322: Type '{ type: "clear"; }' is not assignable to type 'never'.
bad_types.tsx(41,81): error TS2322: Type '"remove"' is not assignable to type '"add" | "clear"'.
bad_types.tsx(44,15): error TS2554: Expected 1 arguments, but got 0.
bad_types.tsx(49,21): error TS2353: Object literal may only specify known properties, and 'title' does not exist in type 'Iterable<ReactNode> | Promise<AwaitedReactNode> | ReactElement<unknown, string | JSXElementConstructor<any>> | ReactPortal'.
```

Decoding each error

#	Error says	What it means	Fix
1	`Property 'label' is missing`	required prop not passed	pass it, or make it optional with a default
2	`"danger"` not assignable to the union	typo or unsupported variant	use an allowed value, or extend the union
3	`'user' is possibly 'null'`	state can still be `null`	narrow: `user ? ... : ...` or `user?.name`
4	`string` not assignable to `never`	`useState([])` inferred `never[]`	`useState<string[]>([])`
5	handler for `HTMLInputElement` on a `<select>`	wrong element type	`ChangeEvent<HTMLSelectElement>` (in `@types/react` 19 event types have a second type parameter for `target`, normally omitted)
6	`Property 'count' is missing ... required in type '{ status: "success"; count: number; }'`	chosen union member needs `count`	pass `count`
7	`string` not assignable to `"star" | "user" | undefined` on `icon`	with `src` given, no member fits both; the message points at one	pass only one of `src` / `icon`
8	`"author"` not assignable to `"id" | "title"`	column names a field the row lacks	fix the key or add the field
9	`{ type: "clear" }` not assignable to `never`	exhaustiveness check caught an unhandled action	add `case 'clear'`
10	`"remove"` not assignable to `"add" | "clear"`	unknown action dispatched	dispatch a valid one or add it to the union
11	`Expected 1 arguments, but got 0`	React 19 types require `useRef`'s initial value	`useRef<HTMLInputElement>(null)`
12	`'title' does not exist in type ... ReactNode ...`	rendering a plain object (would also crash at runtime)	render a field: `{post.title}`

Other common mistakes

- `as` **everywhere** — silences the checker without checking. Only when you truly know more (`e.target.value as V` above), never to hide a mismatch.
- **Non-null assertions** `user!.name` ("trust me, not null", unchecked) — same risk; narrow instead.
- **Duplicating existing types** — use `ComponentProps<typeof X>`, `ReturnType<typeof fn>`, `Parameters<...>`, or export the props type.
- **Hand-typed API responses trusted blindly** — validate at the boundary (Zod) and derive the type from the schema.
- **Over-engineered generics** — a one-type component doesn't need them.

Self-check

1. Why does `useState([])` cause trouble? → Inferred `never[]`, so nothing can be added.
2. Type for `children`? → `ReactNode`.
3. All props of a native `<input>`? → `ComponentProps<'input'>`.
4. Make a reducer fail to compile on an unhandled action? → Assign `action` to a `never` variable in `default`.
5. React 19: how does a function component receive a ref? → Regular `ref` prop typed `Ref<T>`.

6. Does `tsc` run when Vite builds? → No; the bundler only strips types. Run `tsc --noEmit` in CI or the build script.

Chapter 11 TL;DR

Idea	Remember
Setup	`.tsx`, `@types/react(-dom)`, `strict: true`, `jsx: "react-jsx"`; bundler strips types, `tsc --noEmit` checks them (in CI)
Props	`function C({ a, b = 1 }: Props)`; `?` optional, defaults remove `undefined`; literal unions for variants; function types for callbacks
Children	`ReactNode`; `PropsWithChildren<P>`; native props via `ComponentProps<'button'>` & spread the rest
Events	Inline = inferred; otherwise `ChangeEvent<HTMLInputElement>`, `SubmitEvent<HTMLFormElement>` (older: `FormEvent`), `MouseEvent<...>`, `KeyboardEvent<...>` from `react`; `currentTarget` has your element's type
State	Infer; `useState<T | null>(null)`, `useState<string[]>([])`, literal unions
Reducers	Action = discriminated union; `never` exhaustiveness check
Refs	`useRef<HTMLXxxElement>(null)`; argument required in React 19 types; `ref` is a prop typed `Ref<T>`; `useImperativeHandle` for custom handles
Hooks & Context	`as const` tuples or objects; generic Hooks with union results; `createContext<T | null>(null)` + throwing `useX()`
Generic components	`List<T>` infers `T` from props; constraints with `extends`; `keyof`; `<T,>` in arrow functions
Union props	One member per valid combination; narrow before destructuring; `?: never` to forbid
Errors	Read "A is not assignable to B" bottom-up; avoid `any`, `as`, `!` as escape hatches

Next: Chapter 12 — Testing React.

Chapter 12 — Testing React

A test is a small program that uses your code and checks the result, so a computer — not you clicking around — notices when something breaks. The most useful React tests **render the component, interact with it as a user would, and check what the user would see**. That is the idea behind **React Testing Library** (RTL), the standard tool for component tests. General testing ideas — `node:test` (Node's built-in runner), assertions (checks that fail the test when false), test doubles and mocking (fakes that replace real parts) — are in the JavaScript Handbook (Chapter 16).

Code: `code/`. Every file is a **runnable test file**: React 19.3, React Testing Library 16.3, `@testing-library/user-event` 14.6 and jsdom (a pure-JavaScript imitation of the browser DOM for Node.js), run by `node:test` with `node:assert/strict`. Output is Node's `spec` reporter (`node --test-reporter=spec file.mjs`); times vary per run, and summary lines for suites, cancelled, skipped and todo (all 0) are omitted. Real projects usually run the same tests with Vitest or Jest (the two most popular JS test runners) — §12.9 shows that setup (not run here).

12.1 What to test, and the Testing Library philosophy

What it is

RTL is a small set of helpers on **DOM Testing Library**: `render` a component into a (simulated) DOM, **query** elements the way a person finds them (role, label, text), fire **events**, and assert on the resulting DOM. Guiding principle: the closer a test is to how people use the software, the more you can trust it.

Test this (what users experience)	Not this (implementation details)
The heading shows "Ada Lovelace" after loading	The component's `user` state variable
Clicking **Add to cart** updates the badge to 2	That `dispatch` was called with `{ type: 'add' }`
The **Submit** button is disabled until terms are accepted	Which Hook or CSS class name is used
An error message appears when the server returns 404 Not Found	The number of re-renders

Like an inspector who tastes the food rather than reading recipe cards: a kitchen reorganisation (a **refactor** — restructuring without changing behaviour) still passes; worse food fails.

Why it matters

- **Refactor-proof**: tests that ignore internals survive renaming state, splitting components, `useState` → `useReducer`, or enabling React Compiler (Chapter 10).
- **Accessibility for free** (UI usable by everyone, including screen-reader and keyboard-only users): finding by role and accessible name only works if the UI is accessible (§12.3 catches an unlabelled icon button).
- **Confidence**: clicking the real button through the real event system catches wiring bugs (e.g. a handler never attached) that a unit test of the handler misses.

Where component tests sit

Kind	Checks	Tools	Speed
Unit	one function (a reducer, a formatter)	`node:test`, Vitest, Jest	fastest
Component / integration	one component or feature, simulated DOM	RTL + jsdom (this chapter)	fast
End-to-end (E2E)	the whole app in a real browser against a real or staged (test-copy) backend	Playwright, Cypress	slowest

A common strategy (the "testing trophy"): most effort in **integration-style component tests**, a few E2E tests for critical flows (sign-up, checkout), unit tests for tricky pure logic (output depends only on inputs, no side effects) — plus static checks that don't run code (TypeScript, Chapter 11; ESLint).

12.2 Your first component test

How it works

1. **Arrange** — `render(<Counter />)` mounts the component into `document.body` (jsdom provides the DOM).
2. **Act** — find an element (`screen.getByRole('button', { name: 'Increment' })`) and interact (`fireEvent.click(...)` or `user-event`, §12.4).
3. **Assert** — check the DOM: text, disabled state, presence/absence.
4. **Clean up** — unmount after each test so tests don't see each other's DOM. Automatic in Jest and Vitest (with `globals: true`); with `node:test` register `afterEach(cleanup)`.

`screen` has every query bound to `document.body`, so you needn't keep `render`'s return value.

`code/01_first_test.jsx`:

```
// imports: tools_dom.mjs (jsdom globals), node:test, node:assert/strict, react, @testing-library/react
function Counter({ initial = 0, step = 1 }) {
  const [count, setCount] = useState(initial);
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(c => c + step)}>Increment</button>
      <button onClick={() => setCount(initial)} disabled={count === initial}>Reset</button>
    </div>
  );
}

afterEach(cleanup);

test('shows the initial count', () => {
  render(<Counter initial={5} />);
  assert.ok(screen.getByText('Count: 5'));                          // getBy* throws if not found
});

test('increments by step when the button is clicked', () => {
  render(<Counter step={10} />);
  fireEvent.click(screen.getByRole('button', { name: 'Increment' }));
  fireEvent.click(screen.getByRole('button', { name: 'Increment' }));
  assert.equal(screen.getByText(/Count:/).textContent, 'Count: 20');
});

test('Reset is disabled until the count changes', () => {
  render(<Counter />);
  const reset = screen.getByRole('button', { name: 'Reset' });
  assert.equal(reset.disabled, true);
  fireEvent.click(screen.getByRole('button', { name: 'Increment' }));
  assert.equal(reset.disabled, false);
  fireEvent.click(reset);
  assert.equal(screen.getByText(/Count:/).textContent, 'Count: 0');
});
```

Output:

```
✔ shows the initial count (13.84825ms)
✔ increments by step when the button is clicked (50.187958ms)
✔ Reset is disabled until the count changes (3.544833ms)
ℹ tests 3
ℹ pass 3
ℹ fail 0
ℹ duration_ms 70.571583
```

What about `act()`?

React batches updates and runs effects asynchronously. `act(() => { ... })` means "apply everything this caused before I look at the DOM". RTL's `render`, `fireEvent`, `user-event` actions and async utilities already wrap themselves in `act`, so you rarely write it — except for **direct** state updates, e.g. calling a Hook's function in `renderHook` (§12.7) or advancing fake timers. The warning *"An update to X inside a test was not wrapped in act(...)"* usually means something finished **after** your assertions — await it with `findBy` / `waitFor` (§12.5).

With jest-dom (in Vitest/Jest)

`@testing-library/jest-dom` adds readable matchers with good failure messages: `expect(reset).toBeDisabled()`, `expect(el).toHaveTextContent('Count: 20')`, `expect(el).toBeInTheDocument()`. This book uses plain `assert` on the same DOM properties (`.disabled`, `.textContent`).

12.3 Queries and their priority

Query variants

The prefix decides **what happens with zero or several matches**, and whether it waits:

Variant	0 matches	1 match	2+ matches	Waits?	Use it to…
`getBy…`	throws	returns element	throws	no	find something that must be there **now**
`queryBy…`	returns `null`	returns element	throws	no	assert something is **absent**
`findBy…`	rejects	resolves element	rejects	**yes** (retries, default 1000 ms)	wait for something to **appear**
`getAllBy…`	throws	array	array	no	find several
`queryAllBy…`	`[]`	array	array	no	count, possibly zero
`findAllBy…`	rejects	array	array	**yes**	wait for several

`findBy…` returns a **Promise** that resolves with the element or rejects after the timeout — `await` it.

Which query to use (priority order)

The higher, the closer to how people (including screen-reader users) find things:

1. **Accessible to everyone**
    1. `ByRole` — by ARIA role (ARIA, Accessible Rich Internet Applications: attributes telling assistive tools what an element is; the *role* is its kind — button, link, heading, textbox, checkbox, list…) and **accessible name** (`{ name: 'Save' }`). Use for almost everything.
    2. `ByLabelText` — form fields by `<label>`.
    3. `ByPlaceholderText` — only if a field truly has no label (a placeholder isn't a label).
    4. `ByText` — non-interactive text.
    5. `ByDisplayValue` — a field by its current value.
2. **Semantic attributes**: `ByAltText` (images, `alt`); `ByTitle` (`title`, not reliably announced by screen readers).
3. **Test IDs**: `ByTestId` — `data-testid="…"`, only when nothing user-visible identifies the element.

The **accessible name** is what assistive technology announces: a button's text, an input's label, `aria-label` / `aria-labelledby`, an image's `alt`. Matchers can be exact strings (`'Save'`), case-insensitive regexes (`/save/i`) or functions.

`code/02_queries.jsx` (component and key tests):

```
function ProductCard({ product, inCart }) {
  return (
    <article aria-labelledby="p-title">
      <img src={product.image} alt={`Photo of ${product.name}`} />
      <h2 id="p-title">{product.name}</h2>
      <p>{product.price.toFixed(2)} EUR</p>
      <label htmlFor="qty">Quantity</label>
      <input id="qty" type="number" defaultValue={1} />
      <input placeholder="Gift message (optional)" />
      <ul aria-label="features">{product.features.map(f => <li key={f}>{f}</li>)}</ul>
      <button>Add to cart</button>
      <button title="Save for later">♡</button>
      {inCart && <p role="status">Already in your cart</p>}
      <span data-testid="sku">{product.sku}</span>
    </article>
  );
}

// priority 1 (role, label, placeholder, text, display value):
screen.getByRole('heading', { name: 'Desk lamp', level: 2 });
screen.getByRole('article', { name: 'Desk lamp' });            // name from aria-labelledby
screen.getByLabelText('Quantity');  screen.getByPlaceholderText(/gift message/i);
screen.getByText('39.50 EUR');      screen.getByDisplayValue('1');
// priority 2 (alt text, title):
screen.getByAltText(/photo of desk lamp/i);  screen.getByTitle('Save for later');
// A button's accessible name comes from its CONTENT first; title is only a fallback. So a screen
// reader announces this icon button as "♡" - the test just revealed an accessibility bug.
screen.getByRole('button', { name: '♡' });                      // found
screen.queryByRole('button', { name: 'Save for later' });       // null
// Fix in the component: <button aria-label="Save for later">♡</button>
// many matches, scoped search:
within(screen.getByRole('list', { name: 'features' })).getAllByRole('listitem');   // LED, Dimmable, USB-C
// absence, then presence after rerender(<ProductCard product={lamp} inCart />):
screen.queryByRole('status');                                   // null -> then getByRole('status')
// ... plus: a failing getBy's message, logRoles, and getByTestId('sku')
```

Output:

```
    error message (first 3 lines):
    | Unable to find an accessible element with the role "button" and name "Buy now"
    |
    | Here are the accessible roles:
    logRoles output:
    | list:
    | Name "tiny":
    | <ul
    |   aria-label="tiny"
    | />
    | --------------------------------------------------
    | listitem:
    | Name "":
    | <li />
    | --------------------------------------------------
✔ priority 1: queries everyone can perceive (role, label, placeholder, text, display value) (61.198666ms)
✔ priority 2: semantic attributes (alt text, title) (6.331375ms)
✔ priority 3: test IDs, only when nothing user-visible identifies the element (2.198959ms)
✔ getAllBy for many matches, within() to search inside one element (8.569083ms)
✔ queryBy to assert that something is NOT there (4.234708ms)
✔ a failing getBy explains itself (13.676958ms)
✔ logRoles prints every role and accessible name - handy when writing queries (2.698125ms)
ℹ tests 7
ℹ pass 7
ℹ fail 0
```

Notes on the output

- A failing `getBy` lists **every accessible role and name** (cut to 3 lines here), then the DOM — usually telling you at once whether the name is wrong or the element is missing.

- `logRoles(container)` prints the role list on demand; `screen.debug()` prints the DOM. For writing tests, not leaving in them.
- Common roles: `button`, `link`, `heading` (with `level`), `textbox`, `checkbox`, `radio`, `combobox` (a `<select>`), `spinbutton` (number input), `list` / `listitem`, `table` / `row` / `cell`, `dialog`, `alert`, `status`, `img`, `navigation`, `main`.
- The icon button had a `title` but was announced as "♡" — role queries make such bugs visible.

Common mistakes

Mistake	Better
`container.querySelector('.btn-primary')`	`screen.getByRole('button', { name: 'Save' })`
`getByTestId` everywhere	role/label/text first; test IDs last resort
`expect(queryByText('x')).toBeTruthy()` to check presence	`getByText` (throws with a helpful message)
`getByText` to assert absence (it throws)	`queryByText(...) === null`
`getBy` for things that appear later	`findBy` (§12.5)
Exact long text that breaks on copy edits	regex for the stable part: `/order confirmed/i`

12.4 `user-event` — interacting like a user

What it is

`fireEvent` dispatches **one** DOM event. A real user typing "hi" produces focus, click, and per key `keydown`, `keypress`, `input`, `keyup`; a real click on a disabled button does nothing; Tab moves focus in document order. `@testing-library/user-event` simulates those full interactions, exercising the same paths as real users (including bugs that only appear with the full sequence, like a `keydown` handler).

```
import userEvent from '@testing-library/user-event';
const user = userEvent.setup();            // v14 API: create a session, then await each action
await user.type(input, 'Ada');
await user.click(button);
await user.selectOptions(select, 'pro');
await user.keyboard('{Enter}');            // special keys in braces: {Enter} {Escape} {Backspace} {Shift>}...
await user.tab();
await user.clear(input);
await user.upload(fileInput, file);
await user.hover(element);
```

All methods are **async** — always `await` them.

`code/03_user_event.jsx` (abridged; `SignupForm` has a Name input, a Plan select, a terms checkbox, and a submit button disabled until terms and name are set):

```
test('fireEvent vs userEvent: which DOM events does "typing" produce?', async () => {
  render(<input aria-label="box" />);
  const box = screen.getByLabelText('box');
  const seen = [];
  for (const type of ['focus', 'keydown', 'keypress', 'input', 'keyup', 'click'])
    box.addEventListener(type, () => seen.push(type));
  fireEvent.change(box, { target: { value: 'hi' } });
  console.log('    fireEvent.change   ->', seen.join(', ') || '(no listened-to events; only a synthetic change)');
  seen.length = 0; box.value = '';
  const user = userEvent.setup();
  await user.type(box, 'hi');
  console.log('    user.type(box, hi) ->', seen.join(', '));
});

test('fills in and submits the whole form like a user', async () => {
  const user = userEvent.setup();
  const submitted = [];
  render(<SignupForm onSubmit={data => submitted.push(data)} />);
  const button = screen.getByRole('button', { name: 'Create account' });
  assert.equal(button.disabled, true);
  await user.type(screen.getByRole('textbox', { name: 'Name' }), 'Ada');
  await user.selectOptions(screen.getByRole('combobox', { name: 'Plan' }), 'pro');
  await user.click(screen.getByRole('checkbox', { name: 'I accept the terms' }));
  assert.equal(button.disabled, false);
  await user.click(button);
  assert.deepEqual(submitted, [{ name: 'Ada', plan: 'pro', terms: true }]);
});

test('keyboard only: Tab moves focus in DOM order, Enter submits', async () => {
  // user.tab() focuses Name; user.keyboard('Grace'); two more tabs reach the checkbox;
  // user.keyboard(' ') toggles it (Space); '{Enter}' in the text field submits the form
});

test('clicking a disabled button does nothing', async () => {
  const user = userEvent.setup();
  let clicks = 0;
  render(<button disabled onClick={() => clicks++}>Pay</button>);
  await user.click(screen.getByRole('button', { name: 'Pay' }));
  assert.equal(clicks, 0);
});
```

Output:

```
    fireEvent.change   -> (no listened-to events; only a synthetic change)
    user.type(box, hi) -> focus, click, keydown, keypress, input, keyup, keydown, keypress, input, keyup
✔ fireEvent vs userEvent: which DOM events does "typing" produce? (72.232958ms)
✔ fills in and submits the whole form like a user (55.993083ms)
✔ keyboard only: Tab moves focus in DOM order, Enter submits (49.1405ms)
✔ clicking a disabled button does nothing (7.998ms)
ℹ tests 4
ℹ pass 4
ℹ fail 0
```

`fireEvent` VS `user-event`

	`fireEvent`	`user-event`
What it does	dispatches exactly one event	simulates the full interaction (focus, pointer, keyboard, input)
Sync/async	sync	async (`await`)
Respects disabled, `pointer-events: none`, focus order	no	yes
When to use	events user-event doesn't model (`scroll` , custom events), very low-level tests	**default** for anything a user does

The keyboard test doubles as an accessibility check: the form works without a mouse.

12.5 Async tests

The problem it solves

Right after `render`, loaded data isn't there, so `getBy` fails. **Wait for the UI**, not a fixed time — `await sleep(500)` is slow and **flaky** (passes or fails with no code change, e.g. fine locally, failing on a busy CI server).

Tools

Tool	Waits for	Typical use
`await screen.findByRole(...)`	an element to appear (retries a `getBy`)	data loaded, dialog opened
`await waitFor(() => { ...assertions... })`	a callback to stop throwing	text change, a mock being called
`await waitForElementToBeRemoved(() => screen.queryByText('Loading…'))`	an element to disappear	spinners, closing dialogs

All retry every 50 ms (default) until success or timeout (default 1000 ms), wrapping updates in `act`.

`code/04_async.jsx` (abridged):

```
const api = {
  loadOrders: () => new Promise(r => setTimeout(() => r([{ id: 1, total: 20 }, { id: 2, total: 35 }]), 50)),
  save: () => new Promise(r => setTimeout(r, 30)),
};
function Orders() {
  const [orders, setOrders] = useState(null);
  const [saved, setSaved] = useState(false);
  useEffect(() => { api.loadOrders().then(setOrders); }, []);
  if (!orders) return <p>Loading orders…</p>;
  return (<>
    <ul>{orders.map(o => <li key={o.id}>Order {o.id}: {o.total} EUR</li>)}</ul>
    <button onClick={async () => { await api.save(); setSaved(true); }}>Save</button>
    {saved && <p role="status">Saved!</p>}
  </>);
}

test('findBy waits (retries up to 1000 ms by default) until the element appears', async () => {
  render(<Orders />);
  const items = await screen.findAllByRole('listitem');
  assert.deepEqual(items.map(li => li.textContent), ['Order 1: 20 EUR', 'Order 2: 35 EUR']);
});
// waitForElementToBeRemoved(() => screen.queryByText('Loading orders…')) then 2 listitems
test('waitFor retries any assertion until it passes', async () => {
  const user = userEvent.setup();
  render(<Orders />);
  await user.click(await screen.findByRole('button', { name: 'Save' }));
  await waitFor(() => assert.equal(screen.getByRole('status').textContent, 'Saved!'));
});
test('findBy gives up after its timeout with a clear error', async () => {
  render(<Orders />);
  await assert.rejects(screen.findByText('Order 99', {}, { timeout: 200 }), /Unable to find an element with the
text: Order 99/);
});
```

Output:

```
✔ right after render only the loading state exists (14.355875ms)
✔ findBy waits (retries up to 1000 ms by default) until the element appears (98.487708ms)
✔ waitForElementToBeRemoved waits for the spinner to go away (55.889ms)
✔ waitFor retries any assertion until it passes (101.515625ms)
    rejected after ~200 ms
✔ findBy gives up after its timeout with a clear error (202.66725ms)
ℹ tests 5
ℹ pass 5
ℹ fail 0
```

Rules for `waitFor`

- **Only assertions** inside — it may run many times; actions go outside.
- One thing to wait for per `waitFor`; assert the rest normally.
- Prefer `findBy` for elements (reads better, better errors); never wrap `findBy` in `waitFor`.

Common mistakes

- Missing `await` before `findBy` / `waitFor` / `user.*` — the test ends before the UI updates, passes by accident, and prints an `act(...)` warning.
- Arbitrary `sleep`s — wait for a **condition**. For truly time-based code — debounce (act once input has paused), polling (re-asking the server every few seconds) — use fake timers (§12.7).

12.6 Mocking `fetch` (and when to use MSW)

What it is

A **mock** replaces a real dependency with a controllable fake. Tests shouldn't hit a real server (slow, needs network, data changes). Mocking `fetch` lets each test choose the response — success, an HTTP error (a 4xx/5xx status; see System Design Fundamentals), a network failure, a slow reply — and check the UI for each.

With `node:test`, `mock.method(globalThis, 'fetch', impl)` replaces the global `fetch` and records calls; `mock.restoreAll()` restores it. (Vitest: `vi.spyOn(globalThis, 'fetch').mockResolvedValue(...)`; Jest: `jest.spyOn(global, 'fetch')`.) Node 18+ has `Response` built in, so fakes return real `Response` objects.

`code/05_mocking_fetch.jsx` (abridged):

```
function UserProfile({ userId }) {
  const [state, setState] = useState({ status: 'loading' });
  useEffect(() => {
    const controller = new AbortController();
    setState({ status: 'loading' });
    fetch(`/api/users/${userId}`, { signal: controller.signal })
      .then(res => { if (!res.ok) throw new Error(`Server said ${res.status}`); return res.json(); })
      .then(user => setState({ status: 'done', user }))
      .catch(err => { if (err.name !== 'AbortError') setState({ status: 'error', message: err.message }); });
    return () => controller.abort();                  // cancel if userId changes or we unmount
  }, [userId]);
  if (state.status === 'loading') return <p>Loading…</p>;
  if (state.status === 'error') return <p role="alert">Could not load user: {state.message}</p>;
  return <h1>{state.user.name}</h1>;
}

const json = (body, status = 200) => new Response(JSON.stringify(body), { status, headers: { 'Content-Type':
'application/json' } });
afterEach(() => { cleanup(); mock.restoreAll(); });    // put the real fetch back after every test

test('shows the user returned by the server, and calls the right URL', async () => {
  const fetchMock = mock.method(globalThis, 'fetch', async () => json({ id: 7, name: 'Ada Lovelace' }));
  render(<UserProfile userId={7} />);
  assert.equal((await screen.findByRole('heading')).textContent, 'Ada Lovelace');
  assert.equal(fetchMock.mock.calls[0].arguments[0], '/api/users/7');
});
// HTTP error: fetch resolves json({...}, 404) -> alert "Could not load user: Server said 404" (fetch does not
reject on 404)
// network error: fetch throws new TypeError('Failed to fetch') -> alert matches /Failed to fetch/
test('a slow old response cannot overwrite a newer one (race condition)', async () => {
  // user 1 answers after 100 ms, user 2 after 10 ms; honour the abort signal like real fetch does
  const delays = { '/api/users/1': 100, '/api/users/2': 10 };
  mock.method(globalThis, 'fetch', (url, { signal }) => new Promise((resolve, reject) => {
    const t = setTimeout(() => resolve(json({ name: `User from ${url}` })), delays[url]);
    signal.addEventListener('abort', () => { clearTimeout(t); reject(new DOMException('aborted', 'AbortError')); });
}));
  const { rerender } = render(<UserProfile userId={1} />);
  rerender(<UserProfile userId={2} />);                  // user switches before the first answer
  assert.equal((await screen.findByRole('heading')).textContent, 'User from /api/users/2');
  await new Promise(r => setTimeout(r, 150));              // wait past the slow response
  assert.equal(screen.getByRole('heading').textContent, 'User from /api/users/2');
});
// ... plus a test that clicks "Next user" and waits for the second request's result
```

Output:

```
✔ shows the user returned by the server, and calls the right URL (67.915ms)
✔ shows an error for an HTTP error status (fetch does not reject on 404) (6.110375ms)
✔ shows an error when the network fails (4.473667ms)
✔ a slow old response cannot overwrite a newer one (race condition) (168.044584ms)
✔ mock only the network call, then drive the UI as a user (20.320625ms)
ℹ tests 5
ℹ pass 5
ℹ fail 0
```

Does the race test really test something? (A **race condition**: the result depends on which overlapping operation finishes first.) A test that can't fail is worthless. Removing the `controller.abort()` cleanup (in a temporary copy) makes it fail:

```
✖ a slow old response cannot overwrite a newer one (race condition) (168.159708ms)
  + actual - expected
    actual: 'User from /api/users/1',
    expected: 'User from /api/users/2',
```

Checking that a test fails on broken code is a good habit. (Race and `AbortController` fix: Chapter 5.) The last test shows the preferred style: mock **only the network boundary**, drive the real UI with `user-event`.

Mock Service Worker (MSW) — the usual choice in real projects

MSW intercepts requests at the network level (in the browser via a Service Worker — a background script between page and network) instead of replacing `fetch`, so it works for `fetch`, `axios`, TanStack Query and anything else, and the same handlers (functions choosing the fake response per URL) can power the dev server. Sketch (MSW 2 — not run here):

```
// test/server.js
import { setupServer } from 'msw/node';
import { http, HttpResponse } from 'msw';
export const server = setupServer(
  http.get('/api/users/:id', ({ params }) => HttpResponse.json({ id: params.id, name: 'Ada Lovelace' })),
);
// setupTests.js
beforeAll(() => server.listen());
afterEach(() => server.resetHandlers());       // undo per-test overrides
afterAll(() => server.close());
// in one test: server.use(http.get('/api/users/:id', () => new HttpResponse(null, { status: 404 })));
```

Approach	Good for	Drawback
Replace `fetch` (`mock.method`, `vi.spyOn`)	small projects, few tests, no dependency	tied to `fetch`; response setup repeated per test
MSW	real apps; any HTTP client; shared handlers for tests, Storybook (components in isolation) and dev	one more library and setup file
Mock your own API module (`vi.mock('./api')`)	quick isolation	skips your fetch/parse/error code — lower confidence

Components using **TanStack Query** need a `QueryClientProvider` in the wrapper — a **new** `QueryClient` per test with `retry: false`, so tests don't share cache and failures surface immediately (Chapter 9; providers in §12.8).

12.7 Testing custom Hooks

Two approaches

1. **Through a component** using the Hook as the app does — most realistic, often enough.
2. **`renderHook`** for reusable Hooks with a clear API (`useCounter`, `useDebounce`): renders a hidden component calling your Hook and exposes the latest return as `result.current`.

Hooks can't be called outside a component (Chapter 7), so both render something.

`renderHook`, `act`, and `rerender`

- `const { result, rerender, unmount } = renderHook(() => useX(args), { initialProps, wrapper })`.
- Read `result.current` **after** each update — it's replaced every render.
- Wrap state-updating calls in **`act(() => ...)`** (no RTL helper is doing it).
- `rerender(newProps)` re-renders the same Hook instance with new arguments (`initialProps` + a callback taking props).
- Context-dependent Hooks take a `wrapper` as in §12.8.

Fake timers

Time-based Hooks (debounce, throttle — at most once per interval — polling, timeouts) shouldn't make tests wait. **Fake timers** replace `setTimeout` & co. with manually advanced versions: `node:test` `mock.timers.enable({ apis: ['setTimeout'] })` / `mock.timers.tick(ms)`; Vitest `vi.useFakeTimers()` / `vi.advanceTimersByTime(ms)`; Jest `jest.useFakeTimers()` / `jest.advanceTimersByTime(ms)`. Advance inside `act`.

`code/06_testing_hooks.jsx` (abridged):

```
function useCounter(initial = 0, { min = -Infinity, max = Infinity } = {}) {
  const [count, setCount] = useState(initial);
  const increment = useCallback(() => setCount(c => Math.min(max, c + 1)), [max]);
  const decrement = useCallback(() => setCount(c => Math.max(min, c - 1)), [min]);
  const reset = useCallback(() => setCount(initial), [initial]);
  return { count, increment, decrement, reset };
}
function useDebounce(value, delay) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);
  }, [value, delay]);
  return debounced;
}
afterEach(() => { cleanup(); mock.timers.reset(); });

test('useCounter: increments, respects max, resets', () => {
  const { result } = renderHook(() => useCounter(8, { max: 10 }));
  act(() => result.current.increment());              // state updates must be wrapped in act()
  act(() => result.current.increment());
  act(() => result.current.increment());              // would be 11, capped at 10
  assert.equal(result.current.count, 10);
  act(() => result.current.reset());
  assert.equal(result.current.count, 8);
});
test('useCounter: rerender with new arguments', () => {
  const { result, rerender } = renderHook(({ max }) => useCounter(0, { max }), { initialProps: { max: 1 } });
  act(() => { result.current.increment(); result.current.increment(); });
  assert.equal(result.current.count, 1);
  rerender({ max: 5 });                               // same Hook instance, new props
  act(() => result.current.increment());
  assert.equal(result.current.count, 2);
});
test('useDebounce: only the last value within the delay gets through', () => {
  mock.timers.enable({ apis: ['setTimeout'] });
  const { result, rerender } = renderHook(({ v }) => useDebounce(v, 300), { initialProps: { v: 'r' } });
  rerender({ v: 're' });
  act(() => mock.timers.tick(200));
  rerender({ v: 'rea' });                             // restarts the 300 ms timer
  act(() => mock.timers.tick(299));
  assert.equal(result.current, 'r');                  // nothing has settled yet
  act(() => mock.timers.tick(1));
  assert.equal(result.current, 'rea');                // 're' was skipped entirely
});
// + 'useCounter through a component': a <Stepper> with -/+ buttons, clicked with user-event, cannot go below min
```

Output:

```
✔ useCounter: increments, respects max, resets (7.892375ms)
✔ useCounter: rerender with new arguments (1.192583ms)
✔ useDebounce: only the last value within the delay gets through (1.022ms)
✔ useCounter through a component: cannot go below min (79.11725ms)
ℹ tests 4
ℹ pass 4
ℹ fail 0
```

The debounce test covers 800 ms of "time" in about 1 ms.

Common mistakes

- `const { count } = result.current` once, expecting it to update — re-read `result.current.count` after each `act`.
- Calling the Hook directly outside a component → "Invalid hook call".
- Leaving fake timers on — reset in `afterEach`. `user-event` waits between actions with timers, so with fake timers in Vitest/Jest use `userEvent.setup({ advanceTimers: vi.advanceTimersByTime })` (or `jest.advanceTimersByTime`).

- Importing `renderHook` from `@testing-library/react-hooks` — that package is for React 17 and older; since RTL 13.1 it's in `@testing-library/react`.

12.8 Providers, routers and custom render

The problem it solves

Real components depend on providers: context (theme, auth, cart), a router, a TanStack Query client, a Redux store. Rendered alone they crash; repeating the provider tree per test is noisy. So projects write one **custom** `render` wrapping the UI via RTL's `wrapper` option.

`code/07_providers_and_router.jsx` (abridged):

```
export function cartReducer(state, action) {            // pure function: unit-test it directly
  switch (action.type) {
    case 'add': return { ...state, [action.sku]: (state[action.sku] ?? 0) + 1 };
    case 'clear': return {};
    default: throw new Error(`unknown action ${action.type}`);
  }
}
// CartProvider (useReducer + CartContext); CartBadge reads the cart; ProductPage reads :sku and dispatches 'add';
// App = <nav> with <Link>s and <CartBadge/> + <Routes> for "/" and "/p/:sku".

function renderWithProviders(ui, { route = '/', cart = {} } = {}) {
  const Wrapper = ({ children }) => (
    <MemoryRouter initialEntries={[route]}>            {/* a router that keeps history in memory */}
      <CartProvider initial={cart}>{children}</CartProvider>
    </MemoryRouter>
  );
  return { user: userEvent.setup(), ...render(ui, { wrapper: Wrapper }) };
}

test('reducer: plain unit test, no React needed', () => {
  assert.deepEqual(cartReducer({ MUG: 1 }, { type: 'add', sku: 'MUG' }), { MUG: 2 });
  assert.throws(() => cartReducer({}, { type: 'oops' }), /unknown action oops/);
});
test('starts on a given route with a pre-filled cart', () => {
  renderWithProviders(<App />, { route: '/p/LAMP', cart: { MUG: 2 } });
  assert.equal(screen.getByRole('heading').textContent, 'Product LAMP');
  assert.equal(screen.getByLabelText('items in cart').textContent, '2');
});
test('navigates with a link, then adds to cart', async () => {
  const { user } = renderWithProviders(<App />);
  await user.click(screen.getByRole('link', { name: 'Mug' }));
  await user.click(screen.getByRole('button', { name: 'Add to cart' }));
  await user.click(screen.getByRole('button', { name: 'Add to cart' }));
  assert.equal(screen.getByLabelText('items in cart').textContent, '2');
});
test('without the provider the component crashes - which is why the wrapper exists', () => {
  assert.throws(() => render(<CartBadge />), /Cannot destructure property 'cart'/);   // console.error silenced
});
```

Output:

```
✔ reducer: plain unit test, no React needed (0.762041ms)
✔ starts on a given route with a pre-filled cart (58.361708ms)
✔ navigates with a link, then adds to cart (45.55225ms)
✔ without the provider the component crashes - which is why the wrapper exists (2.687916ms)
ℹ tests 4
ℹ pass 4
ℹ fail 0
```

How it works

1. `render(ui, { wrapper: Wrapper })` renders `<Wrapper>{ui}</Wrapper>`; `rerender` keeps the wrapper.

2. `MemoryRouter` (from `react-router`) keeps history in memory; `initialEntries` sets the starting URL. For data-router apps (`createBrowserRouter`, loaders/actions), use `createMemoryRouter(routes, { initialEntries })` + `<RouterProvider>`.
3. Options like `route` and `cart` start each test from a state **through the public API** (provider props), not by poking internals.
4. Put `renderWithProviders` in a shared `test-utils` file. With Redux Toolkit create a **fresh store per test**; with TanStack Query a **fresh** `QueryClient` — shared instances leak state between tests.

Unit tests still have a place

The pure reducer gets a plain, very fast unit test. Test pure logic (reducers, formatters, validation schemas) directly; test **components** through the UI.

12.9 Setting up Vitest or Jest in a real project

Real projects usually use **Vitest** (natural for Vite; Jest-compatible API) or **Jest**. Component tests look the same — only imports, `expect` and mock helpers differ. Standard setups, **not run in this book**.

Vitest (Vite projects)

```
npm install -D vitest jsdom @testing-library/react @testing-library/dom @testing-library/user-event @testing-library/jest-dom
```

```
// vite.config.js (or vitest.config.js)
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',          // simulated browser DOM (alternative: 'happy-dom')
    globals: true,                 // describe/test/expect/afterEach as globals -> RTL auto-cleanup works
    setupFiles: './src/setupTests.js',
  },
});
```

```
// src/setupTests.js
import '@testing-library/jest-dom/vitest';   // adds toBeInTheDocument(), toBeDisabled(), toHaveTextContent()...
```

```
// package.json
{ "scripts": { "test": "vitest", "test:run": "vitest run", "coverage": "vitest run --coverage" } }
```

```
// Counter.test.jsx
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { Counter } from './Counter';

test('increments', async () => {
  const user = userEvent.setup();
  render(<Counter />);
  await user.click(screen.getByRole('button', { name: /increment/i }));
  expect(screen.getByText('Count: 1')).toBeInTheDocument();
});
```

With TypeScript add `"types": ["vitest/globals", "@testing-library/jest-dom"]` to `tsconfig.json`. Without `globals: true`, import `test`, `expect`, `afterEach` from `vitest` and call `afterEach(cleanup)` yourself.

Jest

```
npm install -D jest jest-environment-jsdom babel-jest @babel/preset-env @babel/preset-react \
  @testing-library/react @testing-library/dom @testing-library/user-event @testing-library/jest-dom
```

```
// jest.config.js
module.exports = {
  testEnvironment: 'jsdom',                       // needs jest-environment-jsdom (separate since Jest 28)
  setupFilesAfterEnv: ['<rootDir>/jest.setup.js'], // runs after the test framework is installed
  moduleNameMapper: { '\\.(css|less|svg)$': '<rootDir>/test/fileMock.js' },  // stub non-JS imports
};
// jest.setup.js
import '@testing-library/jest-dom';
// babel.config.js
module.exports = { presets: ['@babel/preset-env', ['@babel/preset-react', { runtime: 'automatic' }]] };
```

TypeScript with Jest needs `ts-jest` or Babel's TypeScript preset. Vitest reuses Vite's transform pipeline, so needs none of this — one reason new Vite projects choose it.

Useful extras

Tool	For
`@testing-library/jest-dom`	readable DOM matchers
MSW	network mocking (§12.6)
`vitest --coverage` / `jest --coverage`	coverage (which lines ran) — a guide to untested code, not a goal
`eslint-plugin-testing-library`, `eslint-plugin-jest-dom`	lint rules catching misuse (`getBy` inside `waitFor`, missing `await`)
Vitest Browser Mode, Playwright component testing	component tests in a real browser when jsdom isn't enough (layout, real CSS, canvas)
Playwright / Cypress	end-to-end tests

What jsdom can't do

jsdom implements DOM and events but **no layout or painting**: sizes/positions are 0, `scrollTop` changes only when you set it (as in Chapter 10's virtualization demo), CSS media queries don't apply, and `IntersectionObserver` (element scrolled into view), `ResizeObserver` (size changes) and `matchMedia` (media query from JS) are absent by default and must be mocked. Real-rendering behaviour belongs in browser-based or E2E tests.

12.10 Good tests, bad tests, and the bigger picture

What makes a good component test

1. **Named after behaviour**: "shows an error when the server returns 404", not "test 3" or "sets error state".
2. **Arrange–act–assert**, one behaviour per test (several asserts fine if one behaviour).
3. **Public interface**: props in, DOM out, user events between, network mocked at the boundary.
4. **Independent**: no shared DOM, mocks, stores or caches; any order works.
5. **Deterministic**: no real network, real time or randomness (or seeded — repeatable from a fixed start value); waits for conditions, never sleeps.
6. **Fails for the right reason**: break the feature → red with a clear message (§12.6's race test).

Brittle tests to avoid

A **brittle** test breaks on harmless code changes even though the feature works.

Brittle	Why	Instead
Asserting on state, props, or Hook internals	breaks on refactor, passes when UI is broken	assert on rendered output
`container.querySelector('.css-1x2y3z')`	class names are implementation details	role/label/text queries
Large **snapshot** tests (saved rendered output; fails on any change) of whole components	any markup change fails; reviewers approve new snapshots unread	explicit assertions; small inline snapshots at most
Mocking child components	no longer checks real integration	render the real tree; mock only network or truly heavy leaf modules
Asserting a handler was called with internal arguments	couples to wiring	assert the handler's visible effect
Testing third-party libraries	not your code	test your usage via the UI

Real-world examples

- Checkout form: "button disabled until valid", "shows field errors", "submits the right payload" (checked at the mocked network), "shows server error", plus one E2E test of the full purchase.
- Design-system `Dialog`: role `dialog`, labelled by its title, focus moves inside on open, Escape closes, focus returns to the trigger — all testable with RTL + user-event, and each an accessibility requirement (Chapter 14).

In the interview

30-second answer: "React Testing Library with Vitest or Jest. Tests render the component, find elements the way users do — role and accessible name first — interact with user-event, and assert on the DOM, not state or internals. Async UI is awaited with `findBy` / `waitFor`, never sleeps. The network is mocked at the boundary, ideally with MSW, covering success, errors and races. Hooks are tested through components or `renderHook` with `act`; provider-dependent components use a custom render with a wrapper. Playwright E2E covers the few critical flows."

- *`getBy` vs `queryBy` vs `findBy`?* — `getBy` throws if missing (must exist now), `queryBy` returns `null` (assert absence), `findBy` returns a retrying promise (async appearance).
- *Why prefer `getByRole`?* — Matches how users and assistive technology find elements, doubles as an accessibility check, survives markup refactors.
- *What does `act` do and when do you need it?* — Flushes React updates and effects before assertions; RTL helpers wrap it, so only for direct updates (`renderHook` calls, advancing fake timers).
- *How do you test a component that fetches?* — Mock the network (MSW or a `fetch` spy), `await findBy…` the loaded UI, test error and loading states too.

Self-check

1. Query for a "Save" button? → `screen.getByRole('button', { name: 'Save' })`.
2. Assert an error is **not** shown? → `assert.equal(screen.queryByRole('alert'), null)` (or `expect(...).not.toBeInTheDocument()`).
3. Why is `await new Promise(r => setTimeout(r, 500))` bad for waiting on data? → Slow and flaky; wait for the condition with `findBy` / `waitFor`.
4. Test `useDebounce` without waiting? → Fake timers advanced inside `act`.
5. Why a fresh store/query client per test? → Shared instances leak state and make tests order-dependent.
6. Why must `node:test` files call `afterEach(cleanup)`? → RTL auto-cleans only when a global `afterEach` exists (Jest, Vitest with globals).

Chapter 12 TL;DR

Idea	Remember
Philosophy	Test what users see and do; ignore implementation details
Basic test	`render` → `screen` query → interact → assert → `cleanup` (automatic in Jest/Vitest)
Query priority	Role (+ name) → label → placeholder → text → display value → alt → title → test ID
Variants	`get` now / `query` maybe-absent / `find` wait; `All` for many; `within` to scope
Interactions	`const user = userEvent.setup(); await user.click/type/selectOptions/keyboard/tab(...)`; prefer over `fireEvent`
Async	`findBy`, `waitFor(() => assertion)`, `waitForElementToBeRemoved`; never sleep
Network	Mock at the boundary (MSW, or a `fetch` spy returning `Response`s); cover success, HTTP error, network error, races; restore mocks
Hooks	Through a component, or `renderHook` + `act` + `result.current`, `rerender` for new args; fake timers for time
Providers	Custom render with `wrapper`: `MemoryRouter`, context, fresh store/query client per test; unit-test pure functions directly
Setup	Vitest (`jsdom`, `globals`, jest-dom setup) or Jest (`jest-environment-jsdom`, Babel/TS, `setupFilesAfterEnv`)
Quality	Behaviour-named, independent, deterministic, fails for the right reason; few snapshots; E2E for critical paths

Next: Chapter 13 — Server Rendering and Frameworks.

Chapter 13 — Server Rendering and Frameworks

This chapter moves React onto the **server**: what the server sends (captured byte by byte), how the browser takes that HTML over (**hydration** — React attaching handlers and state to HTML already on screen), what breaks when the two disagree, how slow parts stream in later (as extra pieces of the same response), and how **React Server Components** run some components *only* on the server. Then the two frameworks that package this — **Next.js** 16 (App Router) and **React Router** 8 (framework mode) — and how to choose. Chapter 1 §1.4 introduced the vocabulary (MPA — multi-page app, a new HTML page per click; SPA/CSR — one page, JavaScript in the client draws every screen; SSR and SSG, recapped below) with a tiny `renderToString` + `hydrateRoot` example; this builds on it.

Code: `code/` — React 19.3 on Node 24. Files `01`–`05` are runnable; `04_ssr_server_hydrate.jsx` starts a real `node:http` server, loads the page in headless Chrome (no window, script-driven) and takes this chapter's screenshots. Next.js and React Router snippets are **(not run)**: they need the framework's bundler. Background: Suspense and `use()` (Chapter 9), Effects (Chapter 5), `useSyncExternalStore` and `useId` (Chapter 7), Form Actions (Chapter 4); HTTP basics: How the web works.

13.1 Rendering strategies: CSR, SSR, SSG, ISR, streaming, PPR

What it is

A **rendering strategy** answers, per page: *where* is HTML produced (browser or server) and *when* (build time — once before publishing — request time, or a mix)?

Strategy	Where / when HTML is made	First HTML contains	Data freshness
CSR (client-side rendering, SPA)	Browser, after JS loads	Empty `<div id="root">`	Live (fetched by the browser)
SSR (server-side rendering)	Server, every request	Full content	Live per request
SSG (static site generation)	Build machine, once per deploy	Full content	As old as the last build
ISR (incremental static regeneration)	Build, then re-built in the background on a timer or on demand	Full content	Stale for at most N seconds, or until you invalidate
Streaming SSR	Server, per request, sent in pieces	Page frame + placeholders first, slow parts later	Live
PPR (partial prerendering)	Static shell at build + dynamic "holes" filled per request	Static shell with placeholders, holes streamed in	Mixed per section
RSC (React Server Components)	Some components run only on the server (build or request time)	Works *with* any of the above	Per component

Restaurant analogy: CSR hands over a kit and recipe; SSR cooks each plate to order; SSG pre-plates sandwiches each morning, ISR re-makes the tray when it's over an hour old; streaming serves the starter now and the main when ready; PPR keeps pre-plated sides and cooks only the main.

Why it matters

It decides how soon something appears (**FCP**, First Contentful Paint), how soon the main content appears (**LCP**, Largest Contentful Paint), when clicks work (hydration done), and data freshness. And cost: SSG pages are files on a **CDN** (Content Delivery Network — caching servers near users; System Design Fundamentals); SSR runs your code on every request.

How ISR works

1. Generated at build and cached like SSG; requests served from cache.
2. After the revalidation window (say 60 s), the next request **still gets the stale page** but triggers a background re-render.
3. The new page replaces the cached copy. You can also invalidate on demand (e.g. an editor publishes).

This is **stale-while-revalidate** (also used by HTTP caches and TanStack Query, Chapter 6 §6.9): serve what you have, refresh behind the scenes. ISR is a framework feature (named by Next.js), not a React API.

Trade-offs

	CSR	SSR	SSG	ISR	Streaming SSR / PPR
Time to first content	Slow	Medium (waits for server render)	Fastest	Fastest	Fast (shell first)
Needs a server	No	Yes	No	Yes (for regeneration)	Yes
Per-user content in HTML	No	Yes	No	No	Yes (in the holes)
Good for	Dashboards behind login	Personalised pages	Docs, blogs	Catalogues, news	Fast frame + slow widgets

Common mistakes

- Thinking SSR makes pages *interactive* sooner — it makes them *visible* sooner; interactivity waits for JS and hydration (§13.4).
- SSG for per-user data (cart prices, "logged in as…") — use a dynamic hole or client fetch.
- Whole-app thinking — frameworks choose **per route**, and with PPR **per component**.

13.2 The server rendering APIs

What it is

`react-dom/server` renders **per request**; `react-dom/static` renders **ahead of time** and waits for all data.

Function	Package	Output	Waits for <Suspense> data?	Use it for
`renderToString(el)`	`react-dom/server`	A string	No — emits the fallback	Legacy code; simple SSR without async data
`renderToStaticMarkup(el)`	`react-dom/server`	A string, no hydration markers	No	Non-interactive HTML: emails, static snippets
`renderToPipeableStream(el, opts)`	`react-dom/server` (Node)	Node stream: `{ pipe, abort }`	Streams it later	**SSR on Node** (Express, `node:http`)
`renderToReadableStream(el, opts)`	`react-dom/server` (Web Streams)	`Promise<ReadableStream>`	Streams it later	SSR on edge runtimes, Deno, Bun, Cloudflare Workers
`prerenderToNodeStream(el, opts)`	`react-dom/static`	`{ prelude }` Node stream	**Yes — waits for everything**	SSG on Node
`prerender(el, opts)`	`react-dom/static`	`{ prelude, postponed }` Web stream	Yes (unless aborted)	SSG with Web Streams; partial prerendering
`resume(el, postponed)` / `resumeToPipeableStream`	`react-dom/server`	Web / Node stream	Streams it	Filling `prerender`'s holes at request time (§13.6)
`resumeAndPrerender` / `…ToNodeStream`	`react-dom/static`	`{ prelude, postponed }`	Yes	Continuing a postponed prerender at build time

A **Node stream** / **Web stream** delivers data piece by piece (Node's format vs the browser standard); **Express** is a popular Node web-server library; **edge runtimes** are lightweight JS servers in many locations near users (e.g. **Cloudflare Workers**); **Deno** and **Bun** are Node.js alternatives.

Browser partner: **`hydrateRoot(domNode, element, options)`** from `react-dom/client` attaches React to existing server HTML (`createRoot` builds DOM from nothing).

Seeing the string APIs

`code/01_render_to_string.jsx` :

```
function Greeting({ name }) {
  const [likes] = useState(3);                    // state works: initial value is rendered
  return <p className="greet">Hello, {name}! Likes: {likes}</p>;
}
console.log('renderToString      :', renderToString(<Greeting name="Ada" />));
console.log('renderToStaticMarkup:', renderToStaticMarkup(<Greeting name="Ada" />));

const never = new Promise(() => {});              // data that never arrives
function Slow() { return <p>{use(never)}</p>; }
console.log(renderToString(<main><h1>Shop</h1><Suspense fallback={<p>Loading…</p>}><Slow /></Suspense></main>));
```

Output:

```
renderToString      : <p class="greet">Hello, <!-- -->Ada<!-- -->! Likes: <!-- -->3</p>
renderToStaticMarkup: <p class="greet">Hello, Ada! Likes: 3</p>
renderToString + pending Suspense:
  <main><h1>Shop</h1><!--$!--><template data-msg="Switched to client rendering because the server rendering
aborted due to: …"></template><p>Loading…</p><!--/$--></main>
```

1. **Hooks run once on the server.** `useState` returns its initial value; Effects **never** run there and no handlers are attached.
2. `<!-- -->` **comments** separate adjacent text so hydration can match each text node. `renderToStaticMarkup` drops them, so its HTML can't be hydrated.
3. `renderToString` **cannot wait**: a suspended component yields the fallback marked `<!--$!-->` ("render on the client"), plus a dev-only hint to use a streaming API.

Common mistakes

- `renderToString` in new code — data components only show the fallback on the server. Stream instead.
- `hydrateRoot` on `renderToStaticMarkup` output — no markers.
- Reading `window`, `document` or `localStorage` during render — `ReferenceError` on the server. Use Effects or handlers (§13.5).

13.3 Streaming SSR with `<Suspense>`

What it is

Streaming sends one HTML response in pieces, cut at `<Suspense>` boundaries: everything **outside** pending boundaries is the **shell**, sent first with fallbacks; each boundary's content follows when ready, with a tiny inline script that swaps it in.

The problem it solves

Plain SSR is all-or-nothing: one slow query (reviews taking 1.5 s) delays the **whole** page. Streaming lets fast parts leave at once, so the browser loads CSS/JS and even hydrates the shell while slow parts render.

How it works

1. `renderToPipeableStream(<App />, options)` starts rendering.
2. When everything outside pending boundaries is rendered, `onShellReady` fires: set the HTTP status code and headers, then `pipe(res)` — the shell goes out with fallbacks.
3. As each boundary resolves, React writes a chunk: content in a **hidden** `<div>` plus a `<script>` that moves it into place and removes the fallback.
4. When all is done, `onAllReady` fires and the response ends.
5. `onShellError` — the shell failed (send an error page). `onError` — any error, including inside boundaries (log it). `abort()` — stop waiting; pending boundaries fall back to client rendering.

Seeing every chunk

`code/02_stream_chunks.jsx` pipes into a fake response printing each chunk with its time; price takes 100 ms, reviews 300 ms:

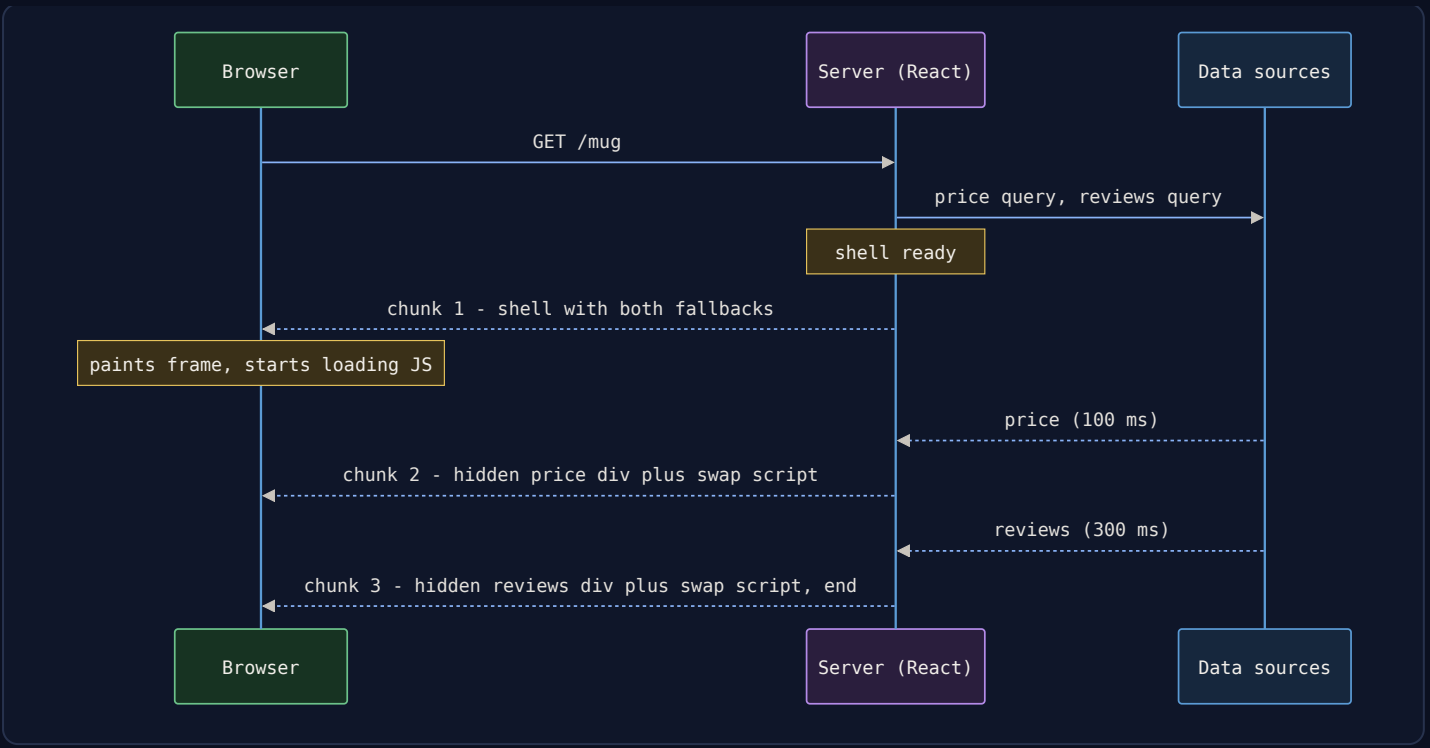
```
const pricePromise = later('$18', 100);                       // fast data
const reviewsPromise = later(['Great mug', 'Too small'], 300); // slow data
function Price() { return <b>{use(pricePromise)}</b>; }
function Reviews() { return <ul>{use(reviewsPromise).map(r => <li key={r}>{r}</li>)}</ul>; }
function Page() {
  return (
    <main>
      <h1>Trail Mug</h1>
      <Suspense fallback={<i>price…</i>}><Price /></Suspense>
      <Suspense fallback={<i>reviews…</i>}><Reviews /></Suspense>
    </main>
  );
}
const { pipe } = renderToPipeableStream(<Page />, {
  onShellReady() { console.log(`onShellReady @ ${ms()} -> pipe(res)`); pipe(res); },
  onAllReady()   { console.log(`onAllReady   @ ${ms()}`); },
  onError(e)     { console.error(e); },
});
```

Output (long scripts shortened; timings vary by a few ms):

```
onShellReady @    3 ms -> pipe(res)
chunk 1 @    5 ms: <main><h1>Trail Mug</h1><!--$?--><template id="B:0"></template><i>price…</i><!--/$--><!--$?-->
<template id="B:1"></template><i>reviews…</i><!--/$--></main><script>/* 74 chars of React's swap code */</script>
chunk 2 @  101 ms: <div hidden id="S:0"><b>$18</b></div><script>/* 862 chars of React's swap code */</script>
onAllReady   @  302 ms
chunk 3 @  302 ms: <div hidden id="S:1"><ul><li>Great mug</li><li>Too small</li></ul></div>
<script>$RC("B:1","S:1")</script>
end      @  302 ms
```

Marker	Meaning
`<!--$?-->` … `<!--/$-->`	A boundary still **pending**; between them is the fallback
`<template id="B:0">`	Marks *where* boundary 0's content will go
`<div hidden id="S:0">`	Boundary 0's finished content ("segment"), sent later, hidden
`$RC("B:1","S:1")`	"React: complete boundary" — moves S:1 into B:1's place, removes the fallback
`<!--$-->`	A boundary **complete** when sent (no swap)
`<!--$!-->`	A boundary that **failed** on the server; the browser renders it

Chunk 2 carries the helper defining `$RC`; chunk 3 only calls it. The shell left at 5 ms instead of 300 ms.



Status codes, errors and crawlers

- The **status code** can only be set before the first byte — decide it in `onShellReady`. A later error inside a boundary can't change the 200; the boundary is client-rendered, and your error boundary shows if it fails again.
- **Crawlers** (search-engine bots) and static generation: wait for `onAllReady` so the whole document arrives at once.
- Always `abort()` **after a timeout**, or a hung query holds the connection open.
- Since React 19.2 the server briefly **batches reveals** of boundaries finishing close together, so they appear in one go instead of flickering in.

Where to put boundaries

Wrap parts whose data is slow **and** not needed for the first view (reviews, recommendations, charts). Keep main content (title, price, article) in the shell, or LCP suffers. Many tiny boundaries "pop" piecemeal; group related slow parts.

In the interview

30-second answer: "The shell — everything outside pending Suspense boundaries — is sent as soon as it's ready, with fallbacks. Each boundary's HTML follows in a hidden div plus a tiny swap script, so one slow query no longer blocks the page and the browser loads JS and hydrates the shell meanwhile. Status goes in `onShellReady`, logging in `onError`, crawlers wait for `onAllReady`, and a timeout calls `abort()`."

- *A component inside a boundary throws on the server?* — The boundary is marked for client rendering (`<!--$!-->`), the fallback stays, the browser tries; if it fails there too, the nearest error boundary shows.
- *Why no 404 after the shell is sent?* — The status line is the first thing in the response; once bytes are out it's fixed.
- *What's in* `<div hidden id="S:1">`*?* — A boundary's finished HTML, hidden until `$RC` moves it into place.

13.4 Hydration end to end

What it is

Hydration is React in the browser "adopting" server-rendered DOM: it renders components in memory, checks the result matches the HTML, **reuses** those nodes, attaches handlers and runs Effects. The server builds and paints the house; hydration is the electrician — visible all along, switches work only after wiring.

How it works

1. Server HTML includes a `<script>` for the client bundle (React adds it via `bootstrapScripts`).
2. The bundle calls `hydrateRoot(document, <App {...sameProps} />)` — the *same* tree with the *same* data as the server.
3. **Selective hydration**: the shell hydrates while boundaries still stream; a click in an unhydrated boundary moves it to the front; each streamed boundary hydrates on arrival.

The full demo — server, stream, Chrome

`code/04_ssr_server_hydrate.jsx` bundles the browser entry `ssr_client.jsx` with esbuild, starts a `node:http` server streaming `shop_app.jsx` (reviews take 1.5 s), then drives headless Chrome: screenshot the shell, click, wait for reviews, click again. `code/shop_app.jsx` (key parts) renders the **whole document**:

```
function AddToCart() {
  const [count, setCount] = useState(0);
  useEffect(() => { window.__hydrated = true; }, []);   // Effects never run on the server: proves hydration
  return <button onClick={() => setCount(c => c + 1)}>Add to cart ({count})</button>;
}
function Reviews({ reviewsPromise }) {
  const reviews = use(reviewsPromise);                  // suspends until the data is ready
  return <ul>{reviews.map(r => <Review key={r.id} r={r} />)}</ul>;
}
export default function App({ reviewsPromise }) {
  // <html><head>…</head><body>: product card with <AddToCart />, then
  // <Suspense fallback={<ReviewsSkeleton />}><Reviews reviewsPromise={reviewsPromise} /></Suspense>
}
```

Browser entry (`code/ssr_client.jsx`):

```
import { hydrateRoot } from 'react-dom/client';
import App from './shop_app.jsx';
const reviewsPromise = fetch('/api/reviews').then(r => r.json());   // the browser needs the data too
hydrateRoot(document, <App reviewsPromise={reviewsPromise} />, {
  onRecoverableError(err) { console.error('recoverable:', err.message); },
});
```

Server (key part):

```
const server = http.createServer((req, res) => {
  // /client.js and /api/reviews served directly; everything else:
  let didError = false;
  const { pipe, abort } = renderToPipeableStream(<App reviewsPromise={slowReviews()} />, {
    bootstrapScripts: ['/client.js'],          // React emits <script src="/client.js" async>
    onShellReady() { res.statusCode = didError ? 500 : 200; res.setHeader('content-type', 'text/html; charset=utf-
8'); pipe(res); },
    onShellError() { res.statusCode = 500; res.end('<h1>Something went wrong</h1>'); },
    onError(err) { didError = true; console.error(err); },
  });
  setTimeout(abort, 10_000);                   // never hang forever on a slow data source
});
```

Output (timings vary):

```
[ 349 ms] server: GET / -> start rendering
[ 354 ms] server: onShellReady -> send shell (with fallback) now
[ 387 ms] browser: shell painted, reviews fallback visible
[ 402 ms] browser: shell hydrated while reviews still loading -> Add to cart (1), fallback still shown: true
[1852 ms] server: onAllReady -> reviews streamed, response ends
[1857 ms] browser: streamed reviews swapped in place of the fallback
[1883 ms] browser: after clicking Helpful -> ["Add to cart (1)","Helpful (5)","Helpful (1)"]
```



Figure 13.1 — About 40 ms after the request: the shell is painted and hydrated ("Add to cart (1)" responded to a click) while the reviews boundary still shows its fallback.



Figure 13.2 — 1.5 s later: the reviews chunk replaced the skeleton; clicking "Helpful" on the first review changed 4 → 5, so the streamed part is hydrated too.

Proven: the shell was **usable 1.4 s before the slow data** (selective hydration), the **streamed part hydrated too**, and hydration **reused the server DOM** (no blank page or flicker).

The double-fetch problem (and how frameworks fix it)

`ssr_client.jsx` **fetches the reviews again** to render the same tree during hydration, though the server had them. Frameworks **serialize the server's data into the page** (as JSON-like text): Next.js embeds the RSC payload (§13.7), React Router embeds loader data. That, plus routing, per-route code splitting and CSS handling, is most of what a framework does — why react.dev recommends one for server rendering.

Common mistakes

- `createRoot` instead of `hydrateRoot` on server HTML — React discards and rebuilds the DOM (a flash).
- A different tree or props on the client (§13.5).
- One giant `<Suspense>` around everything — empty shell.
- Forgetting the page is **not interactive** until the bundle loads — keep client JS small; use real `<a>` and `<form>` so basics work pre-hydration.

In the interview

> **30-second answer:** " `hydrateRoot` renders the same tree in the browser, matches it against the server HTML, reuses those nodes and attaches handlers and Effects. With streaming it's selective: the shell becomes interactive while boundaries load, and a click in an unhydrated boundary gets priority. The client must render exactly what the server did — hence frameworks serialize server data into the page."

- *Clicks between "visible" and "interactive"?* — Before the bundle runs they do nothing unless the element works without JS (links, forms). Once React runs, a click on a not-yet-hydrated part makes React hydrate it first and replay the event.
- *Why hydrate `document` rather than a `<div>`?* — Rendering the whole `<html>` lets React manage `<head>` tags too.
- *Which Hook proved hydration in the demo?* — `useEffect`; Effects never run on the server.

13.5 Hydration mismatches

What it is

A **hydration mismatch**: the HTML React renders in the browser during hydration differs from what the server sent. React can't safely adopt mismatched DOM, so it discards that part and **re-renders it on the client** — costing performance, a visible flash, and an error log.

Seeing one

`code/05_hydration_mismatch.jsx` renders on a pretend "server" (no `window`), then hydrates in jsdom:

```
function BadBanner() {                      // BUG: output depends on the environment
  return <p>{isBrowser() ? 'Welcome back!' : 'Hello, guest'}</p>;
}
function GoodBanner() {                     // FIX 1: same first render, switch in an Effect
  const [mounted, setMounted] = useState(false);
  useEffect(() => setMounted(true), []);
  return <p>{mounted && isBrowser() ? 'Welcome back!' : 'Hello, guest'}</p>;
}
const subscribe = () => () => {};
function GoodBanner2() {                    // FIX 2: useSyncExternalStore with a server snapshot
  const inBrowser = useSyncExternalStore(subscribe, () => isBrowser(), () => false);
  return <p>{inBrowser ? 'Welcome back!' : 'Hello, guest'}</p>;
}
function Greeting() { use(browser()); return <p>Welcome back!</p>; }   // FIX 3 (React 19.3)
function GoodBanner3() { return <Suspense fallback={<p>Hello, guest</p>}><Greeting /></Suspense>; }
function Stamp({ now }) { return <time suppressHydrationWarning>{now}</time>; }
// each: renderToString on the "server", flip to "browser", hydrateRoot(..., { onRecoverableError })
```

Output:

```
BadBanner    server: <p>Hello, guest</p>          after hydrate: <p>Welcome back!</p>
   recoverable error: Hydration failed because the server rendered text didn't match the client. As a result this
tree will be regenerated on the client. This can happen if a SSR-ed Client Component used:
GoodBanner   server: <p>Hello, guest</p>          after hydrate: <p>Welcome back!</p>
GoodBanner2  server: <p>Hello, guest</p>          after hydrate: <p>Welcome back!</p>
GoodBanner3  server: <!--$!--><template…><p>Hello, guest</p><!--/$-->
             after hydrate: <p>Welcome back!</p>  errors: 0
Stamp        server: <time>10:00:00</time>        after hydrate: <time>10:00:00</time>  errors: 0
```

1. `BadBanner` ends right only because React **gave up and re-rendered** on the client, reported via `onRecoverableError` (dev also logs a server/client diff).
2. Fixes 1 and 2 reach the same result **with no error**: hydration reproduces the server output, then a normal update changes it. `useSyncExternalStore`'s third argument (`getServerSnapshot`) is used on the server *and during hydration*.
3. `use(browser())` (React 19.3, from `react-dom`) suspends **only on the server**: the server sends the fallback marked `<!--$!-->` and the browser renders the real content — no mismatch.
4. `suppressHydrationWarning` silences the warning for **one element's own text/attributes**, one level deep, and React **keeps the server text** (`10:00:00`) — it doesn't patch it. Only for expected, harmless differences like timestamps.

Common causes and fixes

Cause	Example	Fix
Environment checks in render	`typeof window !== 'undefined' ? A : B`	Server version first; switch in `useEffect` or via `useSyncExternalStore`
Browser-only data	`localStorage.getItem('theme')` during render	Same; or a cookie (sent with every request) so the server knows it
Time, randomness, locale (language/region formatting)	`Date.now()`, `Math.random()`, `toLocaleDateString()` → `9/28/2026` vs `28/09/2026`	Compute on the server and pass as a prop; explicit `Intl` locale and `timeZone`; `useId` for IDs (Chapter 7 §7.8 — derived from tree position, same on both sides)
Invalid HTML nesting	`<p><div/></p>`, `<a>` inside `<a>`, `<table>` without `<tbody>`	Fix the markup — the browser's HTML parser "repairs" it (e.g. closes `<p>` before `<div>`), so the DOM no longer matches
Different data	Client refetched newer data	Serialize server data into the page (frameworks do)
Browser extensions / proxies (middlemen that may alter traffic)	An extension injects attributes; a CDN minifier strips comments	React 19 tolerates unexpected tags inserted in `<head>` / `<body>`; don't let proxies rewrite HTML

When the difference is intentional

For genuinely browser-specific output (an "offline" badge, a window-size chart) use **two-pass rendering** — a neutral placeholder on the server and during hydration, the real thing after mount (Fix 1 or 2) — or, for a whole browser-only component, `use(browser())` in `<Suspense>` (Fix 3). Next.js also offers `dynamic(() => import('./Chart'), { ssr: false })` inside a Client Component.

In the interview

30-second answer: "The client's first render didn't produce the server's HTML, so React can't adopt that DOM: it client-renders the tree and reports a recoverable error. Causes: environment checks, `localStorage`, dates, randomness, locale, invalid nesting, different data. Fix: make the first client render identical and switch after mount (an Effect, `useSyncExternalStore`'s server snapshot,

or `use(browser())` in 19.3); `suppressHydrationWarning` only for single expected differences like timestamps."

- *Does `suppressHydrationWarning` fix the text?* — No: keeps the server's text and silences the warning, one level deep.
- *Why is `useId` safe for SSR?* — IDs derive from tree position, identical on server and client.
- *Why doesn't `GoodBanner` mismatch?* — Its first client render uses `mounted = false`, like the server.

13.6 Static generation and partial prerendering

What it is

Prerendering renders before any request (at build). `prerenderToNodeStream` and `prerender` (`react-dom/static`) **wait for all Suspense data** and return finished HTML as `prelude`.

Partial prerendering (PPR) prerenders what it can, **stops** at per-request data (cart, cookies), saves a small "where I stopped" object (`postponed`), and at request time **`resume()`** renders only the holes. The static part sits on a CDN; the server does only dynamic work. `resume` arrived in React 19.2.

Seeing both

`code/03_prerender_ssg.jsx`:

```
// 1. SSG
const { prelude } = await prerenderToNodeStream(<Blog />);   // resolves only when everything is ready

// 2. PPR
let cart = new Promise(() => {});                 // per-user data: not available at build time
function Cart() { return <p>Cart: {use(cart)}</p>; }
function Store() {
  return (
    <main>
      <h1>Store</h1><p>Free shipping over $50</p>
      <Suspense fallback={<p>Cart: …</p>}><Cart /></Suspense>
    </main>
  );
}
const controller = new AbortController();
setTimeout(() => controller.abort(), 50);         // "build" stops waiting after 50 ms
const pre = await prerender(<Store />, { signal: controller.signal, onError() {} });
const saved = JSON.stringify(pre.postponed);      // store next to the prelude

cart = Promise.resolve('3 items');                // later, a real request: data is available
const rest = await resume(<Store />, JSON.parse(saved));
```

Output:

```
SSG (prerenderToNodeStream):
  <main><h1>Blog</h1><!--$--><ul><li>Packing list for a weekend hike</li><li>Choosing a camping stove</li></ul><!--/$--></main>

PPR prelude (cache on a CDN):
  <main><h1>Store</h1><p>Free shipping over $50</p><!--$?--><template id="B:0"></template><p>Cart: …</p><!--/$--></main><script>…</script>
postponed state: 610 bytes of JSON

resume() stream appended after the prelude:
  <div hidden id="S:0"><p>Cart: <!-- -->3 items</p></div><script>/* swap code */</script>
```

1. **SSG**: boundary `<!--$-->` (complete) — no fallback or swap script; CDN-ready.
2. **PPR**: the abort left the cart pending — prelude = shell with fallback; `postponed` records what's missing.

3. `resume` rendered **only** that boundary. At request time the server sends the cached prelude at once, then appends the resumed stream (§13.3's hidden-div-plus-swap).

Where you meet this in practice

You rarely call these yourself. Next.js 16 **Cache Components** uses this model: static and `'use cache'` parts form the prerendered shell; anything reading cookies, headers or uncached data sits in `<Suspense>` and streams per request (§13.9). Static-site setups (React Router `prerender`, Astro, custom scripts) call `prerender*` at build.

Common mistakes

- Prerendering per-user data into a static page — every visitor sees the first user's cart.
- Expecting `prerender` to be fast with slow data — it waits. Abort + PPR, or move slow data behind a request-time boundary.

13.7 React Server Components

What it is

A **React Server Component (RSC)** runs **only** on the server (build or request time); its code never ships. It can be `async`, `await` a database query, read files and use secrets (API keys etc.); only its *output* is sent. A **Client Component** is what you've written all book: its code ships, it uses state, Effects and handlers, and SSR still pre-renders it to HTML. In an RSC app (e.g. Next.js App Router) components are **Server Components by default**; `'use client'` opts into client code (§13.8). Analogy: Server Components are cooks using the fridge (database) and knives (secrets), sending out only the plate; Client Components are the salt shaker the diner handles.

RSC is not SSR

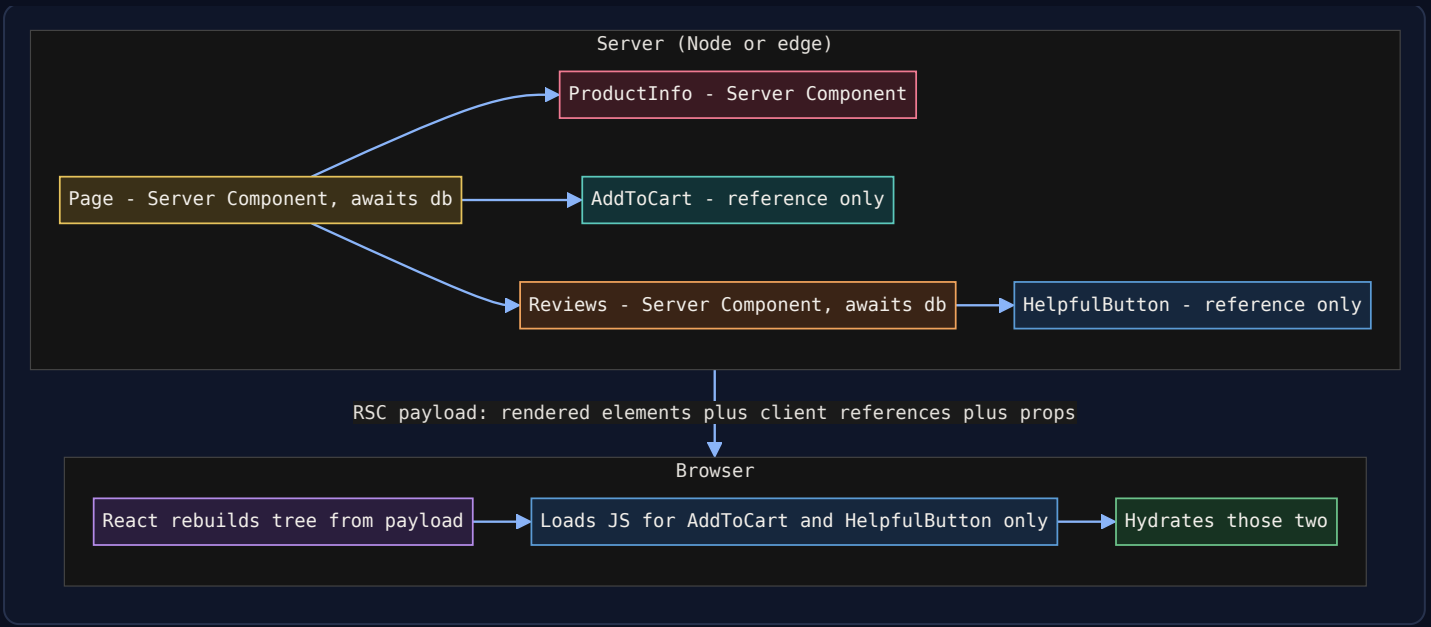
The most common confusion:

	SSR	Server Components
Produces	**HTML** for the first load	A serialized **component tree** (the RSC payload)
Which components	All (client ones included)	Only Server Components
Code shipped to browser	All component code (for hydration)	None for Server Components
Runs again on navigation?	No — after load, navigation is client-side	Yes — new payload per route
State/Effects?	Yes (initial render only on the server)	No
`await` data in render?	Via Suspense + `use()`	Yes, directly (`async function`)

Complementary: Next.js runs Server Components to get the payload, then SSR turns payload + Client Components into first-load HTML.

How it works

1. The server renders the Server Component tree (awaiting data). At a Client Component React does **not** run it; it records a **reference** ("module `LikeButton.js`, export `default`") plus props.
2. The result is serialized into the **RSC payload**: rendered elements, client references, their props.
3. **First load**: SSR turns payload + Client Components into HTML and embeds the payload; the browser hydrates only Client Components.
4. **Client navigation**: the browser fetches the new route's payload (no HTML) and React merges it — **client state is preserved** where the tree is unchanged.



Conceptually the payload is rendered elements (`<main><h1>Trail Mug</h1>…<ul>…reviews…</ul>`) plus holes like *client ref* `AddToCart.js#default`, *props* `{ productId: "mug-1" }`; the real wire format is internal and version-dependent.

Why it matters

- **Smaller bundles** — a Server Component using a 200 KB Markdown parser ships 0 KB of it, only the resulting elements.
- **No client-server waterfalls** (each request waiting on the previous) — instead of "load JS → Effect → fetch → render", the component awaits data where it lives.
- **Safe data access** — queries and API keys stay on the server.
- **Streaming built in** — async Server Components suspend, so `<Suspense>` streams as in §13.3.

Code (Next.js 16 — not run)

```jsx
// app/products/[id]/page.jsx — a Server Component (no directive needed)
import { db } from '@/lib/db';
import AddToCart from './AddToCart';        // a Client Component
import { Suspense } from 'react';

export default async function ProductPage({ params }) {
  const { id } = await params;                          // Next.js 15+: params is a Promise
  const product = await db.product.findUnique({ where: { id } });   // direct DB access
  return (
    <main>
      <h1>{product.name}</h1>
      <p>{product.description}</p>
      <AddToCart productId={product.id} price={product.price} />   {/* serializable props only */}
      <Suspense fallback={<p>Loading reviews…</p>}>
        <Reviews productId={id} />                    {/* async Server Component, streams */}
      </Suspense>
    </main>
  );
}
async function Reviews({ productId }) {
  const reviews = await db.review.findMany({ where: { productId } });
  return <ul>{reviews.map(r => <li key={r.id}>{r.text}</li>)}</ul>;
}
```

`AddToCart.jsx` starts with `'use client'` and is an ordinary component with `useState` and `onClick`.

What can cross the server → client boundary

Props from a Server to a Client Component must be **serializable** by React:

Allowed	Not allowed
string, number, bigint, boolean, `null`, `undefined`	Ordinary functions (event handlers, callbacks)
Plain objects and arrays of allowed values	Class instances (e.g. an ORM — object-relational mapper — model object, a `class User`)
`Date`, `Map`, `Set`, typed arrays, `ArrayBuffer`	Objects with a `null` prototype
JSX / React elements (including Server Components as `children`)	Symbols not created with `Symbol.for`
Promises (read with `use()` on the client)	
Server Functions (`'use server'`, §13.8)	

Composition patterns

1. **Push `'use client'` to the leaves.** Keep pages/layouts as Server Components; make only interactive pieces (buttons, forms, menus) Client Components. The directive marks a module *and everything it imports* as client code, so placing it high ships the whole subtree.
2. **Pass Server Components *through* Client Components as `children`.** A Client Component can't *import* a Server Component, but can *render* one it receives as a prop — the server renders it first.

```jsx
// Modal.jsx — a Client Component (needs state)
'use client';
import { useState } from 'react';
export default function Modal({ children }) {
  const [open, setOpen] = useState(false);
  return <><button onClick={() => setOpen(!open)}>Cart</button>{open && children}</>;
}
```

```jsx
// page.jsx — a Server Component
import Modal from './Modal';
import Cart from './Cart';                // a Server Component that awaits the database
export default function Page() {
  return <Modal><Cart /></Modal>;         // Cart runs on the server; Modal receives its output
}
```

3. **Context is created and read in Client Components.** Server Components can't call `createContext`, `useContext` or `use(Context)`. Define context in a `'use client'` file; since React 19.3 a Server Component may *render* it directly (`<UserContext value={user}>{children}</UserContext>`) — before, you needed a `'use client'` provider wrapping `children`.
4. **Stream data to the client with a promise**: start a fetch *without awaiting*, pass the promise as a prop; the Client Component calls `use(promise)` in `<Suspense>` (Chapter 9).
5. **Keep server code out of client bundles**: `import 'server-only'` in a data module makes the build fail if a Client Component imports it.

When not to use RSC

- Highly interactive screens (editors, canvases, live-filter dashboards) that are nearly all client state.
- Pure static hosting — RSC can run at build time, but no per-request rendering.
- Without a framework: RSC needs deep bundler integration, and React's low-level RSC bundler APIs aren't covered by semver (breaking changes only in major versions). Next.js App Router is the complete implementation; React Router and TanStack Start support is still experimental.

Common mistakes

- Thinking `'use server'` marks a Server Component — there's **no directive** for those; `'use server'` marks Server Functions.

- Passing an `onClick` from a Server to a Client Component — functions aren't serializable (unless Server Functions).
- `useState` / `useEffect` in a component running as a Server Component (no `'use client'` above) — build error.
- Assuming Server Components re-run on client state changes — only on a new server request (navigation, `router.refresh()`, revalidation after a Server Function).

In the interview

> **30-second answer:** "Server Components run only on the server, can be async, read data directly and ship zero JS — the browser gets their output as an RSC payload. Client Components (`'use client'`) ship JS and handle state and events. SSR is different: it turns the tree into first-load HTML; RSC decides which components exist in the browser at all. Together: RSC payload → SSR HTML → hydrate only Client Components. Props crossing must be serializable; functions cross only as Server Functions."

- *Can a Client Component render a Server Component?* — Not by importing it; yes if it receives it as `children` or another JSX prop from a Server Component.
- *Several Server Components sharing one query?* — Wrap the fetcher in React's `cache()` (per-request memoization; Server Components only).
- *Can a Server Component use `useState`?* — No — no state, Effects or handlers.

13.8 Directives: `'use client'` and `'use server'`

What it is

A **directive** is a string literal at the top of a file (or function body), like `'use strict'`. RSC bundlers read them to decide what goes where. React defines exactly two:

Directive	Where	Marks	Meaning
`'use client'`	Top of a **file** (before imports; comments allowed)	That module **and its imports**	"Client code starts here" — the server→client boundary
`'use server'`	Top of a **file**, or first line of an **async function**	Exported async functions (file), or that function	"A **Server Function**: callable from the client, runs on the server"

`'use cache'` (and variants) is a **Next.js directive**, not React's (§13.9).

`'use client'` rules

1. First statement in the file, single or double quotes (not backticks).
2. Marks a **module-graph boundary** (the web of files importing each other): the file and everything it imports becomes client code. Needed only at entry points that Server Components import.
3. Exports used by Server Components must be components or serializable values.
4. A directive-less `Button.jsx` is client code when imported from a `'use client'` file and server code when imported from a Server Component.
5. Libraries put `'use client'` on Hook-using entry points so Server Components can import them.

`'use server'` — Server Functions

A **Server Function** runs on the server but is **called from client code** as if local: the bundler swaps it for a reference; a call POSTs the serialized arguments, runs the function, returns the serialized result. Used as a `<form action>` or with `useActionState` it's often called a **Server Action** (React's name for all of them until September 2024; Next.js still uses it). Form side: Chapter 4 §4.8.

```
// app/actions.js — (Next.js 16 — not run)
'use server';
import { revalidatePath } from 'next/cache';
import { auth } from '@/lib/auth';
import { db } from '@/lib/db';

export async function addReview(prevState, formData) {
  const user = await auth();                                  // 1. authenticate
  if (!user) return { error: 'Please sign in' };
  const text = String(formData.get('text') ?? '').trim();     // 2. validate — never trust input
  if (text.length < 5) return { error: 'Review is too short' };
  await db.review.create({ data: { text, userId: user.id, productId: formData.get('productId') } });
  revalidatePath(`/products/${formData.get('productId')}`);   // 3. refresh server-rendered UI
  return { ok: true };
}
```

```
// Inline Server Function inside a Server Component (not run)
export default async function Page() {
  async function subscribe(formData) {
    'use server';
    await db.newsletter.add(formData.get('email'));
  }
  return <form action={subscribe}><input name="email" type="email" /><button>Subscribe</button></form>;
}
```

Outside a form, wrap the call in a transition for a pending state: `startTransition(async () => { await like(postId); })` (Chapter 7 §7.10).

Security — treat every Server Function as a public API endpoint

(An **endpoint** is one server URL performing one operation; "public" = anyone can call it.)

- Anyone can call it with any arguments (the reference is in the page). **Authenticate and authorise inside**; a hidden button is no protection.
- **Validate** every argument (e.g. Zod, Chapter 4 §4.9).
- Values an inline Server Function captures (closes over) travel to the client and back (Next.js encrypts them) — never close over secrets; re-check anything important.
- Arguments and returns must be serializable (§13.7 table, plus `FormData`).

Common mistakes

- `'use client'` on every file "to be safe" — all RSC benefits lost.
- `'use server'` on a file of private helpers — every exported async function becomes a public endpoint. Use `import 'server-only'`.
- Server Functions for **reading** — they're for mutations; read in Server Components or loaders.

In the interview

30-second answer: " `'use client'` marks where the client module graph starts — that file and its imports ship to the browser. `'use server'` marks Server Functions: async functions that run on the server but are callable from the client, typically as form actions with progressive enhancement (the form works as plain HTML before JS loads). Server Components have no directive — they're the default. Each Server Function is a public POST endpoint: authenticate, authorise and validate inside."

- *Does `'use client'` mean no server rendering?* — No; Client Components are still pre-rendered by SSR; the directive means their code ships and hydrates.
- *Why wrap a Server Function call in `startTransition`?* — For `isPending` and a responsive UI; Actions are transitions.

13.9 Next.js App Router overview

What it is

Next.js (by Vercel) is the framework react.dev lists first and the most complete Server Components implementation. The **App Router** (`app/`, since Next.js 13) is built on RSC, streaming and Server Functions; the older **Pages Router** (`pages/`, `getServerSideProps`) is supported but gets no new features. This describes **Next.js 16** (current major since October 2025; checked at 16.3; Node.js 20.9+). `npx create-next-app@latest` sets up TypeScript, Tailwind (utility-class CSS framework), ESLint, the App Router and Turbopack (Next.js's bundler).

File-system routing

Folders define URL segments; special file names define what each renders.

```
app/
├── layout.jsx            ← root layout: <html><body>; layouts persist (keep state) across navigations
├── page.jsx              ← "/" — a page file makes the path public
├── loading.jsx           ← auto-wraps the page in <Suspense fallback={<Loading/>}>
├── error.jsx             ← error boundary for the segment ('use client'; gets error + retry)
├── not-found.jsx         ← UI for notFound()
├── products/
│   ├── page.jsx          ← "/products"
│   └── [id]/             ← dynamic segment
│       ├── page.jsx      ← "/products/42"   (params.id = "42")
│       └── AddToCart.jsx ← not a route: only page/route files are public
├── (marketing)/          ← route group: organises files, not part of the URL
│   └── about/page.jsx    ← "/about"
├── api/health/route.js   ← Route Handler: GET/POST functions returning Response
└── _components/          ← private folder (underscore): never routed
proxy.js                  ← runs before routes (was middleware.js before v16)
next.config.js
```

Also: `template.jsx` is a layout that re-mounts on every navigation; `[...slug]` / `[[...slug]]` are catch-all / optional catch-all segments.

Pages, layouts and data (not run)

```jsx
// app/layout.jsx
export const metadata = { title: 'Hill & Trail', description: 'Outdoor gear' };  // <head> tags
export default function RootLayout({ children }) {
  return <html lang="en"><body><nav><a href="/">Home</a></nav>{children}</body></html>;
}

// app/products/page.jsx — Server Component fetching data
import Link from 'next/link';
export default async function Products({ searchParams }) {
  const { q = '' } = await searchParams;                    // Promise in Next.js 15+
  const products = await getProducts(q);                    // DB or fetch — on the server
  return <ul>{products.map(p => <li key={p.id}><Link href={`/products/${p.id}`}>{p.name}</Link></li>)}</ul>;
}
```

Per-page `<head>` data: `export async function generateMetadata({ params })`; `notFound()` (from `next/navigation`) renders `not-found.jsx`.

- **`<Link>`** (`next/link`) navigates client-side and **prefetches** routes that come into view.
- **`useRouter`**, **`usePathname`**, **`useSearchParams`** from `next/navigation` (Client Components only); `redirect()` and `notFound()` work on the server.
- **Mutations** use Server Functions (§13.8), then **`revalidatePath('/products')`** or tag-based **`revalidateTag('products', 'max')`**. Next.js 16 added **`updateTag(tag)`** (read-your-own-writes inside a Server Action — the user immediately sees what they saved) and **`refresh()`**.

Rendering and caching in Next.js 16

1. **Previous model (default unless you opt in).** A route is static (prerendered) unless it uses request data (`cookies()` , `headers()` , `searchParams`) or uncached fetches. ISR per route:

```
// app/blog/[id]/page.jsx (not run)
export const revalidate = 60;                     // ISR: regenerate at most every 60 s
export async function generateStaticParams() {     // which [id]s to build ahead of time
  const posts = await getPosts();
  return posts.map(p => ({ id: String(p.id) }));
}
```

2. **Cache Components** (`cacheComponents: true` in `next.config.js` , new in 16). Everything runs at request time **unless** cached with `'use cache'` . The build prerenders a **static shell** — static parts, cached parts, `<Suspense>` fallbacks — and dynamic parts stream per request: partial prerendering (§13.6), the default once enabled.

```
// app/blog/page.jsx — with cacheComponents: true (not run)
import { Suspense } from 'react';
import { cookies } from 'next/headers';
import { cacheLife, cacheTag } from 'next/cache';

export default function BlogPage() {
  return (
    <>
      <h1>Our Blog</h1>                                {/* static: in the shell */}
      <Posts />                                        {/* cached: in the shell */}
      <Suspense fallback={<p>Loading your preferences…</p>}>
        <Preferences />                                {/* per request: streamed */}
      </Suspense>
    </>
  );
}
async function Posts() {
  'use cache';
  cacheLife('hours');                                  // how long the cached result lives
  cacheTag('posts');                                   // invalidate later with revalidateTag / updateTag
  const posts = await db.post.findMany();
  return <ul>{posts.map(p => <li key={p.id}>{p.title}</li>)}</ul>;
}
async function Preferences() {
  const theme = (await cookies()).get('theme')?.value ?? 'light';   // request data → must be inside Suspense
  return <p>Theme: {theme}</p>;
}
```

With Cache Components on, reading request or uncached data **outside** `<Suspense>` is reported in dev and at build, because the route could no longer produce a static shell.

Other pieces worth knowing

Feature	What it does
`proxy.js` (was `middleware.js`)	Redirects, rewrites (serve another URL's content silently) and auth checks before a route runs
`next/image` , `next/font`	Image resizing/lazy-loading; self-hosted fonts without layout shift
Parallel & intercepting routes (`@slot` , `(.)photo`)	Several pages in one layout; modals with their own URL
Static export (`output: 'export'`)	A plain static site (no server features)

Common mistakes

- `'use client'` on `layout.jsx` / `page.jsx` for one Hook — the whole subtree becomes client code. Extract the interactive bit.

- Forgetting `await params` / `await searchParams` / `await cookies()` — Promises since Next.js 15 (sync access removed in 16).
- Expecting fresh data after a mutation without `revalidatePath` / `revalidateTag` / `updateTag`.
- Following Next.js 13–14 tutorials on implicit fetch caching — 15 changed defaults; 16's Cache Components makes caching opt-in with `'use cache'`.

13.10 React Router framework mode

What it is

React Router (maintained by Shopify; Remix's successor — Remix merged into it) has **declarative**, **data** (loaders/actions, Chapter 9) and **framework** modes. Framework mode adds a **Vite plugin** for SSR, prerendering, per-route code splitting, typed route modules and deployment adapters (packages targeting a hosting platform). **React Router 8** (June 2026) needs React 19.2.7+, Node 22.22+ and Vite 7+, is ESM-only (no CommonJS `require`), and drops `react-router-dom` (import from `react-router` and `react-router/dom`). Start with `npx create-react-router@latest`; Chapter 9 §9.7 introduced it — here, server rendering.

Structure (not run)

```ts
// app/routes.ts — the route config
import { type RouteConfig, index, route, layout } from '@react-router/dev/routes';
export default [
  index('routes/home.tsx'),
  layout('routes/shop-layout.tsx', [
    route('products/:id', 'routes/product.tsx'),
  ]),
] satisfies RouteConfig;
```

```ts
// app/routes/product.tsx — a route module
import { Form, useNavigation } from 'react-router';
import type { Route } from './+types/product';        // generated types for this route

export async function loader({ params }: Route.LoaderArgs) {      // runs on the server (SSR / prerender)
  const product = await db.product.find(params.id);
  if (!product) throw new Response('Not found', { status: 404 });
  return { product };
}
export async function action({ request, params }: Route.ActionArgs) { // handles <Form method="post">
  const form = await request.formData();
  await db.review.create({ productId: params.id, text: String(form.get('text')) });
  return { ok: true };                                // loaders re-run automatically afterwards
}
export const meta = ({ loaderData }: Route.MetaArgs) => [{ title: loaderData.product.name }];
export default function Product({ loaderData }: Route.ComponentProps) { /* <h1>, <Form method="post"> with
useNavigation() pending state */ }
export function ErrorBoundary() { return <p role="alert">Something went wrong.</p>; }
```

```ts
// react-router.config.ts — choose the rendering strategy
import type { Config } from '@react-router/dev/config';
export default {
  ssr: true,                                          // false = SPA (CSR only)
  async prerender() { return ['/', '/about']; },      // these URLs become static HTML at build
} satisfies Config;
```

How it maps to this chapter

Concept	React Router framework mode
SSR	`ssr: true`; `loader` runs on the server per request; HTML streams
SSG	`prerender` (boolean, array of paths, or async function)
CSR per route	`clientLoader` instead of `loader` (fetched in the browser)
Hydration data	Loader data serialized into the page — no double fetch (§13.4)
Mutations	`action` + `<Form>`; works without JS (progressive enhancement), then revalidates loaders
Streaming	Return a promise from the loader; read with `<Await>` or `use()` in `<Suspense>`
RSC	Experimental (`unstable_` APIs) in v8 — not yet for production

Next.js vs React Router at a glance

	Next.js App Router	React Router framework mode
Data loading	`async` Server Components	`loader` / `clientLoader` per route
Mutations	Server Functions (`'use server'`)	`action` + `<Form>`
Server Components	Stable, central	Experimental
Bundler	Turbopack (webpack optional)	Vite
Mental model	React-specific (RSC, directives, caching)	Web standards (Request, Response, FormData, HTTP caching)
Can start as a SPA	Static export (limited)	Yes — `ssr: false`, add SSR later

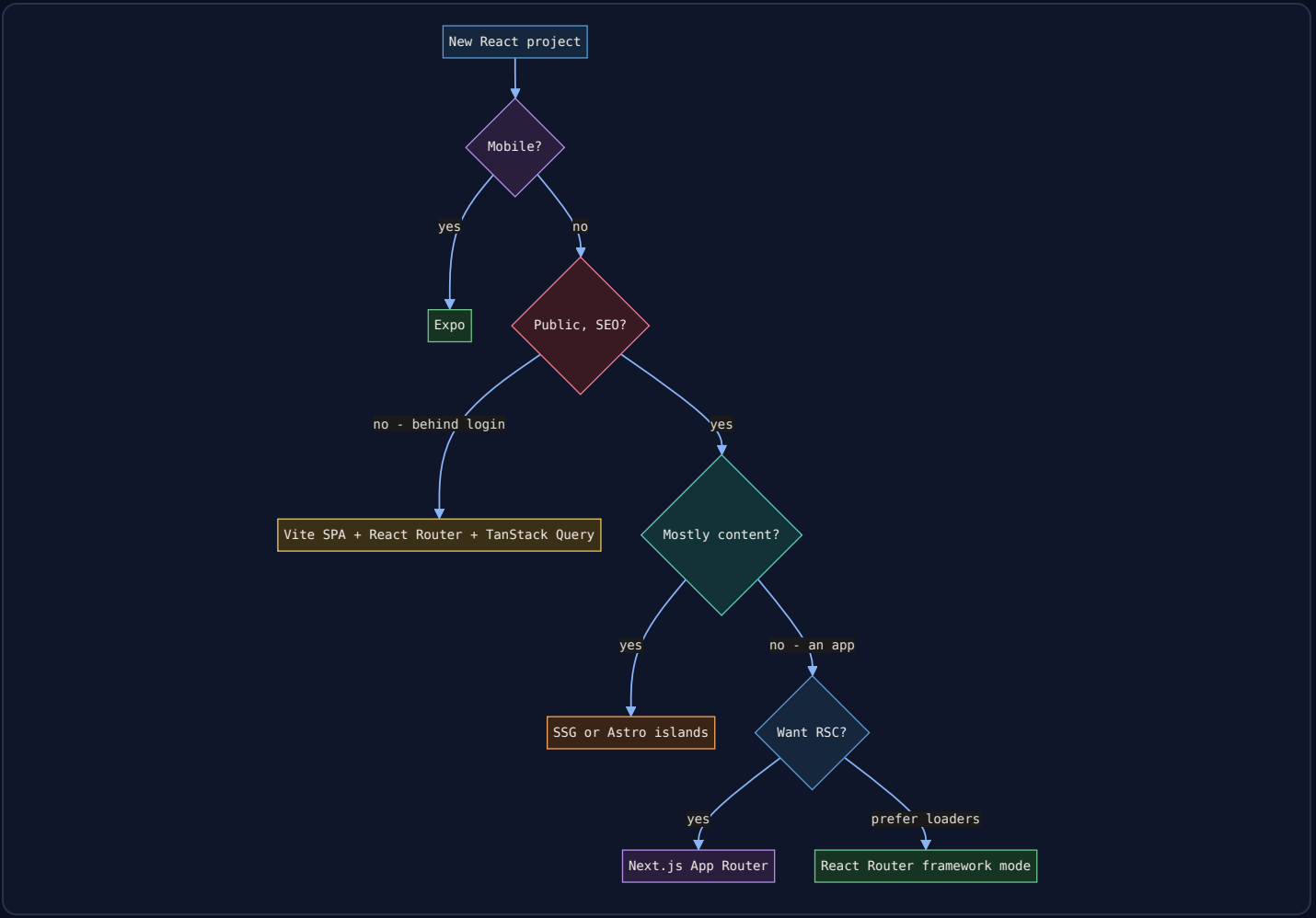
---

13.11 Choosing a framework

The options (September 2026)

Option	What it is	Pick it when
Next.js (App Router)	Full-stack, RSC-first, by Vercel	You want Server Components, SEO-heavy (search-ranking and link-preview sensitive) pages mixing static/dynamic content, one framework for front + back end
React Router (framework mode)	Full-stack, web-standard, Vite-based, by Shopify	You like loaders/actions and HTTP semantics, come from a React Router SPA, or deploy to many platforms
Expo	React Native (React for real mobile apps) framework for iOS, Android and web	A mobile app
TanStack Start	Newer full-stack framework on TanStack Router (type-safe routing, server functions, streaming); v1 release candidate, RSC experimental	End-to-end type safety, already on TanStack tools
Astro (not React-specific)	Content-site framework with React "islands" (small interactive parts in a static page)	Mostly static content with a few widgets
Vite SPA (no framework)	`npm create vite@latest` + React Router data mode + TanStack Query	Internal tools/dashboards behind a login, static hosting, no SEO needs

react.dev recommends a framework (Next.js, React Router or Expo) for new apps, and Vite, Parcel or Rsbuild (frameworkless build tools) when none fits.



Questions to ask

1. **Who must see content before JS runs?** Crawlers, link previews, slow phones → server rendering or SSG.
2. **Where will it run?** Static hosting → SPA or SSG; Node / serverless (code runs on demand on the provider's machines) / edge → any framework (check the adapter).
3. **How fresh must data be?** Static → SSG; hourly → ISR / `'use cache'`; per user → SSR or client fetching.
4. **What does the team know?** React Router data-mode teams move to framework mode easily.
5. **Hosting and lock-in.** All run on Node/Docker (portable app containers), but check self-hosting docs for ISR and caching if not using the vendor's platform.

Common mistakes

- SSR for an internal dashboard "for performance" — a server to run, little gain.
- A SPA for an e-commerce catalogue — slow product pages, empty link previews.
- Choosing by popularity rather than deployment target.

In the interview

30-second answer: "Requirements first. Public pages needing SEO and fast first paint → a framework: Next.js for Server Components and mixed static/cached/streamed content, React Router framework mode for loaders, actions and web standards or an existing React Router app. Internal tool behind a login → Vite SPA + React Router + TanStack Query. Mobile → Expo. Mostly static content → SSG or Astro."

- *What does a framework add over Vite + `react-dom/server`?* — Routing tied to data loading and code splitting, data serialization for hydration (no double fetch), caching/revalidation, server + client bundling and deployment adapters — what §13.4 would otherwise make you build.

Chapter 13 TL;DR

Idea	Remember
Strategies	CSR (browser), SSR (server per request), SSG (build), ISR (SSG + background refresh), streaming (frame now, slow parts later), PPR (static shell + streamed holes) — chosen per route
Server APIs	`renderToPipeableStream` / `renderToReadableStream` (SSR), `prerender*` (build), `resume*` (PPR), `hydrateRoot` in the browser; string APIs are legacy and can't wait for Suspense
Streaming	Shell first; boundaries arrive as hidden divs + `$RC`; status in `onShellReady`, crawlers on `onAllReady`, `abort` on timeout
Hydration	Same tree and data in `hydrateRoot`; reuses DOM; selective; visible ≠ interactive; frameworks avoid the double fetch
Mismatches	First client render = server HTML; Effect, `useSyncExternalStore` or `use(browser())`; fix nesting; `suppressHydrationWarning` for single expected differences
Server Components	Server-only, async, zero JS; RSC payload; not SSR; only serializable props cross; `'use client'` at the leaves; server output as `children`
Server Functions	`'use server'`; public endpoints → auth + validate; for mutations, not reads
Frameworks	Next.js 16 (`app/`, `proxy.js`, Turbopack, Cache Components + `'use cache'`), React Router 8 (`routes.ts`, loaders/actions, `ssr`/`prerender` config, RSC unstable); SPA for tools behind a login

Next: Chapter 14 — Patterns and Architecture.

Chapter 14 — Patterns and Architecture

This chapter is about **arranging** React's building blocks: the reusable patterns found in every codebase and component library — compound components, render props, higher-order components, custom Hooks, portals, error boundaries — how to make components **accessible** (usable by everyone, including screen-reader and keyboard-only users), how to organise **folders** as a project grows, and how a **design system** (a company's shared UI components, styles and usage rules) ties it together. Each pattern answers "how do I share *this* kind of thing?"; the short version: **composition and custom Hooks cover most needs**; the others are for specific jobs.

Code: `code/` — React 19.3 in jsdom with Testing Library; every numbered file prints what a user (or screen reader) would get. Reusable modules: `Tabs.jsx`, `Dialog.jsx`, `ErrorBoundary.jsx`, `Button.jsx`; screenshots from `ds_demo.html` in headless Chrome. Background: composition and `children` (Chapter 2 §2.4), Context (Chapter 6), custom Hooks and `useId` (Chapter 7), refs (Chapter 5), Testing Library queries (Chapter 12 §12.3). Accessibility fundamentals (landmarks — page regions like `<nav>` / `<main>`; ARIA — attributes describing elements to assistive tools; WCAG — the Web Content Accessibility Guidelines) are in the HTML & CSS book's Chapter 3; general design patterns (decorator, observer, strategy…) in the LLD (low-level design) book's structural and behavioural chapters.

14.1 Composition first

What it is

Composition builds a component from components you pass in (`children`, or JSX in named props called **slots**) instead of adding a configuration prop per variation. Introduced in Chapter 2 §2.4; here it's the baseline other patterns are compared against.

The problem it solves — "prop explosion"

```
<Card title="Trail Mug" subtitle="Enamel" showImage imageSrc="mug.png" imagePosition="left"
      showFooter footerButtonLabel="Buy" onFooterButtonClick={buy} compact highlighted />
```

Every need adds a prop, a branch and a test case, and callers can't do anything unforeseen (two footer buttons? a badge by the title?). The composed version exposes **structure**:

```
<Card>
  <Card.Media src="mug.png" alt="Trail Mug" />
  <Card.Title>Trail Mug <Badge>New</Badge></Card.Title>
  <p>Enamel, 350 ml.</p>
  <Card.Footer><Button onClick={buy}>Buy</Button><Button variant="secondary">Save</Button></Card.Footer>
</Card>
```

How to apply it

1. `children` for main content, **slot props** (`header={…}`, `actions={…}`) for secondary areas.
2. **Specialise by wrapping**, not flags: `function DangerDialog(props) { return <Dialog {...props} tone="danger" icon={<WarningIcon/>} />; }`.
3. Parts needing **shared state** (selected tab) → compound components (§14.4).
4. Pass elements down instead of data to avoid prop drilling (threading a prop through layers that don't use it) — middle layers just render `{children}`.

When not to use it

Tiny fixed components (a `Price` formatter) are simpler with a couple of props. Composition pays when callers need different layouts.

14.2 Container / presentational

What it is

One feature split in two:

- **Presentational** ("view") — everything via props, returns markup. No fetching, global state or Effects. Easy to reuse, test and show in Storybook (renders components in isolation in all their states).
- **Container** — knows *where data comes from*: fetches, reads context or a store, handles events, renders the presentational component.

The view is an actor reading whatever script it's handed; the container is the director choosing the script.

History and today

Popular from 2015 (class components and Redux `connect`). After Hooks, its best-known advocate (Dan Abramov) noted he no longer recommends splitting *by rule*, since a custom Hook does the container's job without an extra component. Still valuable for:

1. **Design-system/shared UI** — visual components stay data-agnostic.
2. **Testing and Storybook** — render every visual state (loading, empty, error) via props.
3. **Server Components** (Chapter 13 §13.7) — a Server Component "container" fetches and renders a Client Component "view"; the split is now an architectural boundary.

Runnable version in §14.3: `ProductSearch` (container) = `useProductSearch` (logic) + `ProductSearchView` (presentational).

Common mistakes

- Mechanical `FooContainer` + `Foo` everywhere — twice the files. Split when the view is reused or has many states worth testing alone.
- A "presentational" component secretly calling `useContext(AuthContext)` — no longer reusable.

14.3 Extracting custom Hooks

What it is

Moving stateful logic (state + Effects + derived values) out of a component into a **custom Hook** (Chapter 7); the component keeps only what it renders.

Why it matters

The component becomes markup-only; the logic becomes **reusable** and **testable alone** with `renderHook`; the view becomes testable **without** mocking the network.

How to do it (recipe)

1. Find state + Effects that work together (query → loading → results).
2. Move them into `function useX(inputs)` and **return only what the UI needs** (`{ status, results }`), not internal setters.
3. Pass dependencies in as parameters (the `search` function) — **dependency injection**: code receives what it relies on instead of importing it, so tests can hand in a fake.
4. The component calls the Hook and renders (optionally via a presentational component, §14.2).
5. Check behaviour didn't change (same test before and after).

`code/04_custom_hook_extraction.jsx` — before (abridged):

```
function ProductSearchBefore() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);
  const [status, setStatus] = useState('idle');
  useEffect(() => { /* if (!query) reset; else setStatus('loading'), api.search(query) with an ignore flag */ },
[query]);
  return (/* input + "Searching…" status + results list */);
}
```

After — Hook + view + thin container:

```
export function useProductSearch(query, search = api.search) {        // logic, injectable dependency
  const [state, setState] = useState({ status: 'idle', results: [] });
  useEffect(() => {
    if (!query) { setState({ status: 'idle', results: [] }); return; }
    let ignore = false;                                                // ignore stale responses
    setState(s => ({ ...s, status: 'loading' }));
    search(query).then(results => { if (!ignore) setState({ status: 'done', results }); });
    return () => { ignore = true; };
  }, [query, search]);
  return state;
}
export function ProductSearchView({ query, onQueryChange, status, results }) {   // markup only
  return (
    <section>
      <label>Search <input value={query} onChange={e => onQueryChange(e.target.value)} /></label>
      {status === 'loading' && <p role="status">Searching…</p>}
      {status === 'done' && <ul>{results.map(r => <li key={r}>{r}</li>)}</ul>}
    </section>
  );
}
function ProductSearch() {                                             // wiring
  const [query, setQuery] = useState('');
  const { status, results } = useProductSearch(query);
  return <ProductSearchView query={query} onQueryChange={setQuery} status={status} results={results} />;
}
// tests: same user flow for both versions; then the Hook alone and the view alone
const { result, rerender } = renderHook(({ q }) => useProductSearch(q, fakeSearch), { initialProps: { q: '' } });
render(<ProductSearchView query="x" onQueryChange={() => {}} status="loading" results={[]} />);
```

Output:

```
before: Trail Mug, Trail Stove
after : Trail Mug, Trail Stove
hook, empty query : {"status":"idle","results":[]}
hook, "mug"       : {"status":"done","results":["mug result"]}
view in loading   : Searching… (no API needed)
```

Identical behaviour; the Hook tests with a fake search, the view's loading state renders directly. In a real app the Hook would wrap TanStack Query (Chapter 9) and add debouncing (`useDebounce`, Chapter 7 §7.5).

Common mistakes

- A Hook returning 12 values including internal setters — callers depend on internals. Return a small, intention-revealing API.
- A "Hook" that calls no Hooks — that's a plain function; don't name it `use…`.
- Expecting two components calling one Hook to **share** state — each call has its own; share via context or a store (Chapter 6).

In the interview

30-second answer: "When a component mixes data logic and markup, I extract the logic into a custom Hook that takes inputs (and injectable dependencies) and returns a small API like `{ status, results`

`}` . The component just renders, often via a presentational component. The logic becomes reusable and testable with `renderHook`, the view testable with plain props. Hooks share *logic*, not state."

14.4 Compound components

What it is

Components designed to be used **together**, sharing state **implicitly** via a private context: `<Tabs>`, `<Tabs.List>`, `<Tabs.Tab>`, `<Tabs.Panel>`. The root owns state; parts read it — like HTML's `<select>` / `<option>`.

The problem it solves

`<Tabs tabs={[{ label, content }]}>` can't handle custom layouts (tabs below the panel, an icon or badge in one tab, a shared panel) without ever more props. Compound components give callers the **structure** while the component keeps the **behaviour** (selection, keyboard, ARIA).

How it works

1. The root holds `selected` and puts it plus `select` in a **context**.
2. Each part reads it with a small Hook that **throws a clear error** outside the root.
3. Parts attach as properties (`Tabs.Tab = Tab`) — one import for the family.
4. The root supports **uncontrolled** (`defaultValue`; keeps its own state) and **controlled** (`value` + `onValueChange`; parent owns it) use, like `<input>` (Chapter 3 §3.9).
5. It owns accessibility: roles, `aria-selected`, `aria-controls`, keyboard navigation.

`code/Tabs.jsx` (key parts):

```jsx
const TabsContext = createContext(null);
function useTabs(part) {
  const ctx = useContext(TabsContext);
  if (!ctx) throw new Error(`<Tabs.${part}> must be used inside <Tabs>`);
  return ctx;
}

export function Tabs({ defaultValue, value, onValueChange, children }) {
  const [inner, setInner] = useState(defaultValue);
  const selected = value ?? inner;                                  // controlled wins
  const select = (v) => { if (value === undefined) setInner(v); onValueChange?.(v); };
  const baseId = useId();                                           // stable ids for aria links
  return <TabsContext value={{ selected, select, baseId }}>{children}</TabsContext>;
}

function List({ label, children }) {
  const listRef = useRef(null);
  function onKeyDown(e) {                                           // arrows, Home, End
    const tabs = [...listRef.current.querySelectorAll('[role="tab"]')];
    const i = tabs.indexOf(document.activeElement);
    const next = { ArrowRight: i + 1, ArrowLeft: i - 1, Home: 0, End: tabs.length - 1 }[e.key];
    if (next === undefined) return;
    e.preventDefault();
    const tab = tabs[(next + tabs.length) % tabs.length];
    tab.focus(); tab.click();
  }
  return <div role="tablist" aria-label={label} ref={listRef} onKeyDown={onKeyDown}>{children}</div>;
}

function Tab({ value, children }) {
  const { selected, select, baseId } = useTabs('Tab');
  const isSelected = selected === value;
  return (
    <button role="tab" type="button" id={`${baseId}-tab-${value}`} aria-controls={`${baseId}-panel-${value}`}
      aria-selected={isSelected} tabIndex={isSelected ? 0 : -1} onClick={() => select(value)}>
      {children}
    </button>
  );
}

function Panel({ value, children }) {
  const { selected, baseId } = useTabs('Panel');
  if (selected !== value) return null;
  return <div role="tabpanel" id={`${baseId}-panel-${value}`} aria-labelledby={`${baseId}-tab-${value}`} tabIndex=
{0}>{children}</div>;
}
Tabs.List = List; Tabs.Tab = Tab; Tabs.Panel = Panel;
```

Usage (`code/01_compound_tabs.jsx`), checked with **role queries** (what assistive technology sees):

```jsx
<Tabs defaultValue="details">
  <Tabs.List label="Product information">
    <Tabs.Tab value="details">Details</Tabs.Tab>
    <Tabs.Tab value="reviews">Reviews</Tabs.Tab>
    <Tabs.Tab value="shipping">Shipping</Tabs.Tab>
  </Tabs.List>
  <Tabs.Panel value="details">Enamel mug, 350 ml.</Tabs.Panel>
  <Tabs.Panel value="reviews">4.5 stars from 12 reviews.</Tabs.Panel>
  <Tabs.Panel value="shipping">Ships in 2 days.</Tabs.Panel>
</Tabs>
```

Output (`*` = `aria-selected="true"`):

```
tablist name : Product information
start        : Details* | Reviews | Shipping  panel="Enamel mug, 350 ml."
click Reviews: Details | Reviews* | Shipping  panel="4.5 stars from 12 reviews."
ArrowRight   : Details | Reviews | Shipping*  panel="Ships in 2 days."  focus: Shipping
ArrowRight   : Details* | Reviews | Shipping  panel="Enamel mug, 350 ml."  (wrapped around)
End          : Details | Reviews | Shipping*  panel="Ships in 2 days."
panel labelled by its tab: Ships in 2 days.
tab stops    : -1,-1,0
controlled   : parent state: a | Panel A
misuse       : <Tabs.Tab> must be used inside <Tabs>
```

1. Selection, arrows (wrap-around), `End` — the keyboard behaviour of the WAI-ARIA Authoring Practices (W3C's guide to accessible widgets) "Tabs" pattern.
2. **Roving tabindex** (`-1,-1,0`): only the selected tab is in the Tab order; arrows move within the list — the widget is one Tab stop.
3. The panel's **accessible name** comes from its tab via `aria-labelledby` (and `aria-controls` links tab → panel, ids from `useId`).
4. Works **controlled** by a parent; misuse fails loudly.

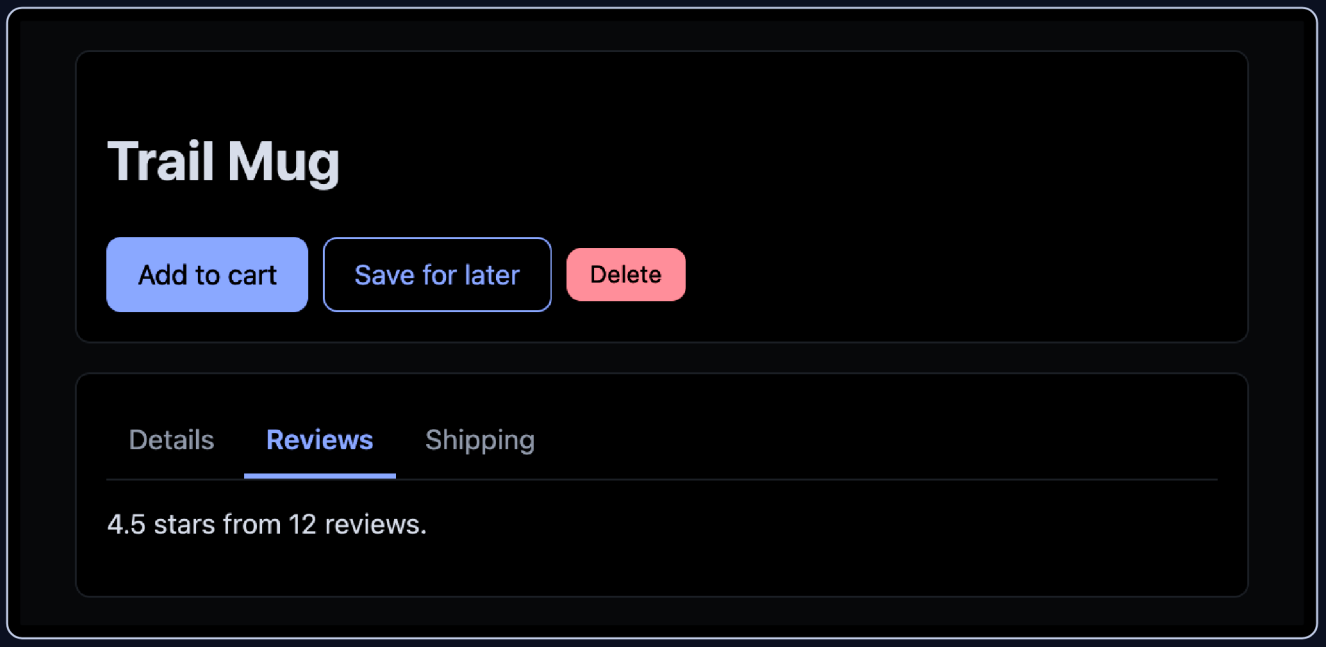


Figure 14.1 — The compound `Tabs` (and the `Button` from §14.12) in Chrome, styled with design tokens (§14.12) (`code/ds_demo.html`).

Trade-offs

	Array/config prop (tabs={[…]})	Compound components
Simple cases	Shortest to use	Slightly more JSX
Custom layout / content	Needs new props	Free — callers arrange parts
Type safety of structure	Easy (typed array)	Harder to enforce order/nesting
Implementation	Simple	Context + part Hooks

Real-world examples

Radix UI (`<Tabs.Root>` , `<Tabs.List>` , `<Tabs.Trigger>` , `<Tabs.Content>`), React Aria Components, Headless UI, Reach UI (accessible, mostly unstyled libraries) — and native `<select>` / `<option>` , `<table>` / `<tr>` , `<details>` / `<summary>` .

Common mistakes

- `React.Children.map` + `cloneElement` (copy an element with changed props) to inject props — breaks once a caller wraps a part in a `<div>` . Context works at any depth.

- No error Hook — parts outside the root fail silently with `undefined` state.
- Rebuilding the context value each render in a large tree (performance — Chapter 6 §6.4; React Compiler handles it).

In the interview

> **30-second answer:** "Compound components are a family like `Tabs`, `Tabs.List`, `Tabs.Tab`, `Tabs.Panel` sharing state through a private context. The root owns the selected value — controlled or uncontrolled — and the parts read it, so callers control layout while the component controls behaviour and accessibility. I use context rather than `cloneElement` so parts nest at any depth, and throw a clear error if a part is used outside its root."

- *Why not `cloneElement`?* — Reaches only direct children, silently breaks with wrappers, and is a legacy API on react.dev.
- *Controlled and uncontrolled?* — Internal state for `defaultValue`, prefer `value` when defined, always call `onValueChange`.
- *Roving tabindex?* — Only the active item has `tabIndex=0`, others `-1`; arrows move focus within the group.

14.5 Render props

What it is

A **render prop** is a prop whose value is a **function returning JSX**; the component owns behaviour/data and **calls your function** to decide what to render. Passed as `children`, it's "function as children".

The problem it solves

Sharing behaviour while each caller chooses markup — before Hooks the main way to reuse stateful logic (React Router v4's `<Route render>`, Formik's `<Formik>{(form) => …}</Formik>`, Downshift). Still the right tool when the component must **control *where* and *how often*** your markup renders: lists (`renderItem`), virtualized rows, table `cell` renderers, empty/loading slots.

Seeing it

`code/02_render_props.jsx`:

```jsx
function Disclosure({ defaultOpen = false, children }) {          // function as children
  const [open, setOpen] = useState(defaultOpen);
  return children({ open, toggle: () => setOpen(o => !o) });
}
function List({ items, renderItem, renderEmpty = () => <p>Nothing here.</p> }) {   // named render props
  if (items.length === 0) return renderEmpty();
  return <ul>{items.map((item, i) => <li key={item.id}>{renderItem(item, i)}</li>)}</ul>;
}
function useDisclosure(defaultOpen = false) {                        // the Hook equivalent
  const [open, setOpen] = useState(defaultOpen);
  return { open, toggle: () => setOpen(o => !o) };
}

<Disclosure>
  {({ open, toggle }) => (
    <><button aria-expanded={open} onClick={toggle}>Shipping details</button>{open && <p>Ships in 2 days.</p>}</>
  )}
</Disclosure>
<List items={products} renderItem={(p, i) => <>{i + 1}. {p.name} — ${p.price}</>} />
```

Output:

```
render prop, closed : <button aria-expanded="false">Shipping details</button>
render prop, open   : <button aria-expanded="true">Shipping details</button><p>Ships in 2 days.</p>
renderItem          : <ul><li>1. Trail Mug — $18</li><li>2. Camp Stove — $45</li></ul>
renderEmpty         : <p>No products match.</p>
same thing via Hook : <button aria-expanded="true">Shipping details</button><p>Ships in 2 days.</p>
```

`Disclosure` never decides the button's look; `List` decides *where* items go (inside `<li>`, keyed) but not what they show.

Trade-offs

- ✔ Explicit about which values the caller gets; works in class components and anywhere JSX is allowed.
- ✘ Nesting several creates "callback pyramids" drifting right.
- ✘ A new function each render can defeat `memo` (React Compiler usually handles it).
- For **sharing stateful logic** only, a custom Hook is simpler (last output line).

Common mistakes

- A render prop just to pass data one level — a normal prop or `children` suffices.
- Calling Hooks inside the render function — it's a plain callback, so the Rules of Hooks forbid it (the linter flags it); put stateful parts in a component.

14.6 Higher-order components (HOCs)

What it is

A **higher-order component** is a function that **takes a component and returns a new one** rendering the original with extra props or behaviour: `const Enhanced = withSomething(Component)`. Named after higher-order functions (JavaScript book, Chapter 4); it's the OOP **decorator pattern** (wrap an object to add behaviour without changing it) (LLD Chapter 9).

Where you still meet them

Redux's `connect(mapState)(Component)`, React Router v5's `withRouter`, `React.memo(Component)` (HOC-like), analytics/feature-flag wrappers (flags toggle features without redeploying), auth guards. New code usually uses Hooks, but you must read and maintain HOCs.

Seeing it

`code/03_hoc.jsx` :

```jsx
function withAuth(Component) {
  function WithAuth(props) {
    const user = useContext(UserContext);
    if (!user) return <p>Please sign in.</p>;
    return <Component {...props} user={user} />;          // pass everything through + inject one prop
  }
  WithAuth.displayName = `withAuth(${Component.displayName || Component.name})`;   // readable in DevTools
  return WithAuth;
}
function withRenderLog(Component) { /* logs each render with its props, same shape */ }
function OrderHistory({ user, limit }) { return <p>{user.name}'s last {limit} orders</p>; }
const ProtectedHistory = withRenderLog(withAuth(OrderHistory));   // apply once, at module level

// PITFALL demo: applying a HOC inside render
function Page({ bad }) {
  const [, force] = useState(0);
  const Wrapped = bad ? withNothing(Counter) : StableCounter;    // bad: a new component type every render
  return <><Wrapped /><button onClick={() => force(x => x + 1)}>re-render page</button></>;
}
```

Output:

```
displayName: withRenderLog(withAuth(OrderHistory))
  [log] withAuth(OrderHistory) rendered with {"limit":3}
signed out : Please sign in.
  [log] withAuth(OrderHistory) rendered with {"limit":3}
signed in  : Asha's last 3 orders
HOC at module    : count 2 -> after parent re-render: count 2
HOC inside render : count 2 -> after parent re-render: count 0
```

1. HOCs **compose** inside-out; `displayName` makes the chain readable in React DevTools.
2. Props flow through (`limit`); the HOC injects its own (`user`).
3. **Never create a HOC during render**: each call returns a *new component type*, so React unmounts the old and mounts a fresh one — state (`count 2`) lost (Chapter 3 §3.10).

Common mistakes (the classic HOC problems)

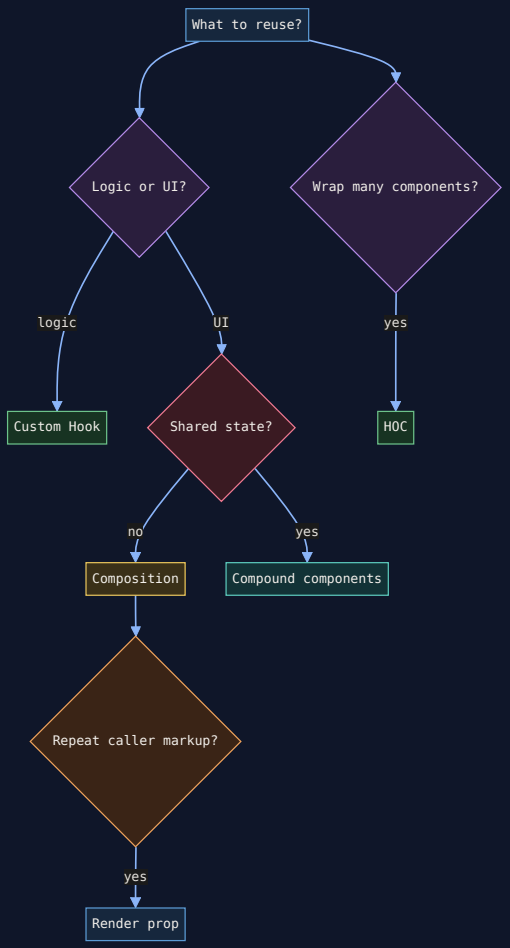
Problem	What happens	Fix
HOC applied inside render	Remount every render, state lost	Apply at module level
Prop name collisions	Two HOCs inject `data`; one silently wins	Namespacing (prefixed names), or Hooks
"Wrapper hell"	Deep anonymous wrappers in DevTools	`displayName`; Hooks
Static methods (functions attached to the component) lost	`Enhanced.someStatic` is undefined	Copy them (`hoist-non-react-statics`)
Refs	Before React 19, `ref` didn't pass through without `forwardRef`	React 19: `ref` is a normal prop for function components; spread it
Hidden dependencies	Reader can't see where `user` came from	A Hook call inside the component is explicit

In the interview

30-second answer: "A HOC takes a component and returns a wrapped one, for cross-cutting concerns (needs spanning many unrelated parts) like auth guards, logging or injecting store data — Redux's `connect` is the famous one. They compose but suffer prop collisions, implicit props, wrapper nesting, and must be created at module level or the component remounts every render. For new code I use a custom Hook; I'd still write a HOC to wrap *rendering* itself, like an auth guard around many route components."

14.7 Choosing a reuse pattern

Need	Best fit	Why
Share **stateful logic** (fetching, form state, subscriptions)	**Custom Hook**	No extra components, explicit, composable
Share **UI structure** with flexible content	**Composition** (`children` , slots)	Simplest; no new concepts
A family of parts sharing **implicit state** (tabs, accordion, menu, select)	**Compound components**	Callers arrange parts; component owns behaviour + a11y
Component controls **where/how often** caller markup renders (lists, tables, virtualized rows)	**Render prop** (`renderItem`)	Component decides placement, caller decides content
Wrap **many components** with the same guard/behaviour, or legacy code	**HOC**	Works on components you don't own; decorator style
Render **outside** the parent's DOM box (modal, tooltip, toast)	**Portal**	Escapes `overflow` / `z-index` while staying in the React tree
Contain crashes of a subtree	**Error boundary**	The only way to catch render errors



14.8 Portals

What it is

`createPortal(children, domNode)` (from `react-dom`) renders `children` into a **different DOM node** — usually `document.body` — while they stay at the **same place in the React tree**: props, context, state and event bubbling behave as if the content were still inside its parent.

The problem it solves

Modals, dropdowns, tooltips and toasts must sit **on top of everything**. Inside a card with `overflow: hidden` or a stacking context (a CSS layering group created by `transform`, `z-index` etc.; `z-index` only compares within one group) they're clipped or covered — CSS can't escape ancestors. Rendering at the end of `<body>` fixes that.

How it works

1. React renders the portal's children as part of the parent's tree.
2. At commit it inserts their DOM nodes into `domNode`.
3. **Context** flows through (the React tree is unchanged).
4. **React events bubble through the React tree**, not the DOM: a click inside the portal triggers `onClick` on React ancestors that aren't DOM ancestors.

Seeing it

`code/Dialog.jsx` — a modal with the accessibility essentials:

```
export function Dialog({ open, onClose, title, children }) {
  const titleId = useId();
  const panelRef = useRef(null);
  const onEscape = useEffectEvent(() => onClose());     // latest onClose, without re-running the Effect
  useEffect(() => {
    if (!open) return;
    const previouslyFocused = document.activeElement;   // remember the trigger
    panelRef.current.focus();                           // move focus into the dialog
    const onKey = (e) => { if (e.key === 'Escape') onEscape(); };
    document.addEventListener('keydown', onKey);
    return () => {
      document.removeEventListener('keydown', onKey);
      previouslyFocused?.focus?.();                     // give focus back on close
    };
  }, [open]);                                          // re-runs only when opened/closed

  if (!open) return null;
  return createPortal(
    <div className="backdrop" onClick={(e) => { if (e.target === e.currentTarget) onClose(); }}>
      <div role="dialog" aria-modal="true" aria-labelledby={titleId} tabIndex={-1} ref={panelRef}
className="dialog">
        <h2 id={titleId}>{title}</h2>
        {children}
        <button type="button" className="btn btn-secondary btn-sm" onClick={onClose}>Close</button>
      </div>
    </div>,
    document.body,
  );
}
```

`code/05_portal.jsx` opens it from a card with `overflow: hidden` whose `onClick` counts clicks (`bubbled`); the dialog contains `<ThemedText />` reading a theme context.

Output:

```
dialog DOM parent        : BODY (not inside .card)
inside .card?            : false
aria-modal               : true
focus moved into dialog  : true
context crosses portal   : theme from context: dark
React events bubble to the React parent: card onClick count 1 -> 2
Escape closes            : true
focus back on trigger    : true
```



Figure 14.2 — The same `Dialog` in Chrome: rendered at the end of `<body>`, so no card's `overflow` or stacking can clip it.

1. The dialog's DOM is under `<body>`, so `overflow: hidden` can't clip it; context still reaches `ThemedText`.
2. The click **inside the dialog** incremented the card's counter — React bubbling follows the component tree. A parent "click outside to close" handler fires for clicks inside the portal; check `e.target`, or `e.stopPropagation()` deliberately.
3. Focus moves in on open, returns to the trigger on close; Escape closes — the minimum modal keyboard behaviour. (This version doesn't trap Tab inside; the Chapter 15 §15.13 modal adds that.) `useEffectEvent` (stable since React 19.2, Chapter 5) lets the Escape listener call the latest `onClose` without re-running the Effect — otherwise a new arrow function each parent render would bounce focus back and forth.

The native `<dialog>` alternative

`<dialog>` with `dialog.showModal()` (called from an Effect via a ref) renders in the browser's **top layer** — above everything, no portal — with Escape-to-close, an inert background (not clickable or focusable), focus moved in on open and **returned to the previously focused element on close** (HTML standard; all major browsers), for free. Supported in all modern browsers and often the simplest correct modal (HTML & CSS book, Chapter 3). Portals remain the tool for tooltips, popovers in older setups, toasts, and rendering into other containers (an iframe, a widget root). The Popover API (`popover` attribute) is another native option for non-modal pop-ups.

Common mistakes

- A home-made modal without focus management — keyboard and screen-reader users end up "behind" it. Move focus in, trap it (or make the background `inert`), return it on close; for production use Radix Dialog, React Aria or `<dialog>`.
- Forgetting portal content still counts as a child for events.
- Rendering a portal during SSR — no `document` on the server; render after mount (or use `<dialog>`).

In the interview

> **30-second answer:** "`createPortal` renders children into another DOM node like `document.body` while keeping them in the same React tree. Modals and tooltips escape `overflow: hidden` and stacking contexts, while context, state and event bubbling still follow the React tree — a click inside a portal bubbles to React ancestors. For modals I also handle focus, Escape and `aria-modal`, or use native `<dialog>` with `showModal()`, which uses the top layer."

14.9 Error boundaries

What it is

An **error boundary** catches errors thrown **while rendering** its descendants (including lifecycle methods and Effect setup during commit) and shows a **fallback UI** instead of the app vanishing. With no boundary, React 19 **unmounts the entire root** on an uncaught render error — a blank page. Boundaries must still be **class components** (no Hook equivalent of the two methods); write one once or use `react-error-boundary`. (Route-level boundaries: Chapter 9; lazy-chunk failures: Chapter 10 §10.8.)

How it works

1. A descendant throws during render.
2. React finds the nearest boundary and calls `static getDerivedStateFromError(error)` — return state so the next render shows the fallback (pure, during render).
3. After commit, `componentDidCatch(error, info)` — for side effects such as logging; `info.componentStack` says where.
4. React 19 also calls the root's `onCaughtError` (caught by a boundary) or `onUncaughtError` (no boundary) — `createRoot` / `hydrateRoot` options, good for one central reporter. (`onRecoverableError` reports errors React recovered from itself, like hydration mismatches — Chapter 13 §13.5.)
5. To retry, reset the boundary's state — a "Try again" button, automatically when inputs change (**reset keys**), or by changing its `key`.

`code/ErrorBoundary.jsx`:

```jsx
export class ErrorBoundary extends Component {
  state = { error: null };
  static getDerivedStateFromError(error) { return { error }; }       // render the fallback next
  componentDidCatch(error, info) { this.props.onError?.(error, info); } // log (Sentry, console…)
  componentDidUpdate(prevProps) {                                    // reset when resetKeys change
    const { resetKeys = [] } = this.props;
    if (this.state.error && resetKeys.some((k, i) => !Object.is(k, prevProps.resetKeys?.[i]))) {
      this.setState({ error: null });
    }
  }
  reset = () => this.setState({ error: null });
  render() {
    if (this.state.error) return this.props.fallback({ error: this.state.error, reset: this.reset });
    return this.props.children;
  }
}
```

What boundaries catch — and what they don't

`code/06_error_boundary.jsx` puts a price and a reviews widget in separate boundaries, plus an event handler that can fail:

```
const root = createRoot(container, {
  onCaughtError(error) { console.log(`   onCaughtError   : "${error.message}"`); },
  onUncaughtError(error) { console.log(`   onUncaughtError : "${error.message}" -> whole root unmounted`); },
});
function ProductPrice({ product }) { return <p>Price: ${product.price.toFixed(2)}</p>; }   // throws on null
function Reviews() {
  const [error, setError] = useState(null);
  if (error) throw error;                                    // re-throw during render → boundary catches
  return <button onClick={() => {
    Promise.reject(new Error('network down')).catch(e => setError(e));   // async error moved into render
  }}>Load reviews</button>;
}
// in Shop:
<ErrorBoundary resetKeys={[id]} fallback={(p) => <Fallback {...p} />} onError={…}><ProductPrice product={product}
/></ErrorBoundary>
<ErrorBoundary fallback={(p) => <Fallback {...p} />}><Reviews /></ErrorBoundary>
<button onClick={() => {
  try { JSON.parse('{bad json'); }                           // event-handler errors: NOT caught by boundaries
  catch { setImportMsg('Could not read the file'); }         // handle them yourself
}}>Import</button>
```

Output:

```
1. start              : Price: $18.00
   onCaughtError   : "Cannot read properties of null (reading 'price')"
   componentDidCatch: stack starts at ProductPrice
2. render error       : Could not show this part (Cannot read properties of null (reading 'price')). Try again
   onCaughtError   : "Cannot read properties of null (reading 'price')"
   componentDidCatch: stack starts at ProductPrice
3. reset, still broken: Could not show this part (Cannot read properties of null (reading 'price')). Try again
4. resetKeys changed  : Price: $21.00
5. handler error      : Could not read the file | price still fine: Price: $21.00
   onCaughtError   : "network down"
6. async error        : Could not show this part (network down). Try again | price still fine: Price: $21.00
7. no boundary        : container HTML = ""
```

- **Step 2**: contained — only the price area shows the fallback.
- **Step 3**: "Try again" re-rendered the same `null` data, so it failed again — reset helps only if something changed.
- **Step 4**: changing `id` (a reset key) cleared the error.
- **Step 5**: **event-handler** errors aren't render errors; boundaries never see them — catch and turn into state.
- **Step 6**: **async** errors (promises, timers) aren't caught unless moved into render by storing and re-throwing (TanStack Query's `throwOnError` and `use()` with a rejected promise do this).
- **Step 7**: no boundary → the whole root unmounts (browser: `onUncaughtError` called; tests: `act()` re-throws).

Caught by error boundaries	Not caught
Errors during render of descendants	Event handlers (use `try/catch`)
Errors in lifecycle methods / Effects of descendants	Async code: `setTimeout`, promise callbacks (move into state and re-throw)
Rejected promises read with `use()` inside Suspense	Errors in the boundary itself (a parent boundary catches those)
Errors from `lazy()` chunk loading	Server-side rendering errors (fall back to client rendering, Chapter 13 §13.3)

Where to put boundaries

- **One at the top** (app shell): "Something went wrong — reload" instead of a blank page.
- **One per route** (Next.js `error.js`, React Router `ErrorBoundary` exports).
- **Around independent widgets** (chart, comments, third-party embed).

- Pair with `<Suspense>`: Suspense = "not ready yet", boundary = "failed".

Common mistakes

- Expecting a boundary to catch `onClick` errors.
- A fallback that can itself throw (reads the same broken data).
- "Try again" that changes nothing — reset with a refetch or new input (reset keys).
- Swallowing errors without logging — report in `componentDidCatch` or `onCaughtError`.

In the interview

> **30-second answer:** "Error boundaries are class components with `getDerivedStateFromError` to show a fallback and `componentDidCatch` to log. They catch errors thrown while rendering their subtree — including `use()` rejections and lazy-loading failures — but not event handlers, async callbacks, SSR, or their own errors. Without one, React 19 unmounts the whole root. I place one at the top, per route and around risky widgets, reset them with reset keys, and report centrally via `createRoot`'s `onCaughtError` / `onUncaughtError`."

- *Why still a class?* — No Hook for `getDerivedStateFromError` / `componentDidCatch`; most teams use `react-error-boundary`, which wraps a class and adds `resetKeys` and a `useErrorBoundary` Hook to push async errors in.
- *Send an async error to a boundary?* — Store it in state and throw during render (or `showBoundary(error)` from `react-error-boundary`).
- *`getDerivedStateFromError` vs `componentDidCatch`?* — The first runs during render to choose the fallback (pure); the second after commit for side effects like logging.

14.10 Accessibility in React

What it is

Accessibility (a11y) means people can use your UI however they perceive or operate it: screen readers, keyboard only, voice control, zoom, reduced motion. React renders the HTML you write; it adds (a) a few JSX spellings, (b) state-driven ARIA attributes, and (c) **focus management** in SPAs, which have no full page loads. Fundamentals (landmarks, headings, ARIA rules, WCAG 2.2): HTML & CSS book Chapter 3; this is the React layer.

React-specific rules

Topic	In React
Labels	`<label htmlFor={id}>` (`for` is a JS keyword); ids from `useId()`, never hard-coded in reusable components (must be unique per page and match between server and client)
ARIA attributes	Hyphenated lower-case: `aria-label`, `aria-expanded={open}`; booleans render as `"true"` / `"false"`
Semantic elements first	`<button>`, `<a href>`, `<nav>`, `<main>`, `<h1>`–`<h6>`, `<ul>`: roles, keyboard support and focusability for free
Fragments (group children without a DOM element)	`<>…</>` instead of wrapper `<div>`s that break list/table semantics (`<ul>` must contain `<li>`)
Focus management	Refs move focus when UI changes: into dialogs, to the new page heading after client navigation, back to the trigger after closing
Announcements	Live region: `role="status"` (polite) or `role="alert"` (urgent), present in the DOM before the text changes
Icon-only buttons	Accessible name `aria-label="Remove item"`; hide the icon with `aria-hidden="true"`
Linting	`eslint-plugin-jsx-a11y` flags missing alt, click handlers on non-interactive elements, invalid ARIA
Automated audits	axe-core (browser extension, `vitest-axe` / `jest-axe`, Playwright's axe integration), Lighthouse (Chrome's page-quality report)

Checking it with role queries

`getByRole` finds elements as assistive technology does — by **role** and **accessible name**; if a test can't find your button by role, a screen-reader user can't either (Chapter 12 §12.3). `code/07_a11y_checks.jsx` audits a bad and a good checkout:

```
function BadCheckout({ onBuy }) {
  return (
    <div>
      <div className="title">Checkout</div>                {/* looks like a heading, isn't one */}
      <img src="mug.png" />                                {/* no alt */}
      <input placeholder="Email" />                        {/* placeholder is not a label */}
      <div className="btn" onClick={onBuy}>Buy now</div>   {/* not focusable, no role */}
      <button><svg aria-hidden="true" /></button>          {/* icon-only: no name */}
    </div>
  );
}
function GoodCheckout({ onBuy }) {
  const emailId = useId(), errId = useId();
  // ... error + done state, submit() validates and calls onBuy ...
  return (
    <form onSubmit={submit} noValidate>
      <h1>Checkout</h1>
      <img src="mug.png" alt="Trail Mug, blue enamel" />
      <label htmlFor={emailId}>Email</label>
      <input id={emailId} name="email" type="email" aria-invalid={!!error} aria-describedby={error ? errId :
undefined} />
      {error && <p id={errId}>{error}</p>}
      <button type="submit">Buy now</button>
      <button type="button" aria-label="Remove item"><svg aria-hidden="true" /></button>
      <p role="status">{done ? 'Order placed' : ''}</p>
    </form>
  );
}
```

Output:

```
--- bad
heading "Checkout"      : false
image with alt text     : missing
textbox named "Email"   : false
button named "Buy now"  : false
button names            : [""]
Tab key visits          : input -> button -> body (the "Buy now" div is never reached)
--- good
heading "Checkout"      : true
image with alt text     : Trail Mug, blue enamel
textbox named "Email"   : true
button named "Buy now"  : true
button names            : ["Buy now","Remove item"]
invalid email -> aria-invalid: true | description: "Enter a valid email"
valid email   -> bought: 1 | aria-invalid: false | status: Order placed
```

1. The bad version **looks** the same with CSS, but a screen reader gets no heading, an unnamed image, an unlabelled field, a nameless button — and the "Buy now" `div` (no button role) is **unreachable by Tab**.
2. In the good version every control has a role and name; the error is the field's **accessible description** (`aria-describedby`), and `aria-invalid` reflects state.
3. A real `<form>` + `<button type="submit">` makes **Enter** submit for free.
4. The `role="status"` live region announces "Order placed" without moving focus.

Focus after client-side navigation

SPA link clicks swap content without a page load, so screen readers may not announce the new page and focus stays on the old link. Fixes: move focus to the new `<h1>` (`tabIndex={-1}`) in an Effect keyed on location, and update `document.title` (React 19 lets any component render `<title>`). Next.js announces route changes; React Router offers scroll restoration and you add focus handling.

Common mistakes

- `<div onClick>` instead of `<button>` — no keyboard, role or focus (you'd need `role="button"`, `tabIndex={0}`, Enter/Space handlers — just use `<button>`).
- `outline: none` without a visible replacement — use `:focus-visible` styles (shown when focus should be visible, e.g. keyboard use).
- ARIA overuse (`role="button"` on a `<button>`, `aria-label` contradicting visible text). First rule of ARIA: native elements when they exist.
- Conditionally rendering the live region itself — only changes to an existing region are announced.
- Testing only with a mouse — Tab through every new screen.

In the interview

30-second answer: "React renders whatever HTML I write, so accessibility is mostly semantic HTML: real buttons, links, headings and labels. The React-specific parts are `htmlFor`, `useId` for label/description ids, state-driven ARIA like `aria-expanded`, live regions for status messages, and focus management with refs — into dialogs, back to triggers, and to the heading after client-side navigation. I test with Testing Library role queries, which fail when elements lack roles or names, plus `eslint-plugin-jsx-a11y`, axe, and manual keyboard and screen-reader checks."

14.11 Folder structure

What it is

File layout decides how easily people find code, how far changes ripple, and whether features can be deleted cleanly. React has **no opinion**; frameworks impose some (Next.js `app/`, React Router `app/routes/`). Start simple (Chapter 1 §1.6); grow deliberately.

Three common layouts

1. By type — fine for small apps (< ~15 components):

```
src/
├── components/   Button.jsx, ProductCard.jsx, CartPanel.jsx, ...
├── hooks/        useCart.js, useProductSearch.js
├── pages/        Home.jsx, Product.jsx
├── services/     api.js
└── utils/        formatPrice.js
```

At scale one feature spreads over five folders and `components/` hits 200 files.

2. By feature — common for medium and large apps:

```
src/
├── app/                ← app shell: providers, router, global styles
├── features/
│   ├── cart/
│   │   ├── components/   CartPanel.jsx, CartItem.jsx
│   │   ├── hooks/        useCart.js
│   │   ├── api.js        cart endpoints
│   │   ├── cart.test.jsx
│   │   └── index.js      ← public API: the only file other features import
│   └── catalog/ ...
└── shared/             ← feature-agnostic: ui/ (design system), lib/ (helpers), config
```

Everything "cart" lives together (**colocation**: files that change together, kept together — tests and styles too). Deleting a feature = deleting a folder.

3. Framework routes + features — thin route files importing from features:

```
app/
├── layout.jsx
├── products/[id]/page.jsx      ← fetch + compose: imports from features/catalog
└── cart/page.jsx
src/features/..., src/shared/...  (or inside app/ in private _folders)
```

Rules that keep it healthy

1. **Dependency direction:** `app → features → shared`. Shared never imports features; features never import the app shell.
2. **Public APIs:** each feature exposes `index.js`; others import `features/cart`, never `features/cart/store.js`, so internals can change freely.
3. **Enforce, don't hope:** ESLint `no-restricted-imports` or `eslint-plugin-boundaries`, or monorepo tool path rules (a monorepo holds several apps/packages in one repository).
4. **Path aliases** (`@/features/cart` instead of `../../../`) via `tsconfig`/`jsconfig` `paths` and the bundler.
5. **Careful with barrel files** (`index.js` re-exporting everything): one per feature is fine; giant barrels slow dev servers and tests, can hurt tree-shaking (dropping unimported code) and create import cycles.

Seeing a boundary check

`code/08_import_boundaries.mjs` scans the sample project in `code/feature-app/` and applies rules 1–2 — a miniature lint rule:

```
if (fl === 'shared' && tl !== 'shared') problems.push([from, spec, 'shared/ must not import from ' + tl + '/']);
if (fl === 'features' && tl === 'app') problems.push([from, spec, 'features must not import app/']);
if (fl === 'features' && tl === 'features' && featureOf(from) !== featureOf(target)
    && target.split(path.sep).length > 2) {                    // deeper than features/<name>
  problems.push([from, spec, `deep import into feature "${featureOf(target)}" (use its index)`]);
}
```

Output (file tree abridged: `app/App.jsx`; `features/cart/{components/CartPanel.jsx, hooks/useCart.js, index.js, store.js}`; `features/catalog/{components/ProductCard.jsx, components/ProductList.jsx, index.js}`;

`shared/{lib/formatPrice.js, ui/Button.jsx}`):

```
2 boundary violation(s):
  features/catalog/components/ProductList.jsx
    imports '../../cart/store.js' -> deep import into feature "cart" (use its index)
  shared/lib/formatPrice.js
    imports '../../features/cart' -> shared/ must not import from features/
```

`ProductCard.jsx` imports `useCart` from `'../../cart'` (the public index) and passes. The equivalent ESLint rule (not run):

```
// eslint.config.js (excerpt)
rules: {
  'no-restricted-imports': ['error', { patterns: [
    { group: ['@/features/*/*'], message: 'Import features through their index (public API).' },
  ] }],
}
```

Common mistakes

- A deep hierarchy on day one — start flat; add `features/` once two or three exist.
- `shared/` / `utils/` as a dumping ground — if one feature uses it, it belongs there.
- Circular imports between features — the shared piece belongs in `shared/`, or one feature should own it.

14.12 Design systems

What it is

A **design system** is the shared set of decisions, components and docs that make a product look and behave consistently. In React, three layers:

Layer	What it is	Example
Design tokens	Named values for design decisions: colours, spacing, radii, typography, shadows, motion	`--color-brand: #3553c9` , `--space-2: 8px`
Primitives / components	Accessible building blocks using the tokens	`Button` , `Tabs` , `Dialog` , `TextField` , `Menu`
Patterns and docs	How to combine them, when to use which	"Destructive actions use `variant="danger"` and a confirm dialog"

Why it matters

Consistency (one Button, not 14), speed (assemble, don't rebuild), accessibility solved once (focus, ARIA, keyboard in every `Menu`), and themeability (change tokens → rebrand or dark mode — Chapter 8 §8.8).

Tokens in practice

`code/ds.css` defines tokens as CSS custom properties (`--something` , read with `var(--something)`); components use only tokens:

```
:root {
  --color-brand: #3553c9; --color-danger: #c62f3b; --color-border: #dde1e7;
  --radius-md: 8px; --space-1: 4px; --space-2: 8px; --space-3: 12px; --space-4: 16px;
  --focus-ring: 0 0 0 3px #9db0ff;
}
.btn-primary { background: var(--color-brand); color: var(--color-brand-text); }
.btn:focus-visible, .tab:focus-visible { outline: none; box-shadow: var(--focus-ring); }
.tab[aria-selected="true"] { color: var(--color-brand); border-bottom-color: var(--color-brand); }
```

The Tabs style keys off `aria-selected` — styling from accessibility state keeps visuals and semantics in sync.

A primitive: `Button`

`code/Button.jsx`:

```
const variants = { primary: 'btn-primary', secondary: 'btn-secondary', danger: 'btn-danger' };
const sizes = { sm: 'btn-sm', md: 'btn-md' };

export function Button({ variant = 'primary', size = 'md', className = '', type = 'button', ref, ...rest }) {
  return <button ref={ref} type={type} className={`btn ${variants[variant]} ${sizes[size]} ${className}`.trim()}
{...rest} />;
}
```

`code/09_design_system_button.jsx`:

```
<Button>Add to cart</Button>
<Button variant="secondary" size="sm" disabled>Save for later</Button>
<Button variant="danger" aria-label="Delete Trail Mug" ref={ref}>×</Button>
<Button type="submit" className="wide">Pay</Button>
```

Output:

```
<button type="button" class="btn btn-primary btn-md">Add to cart</button>
<button type="button" class="btn btn-secondary btn-sm" disabled="">Save for later</button>
<button type="button" class="btn btn-danger btn-md" aria-label="Delete Trail Mug">×</button>
<button type="submit" class="btn btn-primary btn-md wide">Pay</button>
ref points at the DOM node: true BUTTON
found by accessible name  : true
```

Design choices worth copying:

1. **A small, closed set of variants** (`primary | secondary | danger`) — the system decides what's allowed (a TypeScript union, Chapter 11 §11.9).
2. **Safe defaults**: `type="button"` avoids accidental submits (a `<button>` in a form defaults to `submit`).
3. **Pass native props through** (`disabled` , `aria-*` , `onClick`) with `...rest` , plus an extra `className` .
4. `ref` **is a normal prop in React 19** — no `forwardRef` , so focus management and popover positioning work (Chapter 5 §5.10).

Build or adopt?

Few teams should build complex widgets (menus, comboboxes, date pickers) from scratch — keyboard and ARIA details take long to get right.

Approach	Examples	You get	You still do
Headless / unstyled primitives	Radix UI Primitives, React Aria (Adobe), Headless UI, Base UI	Behaviour, keyboard, ARIA, focus — no styles	All visual design (with your tokens)
Copy-in components	shadcn/ui (source copied into your repo; built on headless primitives + Tailwind)	Styled, editable source you own	Maintaining the copied code
Full component libraries	MUI, Chakra UI, Mantine, Ant Design	Styled, themed, many components	Theming to your brand; accept their look/API
Build your own	Large companies' internal systems	Full control	Everything, including accessibility and docs

Common modern stack: tokens (CSS variables or Tailwind theme) + headless primitives + your styled wrappers (`Button` , `Dialog`) exported from `shared/ui` , documented in **Storybook**, tested with Testing Library plus visual-regression tests (before/after screenshot comparison).

API conventions for design-system components

- **Controlled + uncontrolled** (`value` / `defaultValue` / `onValueChange`), as in §14.4.
- **Compound parts** for complex widgets (`Dialog.Title` , `Menu.Item`).
- **Composition over configuration** (§14.1).
- **Polymorphism** to render as another element: an `as` prop (`<Button as="a" href="/cart">`) or Radix's `asChild` (merge behaviour onto the single child). Keep it limited — `<Button as="div">` loses button semantics.
- **Accessible by default**: correct roles, names required for icon-only components, visible focus.

Common mistakes

- Every team adding "just one more" variant — the system fragments; changes go through its owners.
- Hard-coded colours instead of tokens — dark mode and rebrands become rewrites.
- Wrapping a headless library and dropping its accessibility props (not spreading `...rest` , losing `ref`).

In the interview

30-second answer: "A design system is tokens — named colours, spacing, type — plus accessible primitives built on them, plus usage docs. In React I'd expose components like `Button` with a closed set of variants, safe defaults, native props and `ref` passed through, and compound APIs for complex widgets. Rather than hand-rolling menus and dialogs I'd build on headless primitives like Radix or React Aria, style them with our tokens, document them in Storybook and test with Testing Library and visual regression."

14.13 Client architecture patterns: MVC, MVP, MVVM, VIPER — and where React fits

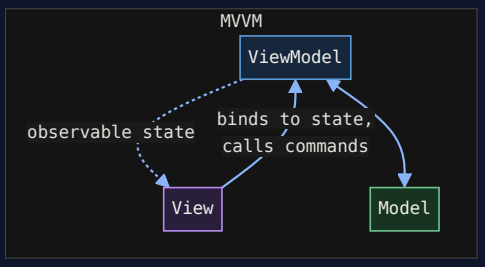
What it is

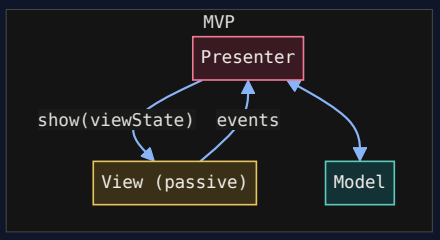
One level above components, every UI codebase (web, iOS, Android, desktop) must decide **where logic lives** between data and pixels. The classic answers all split a screen into a **Model** (data and business rules — e.g. "a cart total includes tax" — no UI), a **View** (what the user sees) and a connecting part; they differ in **who talks to whom** and **how much the view knows**. You'll hear them in mobile interviews, Angular/Vue/SwiftUI/Android codebases, and "how does React compare?" questions. Restaurant analogy: model = kitchen and recipes, view = the guest's table; in **MVC** a head waiter takes orders and the table watches the kitchen's pass; in **MVP** a waiter carries and arranges every plate (passive table); in **MVVM** the table has a self-updating display board.

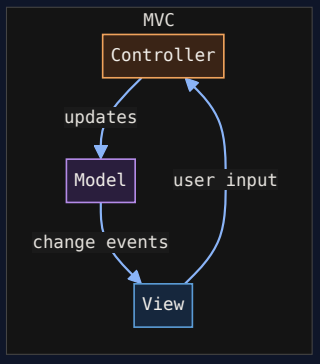
The problem it solves

Without a pattern, screens become "massive view controllers" (iOS's per-screen class) mixing network calls, formatting, validation, navigation and drawing — testable only by UI tests. Each pattern moves logic **out of the view** into something testable with plain function calls, and makes pieces replaceable.

How it works — the four classics







	MVC (Smalltalk, 1979)	**MVP (1990s; Android before 2017)**	**MVVM (Microsoft WPF, 2005; Android Jetpack, SwiftUI, Vue, Angular)**	**VIPER (iOS, 2014)**
Parts	Model, View, Controller	Model, **passive** View, Presenter	Model, View, ViewModel	**V**iew, **I**nteractor, **P**resenter, **E**ntity, **R**outer
Who updates the view	the view observes (subscribes to) the model (or, in web/iOS variants, the controller pushes)	the presenter calls methods on a view interface	the view **binds** to observable ViewModel state; the ViewModel never references the view	presenter → view
Business logic	model / controller	presenter + model	ViewModel + model	**Interactor** (use cases); presenter only formats; **Router** owns navigation
Testing without UI	controller and model	presenter, with a fake view	ViewModel, with no view	every part separately
Weak spot	controllers grow huge ("Massive View Controller" on iOS)	view-interface boilerplate	binding "magic"; ViewModels can grow huge	many files per screen; heavy for small apps

MVC was invented for Smalltalk-80 (Trygve Reenskaug); server frameworks (Rails, Spring MVC, ASP.NET MVC) reused the name for request → controller → model → rendered template — related, not the same. **MVP** makes the view as dumb as possible so the presenter is unit-tested against a fake view. **MVVM** replaces explicit `view.show(...)` calls with **data binding**: the view declares "this label shows `vm.total`" and the

framework re-renders on change. **VIPER** is Clean Architecture (business rules never depend on UI or database) for one iOS screen: use cases in the interactor, plain entities, a router for navigation.

Two newer patterns matter more for React:

- **Unidirectional data flow** (Facebook's **Flux**, 2014; **Redux**; the **Elm Architecture**): the view dispatches an **action**, a pure **reducer** computes the next state, the view re-renders. One direction, so every change is traceable and replayable (Chapter 6 — `useReducer`, Redux Toolkit, Zustand). It appeared because two-way bindings with many models updating many views made changes hard to trace.
- **MVI** (Model–View–Intent, Android/Kotlin): MVVM with a single immutable state object and reducer-style updates — essentially Redux on mobile.

How React fits

React began as "the V in MVC"; today's usage maps onto these patterns without the names:

Classic role	In a React codebase
View	function components returning JSX — ideally presentational, props in (14.2)
ViewModel / Presenter	a **custom Hook** owning state and exposing display-ready values plus commands (14.3): `const { lines, totalText, add } = useCartViewModel()`
Binding	React re-renders from state — one-way binding, `UI = f(state)`
Model / store	server state in TanStack Query or a loader (Chapter 9); client state in `useReducer`, Context or an external store read with `useSyncExternalStore`
Controller / Interactor	event handlers, reducers, service modules; with Server Components and Server Actions (Chapter 13) some moves to the server
Router	React Router / Next.js routing

"Is React MVC or MVVM?" — "React is a view library with one-way data flow. A custom Hook plays the ViewModel role, reducers or a store give Flux-style unidirectional updates, and the component is a pure view of that state."

Seeing it: `code/10_client_architectures.jsx`

One tiny cart ("Add Mug", "Add Tent", a total) built four ways — MVC (a model object the view subscribes to with `useSyncExternalStore`), MVP (a plain-JS presenter pushing view state into a passive view), MVVM (a `useCartViewModel` Hook) and a reducer — all rendering the same `CartView`; then each architecture's logic is tested without rendering:

```
MVC     : Mug $8.50, Tent $120.00 | Total: $128.50
MVP     : Mug $8.50, Tent $120.00 | Total: $128.50
MVVM    : Mug $8.50, Tent $120.00 | Total: $128.50
Reducer : Mug $8.50, Tent $120.00 | Total: $128.50
MVP presenter test, fake view got   : {"lines":["Tent $120.00"],"totalText":"$120.00"}
MVVM view-model test, state         : {"lines":["Mug $8.50"],"totalText":"$8.50"}
Reducer test, replaying 2 actions   : 2 items
all assertions passed
```

Presenter tested with a fake view, ViewModel Hook with `renderHook`, reducer as a pure function (same state + action → same next state) — the shared payoff.

When not to use it

- VIPER-style layering on a five-screen app — a component + custom Hook per feature is enough.
- Imperative (step-by-step) `view.show()` MVP in React — state-driven rendering already gives the binding MVP simulates (the demo does it only for comparison).
- One global store for everything "because Redux" — server state in a query cache, local state local (Chapter 6).

Common mistakes

- **Saying React "is MVC"** → it's a view library; the patterns describe arranging state and logic around it.
- **ViewModels that know about the view** (DOM nodes, refs) → expose state and commands only.
- **Fat views** → formatting (`$8.50`), validation and derived values belong in the Hook/presenter/selector (function deriving a value from stored state).
- **Two-way binding everywhere** → hard to trace who changed what; prefer one-way flow with explicit events.

In the interview

> **30-second answer:** "All separate Model and View and differ in the middle piece. In MVC a controller handles input and updates the model, and the view observes the model. In MVP a presenter does the work and pushes formatted data into a passive view through an interface, so it's easy to unit-test. In MVVM the view binds to observable state on a ViewModel, which never references the view. VIPER splits an iOS screen into View, Interactor, Presenter, Entity and Router. React is a view library with one-way data flow: components are views, custom Hooks act as ViewModels, reducers or stores give Flux-style unidirectional updates."

- *MVP vs MVVM?* → MVP's presenter holds a view reference and calls it; MVVM's ViewModel exposes state the view binds to, with no reference back.
- *Why Flux/Redux?* → Two-way bindings and many models updating many views made changes hard to trace; action → reducer → state → view is predictable and replayable.
- *What does VIPER's R own?* → Navigation between screens.

Sources: Trygve Reenskaug, "MVC — Xerox PARC 1978-79" notes; Martin Fowler, "GUI Architectures" (2006) and "Presentation Model" (2004); John Gossman, "Introduction to Model/View/ViewModel pattern" (Microsoft, 2005); objc.io issue 13, "Architecting iOS Apps with VIPER" (2014); Flux documentation (Meta); Redux documentation ("Three Principles"); Elm guide ("The Elm Architecture"); Android Developers "Guide to app architecture"; React docs (`useSyncExternalStore`, reusing logic with custom Hooks).

Chapter 14 TL;DR

Idea	Remember
Composition	`children` and slots beat boolean-prop explosions; specialise by wrapping
Container / presentational	Props-in views; data access in a Hook, container or Server Component — where it helps, not by rule
Custom Hook extraction	Logic → `useX(inputs)` with a small return API; injectable deps; test with `renderHook`; Hooks share logic, not state
Compound components	Root owns state in a private context; parts read it; controlled + uncontrolled; roving tabindex; throw on misuse
Render props	A function prop the component calls to render; best for lists/tables/slots; Hooks for logic
HOCs	`withX(Component)`; module level only; pass props through; `displayName`; mostly legacy
Portals	DOM elsewhere, React tree unchanged — context and event bubbling follow React; manage focus; consider `<dialog>`
Error boundaries	Class with `getDerivedStateFromError` + `componentDidCatch`; render errors only (not handlers/async); reset keys; root `onCaughtError` / `onUncaughtError`
Accessibility	Semantic elements, `htmlFor` + `useId`, state-driven ARIA, live regions, focus with refs; test by role
Folder structure	Small: by type. Growing: by feature with public `index.js`; `app → features → shared`; enforce with lint; thin framework route files
Design systems	Tokens + accessible primitives + docs; closed variants, safe defaults, pass-through props and `ref`, headless libraries, Storybook
Client architectures	MVC (view observes model), MVP (presenter drives passive view), MVVM (view binds to ViewModel), VIPER (use-case layers + router), Flux (one-way); React: custom Hook ≈ ViewModel, reducers = unidirectional

Next: Chapter 15 — React Interview and Machine Coding.

Chapter 15 — React Interview and Machine Coding

A front-end React interview usually has two technical parts. The **concepts round** asks "why" questions — why keys matter, why an Effect ran twice, why a memoized child still re-rendered — and listens for a precise mental model. The **machine-coding round** has you build a small widget (autocomplete — a text box suggesting matches as you type — modal, tabs…) in 45–90 minutes while explaining your choices: how you split the problem, where state lives, whether the keyboard works, how you handle edge cases (empty data, errors, very fast input).

Part one is **53 questions with short, correct answers** for React 19, each linked to the section that teaches it; the most-disputed answers are *proved* by a runnable program. (Chapter 17 is the wider question bank and links back here rather than repeating these.) Part two is **eight machine-coding builds**, done as in the room: requirements → component tree → state design → code → a React Testing Library + `user-event` test → a Chrome screenshot → follow-ups and edge cases.

Code: `code/` — React 19.3 in jsdom. Each build is one component file (`todo.jsx`, `autocomplete.jsx`, …) plus a test file (`02_todo.test.jsx`, …) using `node:test`, React Testing Library and `@testing-library/user-event`, as in Chapter 12; test listings below show the key parts. Screenshots come from `code/machine-coding.html`: open it to see all eight builds, or add `#todo`, `#autocomplete`, `#infinite`, `#modal`, `#tabs`, `#stars`, `#pagination` or `#wizard` for one. Run everything with `bash tools_run_js.sh react` from the repository root.

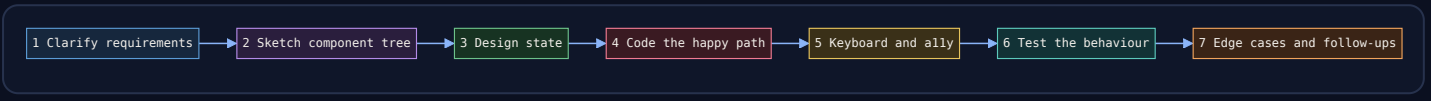
15.1 How React interviews are run

What it is

Round	What happens	What is really being tested
Concepts / "React internals"	30–60 minutes of questions, often with a snippet: "what does this log?"	A correct mental model of rendering, state and effects
Machine coding (UI build)	Build a widget in 45–90 minutes, in a sandbox (online editor) or your own editor	Decomposition, state design, working code, accessibility, testing, talking while coding
JavaScript	Closures, the event loop, `this`, implement-from-scratch utilities such as `debounce`	The language under React — JavaScript Handbook Chapter 17
Front-end system design	Design a news feed, a chat app, a design system	Data flow, caching (System Design Fundamentals), rendering strategy (CSR/SSR/RSC; Q43–Q45), performance, accessibility at scale

The machine-coding method

The same seven steps work for every build. Say each step out loud; interviewers score the process, not only the code.



1. **Clarify requirements** (3–5 min): what must work, then the quiet requirements — keyboard, screen readers, mobile, loading/error states, data size, browsers. Write a checklist; agree what's out of scope.
2. **Sketch the component tree** (2–3 min): who renders whom, props down, callbacks up.
3. **Design state** (3–5 min). For each piece of data: *can it be computed from something else?* (then derived — §3.8); *who needs it?* (closest common parent owns it — §3.7); *does it belong outside React?* (server data, URL, `localStorage`).

4. **Code the happy path** (normal flow, nothing going wrong; 20–30 min): something on screen early, then grow it; prefer native elements (`<button>` , `<input type="radio">` , `<form>`) for free keyboard and screen-reader behaviour.
5. **Keyboard and accessibility** (5–10 min): Tab order, visible focus, Escape, arrows where expected, labels, roles, `aria-*` state.
6. **Test** (5–10 min): one or two user-like tests — find by role and label, act with `user-event` , assert what the user sees (§12.4).
7. **Edge cases and follow-ups** (the rest): empty, loading, error, huge lists, fast typing, double clicks, unmount mid-request, "how would this scale?".

60-minute budget	Minutes
Requirements + tree + state	10
Happy path	25
Keyboard + a11y	8
Test	7
Edge cases, discussion	10

Common mistakes

- Coding before agreeing requirements, then building the wrong thing.
- Storing derived values in state (`filteredTodos` next to `todos`) and fighting to keep them in sync.
- `<div onClick>` instead of `<button>` : not focusable, no Enter/Space, no role.
- Silence — the interviewer can't credit reasoning they didn't hear.
- No time left for edge cases — the part that separates "works" from "senior".

15.2 Interview questions: rendering, reconciliation and keys

How to use sections 15.2–15.9

Each answer takes about 30 seconds to say and links to the section that teaches it. Where an answer is often disputed, `code/01_interview_facts.jsx` proves it and its real output is shown. Answer aloud first, then check.

Q1. What happens, step by step, when you call a state setter? Nothing changes immediately. (1) **Trigger** — the update is queued and a re-render scheduled; (2) **render** — React calls the component (and its children) and compares the new element tree with the previous one (reconciliation); (3) **commit** — React applies only the differences to the DOM, runs layout effects, the browser paints, then `useEffect` Effects run. → §10.1, §1.3

Q2. What is the difference between the render phase and the commit phase? Which one must be pure? The **render phase** calls components to compute the UI. It must be **pure** (same inputs → same JSX, no side effects) because React may call it twice, pause it or discard it (concurrent rendering, StrictMode). The **commit phase** applies DOM changes and runs effects, once per committed update; side effects belong there or in event handlers. → §2.5, §10.1

Q3. What is reconciliation? Is "the virtual DOM" still the right way to describe React? Reconciliation compares the previous element tree with the next to find the DOM changes. A general tree diff is $O(n^3)$; React makes it $O(n)$ with two rules: **a different element type means a different subtree** (`<div>` → `<section>` , or `ComponentA` → `ComponentB` , destroys the old subtree and its state), and **keys identify list children** across renders. "Virtual DOM" is an older name for the in-memory element tree; today's docs speak of *rendering* and *committing*, and what matters is identity by *type* + *position* + *key*. → §1.3, §3.10

Q4. Why do lists need keys, and why is the array index a bad key? A key tells React which item is which between renders, so the right DOM nodes and **state** move with it. With index keys, inserting at the top or re-sorting gives key `0` to a different item, which inherits the first row's DOM and state (typed

text, a checked box, focus). Use a stable ID from the data; index keys only for static lists. Never `Math.random()` — new keys every render remount every item. → §2.8

Q5. What does changing a component's `key` do? React treats it as a **different instance**: the old one unmounts (state gone, Effects cleaned up) and a new one mounts fresh — the idiomatic way to reset a form when the selected user changes: `<Profile key={userId} />`.

```
# a different key = a new component instance = fresh state
  Draft state initialised
  (same key: no new initialisation)
  Draft state initialised
```

→ §3.10

Q6. When does a component re-render? When its own **state** changes, its **parent re-renders** (children re-render by default, props changed or not), or a **context** it reads changes. "Props changed" is not a separate trigger — props change only because the parent re-rendered. A re-render with identical output commits nothing to the DOM. → §10.2

Q7. Why does `{items.length && <List />}` sometimes render a `0`? `&&` returns its left side when falsy (`0`, `''`, `null`, `undefined`, `NaN`, `false`), and React renders the number `0`. Use a boolean (`items.length > 0 && …`) or a ternary; `false`, `null`, `undefined` and `true` render nothing. → §2.7

15.3 Interview questions: state, events and forms

Q8. I call `setCount(count + 1)` three times in one click. Why does the count go up by one? State is a **snapshot**: within one render `count` is a constant, so all three calls queue "replace with 1". An **updater function** builds on the queued value: `setCount(c => c + 1)` three times gives +3. The handler still sees the old value after calling the setter.

```
# snapshot + batching: three setN(n + 1) vs three setN(m => m + 1)
  render n=0
  handler still sees n=0
  render n=1
  render n=4
```

One render per click — **batching** (Q9). → §3.3, §3.5

Q9. What is batching? Did React 18 change it? React applies all updates queued during an event in **one** re-render. Before React 18 only React event handlers were batched; since React 18 (`createRoot`) batching is **automatic everywhere** — timeouts, promises, native listeners. To force a synchronous DOM update (rare: scrolling to a just-added item, focusing a heading that must already show new text) use `flushSync` from `react-dom`. → §3.4

Q10. Why must state be updated immutably? React detects change with `Object.is(old, new)`; a mutated object is the same reference, so React may skip the render, and `memo` and dependency checks miss it too. Mutation also breaks snapshots. Create new objects/arrays (spread, `map`, `filter`, `toSorted`, `with`) or use Immer for deep updates. → §3.6

Q11. What is "derived state", and what's wrong with copying props into state? Anything **computable** from props or state during render — a filtered list, a count, "is the form valid". Store the minimum and compute the rest, or two copies drift apart. `useState(prop)` uses only the prop's **first** value; use the prop directly, name it `initialX` to make that explicit, or reset with a `key`. → §3.8

Q12. Where should a piece of state live? As **low** as possible but **high enough** that all its users are below it — the closest common parent ("lifting state up"), passing down the value and a callback. Colocation keeps re-renders small; lifting avoids out-of-sync copies. → §3.7, §6.1

Q13. Controlled vs uncontrolled inputs — which should I use? Controlled: `value` from state plus `onChange`; React is the source of truth, so you can validate or format per keystroke. **Uncontrolled**: the DOM keeps the value (`defaultValue`), read with a ref or `FormData` on submit — less code, fewer re-renders. Controlled when the UI reacts to each keystroke; uncontrolled for submit-and-read forms, especially with React 19 form Actions (`<form action={fn}>` receives `FormData` and resets uncontrolled

fields after success). Don't switch one input between the two — `value={undefined}` then a string triggers a warning. → §4.2, §4.4

Q14. What do `useActionState`, `useFormStatus` and `useOptimistic` do?

- `useActionState(action, initialState)` → `[state, formAction, isPending]`: the action's return value (e.g. validation errors) becomes state.
- `useFormStatus()`, called in a component *inside* a `<form>`, → `{ pending, data, method, action }` — ideal for a reusable submit button.
- `useOptimistic(state, updateFn)` shows a temporary value while an async action runs, then falls back to the real state when it finishes or fails. → §4.5, §4.6, §4.7

Q15. How are React events different from DOM events? Handlers get a **synthetic event**, a cross-browser wrapper with the usual interface (`target`, `preventDefault()`, `stopPropagation()`). Since React 17 listeners sit on the **root container**, not `document`, and events propagate through the **React** tree (portals — Q49). `onChange` fires on every keystroke (like the DOM `input` event). Returning `false` doesn't prevent default; call `e.preventDefault()`. → §3.1

15.4 Interview questions: effects and refs

Q16. What is `useEffect` for — and when is it the wrong tool? Effects **synchronise a component with an external system** (network, browser, a third-party library): a request tied to what's on screen, a subscription, a timer, a non-React widget, `document.title`. Wrong tool for computing from props/state (do it in render), reacting to a user action (event handler), resetting state on a prop change (`key`) or passing data up (lift state). Each unnecessary Effect costs an extra render and invites bugs. → §5.1, §5.6

Q17. What do no array, `[]` and `[a, b]` mean as the second argument? No array: after **every** commit. `[]`: after mount only (cleanup on unmount). `[a, b]`: after mount and whenever `a` or `b` changed (`Object.is`). Dependencies aren't a choice — every reactive value (props, state, anything computed from them) the Effect reads must be listed (`react-hooks/exhaustive-deps` checks). Objects or functions created during render are new each time and re-run the Effect every render; create them inside the Effect or memoize them. → §5.5

Q18. When does cleanup run, and in what order do Effects run? Cleanup runs **before the Effect re-runs** (seeing the *old* values of its own render) and on **unmount**. Children's Effects run before their parent's, and all layout effects before any passive `useEffect`.

```
# effect order: child before parent, layout before passive, cleanup sees OLD values
  child  layout setup   v=1
  child  effect setup   v=1
  parent effect setup   v=1
  --- re-render with v=2
  child  layout cleanup v=1
  child  layout setup   v=2
  child  effect cleanup v=1
  parent effect cleanup v=1
  child  effect setup   v=2
  parent effect setup   v=2
```

→ §5.3

Q19. Why does my Effect run twice, and my component render twice, in development? `<StrictMode>` deliberately runs, **in development only**, component bodies, initialisers and updaters twice and each Effect's **setup → cleanup → setup** on mount (since React 19, ref callbacks too). It surfaces impure renders and missing cleanups: whatever breaks would also break on a real remount (navigation, `<Activity>`, fast refresh). Fix the cleanup — don't disable StrictMode or add "has run" flags. Production runs everything once.

```
# StrictMode (development only): render twice, effects setup -> cleanup -> setup
  effect setup
  effect cleanup
  effect setup
  component function called 2 times for one mount
```

→ §5.4

Q20. What race condition can happen when you fetch in an Effect, and how do you prevent it? A **race condition**: the result depends on which overlapping operation finishes first. If `userId` goes 1 → 2 quickly, the response for 1 can arrive **after** the one for 2 and overwrite it. In the cleanup, set an `ignore` flag the `.then` checks, or abort with an `AbortController` (`fetch(url, { signal })`). Better: TanStack Query or router loaders, which also cache and de-duplicate. Build 2 (§15.11) tests this race. → §5.7, §7.6

Q21. `useEffect` vs `useLayoutEffect`? `useLayoutEffect` runs after the DOM update but **before the browser paints**, so you can measure (a tooltip's height) and re-position without flicker. It blocks painting — use it only for measure-then-position; everything else goes in `useEffect`, which normally runs after paint. → §5.11

Q22. `useRef` vs `useState`? Both persist across renders. Setting state **re-renders**; changing `ref.current` does **not** and is visible immediately (no snapshot). Refs hold values that don't affect output — timer IDs, an `AbortController`, "request in flight" flags, DOM nodes. Don't read or write `ref.current` during render (except lazy initialisation): render must stay a pure function of props and state.

```
# changing ref.current does not re-render; state does
  render (ref=0)
  (clicked ref++ twice: no renders logged)
  render (ref=2)
```

→ §5.8, §5.9

Q23. How do you pass a ref to your own component in React 19? As an ordinary prop: `function Input({ ref, ...props }) { return <input ref={ref} {...props} /> }`. `forwardRef` is no longer needed (it still works; deprecation is planned). Ref callbacks may return a **cleanup function**. To expose a limited API (only `focus()`), use `useImperativeHandle`. → §5.10

Q24. What problem does `useEffectEvent` solve? An Effect sometimes needs the *latest* value without re-running when it changes — a chat connection that logs with the current theme, an interval calling the latest `onTick`. As a dependency it reconnects on every change; left out it goes stale. `useEffectEvent(fn)` (stable since React 19.2) returns a non-reactive function that always sees the latest props/state, callable only from Effects. → §7.7

15.5 Interview questions: hooks, context and state management

Q25. What are the Rules of Hooks, and why do they exist? Call Hooks only at the **top level** of components and custom Hooks — not in conditions, loops, nested functions or after an early `return`. React stores Hook state in a list matched **by call order**; if the order changes, state attaches to the wrong Hook. `use()` is the exception: allowed in `if` and loops (still only in components/Hooks). → §7.1, §7.2

Q26. What is a custom Hook? Do two components using the same Hook share state? A function named `use…` that calls other Hooks, to reuse **stateful logic** (`useDebouncedValue`, `useLocalStorage`). It shares logic, **not state**: each call has its own state, as if written inline. To share state, lift it or use context or a store. → §7.3

Q27. `useState` or `useReducer`? `useState` for simple independent values. `useReducer` when values change together, the next state depends on the previous in several ways, or you want the logic in one **pure, testable function** with named actions (`{ type: 'toggled', id }`) — easy to move into Redux or Zustand later. Build 1 uses one. → §6.2

Q28. Is Context a state-management tool? No — Context is a **transport** that passes a value to any depth; the state still lives in `useState` / `useReducer` in a provider. Good for theme, current user, locale, a

reducer's `dispatch`. React 19 renders `<ThemeContext value={...}>` directly (`.Provider` still works). → §6.3, §6.5

Q29. What is the performance problem with Context, and how do you fix it? When a provider's `value` changes (`Object.is`), **every** consumer re-renders, `memo` or not. An inline `value={{ user, setUser }}` is a new object each render, so consumers re-render even when nothing changed:

```
# memo does not stop context updates; a new object value re-renders every consumer
  Label rendered (dark)
  Plain rendered
  --- parent re-renders, nothing really changed
  Label rendered (dark)
```

(`Plain` and `Label` are both memoized; only `Label` reads the context, and only it re-rendered.) Fixes: memoize the value (`useMemo`, or the React Compiler); **split** contexts (state vs dispatch, fast vs slow); move state down; or an external store with selectors (Zustand, Redux, `useSyncExternalStore`) for high-frequency shared state. → §6.4

Q30. How do you choose a state-management approach? First classify the state:

Kind	Put it in
UI state used by one component	`useState` / `useReducer`
Shared by a few nearby components	Lift to the common parent
Needed deep in the tree, changes rarely	Context (+ reducer)
Client state shared widely, changes often	Zustand or Redux Toolkit (selectors limit re-renders)
Server data (cache of the backend)	TanStack Query, router loaders, or Server Components
Shareable/bookmarkable (filters, page, tab)	The URL (search params)
Form fields	Form Actions, or a form library (React Hook Form) for big forms

→ §6.6, §6.10

Q31. Why treat server data differently from client state? Server data is a **cache** of something you don't own: it goes stale, is shared, and needs loading/error states, de-duplication, refetching, pagination and invalidation. Hand-writing that with `useEffect` + `useState` is subtle; TanStack Query (or loaders/RSC) solves it once. Keep `useState`/stores for true client state. → §6.9, §9.8

Q32. What is `useSyncExternalStore` for? Subscribing to a store **outside** React (`navigator.onLine`, a hand-written store, a library) safely under concurrent rendering — no "tearing", where parts of the screen show different versions. You pass `subscribe`, `getSnapshot` and optionally `getServerSnapshot`; Zustand and React Redux use it internally. → §7.9

15.6 Interview questions: performance and the React Compiler

Q33. What does `memo` do, and when doesn't it help? `memo(Component)` skips a re-render caused by the parent when every prop is `Object.is`-equal to last time. It doesn't help with new objects/functions each render (`style={{…}}`, `onClick={() => …}`), a changed context, the component's own state changes, or cheap components (the comparison costs too). → §10.4

Q34. `useMemo` vs `useCallback`? Both cache between renders until a dependency changes. `useMemo(() => compute(a), [a])` caches a **value**; `useCallback(fn, [a])` caches the **function** itself (= `useMemo(() => fn, [a])`). They are hints — React may drop the cache. Use them for measured slow calculations or when identity matters (a `memo` child, an Effect dependency). → §10.5, §10.6

Q35. What is the React Compiler, and does it make `memo` / `useMemo` / `useCallback` obsolete? A build-time tool (v1.0 stable since October 2025) that inserts memoization of values, callbacks and JSX automatically, more finely than people do by hand. Manual memoization becomes rarely needed; the Hooks remain escape hatches (e.g. to control an Effect's dependencies). It assumes the Rules of React (pure

render, no mutation), skips code it can't prove safe, and doesn't fix slow algorithms, huge DOMs or big bundles. → §10.7

Q36. A page feels slow. How do you investigate? Measure first: React DevTools **Profiler** (what rendered, why, how long), the browser Performance panel, and a **production** build (dev is slower and StrictMode doubles renders). Usual causes: state too high (a keystroke re-renders the page), new objects breaking `memo`, over-eager context values, expensive render calculations, long lists, big bundles. → §10.3, §10.10

Q37. How does code splitting work in React? `const Chart = lazy(() => import('./Chart.jsx'))` puts `Chart` in a separate file downloaded on first render; `<Suspense fallback={…}>` covers the wait. Split by route first, then heavy rarely-used widgets (editors, charts). → §10.8

Q38. How do you render 10,000 rows? Virtualize: render only rows in or near the viewport, with spacers for the rest (TanStack Virtual, react-window). Or paginate (Build 7) or scroll infinitely (Build 3). Keep keys stable and rows cheap. → §10.9

15.7 Interview questions: Suspense, `use()` and transitions

Q39. What is Suspense, and what can "suspend"? `<Suspense fallback={<Spinner />}>` shows the fallback until everything inside is ready. What suspends: `lazy()` components, `use(promise)` on a pending promise, and Suspense-enabled sources (`useSuspenseQuery`, framework loaders, async Server Components) — **not** data fetched in `useEffect`. Nested boundaries let parts load independently. → §9.11

Q40. What does `use()` do, and how is it different from other Hooks? `use(resource)` reads a **promise** (suspends until it resolves; a rejection goes to the nearest error boundary) or a **context**, and may be called in `if` and loops. In a Client Component the promise must be **cached or created outside render** (from a Server Component, loader or cache) — one created during render is new each time, so the component suspends forever. → §9.11

Q41. `useTransition` vs `useDeferredValue`? Both mark work **non-urgent** so typing stays responsive. `useTransition` → `[isPending, startTransition]`: wrap a **state update you control** (tab switch, filter, navigation) and React renders it interruptibly in the background; in React 19 the function may be async (an "Action"). `useDeferredValue(value)` suits a value handed to you (a prop): the slow part renders with an older copy that catches up when React has time. Neither is for a controlled input's own value, which must update urgently. → §7.10, §7.11

Q42. When content is already on screen and a new update suspends, does the user see the fallback again? Not in a **transition**: React keeps the old content (dim it with `isPending`) until the new content is ready. Without a transition the nearest fallback reappears — which is why routers wrap navigations in transitions. → §9.11, §9.12

15.8 Interview questions: server rendering, hydration and Server Components

Q43. CSR, SSR, SSG, streaming — what's the difference? **CSR**: an almost empty page plus JS; the browser builds the UI. **SSR**: HTML rendered per request, visible before JS loads, then hydrated. **SSG**: HTML rendered once at build time, served as files. **Streaming SSR**: HTML sent in chunks as each `<Suspense>` boundary is ready, so a slow part doesn't hold back the page. → §1.4, §13.1, §13.3

Q44. What is hydration, and what causes a hydration mismatch? Hydration (`hydrateRoot`) makes server HTML interactive: React renders the same tree in the browser, **adopts** the existing DOM and attaches handlers, expecting the first client render to match the server exactly. Mismatches come from `Date.now()`, `Math.random()`, `window` / `localStorage` checks in render, locale formatting, invalid nesting (`<div>` in `<p>`) or extensions editing the DOM; React reports a recoverable error and client-renders that part. Fixes: render the same on both sides and switch after hydration (an Effect, or React 19.3's `use(browser())` inside Suspense); `useId` for IDs; `suppressHydrationWarning` only for a single unavoidable difference like a timestamp. → §13.4, §13.5

Q45. What are React Server Components, and how do they differ from Client Components? Server Components run only on the server (per request or at build), can be `async` and read data directly, and **ship no code** — only their output. No state, Effects, handlers or browser APIs (`window`, `localStorage`). **Client Components** are the React you know: pre-rendered to HTML, then hydrated and run in the browser. `'use client'` marks the **boundary** — that module and its imports become client code. Server → Client props must be **serializable** (data, JSX, promises, Server Functions — not ordinary functions or class instances). A Client Component can't import a Server Component but can receive one as `children`. → §13.7, §13.8

Q46. What does `'use server'` mean? Is it how you mark a Server Component? No — Server Components are the default and need no directive. `'use server'` marks **Server Functions**: async functions that run on the server but are callable from the client (`<form action>`, or in a transition). Each becomes a network endpoint, so validate input and check authorisation inside, like any public API. → §4.8, §13.8

Q47. What goes wrong if you put `'use client'` at the top of the root layout? Everything it imports becomes client code: no zero-JS static parts, no direct data access, a bigger bundle. Push `'use client'` down to interactive **leaves** (a like button, a search box); keep pages and layouts as Server Components. → §13.8, §13.9

15.9 Interview questions: error boundaries, portals, patterns, testing and accessibility

Q48. What do error boundaries catch — and what don't they? A **class** component with `static getDerivedStateFromError` (show a fallback) and `componentDidCatch` (log); no Hook version, so many apps use `react-error-boundary`. It catches errors thrown while **rendering**, in lifecycle methods and in Effects **below** it — **not** in event handlers, async code you started (`setTimeout`, a `.then` outside render), server rendering, or itself. Handle handler errors with `try/catch` + state; to route an async error to a boundary, throw it during render (a rejected `use(promise)`) or inside `startTransition`. React 19 adds root `onCaughtError` / `onUncaughtError` for reporting.

```
# error boundaries catch render errors, not event-handler errors
  render error -> Fallback: render failed
  click error -> reached window 'error' event: click failed
  boundary fallback shown after the click? false
```

→ §14.9, §9.12

Q49. What is a portal, and how do events behave inside one? `createPortal(children, domNode)` renders into a **different DOM node** (usually `document.body`) while keeping the same place in the **React** tree — for modals, tooltips and toasts that must escape `overflow: hidden` or a stacking context. Context still flows in, and events **bubble through React parents**, not DOM parents:

```
# events from a portal bubble through the REACT tree, not the DOM tree
  DOM parent of the button: <body>, inside the React parent's div? false
  React parent onClick fired
```

→ §14.8, Build 4 (§15.13)

Q50. Compound components, render props, HOCs, custom Hooks — when would you use each? Custom Hooks — the default for sharing *logic*. **Compound components** (`<Tabs><Tabs.List/><Tabs.Panel/></Tabs>`, state via context) — flexible *markup*, common in design systems. **Render props** (a function prop) — the caller decides what to render with the component's data; still handy for lists and virtualization. **HOCs** (`withAuth(Page)`) — wrap a component to inject props; mostly replaced by Hooks but common in older code. → §2.4, §14.4–§14.7

Q51. What is the philosophy of React Testing Library? The closer a test is to how people use the UI, the more confidence it gives. Find elements as a user or screen reader would — **role and accessible name first** (`getByRole('button', { name: 'Save' })`), then label and text, `data-testid` last — act with `user-event` (real key presses, focus, clicks) and assert on what's visible. Skip implementation details (state, Hook calls, class names) so tests survive refactors. Use `findBy…` / `waitFor` for async UI; fake the network at the boundary (a function prop, or MSW), not your own modules. → §12.1, §12.3, §12.4, §12.6

Q52. How do you make React components accessible? **Semantic HTML** first (`<button>` , `<a href>` , `<label>` , `<fieldset>` , headings, `<dialog>`) — roles, focus and keyboard for free. Label every input (`htmlFor` + `useId`). Manage **focus** on big changes (into a dialog and back, to a new step's heading, to the first invalid field). Support expected keys (Escape closes, arrows inside tabs/menus/listboxes). ARIA only for what HTML can't say (`aria-expanded` , `aria-current` , `aria-invalid` , `aria-describedby` , live regions) — no ARIA beats wrong ARIA. Check with `eslint-plugin-jsx-a11y` , role-based tests, a keyboard and a screen reader. → §7.8, §14.10, HTML & CSS Chapter 3

Q53. How would you type a component's props, children and events in TypeScript? Props with a `type` / `interface` and destructuring (`function Button({ variant = 'primary', ...rest }: ButtonProps)`); children as `React.ReactNode` ; native props via `React.ComponentProps<'button'>` ; events as `React.ChangeEvent<HTMLInputElement>` or inferred from inline handlers; refs as `useRef<HTMLInputElement>(null)` ; mutually exclusive props as **discriminated unions** (object types told apart by one shared literal field such as `status`). → §11.2, §11.4, §11.9

Self-check (15.2–15.9)

1. A child wrapped in `memo` , with no props, reads a context whose provider passes `value={{ theme }}` . Its parent re-renders. Does the child re-render? → Yes: the provider creates a new object, so the context value changed by `Object.is` , and context updates go around `memo` (Q29).
2. Why is `key={Math.random()}` worse than `key={index}` ? → Index keys only misbehave when the list reorders; random keys remount every item every render, losing state and focus (Q4).
3. What single change stops a stale fetch response from overwriting a newer one? → Effect cleanup that aborts the request or sets an `ignore` flag (Q20).
4. Can a Server Component pass `onClick={() => …}` to a Client Component? → No — ordinary functions aren't serializable. Pass data, or a Server Function (`'use server'`) (Q45–Q46).
5. An error thrown in an `onClick` handler — which error boundary shows it? → None; boundaries only catch errors during rendering, lifecycles and Effects. Catch it in the handler and put it in state (Q48).

15.10 Build 1 — Todo list with filters

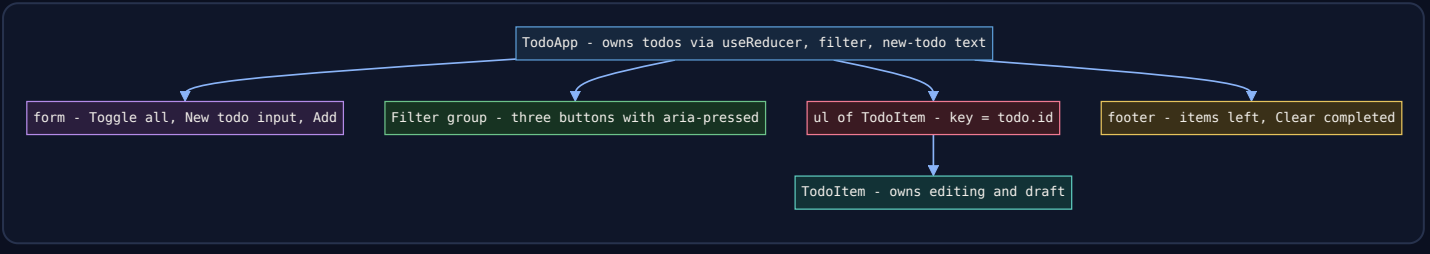
Requirements

The prompt: *"Build a todo app: add, complete, delete, filter by All / Active / Completed."* Clarifying questions turn that into:

Functional	Quiet requirements (ask about them)
Add a todo (Enter or button); ignore blank input	Keyboard-only use works; every control has an accessible name
Toggle done; delete	Survives a page reload (`localStorage`)
Filter: all / active / completed	Counter text is announced ("2 items left")
Edit a title (double-click or ✎); Enter saves, Escape cancels, empty deletes	Toggle-all and "Clear completed"

A simpler version (no reducer, no editing) is built in §3.11; this is the interview-grade one.

Component tree



State design

Data	Kind	Owner	Why
`todos`	state (`useReducer`)	`TodoApp`	many kinds of change → one pure reducer; the list and the counter both need it
`filter`	state	`TodoApp`	UI choice (a follow-up could move it to the URL)
`text`	state	`TodoApp`	controlled "New todo" input
`editing`, `draft`	state	each `TodoItem`	only that row cares — colocated, so typing re-renders one row
`visible`, `left`	**derived**	computed in render	never stored, so never out of sync
next ID	ref	`TodoApp`	needed by handlers only, never displayed

Code

`code/todo.jsx`:

```jsx
// Machine coding 1 — todo list with filters, inline editing and localStorage persistence.
import { useEffect, useReducer, useRef, useState } from 'react';

export const FILTERS = {
  all: () => true,
  active: (t) => !t.done,
  completed: (t) => t.done,
};

// All todo changes live in one pure function: easy to read, easy to unit-test.
export function todosReducer(todos, action) {
  switch (action.type) {
    case 'added':
      return [...todos, { id: action.id, title: action.title, done: false }];
    case 'toggled':
      return todos.map((t) => (t.id === action.id ? { ...t, done: !t.done } : t));
    case 'edited':
      return todos.map((t) => (t.id === action.id ? { ...t, title: action.title } : t));
    case 'deleted':
      return todos.filter((t) => t.id !== action.id);
    case 'toggledAll': {
      const allDone = todos.every((t) => t.done);
      return todos.map((t) => ({ ...t, done: !allDone }));
    }
    case 'clearedCompleted':
      return todos.filter((t) => !t.done);
    default:
      throw new Error(`Unknown action: ${action.type}`);
  }
}

function loadTodos(storageKey) {
  try {
    return JSON.parse(localStorage.getItem(storageKey)) ?? [];
  } catch {
    return [];                                  // corrupt JSON or storage blocked
  }
}

function TodoItem({ todo, dispatch }) {
  const [editing, setEditing] = useState(false);
  const [draft, setDraft] = useState(todo.title);
  const inputRef = useRef(null);
  const cancelled = useRef(false);              // Escape must win over the blur that follows

  useEffect(() => {
    if (editing) { inputRef.current.focus(); inputRef.current.select(); }   // ready to type over
  }, [editing]);

  function startEdit() {
    cancelled.current = false;
    setDraft(todo.title);
    setEditing(true);
  }

  function finish() {
    if (cancelled.current) { cancelled.current = false; return; }
    const title = draft.trim();
    if (!title) dispatch({ type: 'deleted', id: todo.id });   // emptying a todo deletes it
    else dispatch({ type: 'edited', id: todo.id, title });
    setEditing(false);
  }

  if (editing) {
    return (
      <li className="editing">
        <input
          ref={inputRef}
          aria-label={`Edit ${todo.title}`}
          value={draft}
```

```
          onChange={(e) => setDraft(e.target.value)}
          onBlur={finish}
          onKeyDown={(e) => {
            if (e.key === 'Enter') finish();
            if (e.key === 'Escape') { cancelled.current = true; setEditing(false); }
          }}
        />
      </li>
    );
  }
  return (
    <li className={todo.done ? 'done' : undefined}>
      <input
        type="checkbox"
        aria-label={todo.title}
        checked={todo.done}
        onChange={() => dispatch({ type: 'toggled', id: todo.id })}
      />
      <span className="title" onDoubleClick={startEdit}>{todo.title}</span>
      <button className="icon" aria-label={`Edit ${todo.title}`} onClick={startEdit}>✎</button>
      <button className="icon" aria-label={`Delete ${todo.title}`} onClick={() => dispatch({ type: 'deleted', id:
todo.id })}>×</button>
    </li>
  );
}

export default function TodoApp({ storageKey = 'todos' }) {
  const [todos, dispatch] = useReducer(todosReducer, storageKey, loadTodos);   // lazy init: read storage once
  const [filter, setFilter] = useState('all');
  const [text, setText] = useState('');
  const nextId = useRef(Math.max(0, ...todos.map((t) => t.id)) + 1);

  useEffect(() => {
    localStorage.setItem(storageKey, JSON.stringify(todos));                  // sync to an external system
  }, [storageKey, todos]);

  const visible = todos.filter(FILTERS[filter]);          // derived — never stored
  const left = todos.filter((t) => !t.done).length;

  function handleSubmit(e) {
    e.preventDefault();
    const title = text.trim();
    if (!title) return;
    dispatch({ type: 'added', id: nextId.current++, title });
    setText('');
  }

  return (
    <section className="card todo" aria-labelledby="todo-h">
      <h2 id="todo-h">Todos</h2>
      <form onSubmit={handleSubmit} className="row">
        {todos.length > 0 && (
          <button type="button" className="icon" aria-label="Toggle all" onClick={() => dispatch({ type:
'toggledAll' })}>⌄</button>
        )}
        <input aria-label="New todo" placeholder="What needs to be done?" value={text} onChange={(e) =>
setText(e.target.value)} />
        <button>Add</button>
      </form>

      <div className="row" role="group" aria-label="Filter">
        {Object.keys(FILTERS).map((name) => (
          <button key={name} className="chip" aria-pressed={filter === name} onClick={() => setFilter(name)}>
            {name}
          </button>
        ))}
      </div>

      {visible.length === 0
        ? <p className="muted">{todos.length === 0 ? 'Nothing to do yet.' : `No ${filter} todos.`}</p>
        : <ul className="list">{visible.map((t) => <TodoItem key={t.id} todo={t} dispatch={dispatch} />)}</ul>}
```

```
      <footer className="row between">
        <span role="status">{left} {left === 1 ? 'item' : 'items'} left</span>
        {left < todos.length && <button onClick={() => dispatch({ type: 'clearedCompleted' })}>Clear
completed</button>}
      </footer>
    </section>
  );
}
```

What to point out while coding:

- **Reducer** actions name *what happened* (`added` , `toggled`); the reducer is pure, so it's unit-tested without rendering (first test below).
- **Lazy initialiser** (called only on the first render): `useReducer(reducer, storageKey, loadTodos)` reads `localStorage` once. The **Effect** writes back after each change — syncing with an external system, exactly what Effects are for (§7.4 makes it `useLocalStorage`).
- **Keys** are IDs, so filtering never moves the edit box to another item.
- **Escape vs blur** (focus-loss event): removing the focused edit input can fire `blur` , which would call `finish()` and save; the `cancelled` ref makes Escape win.

Test

`code/02_todo.test.jsx` (key parts):

```
test('edit: Enter saves, Escape cancels, empty text deletes', async () => {
  const user = userEvent.setup();
  render(<TodoApp />);
  await user.type(screen.getByRole('textbox', { name: 'New todo' }), 'Wash car{Enter}');
  await user.dblClick(screen.getByText('Wash car'));
  const edit = screen.getByRole('textbox', { name: 'Edit Wash car' });
  assert.equal(document.activeElement, edit);               // focused automatically
  await user.clear(edit);
  await user.type(edit, 'Wash bike{Enter}');
  assert.deepEqual(titles(), ['Wash bike']);
  // ... ' NOPE{Escape}' leaves it unchanged; clearing the text and pressing Enter deletes the todo
});
// persistence: add "Survive reload", tick it, unmount, render <TodoApp storageKey="t1" /> again -> still ticked
```

Output:

```
✔ reducer: toggledAll ticks everything, or unticks if all are done
✔ adds with Enter, ignores blank input, clears the box
✔ toggle + filters + clear completed
✔ edit: Enter saves, Escape cancels, empty text deletes
    stored JSON: [{"id":1,"title":"Survive reload","done":true}]
✔ persists to localStorage and restores on the next mount
ℹ tests 5
ℹ pass 5
ℹ fail 0
```

In the browser

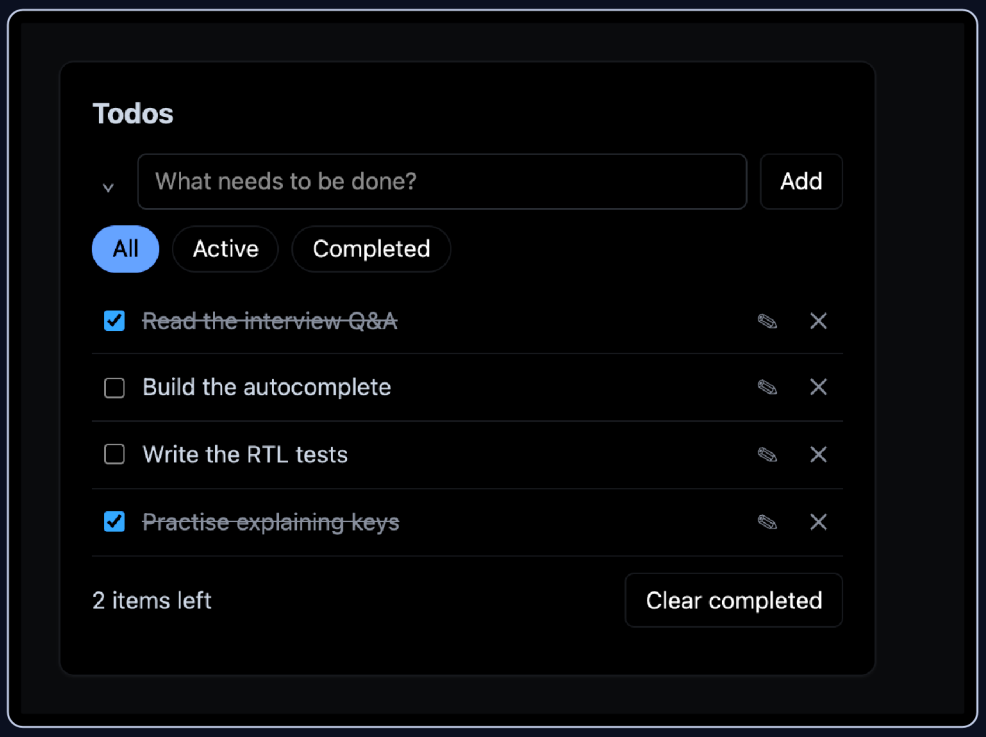


Figure 15.1 — `TodoApp` in Chrome, restored from `localStorage`. The strike-through, the count and the "Clear completed" button are all derived from the one `todos` array.

Follow-ups and edge cases

- **Persist to a server** → send each change to the API, showing it at once with `useOptimistic` and rolling back on failure (§4.7), or a TanStack Query mutation with invalidation (§9.10).
- **Shareable filter** → keep it in the URL (`?filter=active`) via the router's search params (Chapter 9).
- **Two tabs open** → listen for the `storage` event (fired in *other* tabs) and dispatch a `replaced` action, or subscribe with `useSyncExternalStore`.
- **10,000 todos** → virtualize; `memo(TodoItem)` with stable callbacks (or the React Compiler) stops every row re-rendering as you type.
- **Undo** → a `past` stack in the reducer state; `undo` pops it.
- **IDs** → `crypto.randomUUID()` in real apps; a counter ref is fine in an interview if you say why.

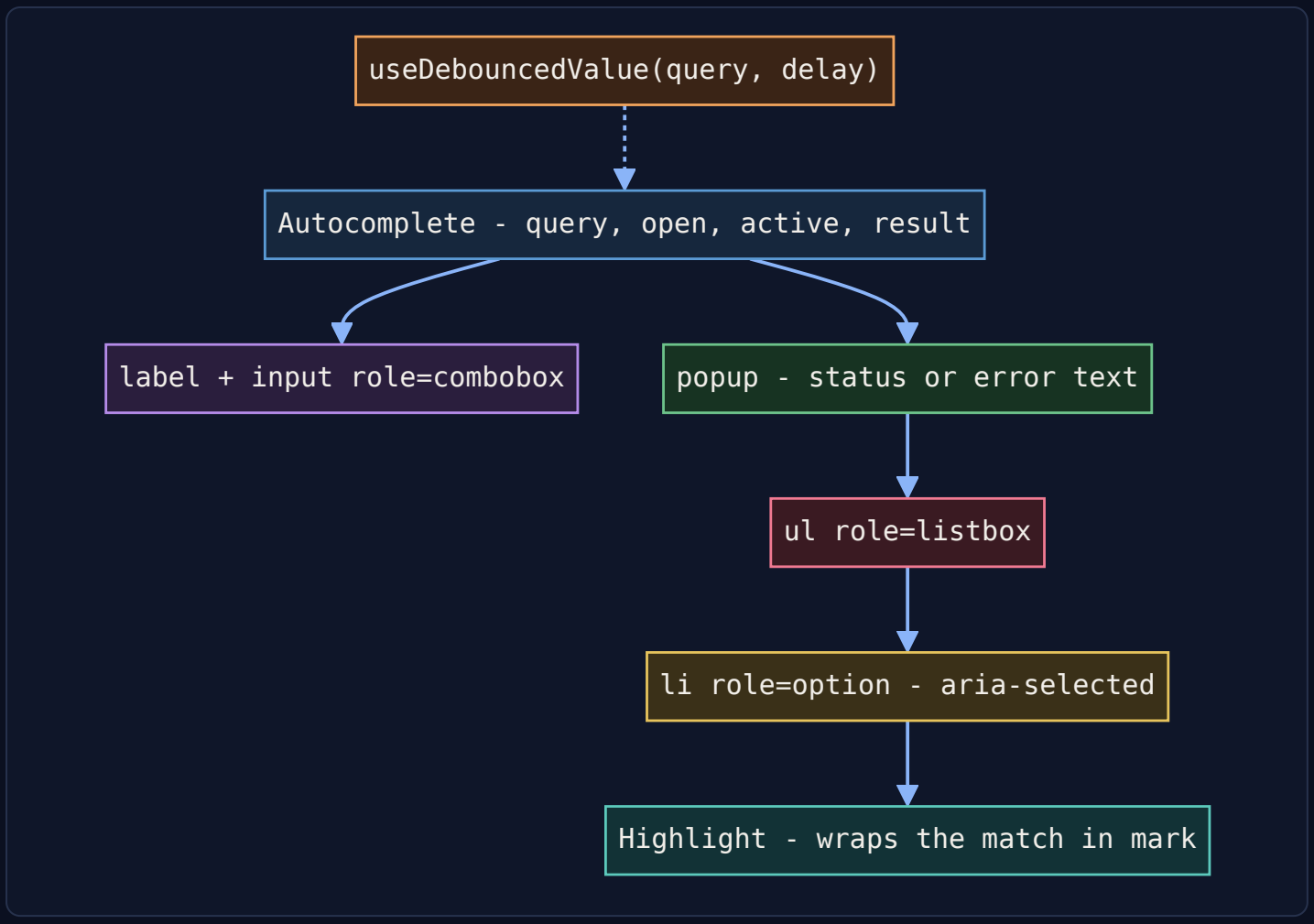
15.11 Build 2 — Autocomplete (debounce, abort, keyboard)

Requirements

Functional	Quiet requirements
Suggestions appear as the user types	Don't call the API on every keystroke — **debounce** (300 ms)
Click or Enter selects a suggestion	Ignore **stale responses** (a slow answer for "ap" must not replace "apr")
ArrowUp/ArrowDown move the highlight, wrapping at the ends; Escape closes	Screen readers announce the list and the highlighted option (ARIA **combobox** pattern — a text input with a popup list)
Loading, empty and error states	The matching part of each suggestion is highlighted

The data source is a prop, `search(query, { signal })` → promise of strings, so the component is API-independent and easy to test.

Component tree



State design

Data	Kind	Why
`query`	state	controlled input — must update on every keystroke (urgent)
`debounced`	state inside `useDebouncedValue`	the value that actually triggers a search, 300 ms after typing stops
`result` = `{ status, items, for }`	state	one object so status and items can't disagree; `for` remembers which query the items belong to
`open`	state	the user can close the list (Escape, blur, selection) without clearing the query
`active`	state	index of the highlighted option, `-1` for none
`showList`, `items` shown	derived	e.g. no list when the query is shorter than `minChars`
option IDs	`useId`	unique on the page, stable between server and client

Debounce — why a Hook, not a `debounce()` wrapper? A debounced *function* (JavaScript Chapter 17 — Debounce, Throttle) must survive re-renders (`useRef` / `useMemo`) and be cancelled on unmount. Debouncing the *value* in an Effect is shorter: each keystroke's cleanup clears the previous timeout (§7.5).

Code

`code/autocomplete.jsx` :

```jsx
import { useEffect, useId, useState } from 'react';

export function useDebouncedValue(value, delay) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);                  // a new keystroke cancels the pending update
  }, [value, delay]);
  return debounced;
}

function Highlight({ text, query }) {
  const i = text.toLowerCase().indexOf(query.toLowerCase());
  if (i === -1 || !query) return text;
  return <>{text.slice(0, i)}<mark>{text.slice(i, i + query.length)}</mark>{text.slice(i + query.length)}</>;
}

export default function Autocomplete({ search, onSelect, label = 'Search', delay = 300, minChars = 1 }) {
  const [query, setQuery] = useState('');
  const [open, setOpen] = useState(false);
  const [active, setActive] = useState(-1);                 // highlighted option index, -1 = none
  const [result, setResult] = useState({ status: 'idle', items: [], for: '' });
  const debounced = useDebouncedValue(query.trim(), delay);
  const id = useId();
  const listId = `${id}-list`;

  useEffect(() => {
    if (debounced.length < minChars) return;
    const controller = new AbortController();
    setResult((r) => ({ ...r, status: 'loading' }));
    search(debounced, { signal: controller.signal })
      .then((items) => { setResult({ status: 'success', items, for: debounced }); setActive(-1); })
      .catch((err) => { if (err.name !== 'AbortError') setResult({ status: 'error', items: [], for: debounced });
});
    return () => controller.abort();                        // newer query (or unmount) cancels this one
  }, [debounced, minChars, search]);

  const tooShort = query.trim().length < minChars;
  const items = tooShort ? [] : result.items;
  const showList = open && !tooShort;

  function choose(item) {
    setQuery(item);
    setOpen(false);
    setActive(-1);
    onSelect?.(item);
  }

  function handleKeyDown(e) {
    if (e.key === 'ArrowDown' || e.key === 'ArrowUp') {
      e.preventDefault();                                   // keep the caret where it is
      setOpen(true);
      if (items.length === 0) return;
      const step = e.key === 'ArrowDown' ? 1 : -1;
      setActive((i) => (i + step + items.length) % items.length);   // wraps around both ends
    } else if (e.key === 'Enter' && showList && active >= 0) {
      e.preventDefault();                                   // don't submit a surrounding form
      choose(items[active]);
    } else if (e.key === 'Escape') {
      setOpen(false);
      setActive(-1);
    }
  }

  return (
    <div className="card ac">
      <label htmlFor={`${id}-input`}>{label}</label>
      <input
        id={`${id}-input`}
        role="combobox"
```

```
        aria-expanded={showList}
        aria-controls={listId}
        aria-autocomplete="list"
        aria-activedescendant={showList && active >= 0 ? `${id}-opt-${active}` : undefined}
        autoComplete="off"
        value={query}
        onChange={(e) => { setQuery(e.target.value); setOpen(true); setActive(-1); }}
        onKeyDown={handleKeyDown}
        onBlur={() => setOpen(false)}
      />
      {showList && (
        <div className="popup">
          {result.status === 'loading' && <p className="muted" role="status">Searching…</p>}
          {result.status === 'error' && <p className="error" role="alert">Something went wrong. Keep typing to
retry.</p>}
          {result.status === 'success' && items.length === 0 && <p className="muted" role="status">No matches for
“{result.for}”</p>}
          <ul id={listId} role="listbox" aria-label={`${label} suggestions`}>
            {items.map((item, i) => (
              <li
                key={item}
                id={`${id}-opt-${i}`}
                role="option"
                aria-selected={i === active}
                onMouseDown={(e) => e.preventDefault()}     // keep focus in the input so blur doesn't close first
                onMouseEnter={() => setActive(i)}
                onClick={() => choose(item)}
              >
                <Highlight text={item} query={result.for} />
              </li>
            ))}
          </ul>
        </div>
      )}
    </div>
  );
}
```

How the tricky parts work:

1. **Debounce → search**: `query` changes every keystroke; `debounced` only after `delay` ms of quiet, and only then does the search Effect run.
2. **Abort**: each search Effect owns an `AbortController`. When `debounced` changes, React runs the old Effect's **cleanup** first, aborting the old request (its `AbortError` is ignored), so an old response can never overwrite a newer one (§5.7). Note the ordering: the abort happens when the *debounced* value changes, not on the keystroke — a response for "ap" that lands during the pause after typing "r" is still shown (labelled by `result.for`) until the "apr" results replace it.
3. **Focus stays in the input.** Options are never focused; `aria-activedescendant` (on the input, pointing at the highlighted option's id) names the active one (the WAI-ARIA combobox pattern), so the user keeps typing.
4. **Mouse vs blur.** Clicking an option would blur the input and close the list *before* the click lands; `onMouseDown={e => e.preventDefault()}` prevents the blur.
5. **Enter** selects only when an option is highlighted, with `preventDefault()` so a surrounding `<form>` doesn't submit.

Test

`code/03_autocomplete.test.jsx` uses a fake `search` whose requests the test resolves **by hand**, so it can make answers arrive out of order:

```
function fakeSearch() {
  const calls = [];
  const search = (q, { signal }) => new Promise((resolve, reject) => {
    const call = { q, signal, resolve: () => resolve(FRUITS.filter((f) =>
f.toLowerCase().includes(q.toLowerCase()))) };
    signal.addEventListener('abort', () => reject(new DOMException('aborted', 'AbortError')));
    calls.push(call);
  });
  return { search, calls };
}

test('abort: a newer query cancels the older request, so stale results never show', async () => {
  const user = userEvent.setup();
  const { search, calls } = fakeSearch();
  render(<Autocomplete search={search} delay={50} label="Fruit" />);
  const box = screen.getByRole('combobox');
  await user.type(box, 'ap');
  await waitFor(() => assert.equal(calls.length, 1));      // request #1 ("ap") is in flight
  await user.type(box, 'r');
  await waitFor(() => assert.equal(calls.length, 2));      // request #2 ("apr") started
  assert.equal(calls[0].signal.aborted, true);             // ...and #1 was aborted by the effect cleanup
  calls[1].resolve();                                      // the answers arrive OUT of order:
  await waitFor(() => assert.deepEqual(options(), ['Apricot']));
  calls[0].resolve();                                      // the late answer for "ap" must be ignored
  await new Promise((r) => setTimeout(r, 20));
  assert.deepEqual(options(), ['Apricot']);
});

// keyboard test: ArrowDown x2 -> aria-activedescendant = second option's id (focus stays in the input);
// ArrowUp x2 wraps to the last; ArrowDown + Enter wraps to the first and selects "Apple"; Escape sets aria-
expanded="false"
```

Output:

```
✔ debounce: typing quickly sends ONE request, for the final text
✔ abort: a newer query cancels the older request, so stale results never show
✔ keyboard: arrows move (and wrap), Enter selects, Escape closes
✔ mouse: clicking an option selects it (mousedown does not blur the input first)
✔ empty and error states
ℹ tests 5
ℹ pass 5
ℹ fail 0
```

The tests pass short real delays (10–100 ms) as a prop, so no fake timers are needed.

In the browser

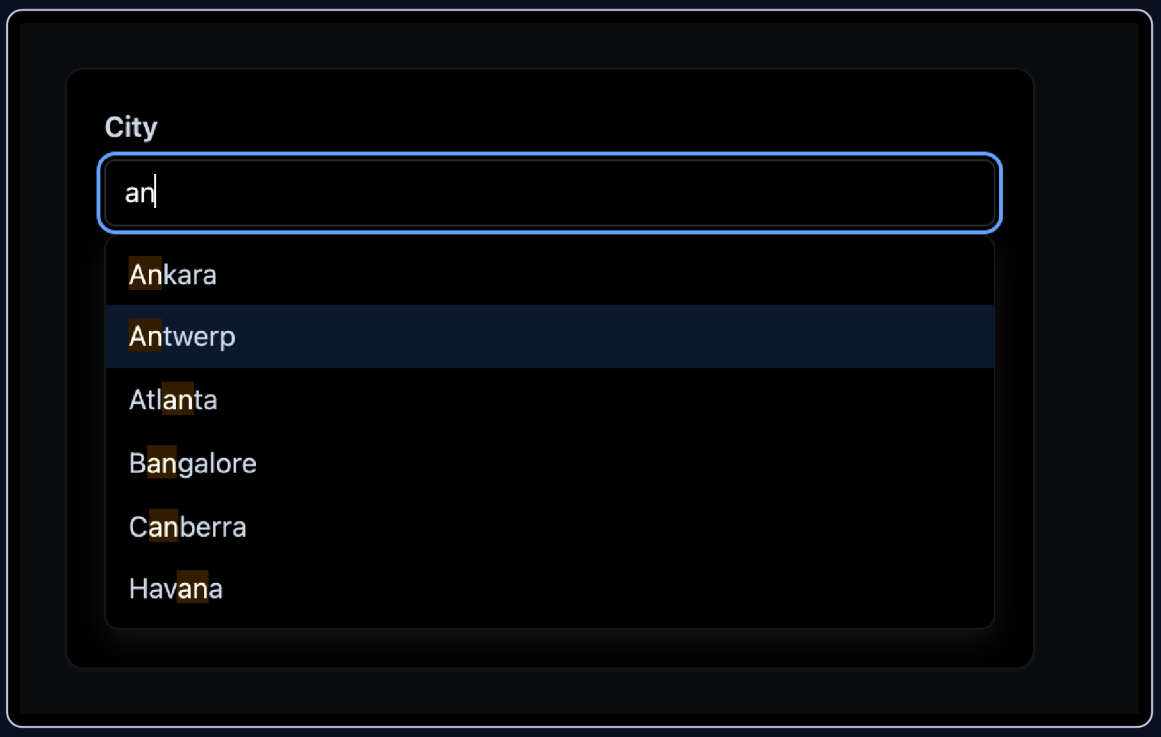


Figure 15.2 — After typing "an" and pressing ArrowDown twice. The matching text is wrapped in `<mark>`; the blue row is the option named by `aria-activedescendant`, while keyboard focus stays in the input.

Follow-ups and edge cases

- **Cache results** → a `Map` of query → results in a ref (or TanStack Query keyed by the query), so Backspace shows earlier results instantly.
- **Why not throttle?** → throttle fires at most once per interval *while* typing (scroll/resize); debounce waits for a pause (search). Both are in JavaScript §17.3.
- **Huge result lists** → cap results (top 10) or render them from `useDeferredValue(query)` (§7.11).
- **Active option out of view** → in an Effect on `active`, `scrollIntoView({ block: 'nearest' })` the option.
- **Selecting searches the chosen text** → harmless here (the list is closed), or skip when `query` equals the last selection.
- **Highlighting user input** → never `dangerouslySetInnerHTML`; splitting the string and rendering `<mark>` in JSX, as `Highlight` does, is XSS-safe (XSS — cross-site scripting — runs one user's text as code in another's browser; JSX escapes text).
- **Rate limits** (caps on requests per time window; see System Design Fundamentals) → debounce plus `minChars`, caching and server-side limits.

15.12 Build 3 — Infinite scroll

Requirements

Functional	Quiet requirements
Show the first page, load the next when the user nears the bottom	**Never** send two requests for the same page (fast scrolling, observer firing twice)
Show "Loading more…"; stop at the end with a message	If one page doesn't fill the box, keep loading until it does
On failure, show an error with Retry	Don't update state after unmount; work inside a scrollable box, not only the window

How it works

An `IntersectionObserver` tells you when an element (here an invisible 1-pixel **sentinel** after the last item) enters a **root** (the scroll box) — no `scroll` listeners or layout reads (which force recalculation) on every scroll event. `rootMargin: '200px'` grows the root so loading starts 200 px *before* the user reaches the end.



State design

Data	Kind	Why
`items`	state	the growing list
`page`	state	last page loaded
`status`	state `'idle' | 'loading' | 'error'`	one variable, so "loading and error at once" is impossible
`hasMore`	state	stop observing at the end
`inFlight`	ref	a **synchronous** guard: `status` only changes on the next render, a ref immediately
`mounted`	ref	ignore a response that arrives after unmount
scroller, sentinel	refs to DOM nodes	the observer needs real elements

Code

`code/infinite-list.jsx` :

```jsx
import { useEffect, useRef, useState } from 'react';

export default function InfiniteList({ fetchPage, renderItem = (item) => item.name, rootMargin = '200px' }) {
  const [items, setItems] = useState([]);
  const [page, setPage] = useState(0);                      // last page loaded (0 = none yet)
  const [status, setStatus] = useState('idle');             // 'idle' | 'loading' | 'error'
  const [hasMore, setHasMore] = useState(true);
  const scrollerRef = useRef(null);
  const sentinelRef = useRef(null);
  const mounted = useRef(false);
  const inFlight = useRef(false);                           // sync guard: state updates aren't instant

  useEffect(() => {
    mounted.current = true;
    return () => { mounted.current = false; };              // late responses after unmount are dropped
  }, []);

  // Re-created whenever status/page/hasMore change. A new observer reports the sentinel's current
  // state immediately, so if page N didn't fill the box, page N+1 loads without any scrolling.
  useEffect(() => {
    if (status !== 'idle' || !hasMore) return;              // never two requests at once, stop at the end
    const observer = new IntersectionObserver(
      ([entry]) => {
        if (!entry.isIntersecting || inFlight.current) return;
        inFlight.current = true;
        observer.disconnect();
        setStatus('loading');
        fetchPage(page + 1).then(
          (res) => {
            inFlight.current = false;
            if (!mounted.current) return;
            setItems((prev) => [...prev, ...res.items]);
            setPage(page + 1);
            setHasMore(res.hasMore);
            setStatus('idle');
          },
          () => { inFlight.current = false; if (mounted.current) setStatus('error'); },
        );
      },
      { root: scrollerRef.current, rootMargin },            // start loading 200px before the end
    );
    observer.observe(sentinelRef.current);
    return () => observer.disconnect();
  }, [status, page, hasMore, fetchPage, rootMargin]);

  return (
    <section className="card feed" aria-labelledby="feed-h">
      <h2 id="feed-h">Feed <small className="muted">({items.length} loaded)</small></h2>
      <div className="scroller" ref={scrollerRef} tabIndex={0} aria-label="Feed items">
        <ul className="list" aria-busy={status === 'loading'}>
          {items.map((item) => <li key={item.id}>{renderItem(item)}</li>)}
        </ul>
        <div ref={sentinelRef} aria-hidden="true" className="sentinel" />
        {status === 'loading' && <p role="status" className="muted">Loading more…</p>}
        {status === 'error' && (
          <p role="alert" className="error">
            Couldn’t load more. <button onClick={() => setStatus('idle')}>Retry</button>
          </p>
        )}
        {!hasMore && <p className="muted">You’ve reached the end.</p>}
      </div>
    </section>
  );
}
```

Design notes:

- **The Effect is the loop.** It observes only while `status === 'idle'` and `hasMore`. After each page the dependencies change, cleanup disconnects the old observer and a new one immediately reports whether the sentinel is visible — so short pages chain until the box is full.

- **Why `inFlight` too?** An observer callback can run several times before React re-renders with `status: 'loading'`; state is a snapshot (Q8), a ref isn't.
- **Retry** sets `status` back to `'idle'`, so the Effect observes again.
- **`aria-busy`** and a `role="status"` message tell screen-reader users more is coming.

Test

jsdom does no layout, so it has no `IntersectionObserver`. The test installs a tiny fake and an `intersect()` helper that plays the part of the user scrolling:

```
const observers = new Set();
class FakeIntersectionObserver {
  constructor(callback, options) { this.callback = callback; this.options = options; this.targets = []; }
  observe(el) { this.targets.push(el); observers.add(this); }
  unobserve(el) { this.targets = this.targets.filter((t) => t !== el); }
  disconnect() { this.targets = []; observers.delete(this); }
}
globalThis.IntersectionObserver = window.IntersectionObserver = FakeIntersectionObserver;

async function intersect() {                  // "scroll" until the sentinel is visible
  await act(async () => {
    for (const o of [...observers]) o.callback(o.targets.map((target) => ({ target, isIntersecting: true })), o);
    await new Promise((r) => setTimeout(r, 0));
  });
}

test('no duplicate requests while one is in flight', async () => {
  // fetchPage never resolves until release() is called
  render(<InfiniteList fetchPage={fetchPage} />);
  const [o] = observers;
  await act(async () => { for (let i = 0; i < 3; i++) o.callback([{ isIntersecting: true }], o); });  // fires 3x quickly
  assert.equal(screen.getByRole('status').textContent, 'Loading more…');
  assert.deepEqual(calls, [1]);
});
```

Output:

```
✔ loads the next page each time the sentinel becomes visible, then stops
✔ no duplicate requests while one is in flight
✔ error shows Retry, and Retry resumes loading
ℹ tests 3
ℹ pass 3
ℹ fail 0
```

Without the `inFlight` ref this test fails with `calls` = `[1, 1, 1]` — three requests for page 1, the classic infinite-scroll bug.

In the browser

In Chrome the real `IntersectionObserver` drives it. The demo page scrolls the box to the bottom twice:

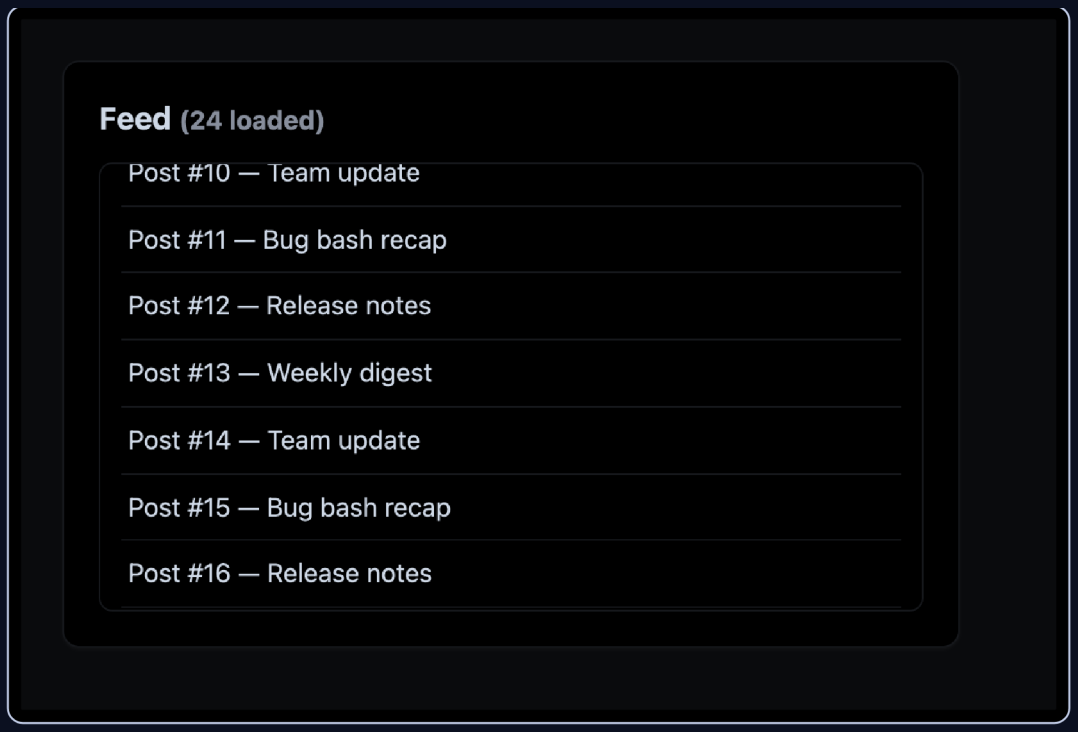


Figure 15.3 — The real `IntersectionObserver` in Chrome: two scrolls to the bottom loaded pages 2 and 3 (24 posts). The third page arrived while the box still showed posts 10–16, so the user never waits at the bottom.

Follow-ups and edge cases

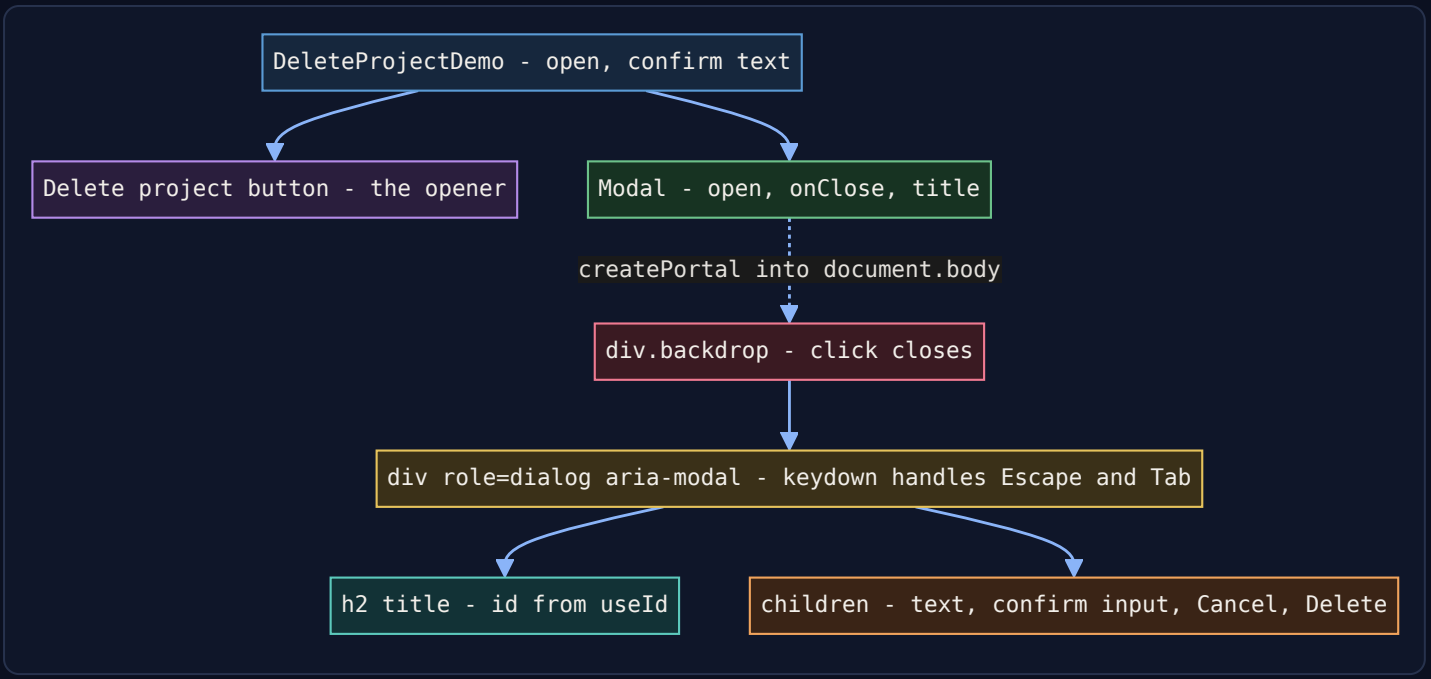
- **Library** → TanStack Query's `useInfiniteQuery` handles pages, caching and `hasNextPage`; an observer still calls `fetchNextPage()` (§9.9).
- **Thousands of items** → add virtualization (§10.9).
- **Cursor vs page numbers** → inserts while scrolling shift page numbers (duplicates, gaps); use a cursor ("after the last item I saw", e.g. `?after=<lastId>`) and de-duplicate by `id`.
- **Back-navigation** → cache the data (TanStack Query) and restore `scrollTop`, or open details in a modal that keeps the list mounted.
- **Accessibility** → the footer becomes unreachable; offer a "Load more" button too (it can be the sentinel) and announce new items via `role="status"`.
- **Abort on unmount** → pass an `AbortSignal` to `fetchPage` and abort in a mount-only Effect's cleanup.

15.13 Build 4 — Modal dialog (portal, focus trap, Escape)

Requirements

Functional	Quiet requirements (the real test)
Open from a button, show a title, content and actions	Rendered at the top level so no parent's `overflow` or `z-index` clips it
Close with Cancel, Escape, or a click on the backdrop	Focus moves **into** the dialog on open and **back** to the opener on close
A destructive confirm: "type the name to enable Delete"	**Tab** and **Shift+Tab** cycle inside the dialog; the page behind doesn't scroll
	Screen readers announce a dialog with its title: `role="dialog"`, `aria-modal`, `aria-labelledby`

Component tree



State design

The `Modal` is **controlled**: the parent owns `open` and passes `onClose`. That lets the parent decide what closing means (reset the confirm text, save a draft) and makes the modal reusable. The modal itself holds no state — only refs (the dialog node, and the element that had focus before opening, captured inside the Effect).

Code

`code/modal.jsx` :

```jsx
// Machine coding 4 — modal dialog: portal, focus trap, Escape, backdrop click, focus restore, scroll lock.
import { useEffect, useId, useRef, useState } from 'react';
import { createPortal } from 'react-dom';

const FOCUSABLE = 'a[href], button:not([disabled]), input:not([disabled]), select:not([disabled]),
textarea:not([disabled]), [tabindex]:not([tabindex="-1"])';

export function Modal({ open, onClose, title, children }) {
  const dialogRef = useRef(null);
  const titleId = useId();

  useEffect(() => {
    if (!open) return;
    const previouslyFocused = document.activeElement;          // usually the button that opened us
    const first = dialogRef.current.querySelector(FOCUSABLE);
    (first ?? dialogRef.current).focus();                       // move focus INTO the dialog
    const { overflow } = document.body.style;
    document.body.style.overflow = 'hidden';                    // stop the page behind from scrolling
    return () => {
      document.body.style.overflow = overflow;
      previouslyFocused?.focus?.();                             // give focus back on close
    };
  }, [open]);

  if (!open) return null;

  function handleKeyDown(e) {
    if (e.key === 'Escape') {
      e.stopPropagation();                                      // a parent modal shouldn't close too
      onClose();
      return;
    }
    if (e.key !== 'Tab') return;
    const nodes = [...dialogRef.current.querySelectorAll(FOCUSABLE)];
    if (nodes.length === 0) { e.preventDefault(); return; }
    const first = nodes[0];
    const last = nodes[nodes.length - 1];
    const outside = !nodes.includes(document.activeElement);    // e.g. the dialog box itself, after a click on
its text
    if (e.shiftKey && (outside || document.activeElement === first)) { e.preventDefault(); last.focus(); }
    else if (!e.shiftKey && (outside || document.activeElement === last)) { e.preventDefault(); first.focus(); }
  }

  return createPortal(
    <div className="backdrop" onMouseDown={(e) => { if (e.target === e.currentTarget) onClose(); }}>
      <div
        ref={dialogRef}
        role="dialog"
        aria-modal="true"
        aria-labelledby={titleId}
        tabIndex={-1}
        className="dialog"
        onKeyDown={handleKeyDown}
      >
        <h2 id={titleId}>{title}</h2>
        {children}
      </div>
    </div>,
    document.body,                                              // escape parents' overflow/z-index
  );
}
```

`DeleteProjectDemo` in the same file (the page used by the tests and the screenshot) owns `open` and the confirm text, and passes `onClose={close}` to `Modal`; Delete is `disabled` until the input says "Apollo".

Walk-through:

1. **Portal**: `createPortal(…, document.body)` puts the DOM at the end of `<body>`, but React-wise the modal is still a child of `DeleteProjectDemo`, so state, context and event bubbling work normally (Q49).

2. **Focus in, focus back**: the Effect runs when `open` becomes `true`: it remembers `document.activeElement` (the opener), focuses the first focusable element, and its **cleanup** — which runs when `open` becomes `false` or the modal unmounts — restores focus and the body's `overflow`.
3. **Focus trap**: Tab on the last focusable element wraps to the first; Shift+Tab on the first wraps to the last. If focus is on none of them — clicking the dialog's text focuses the `tabIndex={-1}` box itself — Shift+Tab would otherwise leave the dialog, so it also wraps. The list is re-queried on every key press, so it stays right when Delete becomes enabled.
4. **Escape** closes; `stopPropagation()` stops an outer modal (if any) from closing too.
5. **Backdrop**: close only when the mousedown target *is* the backdrop, not a child — otherwise selecting text inside the dialog and releasing outside would close it.

Test

`code/05_modal.test.jsx` (key parts):

```
test('renders into document.body through a portal, labelled by its title', async () => {
  const { container, dialog } = await openModal();
  assert.equal(container.contains(dialog), false);        // not inside the component's own DOM
  assert.equal(document.body.contains(dialog), true);
  assert.equal(dialog.getAttribute('aria-modal'), 'true');
  assert.equal(document.body.style.overflow, 'hidden');    // background scroll locked
});

test('focus moves in, and Tab / Shift+Tab stay trapped inside', async () => {
  const { user } = await openModal();
  assert.equal(document.activeElement, screen.getByRole('textbox'));   // first focusable element
  await user.tab();
  assert.equal(focused(), 'Cancel');
  await user.tab();                                        // Delete is disabled, so Cancel is last:
  assert.equal(document.activeElement, screen.getByRole('textbox'));   // wraps to the first
  await user.tab({ shift: true });                         // Shift+Tab on the first wraps to the last
  assert.equal(focused(), 'Cancel');
});
// Escape test: dialog gone, focus back on the trigger, body overflow restored to ''
```

Output:

```
✔ renders into document.body through a portal, labelled by its title
✔ focus moves in, and Tab / Shift+Tab stay trapped inside
✔ Escape closes and focus returns to the button that opened it
✔ clicking the backdrop closes; clicking inside does not
✔ the confirm flow: Delete stays disabled until the name is typed
ℹ tests 5
ℹ pass 5
ℹ fail 0
```

`user.tab()` respects `preventDefault()` on the `keydown`, like a browser, so the trap is genuinely tested.

In the browser

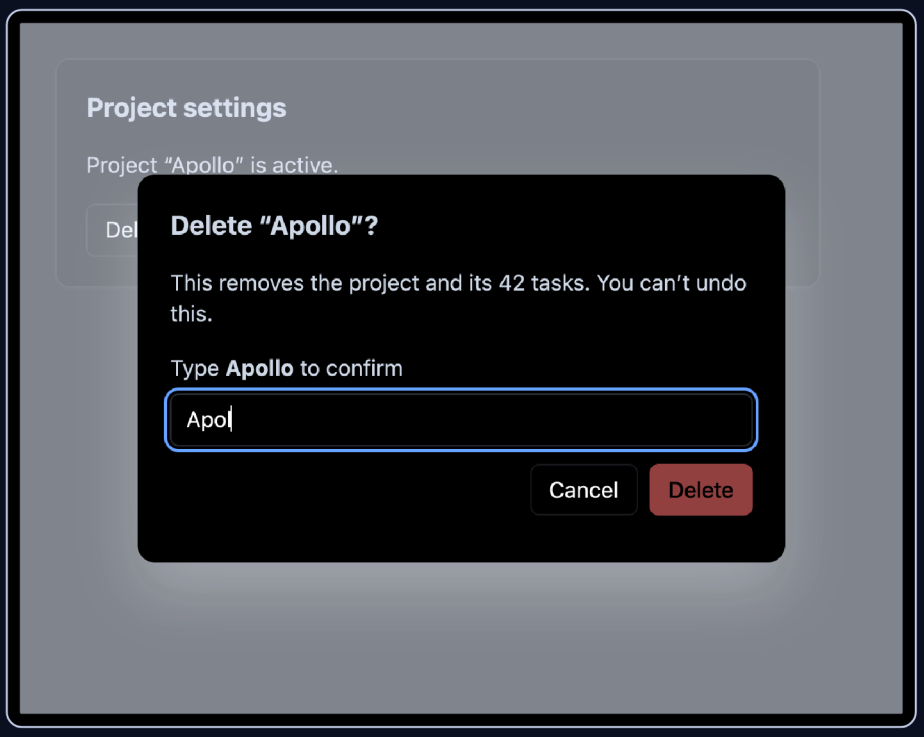


Figure 15.4 — The modal in Chrome, rendered into `<body>` over a dimmed page. "Delete" stays disabled (and outside the focus cycle) until the full name is typed.

Follow-ups and edge cases

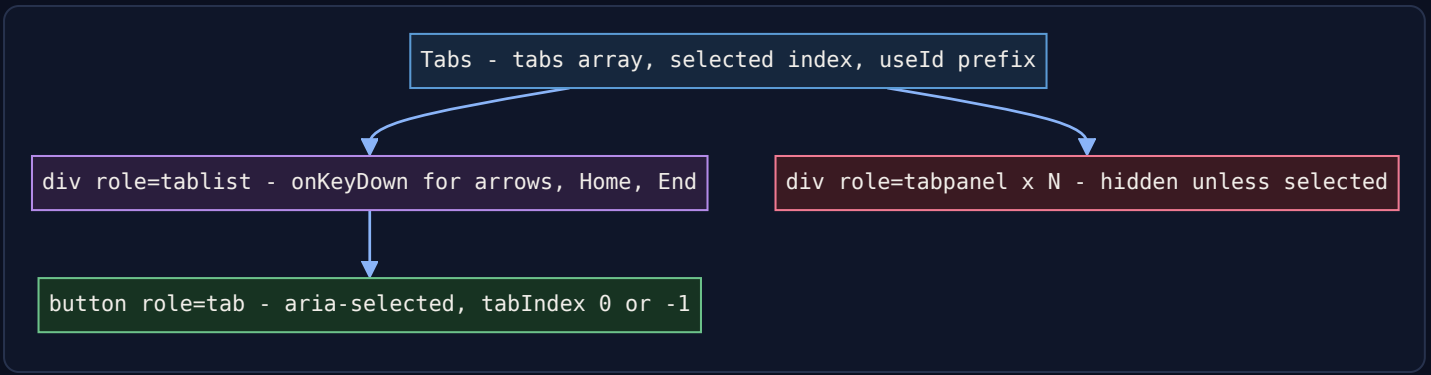
- **Why not native `<dialog>`?** → In real apps, prefer it (§14.8): **top layer**, **inert** background, Escape (a `cancel` event) and focus return for free. Call `showModal()` / `close()` in an Effect when `open` changes and listen for `close` to sync React state. Interviewers still ask for the hand-built version to see you know what the browser does.
- **Screen readers can reach the page behind** a hand-built modal; `aria-modal="true"` helps, but `inert` on the app root is the robust fix (`<dialog>` does it for you).
- **Nested modals** → keep a stack; only the top one handles Escape (the `stopPropagation` above) and traps focus.
- **Exit animations** → keep it mounted in a `closing` state until `transitionend` (CSS transition finished).
- **SSR** → no `document` on the server; create the portal only after mount.
- **Focusable detection** → the selector misses `contenteditable`, `<details>` and similar; `focus-trap` or Radix/React Aria handle the long tail.

15.14 Build 5 — Accessible tabs

Requirements

Functional	Quiet requirements (WAI-ARIA tabs pattern)
A row of tabs; clicking one shows its panel	`role="tablist"`, `role="tab"` with `aria-selected` and `aria-controls`, `role="tabpanel"` with `aria-labelledby`
Optional initial tab and `onChange` callback	**One** Tab-key stop for the whole tab row (roving `tabIndex`), then Tab moves into the panel
Disabled tabs are skipped	ArrowLeft/ArrowRight move between tabs and wrap; Home/End jump to the ends
Switching tabs keeps what the user typed in other panels	Selection follows focus ("automatic activation" — moving to a tab also selects it)

Component tree



State design

Only one piece of state: `selected` (an index). The IDs that connect each tab to its panel come from `useId`, so two `Tabs` on one page never clash. `tabRefs` (DOM nodes collected by **ref callbacks** — functions passed as `ref`, called with the node) lets the keyboard handler move focus. The `tabs` array is a prop, so what is shown is derived from it.

Roving tabindex (defined in §14.4): in a composite widget (tabs, toolbar, radio group, menu) only the current item has `tabIndex={0}`, others `-1` — one Tab stop, arrows move within.

Code

`code/tabs.jsx`:

```jsx
// Machine coding 5 — accessible tabs (WAI-ARIA tabs pattern, automatic activation, roving tabindex).
import { useId, useRef, useState } from 'react';

export default function Tabs({ tabs, defaultIndex = 0, onChange, label }) {
  const [selected, setSelected] = useState(defaultIndex);
  const tabRefs = useRef([]);                                  // one DOM node per tab, for focus()
  const id = useId();

  function select(i) {
    setSelected(i);
    tabRefs.current[i].focus();
    onChange?.(i);
  }

  // Find the next enabled tab in `step` direction, wrapping around the ends.
  function nextEnabled(from, step) {
    for (let k = 1; k <= tabs.length; k++) {
      const i = (from + step * k + tabs.length) % tabs.length;
      if (!tabs[i].disabled) return i;
    }
    return from;
  }

  function handleKeyDown(e) {
    const moves = {
      ArrowRight: () => nextEnabled(selected, 1),
      ArrowLeft: () => nextEnabled(selected, -1),
      Home: () => nextEnabled(-1, 1),                          // first enabled
      End: () => nextEnabled(tabs.length, -1),                 // last enabled
    };
    if (!moves[e.key]) return;
    e.preventDefault();                                        // Home/End would scroll the page
    select(moves[e.key]());
  }

  return (
    <div className="card tabs">
      <div role="tablist" aria-label={label} onKeyDown={handleKeyDown}>
        {tabs.map((tab, i) => (
          <button
            key={tab.label}
            ref={(el) => { tabRefs.current[i] = el; }}
            role="tab"
            id={`${id}-tab-${i}`}
            aria-selected={i === selected}
            aria-controls={`${id}-panel-${i}`}
            tabIndex={i === selected ? 0 : -1}                 // roving tabindex: only one tab stop
            disabled={tab.disabled}
            onClick={() => select(i)}
          >
            {tab.label}
          </button>
        ))}
      </div>
      {tabs.map((tab, i) => (
        <div
          key={tab.label}
          role="tabpanel"
          id={`${id}-panel-${i}`}
          aria-labelledby={`${id}-tab-${i}`}
          tabIndex={0}
          hidden={i !== selected}                              // keep panels mounted: their state survives
        >
          {tab.content}
        </div>
      ))}
    </div>
  );
}
```

Notes:

- `select(i)` sets state **and** focuses the tab, so focus and selection stay together.
- `nextEnabled` walks one direction with wrap-around, skipping disabled tabs; `Home` / `End` reuse it by starting just outside the ends; `preventDefault()` stops them scrolling the page.
- Panels use `hidden` instead of conditional rendering, so **inactive panels stay mounted** and keep their state (the last test proves it). For heavy panels render only the active one and accept losing its state.

Test

`code/06_tabs.test.jsx` (key parts):

```
test('arrow keys move selection AND focus, wrap, and skip disabled tabs', async () => {
  const user = userEvent.setup();
  const changes = [];
  render(<Tabs tabs={TABS} label="Account" onChange={(i) => changes.push(i)} />);
  await user.click(screen.getByRole('tab', { name: 'Overview' }));
  await user.keyboard('{ArrowRight}');
  assert.equal(selectedTab(), 'Billing');
  assert.equal(document.activeElement.textContent, 'Billing');
  await user.keyboard('{ArrowRight}');                    // Audit log is disabled -> Members
  assert.equal(selectedTab(), 'Members');
  await user.keyboard('{ArrowRight}');                    // wraps to the start
  assert.equal(selectedTab(), 'Overview');
  // ... ArrowLeft wraps to the end, Home, End
});

// Tab-key test: Tab from a "before" button lands on the SELECTED tab (Billing), the next Tab goes into the
tabpanel
```

Output:

```
✔ ARIA wiring: tablist > tab[aria-selected], tabpanel labelled by its tab
✔ arrow keys move selection AND focus, wrap, and skip disabled tabs
✔ Tab key: one stop for the whole tablist, then into the panel
✔ panels stay mounted, so typed text survives switching tabs
ℹ tests 4
ℹ pass 4
ℹ fail 0
```

Note `getByRole('tabpanel')` finds exactly one panel: elements with `hidden` are removed from the accessibility tree (the page as exposed to screen readers), and Testing Library's role queries ignore them by default — just like a screen reader.

In the browser

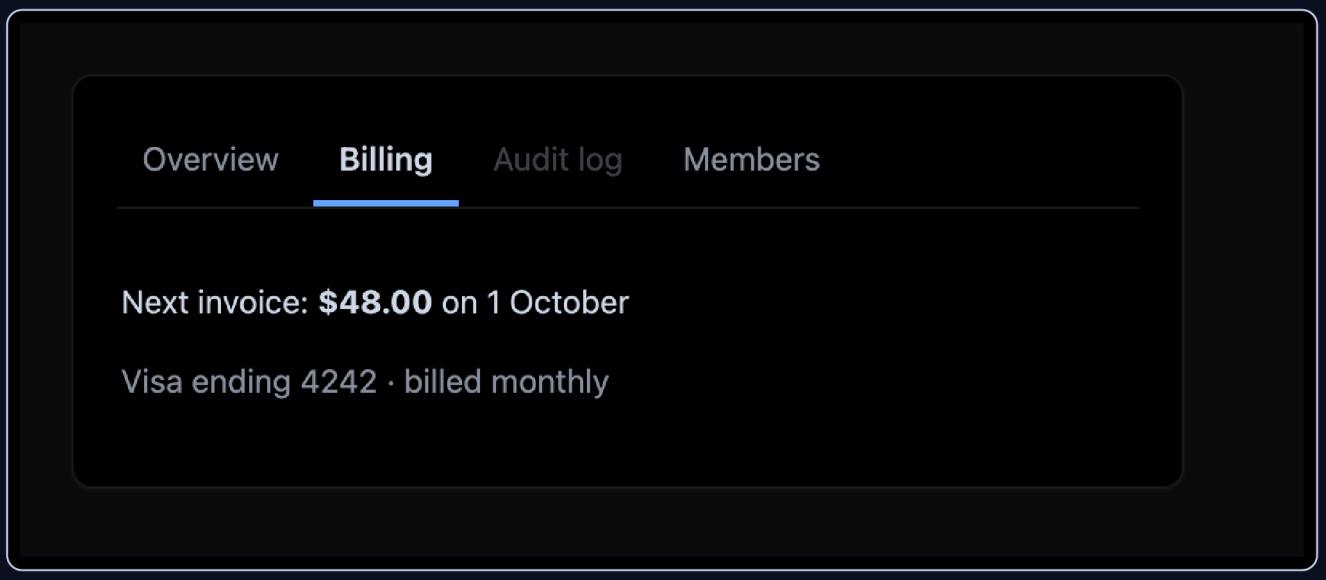


Figure 15.5 — `Tabs` with the second tab selected and a disabled third tab that arrow keys skip.

Follow-ups and edge cases

- **Manual activation** (slow panels) → arrows only *move focus*, Enter/Space selects; keep a separate `focused` index.
- **Controlled version** (e.g. synced with `?tab=billing`) → use the `value` prop when given, else internal state.
- **Compound API** (`<Tabs><TabList><Tab/></TabList><TabPanel/></Tabs>`) → share `selected` and the ID prefix through context (§14.4).
- **Disabled tabs** → ARIA guidance allows keeping them *focusable* with `aria-disabled="true"` so screen-reader users discover them; native `disabled`, used here, removes them from focus. Say which you chose.
- **Heavy panels that keep state** → React 19.2's `<Activity mode="hidden">` hides content and unmounts its Effects while keeping state, and can pre-render at low priority.
- **Vertical tabs** → `aria-orientation="vertical"` and ArrowUp/ArrowDown.

15.15 Build 6 — Star rating

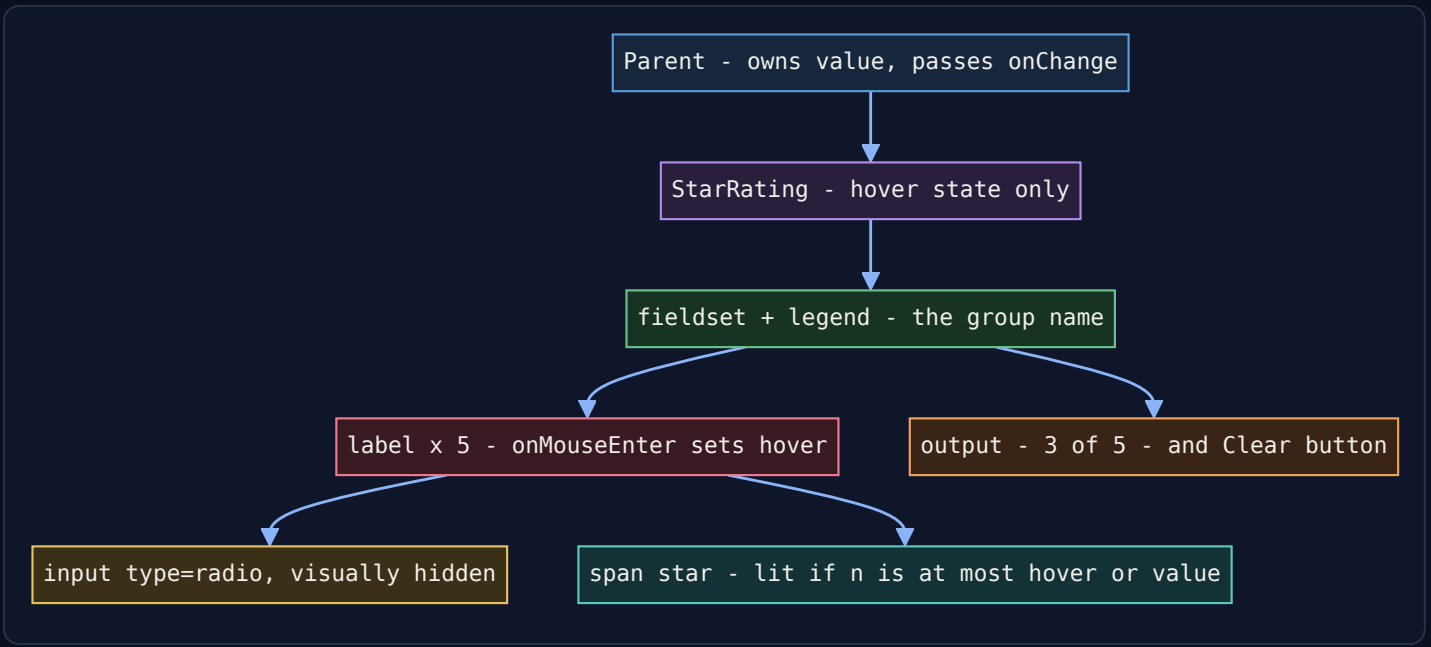
Requirements

Functional	Quiet requirements
5 stars; click to rate; show "3 of 5"	Works with the keyboard: Tab to the group, arrows to change
Hover previews a rating without committing it	Screen readers announce the group name, each star's label ("3 stars") and which one is checked
Clear the rating	Read-only mode for showing an average (one image with a text alternative)
Controlled: parent owns the number	`max` configurable

The key design decision

A rating is **one choice out of N** — a **radio group**. Real `<input type="radio">` elements, hidden with the standard `visually-hidden` CSS (invisible but still focusable and announced), give you one Tab stop, arrow keys, the checked state and correct screen-reader output for free. `<span onClick>` stars give none of that.

Component tree and state



Data	Kind	Owner
`value`	prop (controlled)	parent
`hover`	state	`StarRating` — purely visual preview
`shown = hover ?? value`	derived	computed in render
group `name`	`useId`	radios in one group share a name

Code

`code/star-rating.jsx`:

```jsx
// Machine coding 6 — star rating built on native radio buttons (keyboard + screen readers for free).
import { useId, useState } from 'react';

export default function StarRating({ value, onChange, max = 5, label = 'Rating', readOnly = false }) {
  const [hover, setHover] = useState(null);                 // preview while the pointer is over a star
  const name = useId();
  const shown = hover ?? value;                             // what the stars display right now

  if (readOnly) {
    return (
      <span className="stars" role="img" aria-label={`${label}: ${value} out of ${max}`}>
        {Array.from({ length: max }, (_, i) => <span key={i} className={i < value ? 'star on' : 'star'} aria-
hidden="true">★</span>)}
      </span>
    );
  }

  return (
    <fieldset className="stars" onMouseLeave={() => setHover(null)}>
      <legend>{label}</legend>
      {Array.from({ length: max }, (_, i) => {
        const n = i + 1;
        return (
          <label key={n} onMouseEnter={() => setHover(n)}>
            <input
              type="radio"
              className="visually-hidden"
              name={name}
              value={n}
              checked={value === n}
              onChange={() => onChange(n)}
              aria-label={`${n} ${n === 1 ? 'star' : 'stars'}`}
            />
            <span className={n <= shown ? 'star on' : 'star'} aria-hidden="true">★</span>
          </label>
        );
      })}
      <output className="muted">{value ? `${value} of ${max}` : 'Not rated'}</output>
      {value > 0 && <button type="button" className="link" onClick={() => onChange(0)}>Clear</button>}
    </fieldset>
  );
}
```

Test

`code/07_star_rating.test.jsx` (key parts). One jsdom gap: `user-event` calls `CSS.escape()` to find the other radios of a group, and jsdom has no `CSS` object, so the test adds a minimal stand-in.

```jsx
window.CSS ??= { escape: (s) => s.replace(/[^\w-]/g, (c) => `\\${c}`) };

test('hover previews without changing the value; leaving restores it', async () => {
  const user = userEvent.setup();
  render(<Harness initial={4} />);
  const labels = document.querySelectorAll('fieldset label');
  await user.hover(labels[1]);                              // pointer over star 2
  assert.equal(lit(), 2);
  assert.equal(screen.getByText('4 of 5').tagName, 'OUTPUT');   // value unchanged
  await user.unhover(screen.getByRole('group'));
  assert.equal(lit(), 4);
});

// keyboard test: Tab lands on the CHECKED radio ("2 stars"); ArrowRight checks "3 stars"; ArrowLeft x2 -> 1 lit
```

Output:

```
✔ clicking a star selects it; the group is a labelled set of radios
✔ hover previews without changing the value; leaving restores it
✔ keyboard: arrow keys move between stars like any radio group
✔ Clear resets to 0; read-only mode is a single image with a text alternative
ℹ tests 4
ℹ pass 4
ℹ fail 0
```

The component has **no keyboard code at all**; the keyboard test passes because native radios already behave that way.

In the browser

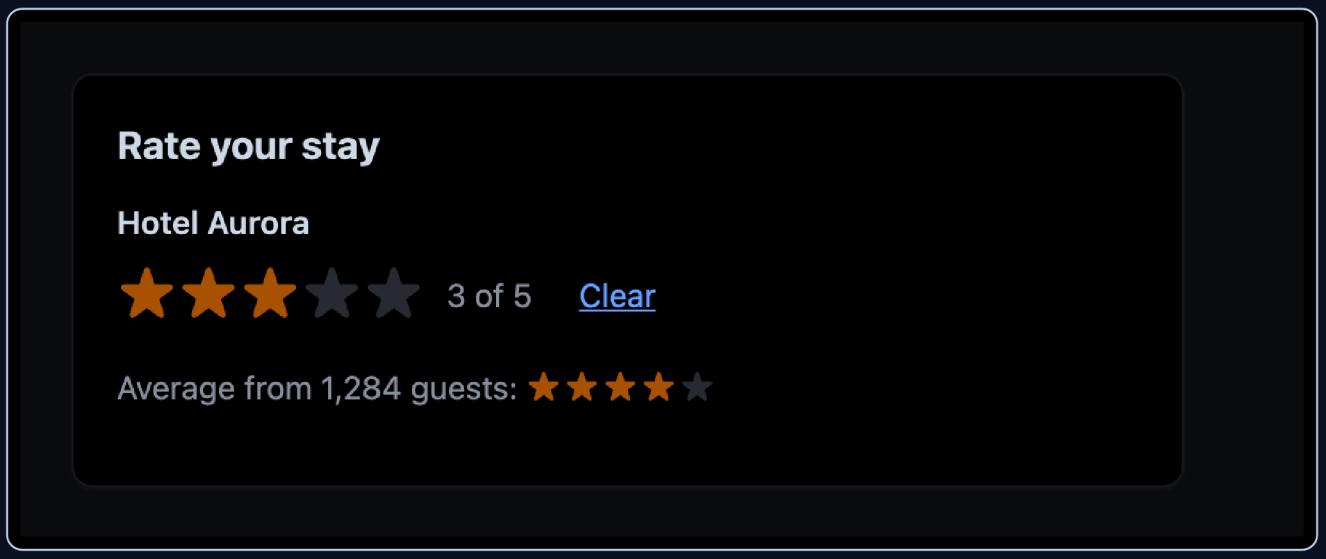


Figure 15.6 — An interactive rating (3 of 5) and a read-only average rendered as one `role="img"` with the label "Average: 4 out of 5".

Follow-ups and edge cases

- **Half stars** → 10 radios (0.5…5), each half drawn by clipping; a read-only 4.3 can be an 86%-wide overlay.
- **Visible focus** → the radio is hidden, so style the star after it (`input:focus-visible + .star` , as the demo CSS does).
- **Uncontrolled version** → `defaultValue` + internal state, and a `name` so a `<form>` submits it (Form Actions read it from `FormData`).
- **Touch devices** have no hover; the preview simply doesn't happen.
- **Why `<fieldset>` / `<legend>` ?** → they give the group its accessible name ("Food" in the test, "Hotel Aurora" in the demo); `role="radiogroup"` + `aria-labelledby` is the ARIA alternative.

15.16 Build 7 — Pagination

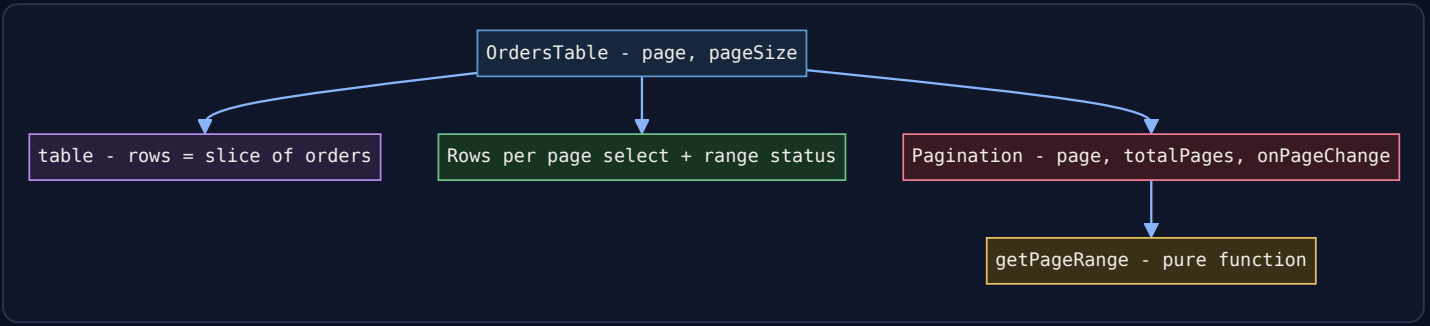
Requirements

Functional	Quiet requirements
Show 10 rows per page; Previous / Next; numbered page buttons	For many pages show `1 … 4 5 6 … 20`, with first and last always visible
Choose rows per page (5/10/25)	The number of buttons stays constant so they don't jump under the mouse
"21–25 of 47" range text	`<nav aria-label="Pagination">`, `aria-current="page"`, disabled ends
	Never show an empty page when the data shrinks (e.g. after filtering)

Split the problem in two

1. A **pure function** `getPageRange(current, total, siblings)` returning what to draw, e.g. `[1, '…', 9, 10, 11, '…', 20]` — easy to test with a table of cases (the heart of every "usePagination" Hook).
2. A **controlled** `<Pagination page totalPages onPageChange>` that draws it; the table owns `page` and `pageSize`.

Component tree and state



Data	Kind	Why
`page`, `pageSize`	state in `OrdersTable`	user choices
`totalPages`, `rows`, range text	derived	from `orders.length`, `page`, `pageSize`
`current = min(page, totalPages)`	derived (clamped)	if the list shrinks, show the last real page — no Effect needed to "fix" state

Code

`code/pagination.jsx`:

```jsx
// Machine coding 7 — pagination: a pure page-range function, a controlled <Pagination>, and a table using it.
import { useState } from 'react';

export const DOTS = '…';

// Pages to show: always first and last, `siblings` on each side of the current page, DOTS for gaps.
// The result always has the same length once there are enough pages, so buttons don't jump around.
export function getPageRange(current, total, siblings = 1) {
  const range = (a, b) => Array.from({ length: b - a + 1 }, (_, i) => a + i);
  const slots = siblings * 2 + 5;                          // first, last, current, 2 dots, siblings
  if (total <= slots) return range(1, total);

  const left = Math.max(current - siblings, 1);
  const right = Math.min(current + siblings, total);
  const showLeftDots = left > 3;                           // a gap of at least 2 pages before `left`
  const showRightDots = right < total - 2;

  if (!showLeftDots) return [...range(1, 3 + 2 * siblings), DOTS, total];
  if (!showRightDots) return [1, DOTS, ...range(total - (2 + 2 * siblings), total)];
  return [1, DOTS, ...range(left, right), DOTS, total];
}

export function Pagination({ page, totalPages, onPageChange, siblings = 1 }) {
  if (totalPages <= 1) return null;
  return (
    <nav aria-label="Pagination">
      <ul className="pager">
        <li><button onClick={() => onPageChange(page - 1)} disabled={page === 1} aria-label="Previous
page">‹</button></li>
        {getPageRange(page, totalPages, siblings).map((p, i) =>
          p === DOTS
            ? <li key={`dots-${i}`} aria-hidden="true" className="dots">{DOTS}</li>
            : (
              <li key={p}>
                <button onClick={() => onPageChange(p)} aria-current={p === page ? 'page' : undefined} aria-label=
{`Page ${p}`}>
                  {p}
                </button>
              </li>
            ))}
        <li><button onClick={() => onPageChange(page + 1)} disabled={page === totalPages} aria-label="Next
page">›</button></li>
      </ul>
    </nav>
  );
}

// Client-side pagination of an in-memory list.
export default function OrdersTable({ orders, initialPageSize = 10 }) {
  const [page, setPage] = useState(1);
  const [pageSize, setPageSize] = useState(initialPageSize);
  const totalPages = Math.max(1, Math.ceil(orders.length / pageSize));
  const current = Math.min(page, totalPages);              // clamp, e.g. after the list shrinks
  const rows = orders.slice((current - 1) * pageSize, current * pageSize);

  return (
    <section className="card">
      <h2>Orders</h2>
      <table>
        <thead><tr><th>#</th><th>Customer</th><th>Total</th></tr></thead>
        <tbody>{rows.map((o) => <tr key={o.id}><td>{o.id}</td><td>{o.customer}</td><td>${o.total.toFixed(2)}</td>
</tr>)}</tbody>
      </table>
      <div className="row between">
        <label>
          Rows per page{' '}
          <select value={pageSize} onChange={(e) => { setPageSize(Number(e.target.value)); setPage(1); }}>
            {[5, 10, 25].map((n) => <option key={n} value={n}>{n}</option>)}
          </select>
        </label>
```

```
        <span className="muted" role="status">
          {orders.length === 0 ? 'No orders' : `${(current - 1) * pageSize + 1}–${Math.min(current * pageSize,
orders.length)} of ${orders.length}`}
        </span>
      </div>
      <Pagination page={current} totalPages={totalPages} onPageChange={setPage} />
    </section>
  );
}
```

How `getPageRange` thinks: with `siblings = 1` there are 7 slots — first, last, current, one neighbour each side, and two places that are either a page or `…`. A `…` only replaces **two or more** hidden pages (showing `1 … 3` would hide just page 2, so the function shows `1 2 3` instead). Near the start it shows the first five pages, near the end the last five, otherwise the current page with its neighbours.

Test

`code/08_pagination.test.jsx` (key parts):

```
test('getPageRange: first/last always shown, dots for gaps, constant length', () => {
  const cases = [[1, 5], [3, 7], [1, 20], [4, 20], [5, 20], [10, 20], [16, 20], [17, 20], [20, 20]];
  for (const [page, total] of cases) console.log(`    page ${String(page).padStart(2)} of ${total}:`,
getPageRange(page, total).join(' '));
  assert.deepEqual(getPageRange(10, 20), [1, '…', 9, 10, 11, '…', 20]);
  assert.deepEqual(getPageRange(10, 20, 2), [1, '…', 8, 9, 10, 11, 12, '…', 20]);
  for (let p = 1; p <= 20; p++) assert.equal(getPageRange(p, 20).length, 7);   // nothing jumps
});

// clamp test: on page 5, rerender with orders.slice(0, 12) -> status "11–12 of 12" (page 2, not an empty page 5)
```

Output:

```
    page  1 of 5: 1 2 3 4 5
    page  3 of 7: 1 2 3 4 5 6 7
    page  1 of 20: 1 2 3 4 5 … 20
    page  4 of 20: 1 2 3 4 5 … 20
    page  5 of 20: 1 … 4 5 6 … 20
    page 10 of 20: 1 … 9 10 11 … 20
    page 16 of 20: 1 … 15 16 17 … 20
    page 17 of 20: 1 … 16 17 18 19 20
    page 20 of 20: 1 … 16 17 18 19 20
✔ getPageRange: first/last always shown, dots for gaps, constant length
    pager on page 5 of 5: ‹ 1 2 3 4 5 ›
✔ Next / Previous / page buttons, aria-current and disabled ends
    pager with 5 per page: ‹ 1 2 3 4 5 … 10 ›
✔ changing rows-per-page goes back to page 1
✔ clamps the page when the data shrinks under it
ℹ tests 4
ℹ pass 4
ℹ fail 0
```

In the browser

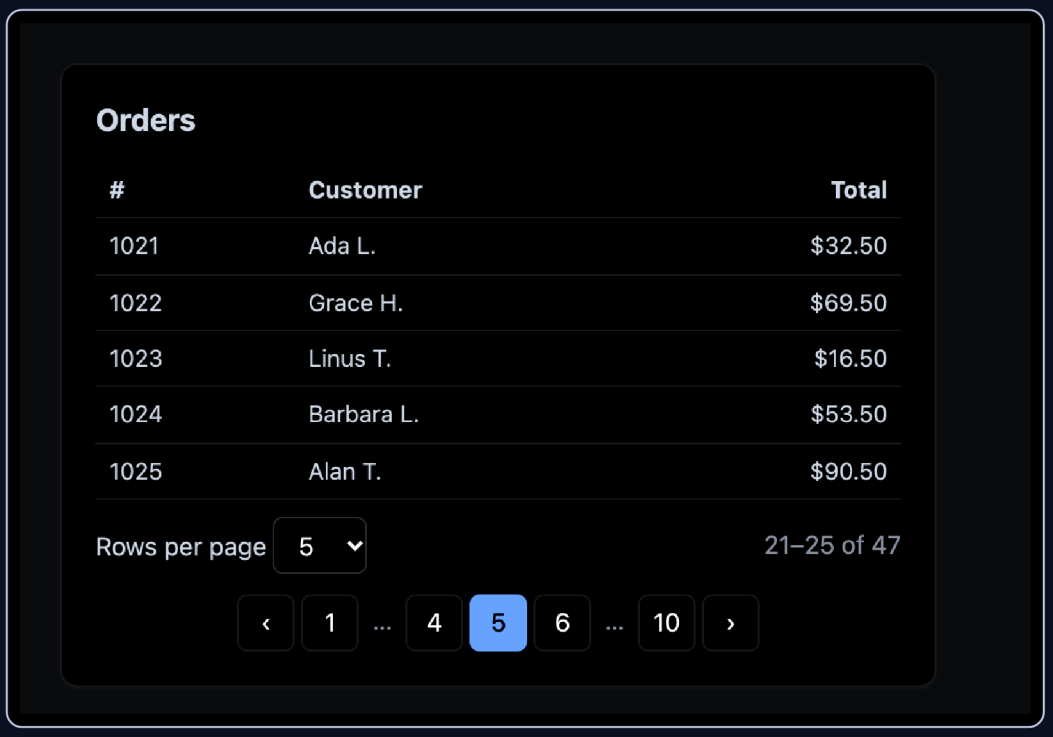


Figure 15.7 — Page 5 of 10 with five rows per page: first and last pages, the current page with one neighbour each side, and ellipses for the gaps.

Follow-ups and edge cases

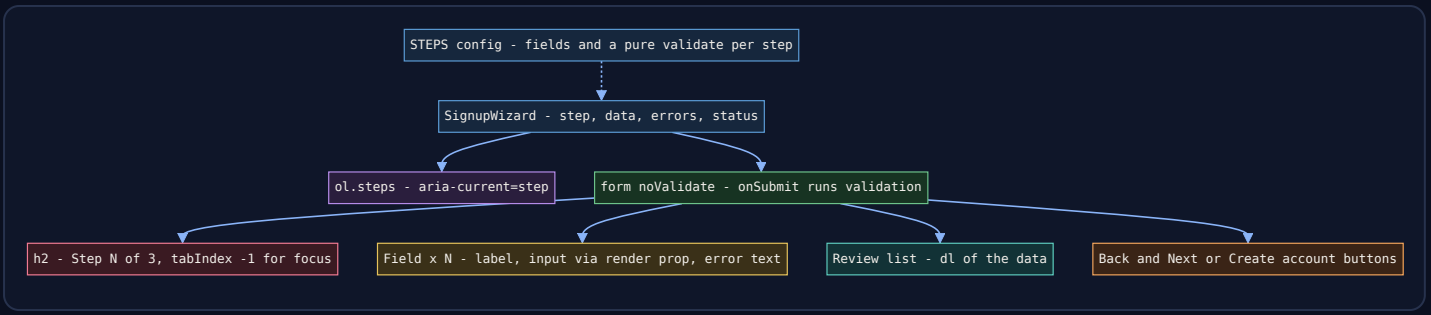
- **Server-side pagination** (server sends one page at a time) → the API returns one page plus a total (`?page=5&limit=5` or a cursor); TanStack Query keyed by `['orders', page, pageSize]` with `placeholderData: keepPreviousData` keeps the old page visible while the next loads (§9.9).
- **Page in the URL** (`?page=5`) → shareable links and a working Back button.
- **Offset vs cursor** → offsets ("skip N items") allow "jump to page 17" but slow down on deep pages and shift on inserts; cursors are stable and fast but only next/previous.
- **Clamping keeps the stored page** → `page` stays 5 while page 2 is shown, so if the data grows back before the user pages again, the view jumps to 5. Resetting to page 1 whenever the filter changes (as rows-per-page does) avoids it.
- **Focus after a page change** → move focus to the table heading and scroll to it, so screen-reader users hear the new rows.
- **Huge counts** → counting is expensive; show "Next" without a total, or an estimate.

15.17 Build 8 — Multi-step form wizard with validation

Requirements

Functional	Quiet requirements
Three steps: Account (email, password) → Profile (name, plan) → Review → Create account	Validate **per step** when Next is pressed; don't advance while errors exist
Back keeps everything entered	Errors tied to fields: `aria-invalid`, `aria-describedby`; focus moves to the **first** invalid field
A step indicator	The indicator marks the current step with `aria-current="step"`; on step change, focus moves to the new step's heading
Submit shows a pending state and a success or error message	No double submit; a failed submit can be retried

Component tree



State design

Data	Kind	Why
`data`	one state object for **all** steps	Back/Next never lose input; the Review step and the submit read one object
`step`	state	which step is showing
`errors`	state `{ field: message }`	set on Next; a field's error clears as the user edits it
`status`	state `'editing' | 'submitting' | 'done' | 'failed'`	one variable instead of several booleans
step definitions (`STEPS`)	constant outside the component	fields + a **pure** `validate(data)` per step — testable without rendering

The wizard is one `<form>`, so Enter in any field "clicks" the submit button — Next on steps 1–2, Create account on step 3. `noValidate` turns off the browser's built-in validation bubbles so messages are consistent and accessible; `type="email"` and `autoComplete` still help mobile keyboards and password managers.

Code

`code/wizard.jsx`:

```jsx
// Machine coding 8 — multi-step form wizard: one data object, per-step validation, focus management.
import { useRef, useState } from 'react';
import { flushSync } from 'react-dom';

// Each step lists its fields and a pure validate(data) -> { field: message }.
export const STEPS = [
  {
    title: 'Account',
    fields: ['email', 'password'],
    validate: (d) => ({
      ...(!d.email ? { email: 'Email is required' } : !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(d.email) ? { email:
'Enter an email like name@example.com' } : {}),
      ...(d.password.length < 8 ? { password: 'Use at least 8 characters' } : {}),
    }),
  },
  {
    title: 'Profile',
    fields: ['name', 'plan'],
    validate: (d) => ({
      ...(!d.name.trim() ? { name: 'Name is required' } : {}),
      ...(!d.plan ? { plan: 'Choose a plan' } : {}),
    }),
  },
  { title: 'Review', fields: [], validate: () => ({}) },
];

const INITIAL = { email: '', password: '', name: '', plan: '' };

function Field({ name, label, error, children }) {
  const errorId = `${name}-error`;
  return (
    <div className="field">
      <label htmlFor={name}>{label}</label>
      {children({ id: name, name, 'aria-invalid': error ? true : undefined, 'aria-describedby': error ? errorId :
undefined })}
      {error && <p id={errorId} className="error">{error}</p>}
    </div>
  );
}

export default function SignupWizard({ onSubmit, initialStep = 0, initialData = INITIAL }) {
  const [step, setStep] = useState(initialStep);
  const [data, setData] = useState(initialData);
  const [errors, setErrors] = useState({});
  const [status, setStatus] = useState('editing');          // 'editing' | 'submitting' | 'done' | 'failed'
  const formRef = useRef(null);
  const headingRef = useRef(null);

  function goTo(next) {
    flushSync(() => {                                       // update the DOM now...
      setErrors({});
      setStep(next);
    });
    headingRef.current.focus();                             // ...so the focused heading already has the new
  }                                                         // text when a screen reader announces it

  const update = (field) => (e) => {
    setData((d) => ({ ...d, [field]: e.target.value }));
    if (errors[field]) setErrors(({ [field]: _gone, ...rest }) => rest);   // clear that error as they fix it
  };

  async function handleSubmit(e) {
    e.preventDefault();
    const found = STEPS[step].validate(data);
    if (Object.keys(found).length > 0) {
      setErrors(found);
      const firstBad = STEPS[step].fields.find((f) => found[f]);
      formRef.current.elements[firstBad].focus();           // take the user to the first problem
      return;
    }
```

```
    if (step < STEPS.length - 1) { goTo(step + 1); return; }
    setStatus('submitting');
    try {
      await onSubmit(data);
      setStatus('done');
    } catch {
      setStatus('failed');
    }
  }

  if (status === 'done') return <section className="card"><h2>Welcome, {data.name}!</h2><p>Your {data.plan}
account is ready.</p></section>;

  return (
    <section className="card wizard">
      <ol className="steps">
        {STEPS.map((s, i) => (
          <li key={s.title} aria-current={i === step ? 'step' : undefined} className={i < step ? 'complete' :
undefined}>{s.title}</li>
        ))}
      </ol>
      <form ref={formRef} onSubmit={handleSubmit} noValidate>
        <h2 ref={headingRef} tabIndex={-1}>Step {step + 1} of {STEPS.length}: {STEPS[step].title}</h2>

        {step === 0 && (
          <>
            <Field name="email" label="Email" error={errors.email}>
              {(p) => <input {...p} type="email" autoComplete="email" value={data.email} onChange=
{update('email')} />}
            </Field>
            <Field name="password" label="Password" error={errors.password}>
              {(p) => <input {...p} type="password" autoComplete="new-password" value={data.password} onChange=
{update('password')} />}
            </Field>
          </>
        )}
        {step === 1 && (
          <>
            <Field name="name" label="Full name" error={errors.name}>
              {(p) => <input {...p} autoComplete="name" value={data.name} onChange={update('name')} />}
            </Field>
            <Field name="plan" label="Plan" error={errors.plan}>
              {(p) => (
                <select {...p} value={data.plan} onChange={update('plan')}>
                  <option value="">Choose…</option><option value="free">Free</option><option
value="pro">Pro</option>
                </select>
              )}
            </Field>
          </>
        )}
        {step === 2 && (
          <dl className="summary">
            <dt>Email</dt><dd>{data.email}</dd>
            <dt>Name</dt><dd>{data.name}</dd>
            <dt>Plan</dt><dd>{data.plan}</dd>
          </dl>
        )}

        {status === 'failed' && <p role="alert" className="error">Sign-up failed. Please try again.</p>}
        <div className="row between">
          <button type="button" onClick={() => goTo(step - 1)} disabled={step === 0 || status ===
'submitting'}>Back</button>
          <button type="submit" disabled={status === 'submitting'}>
            {step < STEPS.length - 1 ? 'Next' : status === 'submitting' ? 'Creating account…' : 'Create account'}
          </button>
        </div>
      </form>
    </section>
  );
}
```

Focus management, the part most candidates skip:

- **On a validation error**, focus jumps to the first invalid field, whose error is read out via `aria-describedby`.
- **On a step change**, `goTo` updates state inside `flushSync` so the DOM already shows the new step, then focuses the `<h2>` (`tabIndex={-1}`), announced as "Step 2 of 3: Profile". Doing it in the handler, not an Effect, keeps it off the first mount (and StrictMode's double run).
- **While submitting**, both buttons are disabled (implicit Enter submission is blocked too) and the label reads "Creating account…", so nothing submits twice.

Test

`code/09_wizard.test.jsx` (key parts):

```
test('Next on an empty step shows errors, marks fields invalid and focuses the first one', async () => {
  const user = userEvent.setup();
  render(<SignupWizard onSubmit={async () => {}} />);
  await user.click(next());
  assert.equal(heading(), 'Step 1 of 3: Account');                  // did not advance
  const email = screen.getByLabelText('Email');
  assert.equal(email.getAttribute('aria-invalid'), 'true');
  assert.equal(screen.getByText('Email is required').id, email.getAttribute('aria-describedby'));
  assert.equal(document.activeElement, email);
  // ... typing clears the error; "ada" then shows the format message
});

// full journey: after Next, focus is on the step <h2>; Back keeps 'ada@example.com'; while submitting the
// "Creating account…" button is disabled (no double submit); finally "Welcome, Ada Lovelace!" appears
```

Output:

```
✔ validators are pure functions: easy to unit-test on their own
✔ Next on an empty step shows errors, marks fields invalid and focuses the first one
    review: Email=ada@example.com, Name=Ada Lovelace, Plan=pro
✔ full journey: steps, Back keeps data, review, submitting state, success
✔ a failed submit shows an alert and lets the user retry
ℹ tests 4
ℹ pass 4
ℹ fail 0
```

In the browser

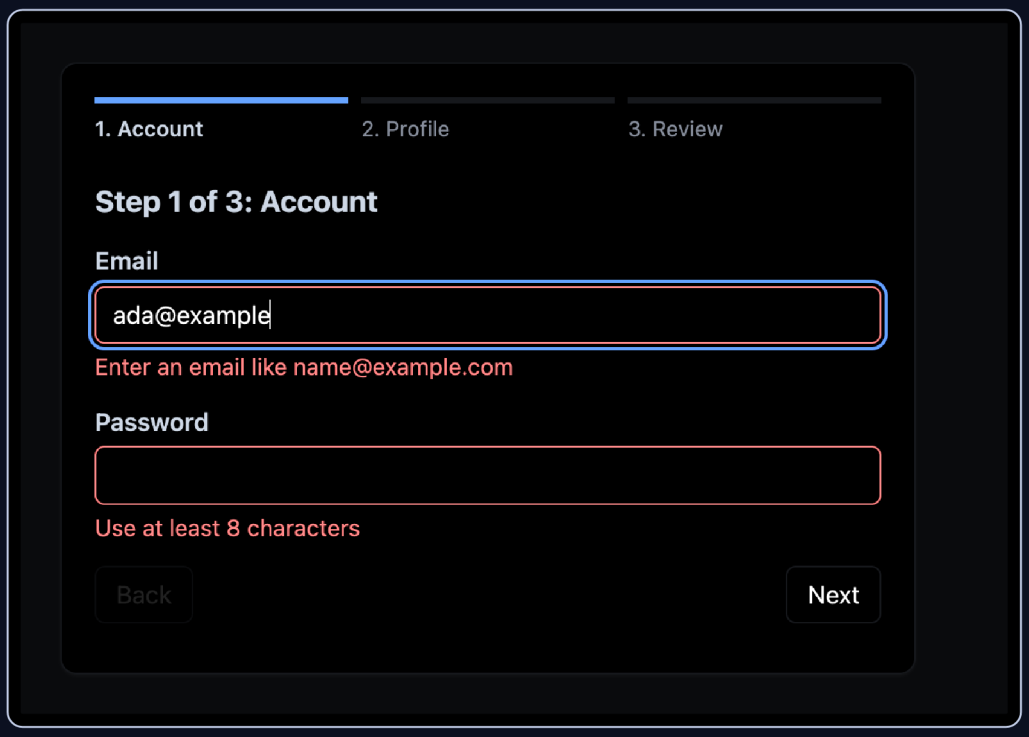


Figure 15.8 — Pressing Next with an incomplete email and no password: both fields are marked invalid, each message is linked to its field, and focus (blue ring) is on the first problem.

Follow-ups and edge cases

- **React 19 form Actions** → `<form action={createAccount}>` with `useActionState` for server errors and `useFormStatus` for the pending button (§4.5); Actions reset only *uncontrolled* fields, so these controlled ones keep their values.
- **Big forms** → React Hook Form with a Zod schema per step; `trigger(['email', 'password'])` validates one step (§4.9).
- **Validate again on the server** — client validation is convenience, not security.
- **Save progress** → `data` (without the password!) in `sessionStorage` (cleared when the tab closes), or `step` in the URL so refresh and Back work.
- **Conditional steps** (e.g. "Company details" for Pro) → compute the visible steps from `data`.
- **IDs** → field names as `id`s clash with two wizards on a page; prefix with `useId()`.
- **Async validation** ("email already taken") → a debounced check on blur or Next, with a pending state on the field.
- **A failed submit, then Back** → the "Sign-up failed" alert stays visible on earlier steps; reset `status` to `'editing'` in `goTo` if that bothers you.

15.18 Other common prompts — which techniques they need

Prompt	Core state	Techniques from this chapter
Accordion	`openIds` (a `Set`, or one ID)	`<button aria-expanded aria-controls>`; or native `<details>` / `<summary>`; like Tabs
Dropdown / menu button	`open`, `active`	Autocomplete's keyboard handling + Modal's Escape/click-outside + focus return
Toast notifications	queue of `{ id, text }`	portal, `role="status"` live region, timers cleared in Effect cleanup
Tooltip / popover	`open`	portal, `useLayoutEffect` to measure and position, `aria-describedby`; or the native Popover API
Countdown / stopwatch	`startedAt`, `elapsed`	`setInterval` in an Effect with cleanup; compute from timestamps, not by counting ticks
Image carousel	`index`	wrap-around like Tabs; pause autoplay on hover/focus; `aria-roledescription="carousel"`
File explorer / nested tree	`expanded` set; tree data as a prop	recursive components (rendering themselves per nested level) with keys; ARIA `tree` roles and arrow keys
Sortable, filterable data table	`sort`, `filters`, `page`	all derived with `useMemo` if large; Pagination; URL state
Kanban board (drag and drop)	columns → card IDs	normalised state (items stored once by ID, referenced by ID) in a reducer; `@dnd-kit` or native drag events; keyboard alternative
OTP (one-time password) / PIN input	array of digits	refs array to move focus; paste handling; `autoComplete="one-time-code"`
Progress bar / file upload	`progress`, `status`	`<progress>` element; abort with `AbortController`
Chat window	messages; `status` per message	`useOptimistic` for sending; auto-scroll with a ref; virtualise long histories

---

Chapter 15 TL;DR

Idea	Remember
Machine-coding method	Requirements → tree → state → happy path → keyboard/a11y → test → edge cases; talk throughout
State design	Minimal state, derive the rest; colocate, lift only when shared; one `status` value, not several booleans
Rendering & keys	Trigger → render (pure) → commit; identity = type + position + key; stable ID keys; new key = fresh instance
State updates	Snapshot per render; batched everywhere since React 18; updater functions build on queued values
Effects	Only to sync with outside systems; cleanup runs before re-run and on unmount; StrictMode runs setup → cleanup → setup in dev
Refs	Mutable, don't re-render, not for render output; `ref` is a normal prop in React 19
Context	A transport, not a store; every consumer re-renders on a new value; split or memoize
Performance	Measure first; `memo` / `useMemo` / `useCallback` or the React Compiler; virtualize long lists; split code with `lazy`
Concurrency	Suspense shows fallbacks; `use()` reads cached promises; transitions keep old UI while new UI loads
Server	SSR → HTML then hydrate; mismatches from non-deterministic render; Server Components ship no JS; `'use client'` marks the boundary, `'use server'` marks Server Functions
Errors & portals	Boundaries catch render/Effect errors, not event handlers; portal events bubble through the React tree
Testing	Roles and labels, `user-event`, assert what users see; fake the network at the boundary
Autocomplete	Debounced value + Effect with `AbortController`; `aria-activedescendant`; `mousedown` preventDefault
Infinite scroll	Sentinel + `IntersectionObserver` inside the scroll box; ref guard against duplicate requests
Modal	Portal, focus in/trap/restore, Escape, backdrop mousedown; prefer native `<dialog>` in production
Tabs, stars, pagination, wizard	Roving tabindex; native radios; pure range function + clamping; per-step pure validators and focus management

Next: Chapter 16 — Cheat Sheets and Glossary.

Chapter 16 — Cheat Sheets and Glossary

This last chapter squeezes Chapters 1–15 into pages you can scan in a minute. There is one **cheat sheet** per area (JSX, components, every Hook, state, the render/commit lifecycle, Effects, refs, forms, state management, routing and data, performance, styling, TypeScript, testing, server rendering, patterns, accessibility), a **"common errors and fixes"** table built from the real messages React printed in earlier chapters, an **index of the 53 interview questions** in Chapter 15, a **"where to go next"** plan, and an alphabetical **glossary** of 532 terms, each linked to the section that explains it properly.

Nothing here is new: follow a line's link for the full explanation. Every technical term — including acronyms such as SSR, RSC, CI or ARIA — is defined once, in the glossary (§16.21); the sheets don't repeat definitions. **§9.5** means Chapter 9, section 5.

How to use it: read a sheet, cover the right-hand column, say the answer, then look. Before an interview, if you can't define a glossary term in one sentence, follow its link.

How the facts were checked. Every concrete behaviour quoted on these sheets — batching, updater queues, key resets, StrictMode double calls, Effect order, default values, error messages — is asserted by `code/01_cheatsheet_checks.jsx`. It runs React 19.3 in jsdom (plus React Router 8.4 and TanStack Query 5.104) and fails loudly if React behaves differently from a sheet. Run it with `bash tools_run_js.sh react` from the repository root. Real output (abridged):

```
OK   16.1 JSX rules
OK   16.2 components and props
OK   16.3 hooks table (identity, lazy init, memo caches, useId, deferred value)
OK   16.4 state: snapshot, updaters, batching, bail-out, immutability, position & key
OK   16.5 render/commit lifecycle order
OK   16.6 effects: dependency arrays, cleanup, StrictMode double run
OK   16.7 refs: no re-render, null during first render, callback cleanup, ref as a prop
OK   16.8 forms: FormData rules, form actions, useActionState, useFormStatus, useOptimistic
OK   16.9 context: default, nearest provider, memo does not block it, use() may be conditional
OK   16.10 React Router params and ranking; TanStack Query defaults and de-duplication
OK   16.11 performance: memo, unstable props, children from above, useMemo, lazy
OK   16.14 testing: get/query/find semantics, role + name, user-event vs fireEvent
OK   16.15 server rendering: markers, no effects on the server, suspended boundaries, hydration reuse,
use(browser()), resume APIs
OK   16.16 patterns: portals bubble through React, boundaries catch render errors only, HOC in render remounts
OK   16.18 common errors: the exact messages React 19.3 produces
all 15 cheat-sheet checks passed (React 19.3.0)
```

(It also prints every error message quoted in §16.18. §16.12 Styling, §16.13 TypeScript and §16.17 Accessibility summarise results Chapters 8, 11 and 14 verified with a real browser, `tsc` and role queries, so they have no separate check.)

16.1 JSX rules

Rule	Write	Not	Why
One root element	`<>…</>` or `<div>…</div>`	two siblings at the top	A function returns one value; JSX becomes one object
Close every tag	`<img />`, ` `, `<input />`	`<img>`	JSX is stricter than HTML
camelCase attributes	`className`, `htmlFor`, `onClick`, `tabIndex`	`class`, `for`, `onclick`	Attributes become JS object keys; `class` / `for` are reserved. `aria-*` and `data-*` keep dashes
`{}` holds an **expression**	`{user.name}`, `{a ? b : c}`, `{list.map(…)}`	`{if (…) …}`	Statements (`if`, `for`, `const`) go above the `return`
Strings in quotes, everything else in braces	`alt="text"`, `size={50}`, `admin` (= `true`)	`size="50"` for a number	Quotes always pass a string
`style` is an object	`style={{ fontSize: 14, lineHeight: 1.5 }}` → `font-size:14px;line-height:1.5`	`style="font-size:14px"`	Numbers get `px`, except unitless properties (`lineHeight`, `zIndex`, `opacity`, `flexGrow`)
Comments	`{/* note */}`	`<!-- note -->`	JSX is JavaScript
Keyed fragment	`<Fragment key={id}>…</Fragment>`	`<key={id}>`	The short `<>` takes no props

What renders what

Value in `{}`	Screen shows
`null`, `undefined`, `true`, `false`	nothing
`0`, `NaN`	"0", "NaN" — the **`{count && <Badge/>}` trap**; write `count > 0 &&`
`''`	an empty text node (invisible)
strings, numbers	the text, **HTML-escaped** (`<script>` shows as text — built-in XSS protection)
arrays of elements	each element (give each a `key`)
a plain object	throws *"Objects are not valid as a React child"*

Remember

- JSX compiles to `jsx(type, props)` / `createElement(type, props, ...children)` and returns an **immutable element object** `{ type, props, key }`. Since React 17 (automatic runtime) you don't `import React`. §1.8
- Capitalised tag = your component (`<Profile />`); lowercase = HTML tag (`<profile>` is just an unknown element, and React warns). §1.7, §2.1
- Raw HTML only through `dangerouslySetInnerHTML={{ __html: sanitised }}`. React 19 replaces a `javascript:` URL in `href` with one that throws. §1.8
- Conditional rendering: `if` + early `return` for big branches, `? :` for either/or, `&&` with a **boolean** on the left, `null` for nothing. §2.7

16.2 Components and props

```
// A component: capitalised function, one props object in, JSX out, declared at module top level.
export function Avatar({ person, size = 100, className, ...rest }) {   // destructure, default, collect the rest
  return <img className={className} src={person.url} alt={person.name} width={size} {...rest} />;
}
<Avatar person={ada} size={50} />          // strings in quotes, other values in braces
```

Topic	Rule
Name and place	Capitalised; defined at **module top level**, never inside another component (a new type every render → state and DOM reset) §1.7
Use	`<Profile />`, never `Profile()` — calling it directly gives it no place in the tree and its Hooks belong to the caller §2.1
Props	One **read-only** object from the parent; frozen in development (mutating throws `TypeError: Cannot assign to read only property`) §2.2
Defaults	Destructuring default `({ size = 100 })` applies when the prop is **missing or** `undefined` — not for `null` or `0`. `defaultProps` / `propTypes` are gone for function components in React 19 §2.2
Special props	`key` is consumed by React (`props.key` is `undefined`); `children` = whatever is nested; `ref` is an ordinary prop in React 19 §2.2, §5.10
Forwarding	`({ variant, ...rest }) => <button {...rest} />`; later props override earlier ones §2.2
`children`	Anything renderable; render it as `{children}` to build wrappers (`Card`, `Layout`) §2.3
Composition	Pass JSX as `children` or **slot** props (`header={…}`) instead of inheritance or boolean-prop explosions; first cure for prop drilling §2.4, §14.1
Purity	Same props/state/context → same JSX; no changes to outside variables during render (local mutation of values created in this render is fine) §2.5
Size	One job per component; extract when JSX has a name, repeats, has its own logic or needs its own state §2.6
Lists	`items.map(i => <Row key={i.id} … />)`; key on the outermost element returned by `map` §2.8

Keys in one table §2.8, §3.10

Key choice	Result
Stable ID from the data	Correct: items keep DOM, state and focus when moved
Array index	OK only for static lists; on insert/sort/filter, state stays at the **position** (the "note moves to Eggs" bug)
`Math.random()` / `crypto.randomUUID()` in JSX	Every item remounts every render — slow, loses input focus
Missing	Warning, index used implicitly
Duplicate	Warning, items duplicated or dropped
Changing the key of one component on purpose	Fresh instance: the idiomatic way to **reset** state (`<Profile key={userId} />`)

16.3 Every Hook in one table

"Since" = the React version that made it stable. All Hooks follow the Rules of Hooks (top level of a component or custom Hook, same order every render) — except `use`, which may be called in `if`/loops (not in `try/catch`). §7.2

Hook	Signature	What it's for	Gotchas	Since
`useState`	`const [v, setV] = useState(init | () => init)`	Memory that changes with interaction; setter schedules a re-render	Value is a snapshot; initial value used on first render only; pass a function for lazy init; setter identity is stable; `Object.is`-equal value → bail-out §3.2	16.8
`useReducer`	`const [s, dispatch] = useReducer(reducer, initialArg, init?)`	Many related updates in one pure `(state, action) => next`	Reducer must be pure (StrictMode calls it twice); `dispatch` is stable; throw on unknown actions §6.2	16.8
`useContext`	`const v = useContext(Ctx)`	Read the nearest provider's value	Default only when **no** provider; re-renders on every value change, even through `memo` §6.3, §6.4	16.8
`use`	`const v = use(promise | Ctx)`	Read a promise (suspends; rejection → error boundary) or a context	Promise must be cached/created outside render — a new promise each render triggers "suspended by an uncached promise"; allowed in `if`/loops §9.11	19.0
`useRef`	`const r = useRef(init)` → `{ current }`	Values that don't affect rendering (timer IDs, DOM nodes, instances)	Changing `r.current` never re-renders; don't read/write it during render (except lazy init); DOM refs are `null` until commit §5.8, §5.9	16.8
`useImperativeHandle`	`useImperativeHandle(ref, () => api, deps?)`	Expose a limited API (`focus` , `clear`) instead of the DOM node	Pair with `ref` as a prop (React 19) §5.10	16.8
`useEffect`	`useEffect(() => { setup; return cleanup }, deps?)`	Synchronise with an external system after commit	No array = every commit; `[]` = mount only; `[a]` = when `a` changes (`Object.is`); setup can't be `async`; doesn't run on the server §5.2	16.8
`useLayoutEffect`	same as `useEffect`	Measure layout and fix it **before paint** (no flicker)	Blocks painting; does nothing during SSR; default to `useEffect` §5.11	16.8
`useInsertionEffect`	same shape	CSS-in-JS libraries inserting `<style>` before layout effects	Not for app code §5.11	18.0
`useEffectEvent`	`const onX = useEffectEvent(fn)`	Non-reactive logic inside an Effect that must	Call only from Effects; never list it in dependencies; don't pass	19.2

Hook	Signature	What it's for	Gotchas	Since
		see the **latest** props/state	it to other components §5.5, §7.7	
`useMemo`	`const v = useMemo(() => calc(a), [a])`	Cache an expensive result (~1 ms+) or keep an object's identity stable	Must be pure; a cache, not a guarantee; unstable deps make it useless §10.6, §10.5	16.8
`useCallback`	`const fn = useCallback(fn, [a])`	Same function identity for `memo` children / Effect deps	= `useMemo(() => fn, deps)` ; missing deps → stale closure §10.5	16.8
`useTransition`	`const [isPending, startTransition] = useTransition()`	Mark updates non-urgent; keep old UI during Suspense	Can't wrap a controlled input's value; since 19 the callback may be `async` (an Action) — updates **after an** `await` need their own `startTransition` §7.10	18.0
`useDeferredValue`	`const d = useDeferredValue(v, initial?)`	A lagging copy of a value for a slow child	Needs `memo` on the slow child; not a debounce (no timer, no fewer requests); `initialValue` added in 19 §7.11	18.0
`useId`	`const id = useId()`	IDs for `htmlFor` / `aria-*` that match between server and client	One call + suffixes (`${id}-hint`); never for list keys; format is not stable API §7.8	18.0
`useSyncExternalStore`	`useSyncExternalStore(subscribe, getSnapshot, getServerSnapshot?)`	Subscribe to data outside React without tearing	`getSnapshot` must return the **same** value if nothing changed (else "should be cached" + infinite loop); keep `subscribe` stable §7.9	18.0
`useDebugValue`	`useDebugValue(label)`	Label a custom Hook in React DevTools	Library Hooks only §7.3	16.8
`useActionState`	`const [state, action, isPending] = useActionState(fn, initial, permalink?)`	Result of the last Action (errors, success) + pending flag	`fn(prevState, payload)` — previous state comes **first**; calls are queued; manual dispatch must be inside `startTransition` §4.5	19.0
`useFormStatus` (`react-dom`)	`const { pending, data, method, action } = useFormStatus()`	Status of the **parent** `<form>` from any child	Always `pending: false` in the component that renders the `<form>` ; import from `react-dom` §4.6	19.0
`useOptimistic`	`const [opt, addOpt] = useOptimistic(value, reducer?)`	Show the expected result while an Action runs	Call `addOpt` inside an Action/transition; rolls back automatically when the Action ends; reducer must be pure §4.7	19.0

Related non-Hook APIs: `memo` §10.4, `lazy` + `<Suspense>` §10.8, `startTransition` (standalone) §7.10, `createContext` / `<Ctx value>` §6.3, `createPortal` §14.8, `flushSync` §3.4, `<Profiler>` §10.3, `<StrictMode>` §5.4, `<Activity>` (19.2) §5.4, `cache()` for Server Components §13.7. Library Hooks (React Router, TanStack Query, Redux, Zustand) are in §16.9 and §16.10.

16.4 State update rules

Rule	Example	Result
State is a **snapshot** per render	`setN(n + 1); console.log(n)`	Logs the old value; a timeout created in that render also sees the old value forever §3.3
Plain value = **replace**	three × `setN(n + 1)` from 0	**1**
Updater = **compute from the queued value**	three × `setN(m => m + 1)` from 0	**3**
Mixed queue, processed in order	`setN(n + 5); setN(m => m + 1)` from 0	**6**
Last replace wins	`setN(m => m + 1); setN(n + 5)` from 0	**5** §3.5
Batching	several setters in one event — also inside `setTimeout`, promises, native listeners (React 18+)	**one** re-render §3.4
Opt out of batching	`flushSync(() => setItems(next))`	DOM updated before the next line (e.g. to scroll to a new item)
Same value	`setN(n)`	Bail-out: no re-render of that subtree (React may call the component once more and discard it) §3.2, §10.1
Never mutate	`p.name = 'x'; setP(p)`	Same reference → **no re-render**, stale screen. Use `setP({ ...p, name: 'x' })` §3.6
State lives at a **position**	same component type at the same place	State kept, even with different props
…	different type at that place, or a **different** `key`	State destroyed, fresh instance §3.10

Immutable update recipes §3.6

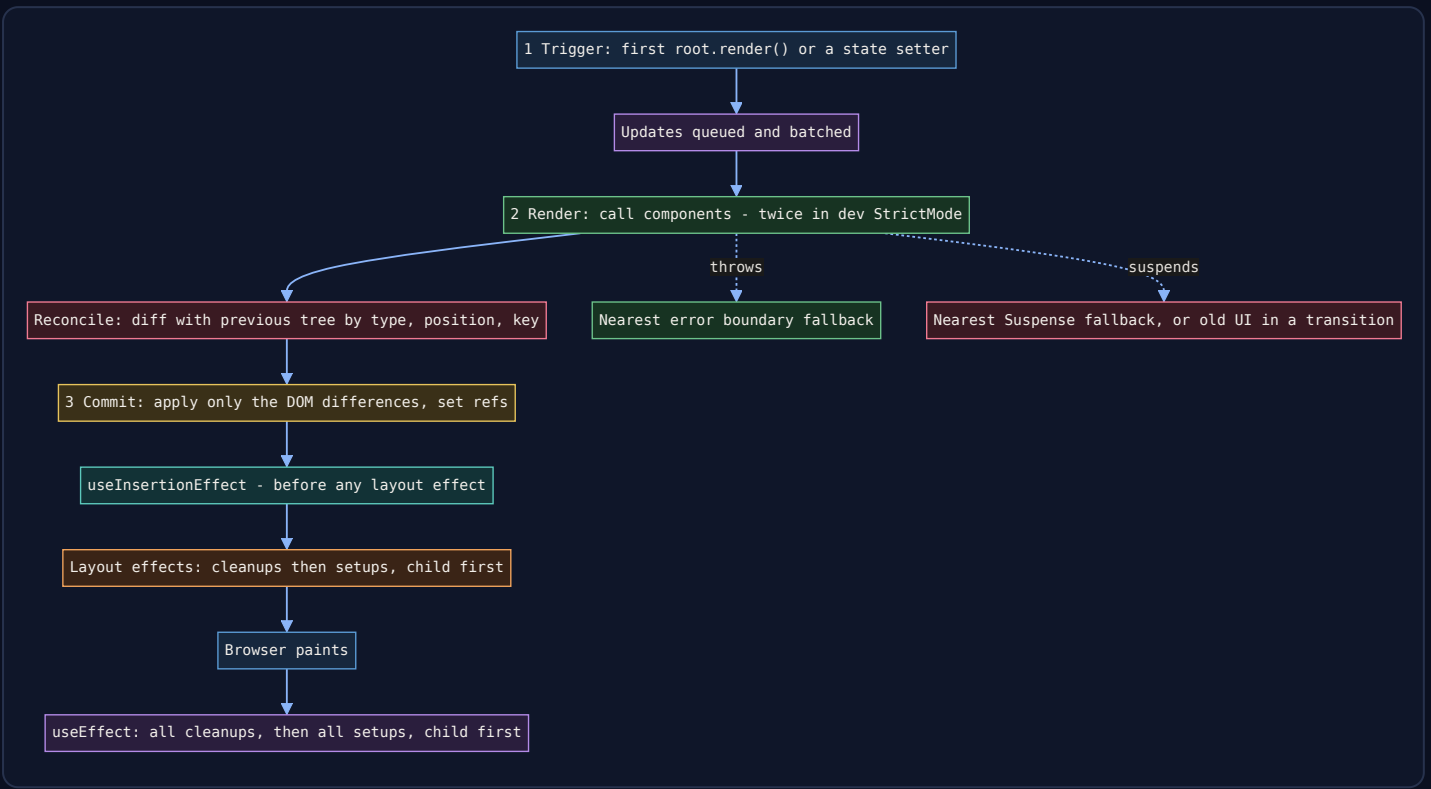
Goal	Use (returns new)	Avoid (mutates)
Change a field	`{ ...obj, field: v }` ; nested: copy every level `{ ...u, address: { ...u.address, zip } }`	`obj.field = v`
Add	`[...arr, x]` , `[x, ...arr]`	`push` , `unshift`
Insert at `i`	`arr.toSpliced(i, 0, x)` or `[...arr.slice(0, i), x, ...arr.slice(i)]`	`splice`
Remove	`arr.filter(it => it.id !== id)`	`splice` , `pop` , `shift`
Update one	`arr.map(it => it.id === id ? { ...it, done: true } : it)` , `arr.with(i, x)`	`arr[i] = x`
Sort / reverse	`toSorted()` , `toReversed()` , or `[...arr].sort()`	`sort()` , `reverse()` in place
Deep changes	Immer: `produce(draft => { draft.a.b = 1 })` , `useImmer`	hand-copying 4 levels

Choosing what to store §3.8, §6.1

- Keep the **minimal** state; compute the rest in render (full names, totals, filtered lists, "is valid").
- Store **IDs**, not copies of objects; don't mirror props into state (`useState(prop)` only reads the first value — name it `initialX` if that's the intent).
- One `status: 'idle' | 'sending' | 'sent'` instead of contradictory booleans.
- State as **low as possible, as high as necessary**: lift shared state to the closest common parent §3.7.
- `useState` for independent values; `useReducer` when many handlers change the same state in related ways §6.2.

16.5 The render and commit lifecycle

Every update goes **trigger** → **render** → **commit**. "Re-render" means *React called your function again* — the DOM changes only where the output differs. §10.1, §1.3



What the check program observed for a `Parent` → `Child` pair when a prop changes from 1 to 2 (§16.5 check):

Step	Order
Mount	render parent → render child → child **layout** setup → child **effect** setup → parent **effect** setup
Update	render parent → render child → child layout cleanup (v=1) → child layout setup (v=2) → child effect cleanup (v=1) → parent effect cleanup (v=1) → child effect setup (v=2) → parent effect setup (v=2)
Unmount	every remaining cleanup runs, with the values of the render that created it

Phase	Must be	What happens there
Render	**Pure** — may run twice, be paused (concurrent rendering) or thrown away	Call components, compute JSX and derived values
Commit	Runs once per committed update	DOM mutations, refs set, layout effects
After paint	—	`useEffect` setups (subscriptions, fetching, logging)

What triggers a re-render — own state change; **parent re-rendered** (all children by default, props are *not* compared unless `memo`); a **context** it reads changed; a subscribed **external store** changed. Not: mutating props, changing `ref.current`, changing a plain variable. §10.2

16.6 Effects: decision guide

Ask "why does this code run?" §5.1, §5.6

Go down the list and stop at the first "yes":

#	Question	If yes
1	Can it be computed from props or state?	Compute it during render (`useMemo` only if expensive)
2	Is it caused by a specific user action (click, submit)?	Event handler or form Action
3	Must **all** state reset when a prop changes?	Give the component a `key`
4	Is it reading data that lives outside React and changes on its own?	`useSyncExternalStore`
5	Is it fetching server data?	TanStack Query or a router loader — or an Effect **with cleanup**
6	Must it stay in sync with an external system while the component is on screen?	`useEffect` with a cleanup
7	None of the above	Probably no Effect at all

You might not need an Effect §5.6

You want to…	Instead of an Effect
Compute a value from props/state	A variable in render (an Effect costs an extra render and a stale frame)
Cache an expensive calculation	`useMemo`
Reset all state when `userId` changes	`<Profile key={userId} />`
Adjust some state when a prop changes	Derive it (store an ID, look the object up)
Run logic because the user clicked/submitted	The event handler / form Action
Chain updates (A → B → C)	Compute all next state in one handler
Notify the parent of a change	Call its callback in the same handler, or lift the state
Initialise the app once	Module-level code or a `didInit` flag
Subscribe to an external store	`useSyncExternalStore`

What legitimately needs an Effect: sockets and connections, browser events (`resize` , `online` , key shortcuts), timers while visible, non-React widgets (maps, video, `dialog.showModal()`), `document.title` , logging an impression because something was shown, fetching (with cleanup).

Dependencies §5.5

Situation	Fix
The linter wants a value you don't want to react to	Don't silence it — split the logic: reactive part in the Effect, non-reactive part in `useEffectEvent`
Object/function created in render is a dependency	New every render → Effect runs every render. Create it **inside** the Effect or outside the component; depend on primitives
Interval reads state	`setCount(c => c + 1)` so `count` isn't a dependency
Two unrelated jobs in one Effect	Two Effects
Can be left out	setters, `dispatch` , refs, module constants

Cleanup pairs §5.3: `connect` / `disconnect` · `setInterval` / `clearInterval` · `addEventListener` / `removeEventListener` (same function reference) · `subscribe` / `unsubscribe` · `new AbortController` + `fetch(url, { signal })` / `controller.abort()` · `observer.observe` / `observer.disconnect` · `showModal` / `close` .

StrictMode (development only) — you opt in by wrapping the app in `<StrictMode>` , which Vite and most templates do — calls component bodies, state initialisers, updaters and reducers **twice**, runs every Effect **setup → cleanup → setup** on mount, and re-runs ref callbacks. Production runs once. The fix for a visible double is a correct cleanup, never a "has run" flag. §5.4, §2.5

Fetching in an Effect §5.7, §7.6: loading/error/success in one `status` value; `ignore` flag or `AbortController` in cleanup (else the slower old response wins); check `res.ok` (`fetch` doesn't reject on 404); ignore `AbortError` . For real apps prefer TanStack Query or loaders (cache, dedupe, retries, no waterfalls).

16.7 Refs

	useState	useRef
Returns	`[value, setValue]`	`{ current }` — the same object every render
Changing it	setter → **re-render**	`ref.current = x` → **no re-render**, visible immediately
Read during render?	Yes (a snapshot)	No — except lazy init `if (r.current === null) r.current = new Engine()`
Use for	anything shown on screen	timer IDs, DOM nodes, `AbortController`, third-party instances, "in flight" flags

§5.8

DOM refs §5.9, §5.10

- `const inputRef = useRef(null)` + `<input ref={inputRef} />` → `inputRef.current` is the node **after commit** (`null` during the first render), `null` again after removal.
- Good uses: focus, scroll into view, measure (`getBoundingClientRect` in `useLayoutEffect`), media `play()`, `dialog.showModal()`, non-React libraries. Not for changing text/classes React renders.
- Lists: one ref holding a `Map` + a **ref callback** per item. In React 19 a ref callback may **return a cleanup**; use a block body (`ref={n => { x = n; }}`), because an implicit-return arrow now looks like a cleanup to TypeScript.
- To scroll to an item you just added: `flushSync(() => setItems(next))`, then scroll.

Refs to your own components

React 19	Legacy
`function Field({ ref, ...p }) { return <input ref={ref} {...p} /> }`	`forwardRef((props, ref) => …)` — still works, planned for deprecation
`useImperativeHandle(ref, () => ({ focus, clear }))` to expose a limited API	same
`<Fragment ref={r}>` (19.3) → `FragmentInstance` with `focus()`, `addEventListener`, `observeUsing`	wrapper `<div>` just to hold a ref

---

16.8 Forms and Actions (React 19)

Two ways to own a field §4.2, §3.9

	Controlled	Uncontrolled
Value lives in	React state: `value={v}` + `onChange` (`checked` for boxes/radios)	The DOM: `defaultValue` / `defaultChecked`
Read it	From state	`FormData` on submit, or a ref
Re-renders	Every keystroke	None while typing
Good for	Live formatting, limits, dependent fields, counters	Most "fill in and submit" forms; pairs with form Actions
Traps	`value` without `onChange` → read-only + warning; `useState()` (undefined) → "uncontrolled to controlled" warning — start with `''`	Changing `defaultValue` later does nothing (remount with `key`); fields without `name` are skipped

`<input type="file">` is always uncontrolled. React's `onChange` fires on every keystroke (like the DOM `input` event).

`FormData` **rules** §4.1: only fields with a `name` are sent · every value is a **string** (`Number(fd.get('age'))`) · an **unchecked** checkbox is **absent** · a checked box without `value` sends `"on"` · `getAll('topic')` for repeated names · `Object.fromEntries(fd)` keeps only the last of repeated names · disabled fields are skipped.

The React 19 form toolkit

API	Shape	Remember
Form Action	`<form action={fn}>` → `fn(formData)`	React prevents navigation, runs `fn` in a Transition (may be async), **resets uncontrolled fields after success**; throwing goes to the error boundary — return expected errors instead §4.4
`formAction`	`<button formAction={saveDraft}>`	Per-button override of the form's action §4.4
`useActionState`	`const [state, action, isPending] = useActionState(async (prev, formData) => next, initial)`	Return errors **and** submitted values (use them as `defaultValue` s so a failed submit doesn't wipe the form); calls queue sequentially §4.5
`useFormStatus`	`const { pending, data } = useFormStatus()` (from `react-dom`)	Only in a **child** of the `<form>` — perfect for a reusable Submit button §4.6
`useOptimistic`	`const [opt, add] = useOptimistic(real, (cur, x) => …)`	Call `add` inside the Action; update the real state on success; rollback is automatic §4.7
`requestFormReset(form)`	from `react-dom` , inside a Transition	Reset by hand §4.4
Server Function	`'use server'` async function used as `action`	Works before hydration (progressive enhancement); arguments must be serializable; authenticate and validate inside §4.8, §13.8

Validation §4.3: three layers — HTML constraints (`required` , `type="email"` , `minLength` , `pattern`), a **pure** `validate(values)` computed during render, and **server validation (mandatory)**. Show errors on blur or after the first submit, then live while fixing; `aria-invalid="true"` + `aria-describedby` to the message; focus the first invalid field; `noValidate` hands the UI to you while `validity` keeps working. Big forms: React Hook Form + Zod (`zodResolver`), with the same schema on the server §4.9.

16.9 Context and choosing state management

Context in three steps §6.3: `const Ctx = createContext(default)` → `<Ctx value={v}>…</Ctx>` (React 19; `<Ctx.Provider>` still works) → `useContext(Ctx)` or `use(Ctx)` (conditional allowed). Nearest provider wins; the default applies **only when there is no provider** (a provider with `value={undefined}` gives `undefined`).

Context performance §6.4, §10.10

Problem	Fix
`value={{ user, setUser }}` is a new object every render → every consumer re-renders, `memo` can't stop it	`useMemo` the value (`useCallback` for functions) — or let the React Compiler do it
Consumers that need only actions re-render when data changes	Split into a state context and an actions/dispatch context
One big context with unrelated values	One context per concern
High-frequency values (typing, mouse, timers)	Keep local, or use a store with selectors

Reducer + context store §6.5: `useReducer` in a provider component, two contexts (state, dispatch), and `useTasks()` / `useTasksDispatch()` Hooks that **throw** outside the provider (use `null` as the default so the check is possible).

Where does this state live? §6.1, §6.10

Kind of state	Examples	Home
Local UI	open/closed, current tab, input text	`useState` in the component
Complex local	wizard, editor	`useReducer`
Shared by nearby components	selected item	Lift to the closest common parent
App-wide, changes rarely	theme, locale, current user	Context
App-wide, changes often, read by many	cart, editor document, dashboard filters	**Zustand** (small, no provider) or **Redux Toolkit** (structure, DevTools, middleware)
Server data	lists, profiles, search results	**TanStack Query** / RTK Query / router loaders / Server Components
Shareable / bookmarkable	filters, sort, page, selected ID	The **URL** (`useSearchParams` , route params)
Form fields	values, errors, pending	Form Actions + `useActionState` , or React Hook Form
Derived	totals, filtered lists	**Not state** — compute in render

Library essentials §6.7, §6.8

Redux Toolkit	Zustand
`createSlice({ name, initialState, reducers })` → reducer + action creators; reducers may "mutate" (Immer) — mutate the draft **or** return new state, not both	`const useStore = create((set, get) => ({ n: 0, inc: () => set(s => ({ n: s.n + 1 })) }))` — `set` merges one level
`configureStore` adds thunks, DevTools, mutation/serializability checks	No provider; `useStore.getState()/setState()/subscribe()` work outside React
`useSelector(sel)` re-renders only when the selected value changes; `useDispatch()`	`useStore(s => s.n)` ; several values → wrap in `useShallow`
Selector returning a new array/object → re-render on every dispatch; memoise with `createSelector`	Selector returning a new object without `useShallow` → "Maximum update depth exceeded" loop
Async: `createAsyncThunk` (pending/fulfilled/rejected) or **RTK Query**	Middleware: `persist` , `devtools` , `immer`

16.10 Routing (React Router v8) and TanStack Query

React Router v8 — one package (`npm i react-router` ; `react-router-dom` is gone), requires React 19.2.7+ and Node 22.22+, ESM-only, middleware on by default. `RouterProvider` comes from `react-router/dom` . §9.2

Mode	You write	You get
Declarative	`<BrowserRouter><Routes><Route path element/></Routes></BrowserRouter>`	Matching, links, params
Data	`createBrowserRouter([{ path, Component, loader, action, ErrorBoundary, children }])` + `<RouterProvider>`	+ loaders, actions, pending UI, route error boundaries, fetchers, middleware
Framework	`app/routes.ts` + route modules + Vite plugin	+ SSR / SPA / prerender, per-route code splitting, generated types §13.10

Task	API §9.3–§9.7
Dynamic segment	`path="/products/:id"` → `useParams()` → `{ id: '42' }` (**always strings**)
Ranking	Static segments beat `:params`, which beat `*` — order in the file doesn't matter; `path="*"` = 404
Links	`<Link to>`; `<NavLink>` gets `isActive` (`end` for exact); relative links resolve against the **route**, `..` = parent route
Query string	`const [params, setParams] = useSearchParams()` — shareable UI state
Navigate in code	`const navigate = useNavigate(); navigate('/x')`, `navigate(-1)`, `{ replace: true }`; or render `<Navigate to>`
Layouts	Parent renders `<Outlet />`; `index` route for the default child; pathless route for shared UI; `<Outlet context>` + `useOutletContext()`
Load before render	`loader({ params, request, context })` → `useLoaderData()`; loaders of all matched routes run **in parallel**; `useNavigation().state` = `'idle' | 'loading' | 'submitting'`
Errors	`throw data('Not found', { status: 404 })` → route `ErrorBoundary`, check `isRouteErrorResponse(error)`
Writes	`<Form method="post">` → route `action` → `return data({ errors }, { status: 400 })` (`useActionData`) or `redirect('/list')`; loaders **revalidate** automatically
Without navigating	`useFetcher()` — `fetcher.Form`, `fetcher.state`, `fetcher.data`
Guards, logging	`middleware: [fn]` with `await next()`; `createContext()` + `context.get/set` (UX only — the server still authorises)
Code splitting	`{ path, lazy: () => import('./route.jsx') }`
Tests	`MemoryRouter initialEntries` / `createMemoryRouter`

TanStack Query essentials §6.9, §9.8–§9.10

Concept	Remember
Setup	One `QueryClient` created **once** (module scope or `useState(() => new QueryClient())`), `<QueryClientProvider client>`
Read	`useQuery({ queryKey: ['product', id], queryFn })` — every input of `queryFn` goes in the key; `queryFn` must **throw** on failure (check `res.ok`)
Defaults	`staleTime` **0** (stale at once → background refetch on mount, window focus, reconnect), `gcTime` **5 min**, **3 retries** in the browser with backoff
Flags	`isPending` = no data yet · `isFetching` = request in flight (incl. background) · `isLoading` = both · `isError`, `error`, `refetch()`
Dedupe & cache	Same key at the same time → one request; remount within `staleTime` → no request
Keys	Arrays, compared by value, hierarchical: invalidating `['products']` also hits `['products', 7]`
Reuse	`queryOptions({ queryKey, queryFn })` shared by `useQuery`, `useSuspenseQuery`, `ensureQueryData`, `prefetchQuery`
Pagination / dependent	`placeholderData: keepPreviousData` (`isPlaceholderData`) · `enabled: !!id` · `useInfiniteQuery` for "load more"
Writes	`useMutation({ mutationFn, onSuccess: () => qc.invalidateQueries({ queryKey }) })`
Optimistic	`onMutate`: `cancelQueries` → snapshot → `setQueryData` → return rollback · `onError`: restore · `onSettled`: invalidate
With a router	Loader calls `queryClient.ensureQueryData(opts)` (critical) or `prefetchQuery` (non-critical); component reads the same `opts` §9.13
Tests	New `QueryClient` per test with `retry: false` §12.6

Suspense and `use()` §9.11: `<Suspense fallback>` shows a placeholder while `lazy` code, `use(promise)` or Suspense-enabled data (`useSuspenseQuery`, loaders, async Server Components) isn't ready — data fetched in `useEffect` does **not** suspend. A **transition** keeps already-visible content instead of flashing the fallback. Pair each important boundary with an error boundary. Loading UX: skeletons for first loads, old content + progress bar for navigation, nothing for fast refreshes, inline errors for validation, Retry buttons for failures §9.12.

16.11 Performance: checklist, memo and the React Compiler

Checklist — in this order, stop when fast enough §10.11

1. Reproduce with a **production build**, CPU throttled, realistic data.
2. Find the slow interaction: React DevTools **Profiler** (enable "why did this render"), "Highlight updates", Chrome Performance panel with React tracks (19.2+).
3. Slow **load**: bundle analyser → `lazy` routes and heavy widgets, images, SSR/SSG.
4. Slow **interaction**, too much re-renders → **structure first**: colocate state, pass JSX as `children`, stable keys, split/memoise context; then `memo` + stable props, or the React Compiler.
5. One slow component → `useMemo` the calculation (≥ ~1 ms) or fix the algorithm.
6. Too many DOM nodes → virtualize (TanStack Virtual, react-window) or paginate.
7. Typing lags behind results → `useDeferredValue` / `useTransition` with a `memo`'d slow child.
8. Measure again; keep only changes that made a measurable difference.

`memo` rules §10.4, §10.5

- `memo(C)` skips a re-render **caused by the parent** when every prop is `Object.is`-equal. It does not block the component's own state changes or context changes.
- Inline objects, arrays, functions and JSX (`style={{…}}`, `onAdd={() => …}`, `children`) are new every render → `memo` does nothing (V2 in §10.4). Stabilise with `useCallback` / `useMemo`, module constants, or

primitives.

- Don't bother for cheap components or props that change anyway; a custom `arePropsEqual` that ignores a callback causes stale closures.
- Free wins without `memo`: state moved down, and **elements passed as `children` from above** keep their identity (the check in §16.11 shows `Shell` re-rendering with 0 child renders).

The four common performance bugs §10.10: component declared inside a component (remount every render) · unstable keys · a new context object each render · state owned too high (every keystroke re-renders the page). Also: Effects that set state every render, fetch waterfalls, huge images/libraries, development builds in production.

React Compiler §10.7

Fact	Detail
What	Build-time Babel plugin (`babel-plugin-react-compiler`); v1.0 stable 7 October 2025; best on React 19 (17/18 need `react-compiler-runtime`)
Does	Memoizes values, callbacks and JSX per component with a cache (`useMemoCache` from `react/compiler-runtime`) — `memo` / `useMemo` / `useCallback` become mostly unnecessary
Needs	The Rules of React: pure render, no mutation of props/state, no `ref.current` during render — it **skips** components that break them
Verify	"Memo ✨" badge in React DevTools; lint rules ship in `eslint-plugin-react-hooks`
Escape hatches	`"use no memo"` directive; keep `useMemo` / `useCallback` when an Effect dependency's identity must be guaranteed
Doesn't fix	Slow algorithms, huge DOM trees, big bundles

Code splitting §10.8: `const Chart = lazy(() => import('./Chart.jsx'))` at **module level**, rendered inside `<Suspense fallback>` with an error boundary for failed chunks; `lazy` needs a **default export** (map named ones: `.then(m => ({ default: m.Chart }))`); preload on hover/focus by calling the same `import()`.

Virtualization math (fixed rows) §10.9: `first = floor(scrollTop / rowHeight) − overscan`, `count = ceil(height / rowHeight) + 2 × overscan`, absolutely positioned rows inside a spacer `items.length × rowHeight` tall — 300 px / 30 px rows / overscan 3 → **16 DOM rows** for 10,000 items.

16.12 Styling

Approach	Write	Scoping	Runtime	Pick when §8.10
Plain CSS	`import './Card.css'`, `className="card"`	**None** — imports are global; BEM or `@layer`	none	Content sites, small apps
Inline `style`	`style={{ width: `${p}%` }}`	Per element	tiny	Truly dynamic values, ideally setting a CSS variable (`style={{ '--progress': … }}`); no `:hover` /media queries
CSS Modules	`Card.module.css`, `className={styles.card}`	Automatic per file (`composes`, `:global`)	none	Teams that like writing CSS
Tailwind v4	`className="p-4 rounded-xl hover:bg-blue-700"`	Nothing to scope	none	Fast iteration; configure tokens with `@theme`
Runtime CSS-in-JS	`styled.button`…`` (styled-components, Emotion)	Automatic	**yes**	Existing codebases; styled-components is in maintenance mode (March 2025); awkward with Server Components
Zero-runtime CSS-in-JS	vanilla-extract, StyleX, Panda	Automatic	none	Typed, colocated styles without the runtime

Rules worth memorising §8.3–§8.8

- `style` numbers get `px` except unitless properties; vendor prefixes are capitalised (`WebkitLineClamp`, but `msTransform`); inline styles beat stylesheets (only `!important` overrides them).
- Build class strings with `clsx`; always accept a `className` prop; use variant maps, not ternary chains. Tailwind: write **complete** class names (`bg-${color}-500` is never generated); resolve conflicts with `tailwind-merge`.
- Theming: tokens as CSS custom properties on `:root`, overridden under `[data-theme="dark"]`; a provider writes the attribute; a blocking `<head>` script prevents the flash of the wrong theme; `color-scheme` for native controls; CSS recolours without re-rendering components.
- React 19 hoists `<link rel="stylesheet" precedence>` and `<style href precedence>` into `<head>` and de-duplicates them.
- Component libraries: styled kits (MUI, Mantine) for speed; headless (Radix, React Aria, Headless UI) for custom design with accessibility; copy-in (shadcn/ui) to own the code §8.9.

16.13 TypeScript + React types

Bundlers (Oxc, esbuild, SWC) only **strip** types; `tsc --noEmit` **checks** them — run it in CI. Use `strict: true` and `"jsx": "react-jsx"`. §11.1

Need	Type / pattern
Props	`type Props = { label: string; count?: number; tone?: 'info' | 'error' }; function Badge({ label, count = 0 }: Props)` §11.2
Callback prop	`onSelect?: (id: string) => void`, called as `onSelect?.(id)`
Children, slots	`children: ReactNode` (anything renderable) · `PropsWithChildren<P>` · `ReactElement` only when it must be one element §11.3
Native element props	`ComponentProps<'button'> & { variant?: 'primary' | 'ghost' }` (includes `ref`) · `ComponentPropsWithoutRef<'input'>` · `ComponentProps<typeof MyComp>`
Change event	`ChangeEvent<HTMLInputElement>` / `HTMLSelectElement` / `HTMLTextAreaElement` — or `ChangeEventHandler<…>` §11.4
Submit	`SubmitEvent<HTMLFormElement>` in recent `@types/react` 19 (`FormEvent` still compiles, now deprecated)
Click, keys, focus	`MouseEvent<HTMLButtonElement>`, `KeyboardEvent<HTMLInputElement>`, `FocusEvent<…>` — imported from `react`, not the DOM globals
Inline handler	No annotation — contextually typed
State	Inferred from the initial value; add `<T>` for `useState<User | null>(null)`, `useState<string[]>([])` (else `never[]`), literal unions §11.5
Setter as a prop	`Dispatch<SetStateAction<T>>`
Reducer actions	Discriminated union `{ type: 'add'; sku: string } | { type: 'clear' }` + `const _: never = action` in `default` §11.5
DOM ref	`useRef<HTMLInputElement>(null)` → `current: HTMLInputElement | null`; React 19 types **require** an argument §11.6
`ref` prop / handle	`ref?: Ref<HTMLInputElement>`; custom handle `Ref<Handle>` + `useImperativeHandle`
Tuple from a Hook	`return [on, toggle] as const` — or return an object §11.7
Context with no default	`createContext<Auth | null>(null)` + `useAuth(): Auth` that throws outside the provider
Generic component	`function List<T>({ items, renderItem }: { items: T[]; renderItem: (t: T) => ReactNode })`; `<T,>` in arrow functions; constraints with `extends`; field names with `keyof T & string` §11.8
Mutually exclusive props	Union per case; `prop?: never` forbids a prop; narrow `props` before destructuring §11.9
Untrusted data	`unknown` + narrowing, or a Zod schema — `as T` is not validation §11.7

Read type errors bottom-up: *"Type A is not assignable to type B"*, then the last indented line names the property that differs §11.10.

16.14 Testing: query priority and user-event

Philosophy §12.1: render → find like a user → act like a user → assert on what the user sees. Don't assert on state, props, Hook calls or class names.

Query priority (Testing Library) §12.3

1. `getByRole(role, { name })` — almost everything (`button`, `link`, `heading` + `level`, `textbox`, `checkbox`, `combobox`, `dialog`, `alert`, `status`, `list` / `listitem` …)
2. `getByLabelText` — form fields
3. `getByPlaceholderText` — only if there's truly no label

4. `getByText` — non-interactive text
5. `getByDisplayValue` — a field by its current value
6. `getByAltText`, `getByTitle`
7. `getByTestId` — last resort

Variant	0 matches	1	2+	Waits?	Use to…
`getBy…`	throws	element	throws	no	find what must be there now
`queryBy…`	`null`	element	throws	no	assert absence
`findBy…`	rejects	element	rejects	yes (default 1000 ms)	wait for something to appear
`getAllBy…` / `queryAllBy…` / `findAllBy…`	throws / `[]` / rejects	array	array	as above	several

user-event §12.4: `const user = userEvent.setup();` then `await` `user.click`, `user.type(el, 'hi{Enter}')`, `user.keyboard('{Escape}')`, `user.tab()`, `user.selectOptions`, `user.clear`, `user.upload`, `user.hover`. It produces the full sequence (focus, keydown, input, keyup…), respects `disabled` and focus order; `fireEvent` dispatches exactly one event (keep it for `scroll` and custom events).

Async §12.5: `await screen.findByRole(…)` (appear) · `await waitForElementToBeRemoved(() => screen.queryByText('Loading…'))` (disappear) · `await waitFor(() => assertion)` (anything else — assertions only inside) · never `sleep`. Forgetting `await` → passes by accident + `act(...)` warning.

Everything else §12.6–§12.9: mock the **network boundary** (MSW, or `mock.method(globalThis, 'fetch')` / `vi.spyOn` returning real `Response`s) and cover success, HTTP error, network error and races · Hooks through a component or `renderHook` + `act` + `result.current` · fake timers for debounce (`advanceTimers` option for user-event) · a `renderWithProviders` with `wrapper` creating a **fresh** store / `QueryClient` per test · Vitest: `environment: 'jsdom'`, `globals: true`, setup file importing `@testing-library/jest-dom/vitest` · jsdom has no layout (sizes are 0; `IntersectionObserver`, `matchMedia` need mocks).

16.15 Rendering strategies and directives

Strategy	HTML made where / when	First HTML has	Needs a server	Good for
CSR (SPA)	Browser, after JS loads	empty `<div id="root">`	No (static hosting)	Dashboards and tools behind a login
SSR	Server, every request	full content	Yes	Personalised or fast-changing public pages
SSG	Build machine, once per deploy	full content	No (CDN)	Docs, blogs, marketing
ISR	Build, then re-built in the background after a window or on demand (framework feature)	full content, stale for at most N s	Yes	Catalogues, news
Streaming SSR	Server, in chunks per `<Suspense>` boundary	shell + fallbacks, slow parts later	Yes	Fast frame, slow widgets
PPR	Static shell at build + dynamic holes per request (`prerender` → `postponed` → `resume`)	static shell with placeholders	Yes	Mixed static/per-user pages (Next.js Cache Components)
RSC	Some components run **only** on the server (build or request)	works with any row above	Yes (or build time)	Data-heavy pages with little interactivity; zero JS for server parts

§1.4, §13.1, §13.6

Server APIs §13.2

Function	Package	Waits for Suspense data?	Use
`renderToString`	`react-dom/server`	No — emits the fallback marked `<!--$!-->` (client renders it)	Legacy / simple pages
`renderToStaticMarkup`	`react-dom/server`	No; no hydration markers (`<!-- -->`)	Emails, static snippets (can't be hydrated)
`renderToPipeableStream`	`react-dom/server` (Node)	Streams later	SSR on Node: status in `onShellReady`, `onAllReady` for crawlers, always `abort()` on a timeout
`renderToReadableStream`	`react-dom/server` (Web Streams)	Streams later	Edge, Deno, Bun, Workers
`prerenderToNodeStream` / `prerender`	`react-dom/static`	**Yes**	SSG; `prerender` + abort → PPR
`resume` / `resumeToPipeableStream`	`react-dom/server` (Web / Node)	Streams	Fill PPR holes per request
`resumeAndPrerender` / `resumeAndPrerenderToNodeStream`	`react-dom/static`	**Yes**	Continue a postponed prerender at build time
`hydrateRoot(domNode, <App/>)`	`react-dom/client`	—	Adopt server HTML in the browser (never `createRoot` on server HTML)

Streaming markers §13.3: `<!--$-->` complete boundary · `<!--$?-->` pending (fallback inside) · `<!--$!-->` failed on the server → client renders it · `<template id="B:0">` where content will go · `<div hidden id="S:0">` finished content · `$RC("B:0","S:0")` swap script.

Hydration §13.4, §13.5: same tree + same data on the client; React **reuses** the DOM and attaches handlers; streaming allows **selective hydration** (the shell is interactive before slow parts arrive; clicked boundaries hydrate first); visible ≠ interactive. Mismatch sources: `window` / `localStorage` checks, `Date.now()`, `Math.random()`, locale/time zone, invalid nesting (`<div>` in `<p>`), refetched data, extensions. Fix: render the server version first and switch after mount (`useEffect` flag, or `useSyncExternalStore` with `getServerSnapshot`); for a whole browser-only component, `use(browser())` (React 19.3, `react-dom`) inside `<Suspense>` — it suspends only on the server, so the client renders it; `useId` for IDs; `suppressHydrationWarning` only for one expected difference (it keeps the **server** text). Hooks run on the server once; **Effects never run on the server**.

Server vs Client Components §13.7

	Server Component	Client Component
Marked by	Nothing — the default in RSC frameworks	A `'use client'` entry file (and everything it imports)
Runs	Server only (build or request)	Server for SSR HTML **and** the browser
Code shipped	None	Yes
Can	be `async`, `await` a database, use secrets and files	use state, Effects, event handlers, browser APIs, context
Can't	use state, Effects, handlers, `createContext` / `useContext` / `use(Ctx)` — but since 19.3 it may **render** a context provider imported from a `'use client'` module	import a Server Component (but can render one passed as `children`)

Crossing the boundary — props from server to client must be serializable: primitives (strings, numbers, booleans, `null`, `undefined`, bigint), plain objects/arrays, `Date`, `Map`, `Set`, typed arrays, JSX, **promises** (read with `use`), **Server Functions**. Not: ordinary functions, class instances, null-prototype objects, local symbols.

Directives §13.8

Directive	Where	Means
`'use client'`	First statement of a **file**	Start of client code: the module and its imports ship to the browser. Push it to the leaves
`'use server'`	Top of a file, or first line of an `async` function	**Server Function**: runs on the server, callable from the client (a public POST endpoint — authenticate, authorise, validate)
(none)	—	Server Component (there is **no** "use server component" directive)
`'use cache'`	Next.js only	Cache a function/component under Cache Components §13.9
`'use no memo'`	Component/file	Opt out of the React Compiler §10.7
`import 'server-only'`	Module	Build fails if a Client Component imports it

Frameworks §13.9–§13.11: **Next.js 16** App Router — folders are routes; `page`, `layout`, `loading`, `error` (client), `not-found`, `route` files; `await params`; `revalidatePath` / `revalidateTag` / `updateTag`; `proxy.js` (was `middleware.js`); opt-in Cache Components. **React Router 8 framework mode** — `routes.ts`, route modules with `loader` / `clientLoader` / `action` / `ErrorBoundary`, `ssr` / `prerender` config, RSC experimental. **TanStack Start** — full-stack on TanStack Router, v1 release candidate, RSC experimental. **Choose**: public + SEO → Next.js or React Router framework; behind a login → Vite SPA + React Router + TanStack Query; mobile → Expo; mostly content → SSG/Astro.

16.16 Patterns

Need	Pattern	Shape	Watch out
Flexible UI structure	**Composition**	`children` + slot props	Prefer to growing lists of boolean props §14.1
Share **stateful logic**	**Custom Hook**	`useProductSearch(query)` → small API	Shares logic, not state; name `use…` only if it calls Hooks §7.3, §14.3
Separate data from looks	**Container / presentational**	Hook or Server Component + props-only view	Not by rule — where reuse/testing benefits §14.2
Parts sharing implicit state	**Compound components**	`<Tabs><Tabs.List><Tabs.Tab/></Tabs.List><Tabs.Panel/></Tabs>` via private context	Throw a clear error outside the root; support controlled + uncontrolled §14.4
Component decides where/how often caller markup renders	**Render prop**	`renderItem={(item) => …}`, `children` as a function	Callback pyramids; a Hook is simpler for logic only §14.5
Wrap many components with the same behaviour	**HOC**	`withAuth(Page)`	Create at module level (in render → remount every time); `displayName`; prop collisions §14.6
Escape `overflow`/stacking	**Portal**	`createPortal(node, document.body)`	Events bubble through the **React** tree; manage focus; read the latest `onClose` in the Escape-key Effect with `useEffectEvent`; consider native `<dialog>` §14.8
Contain crashes	**Error boundary**	Class with `getDerivedStateFromError` + `componentDidCatch` (or `react-error-boundary`)	Catches render/lifecycle/Effect errors below it, `use()` rejections and `lazy` failures — **not** event handlers, async callbacks, SSR or itself §14.9
Wait for code/data	**Suspense boundary**	`<Suspense fallback>`	Pair with an error boundary; transitions keep old UI §9.11
Organise files	**By feature**	`features/cart/index.js` public API; `app → features → shared`	Enforce with lint; avoid giant barrels §14.11
Consistency	**Design system**	Tokens → accessible primitives → docs; closed variants, `type="button"` default, pass-through props and `ref`	Build on headless primitives §14.12

React 19 root error options: `createRoot(el, { onCaughtError, onUncaughtError, onRecoverableError })`. With **no** boundary, an uncaught render error unmounts the **whole root** §14.9.

16.17 Accessibility checklist

§14.10, §7.8, §4.3

- ☐ **Semantic elements first**: `<button>` (not `<div onClick>`), `<a href>`, `<nav>`, `<main>`, headings in order, lists, `<form>` + `type="submit"` (Enter works for free).
- ☐ Every input has a **label**: `<label htmlFor={id}>` with `id` from `useId()` (never hard-coded in reusable components); placeholders aren't labels.

- [ ] Images have `alt` (empty `alt=""` for decoration); icon-only buttons have `aria-label` and `aria-hidden="true"` on the icon.
- [ ] ARIA driven by state and spelled with dashes: `aria-expanded={open}`, `aria-selected`, `aria-current`, `aria-controls`, `aria-invalid`, `aria-describedby`. First rule of ARIA: use a native element when one exists.
- [ ] **Live regions** (`role="status"` polite, `role="alert"` urgent) exist in the DOM *before* their text changes.
- [ ] **Focus management** with refs: into a dialog on open, trapped (or background `inert`), back to the trigger on close; to the first invalid field on a failed submit; to the new page's `<h1>` (`tabIndex={-1}`) after client-side navigation.
- [ ] Keyboard patterns: Escape closes, arrow keys move inside tabs/menus/listboxes (roving `tabIndex`), everything reachable with Tab, visible `:focus-visible` styles (never remove outlines without a replacement).
- [ ] Contrast ≥ 4.5:1 for normal text (WCAG 2.2), in both themes; don't signal errors by colour alone.
- [ ] Update `document.title` (React 19 lets any component render `<title>`).
- [ ] Test with `getByRole(role, { name })`, `eslint-plugin-jsx-a11y`, axe (`vitest-axe`, Playwright), and a real keyboard + screen reader.
- [ ] Complex widgets (menus, comboboxes, date pickers): use headless libraries (Radix, React Aria) rather than hand-rolling ARIA.

16.18 Common errors and fixes

Every message below appeared in the output of an earlier chapter's program. The React, Testing Library and props messages are reproduced verbatim by the check program at the top of this chapter (first sentence shown); the Hook-order warning comes from Chapter 7's run and the TypeScript errors from Chapter 11's `tsc` run.

Message (React 19.3, development)	Usual cause	Fix	Read
Too many re-renders. React limits the number of renders to prevent an infinite loop.	A setter called **during render** — often `onClick={setOpen(true)}` or `onClick={handle()}`	Pass a function: `onClick={() => setOpen(true)}` ; move logic to a handler	§3.1
Objects are not valid as a React child (found: object with keys {…}).	Rendering an object (`{user}` , a `Date` , a Promise)	Render a field `{user.name}` , format it, or `map` an array	§1.8
Each child in a list should have a unique "key" prop.	`map` without `key` , or key on the wrong element	`key={item.id}` on the outermost element returned by `map`	§2.8
Encountered two children with the same key, `1` .	Duplicate IDs, or keys built from non-unique fields	Use a truly unique, stable ID	§2.8
The tag <profile> is unrecognized in this browser.	Component name starts with a lowercase letter	Capitalise: `Profile`	§2.1
Invalid DOM property `class` . Did you mean `className` ?	HTML attribute names in JSX	`className` , `htmlFor` , camelCase events	§1.8
Unsupported style property @media (max-width: 600px).	Selectors/media queries in the `style` object	Use CSS (a class, CSS Module, Tailwind variant)	§8.3
You provided a `value` prop to a form field without an `onChange` handler.	Controlled `value` with no handler → read-only field	Add `onChange` , or use `defaultValue` , or `readOnly`	§3.9
A component is changing an uncontrolled input to be controlled.	`value` was `undefined` / `null` , later a string	Initialise with `''`	§4.2
Rendered more hooks than during the previous render. / Rendered fewer hooks than expected. This may be caused by an accidental early return statement.	A Hook inside an `if` , loop, or after an early `return`	Call every Hook unconditionally at the top; put the condition inside the Hook	§7.2
React has detected a change in the order of Hooks called by …	Same count, different Hook kinds between renders (conditional Hooks)	Same fix — stable call order	§7.2
Invalid hook call. Hooks can only be called inside of the body of a function component.	Hook called in a plain function, handler or module scope; calling a Hook outside `renderHook` in tests; **two copies of React** in the bundle	Call Hooks only in components/custom Hooks; dedupe React (`npm ls react`)	§7.2, §12.7
The result of getSnapshot should be cached to avoid an infinite loop → Maximum update depth exceeded.	`useSyncExternalStore` `getSnapshot` returns a new object each call (same with a Zustand selector returning a new object)	Return the same reference until the store changes; select primitives; `useShallow`	§7.9, §6.8
A component was suspended by an uncached promise.	`use(fetch(...))` — a new promise on every render	Create the promise once (cache, loader, Server Component prop) or use `useSuspenseQuery`	§9.11
Element type is invalid. Received a promise that resolves to: undefined. Lazy element type must resolve to a class or function.	`lazy(() => import('./X'))` where `X` has no **default** export	`.then(m => ({ default: m.X }))`	§10.8

Message (React 19.3, development)	Usual cause	Fix	Read
Hydration failed because the server rendered text didn't match the client.	Render output differs between server and client (`window` , dates, random, locale, invalid nesting)	Same first render; switch after mount; `useId` ; fix the HTML nesting	§13.5
An update to T inside a test was not wrapped in act(...).	Something updated state after the test stopped waiting (missing `await` , un-awaited timer or fetch)	`await findBy…` / `waitFor` ; `act` around direct Hook calls and fake-timer advances	§12.2, §12.5
React has blocked a javascript: URL as a security precaution. (in the rewritten `href`)	A `javascript:` URL in `href`	Validate URLs; use a button with a handler	§1.8
`TypeError: Cannot assign to read only property 'title' of object`	Mutating props (frozen in development)	Ask the parent via a callback, or use local state	§2.2
`Cannot read properties of null (reading 'price')` inside a boundary	Rendering before data exists / missing provider	Handle loading states; error boundary + reset keys; context Hook that throws a clear message	§14.9, §6.5
`useTasks must be used inside <TasksProvider>` *(your own message)*	Component rendered outside its provider	Wrap the tree (or the test) in the provider	§6.5, §12.8
RTL: *Unable to find an accessible element with the role "button" and name "…"*	Wrong name — or the element isn't accessible (a `<div onClick>` , an unlabelled icon button)	Read the printed role list; fix the query **or** the markup	§12.3, §14.10
RTL: *Found multiple elements with the role "button"*	Query not specific enough; DOM left over from a previous test	Add `{ name }` , use `within()` , run `cleanup` after each test	§12.3, §12.2
TS2322 *Type 'string' is not assignable to type 'never'.*	`useState([])` inferred `never[]`	`useState<string[]>([])`	§11.10
TS18047 *'user' is possibly 'null'.*	Using nullable state without narrowing	`user ? … : …` , `user?.name`	§11.10
TS2554 *Expected 1 arguments, but got 0.*	`useRef<HTMLInputElement>()` with React 19 types	`useRef<HTMLInputElement>(null)`	§11.10
TS2741 *Property 'label' is missing in type '{}' but required in type 'BadgeProps'.*	Required prop not passed	Pass it, or make it optional with a default	§11.10

Silent bugs (no message at all) — the ones to recognise by symptom:

Symptom	Cause	Fix
Counter goes up by 1 instead of 3	Snapshot + plain values	Updater functions §3.5
Screen doesn't update after "changing" state	Mutation, same reference	New object/array §3.6
Typed text jumps to another row	Index keys + insert/sort	Stable IDs §2.8
Input loses focus on every keystroke	Component defined inside a component, or random keys	Top-level components, stable keys §10.10
Effect runs twice in development	StrictMode double run	Correct cleanup §5.4
Old search results overwrite new ones	Fetch race	`AbortController` / ignore flag §5.7
`memo` child still re-renders	Inline object/function props, or context	Stable props, split/memoise context §10.4
Interval always sees the first value	Stale closure (`[]` deps)	Updater, or `useEffectEvent` §7.7
Changing a prop doesn't reset the child's state	Same type at same position keeps state	`key` §3.10
Clicks inside a modal trigger the parent's "click outside"	Portal events bubble through React	Check `e.target`, or stop propagation deliberately §14.8
Error in `onClick` doesn't show the fallback	Boundaries don't catch handler errors	`try/catch` → state §14.9
Page reloads on submit, state lost	No `preventDefault` / no function `action`	`e.preventDefault()` or `<form action={fn}>` §4.1
404 when refreshing `/products/3` in an SPA	Server doesn't fall back to `index.html`	Configure the fallback §9.3

16.19 Interview question index

The 53 questions of Chapter 15, grouped as there. Each link opens the section that holds the question; the question numbers are in bold there.

Area	Questions
Rendering, reconciliation, keys	Q1 what happens after a setter · Q2 render vs commit, which must be pure · Q3 reconciliation and "virtual DOM" · Q4 why keys, why not the index · Q5 changing a component's `key` · Q6 when a component re-renders · Q7 why `{items.length && …}` renders 0
State, events, forms	Q8 three `setCount(count + 1)` · Q9 batching and React 18 · Q10 immutability · Q11 derived state, props copied into state · Q12 where state should live · Q13 controlled vs uncontrolled · Q14 `useActionState`, `useFormStatus`, `useOptimistic` · Q15 React vs DOM events
Effects and refs	Q16 what `useEffect` is for · Q17 no array vs `[]` vs `[a, b]` · Q18 cleanup timing and Effect order · Q19 why things run twice in development · Q20 fetch race conditions · Q21 `useEffect` vs `useLayoutEffect` · Q22 `useRef` vs `useState` · Q23 refs to your own components in React 19 · Q24 `useEffectEvent`
Hooks, context, state management	Q25 Rules of Hooks · Q26 custom Hooks share state? · Q27 `useState` or `useReducer` · Q28 is Context state management? · Q29 Context performance · Q30 choosing an approach · Q31 server vs client state · Q32 `useSyncExternalStore`
Performance and the Compiler	Q33 `memo` · Q34 `useMemo` vs `useCallback` · Q35 React Compiler · Q36 investigating a slow page · Q37 code splitting · Q38 10,000 rows
Suspense, `use()`, transitions	Q39 what can suspend · Q40 `use()` · Q41 `useTransition` vs `useDeferredValue` · Q42 fallback on a suspending update
Server rendering and RSC	Q43 CSR/SSR/SSG/streaming · Q44 hydration and mismatches · Q45 Server vs Client Components · Q46 `'use server'` · Q47 `'use client'` on the root layout
Errors, portals, patterns, testing, a11y, TS	Q48 what error boundaries catch · Q49 portals and event bubbling · Q50 compound vs render props vs HOCs vs Hooks · Q51 Testing Library philosophy · Q52 accessible components · Q53 typing props, children, events

Machine-coding builds (each with requirements, state design, code, an RTL test and a screenshot): §15.10 todo list with filters · §15.11 autocomplete (debounce, abort, combobox keyboard) · §15.12 infinite scroll (`IntersectionObserver`) · §15.13 modal (portal, focus trap, Escape) · §15.14 accessible tabs (roving tabindex) · §15.15 star rating (native radios) · §15.16 pagination (pure range function) · §15.17 multi-step form wizard · §15.18 ten more prompts mapped to techniques. The method: requirements → component tree → state design → happy path → keyboard/a11y → test → edge cases §15.1.

16.20 Where to go next

Step 1 — Lock in the core (1–2 weeks). Rebuild three of the Chapter 15 builds from a blank file without looking, with tests. If you get stuck, the sheets in this chapter tell you which section to reread. Make sure you can explain snapshot + updaters, keys, Effects vs handlers and Context re-renders out loud — they are the most common interview topics.

Step 2 — Build one real app (2–4 weeks). Something with routing, server data, a form and auth — for example a reading list or a small shop:

- Vite + React Router data mode + TanStack Query + one form with `useActionState`, deployed as an SPA;
- then move it to a framework (React Router framework mode or Next.js App Router) and compare SSR, loaders vs Server Components, and the bundle size;
- add TypeScript (`strict`), Vitest + Testing Library tests, and turn on the React Compiler.

Step 3 — Go deeper where your job needs it.

Direction	Read / do next
Framework work	Next.js docs (App Router, Cache Components), React Router framework docs; Chapter 13 again
Performance	React DevTools Profiler on your own app; web.dev on Core Web Vitals (LCP, INP, CLS) and the last chapter of the HTML & CSS book (critical rendering path, Core Web Vitals); responsive images
Design systems / accessibility	WAI-ARIA Authoring Practices patterns; Radix / React Aria source; the HTML & CSS book's accessibility chapter
JavaScript depth	The JavaScript book: closures, async and the event loop, its cheat sheets
Front-end system design	Component architecture and machine coding in the LLD book: UI components case, machine coding
Mobile	Expo (React Native) — the same components-and-Hooks model on iOS and Android

Step 4 — Keep up to date. Primary sources, in this order: **react.dev** (Learn + Reference + the blog, where every release and deprecation is announced), the React GitHub releases, then the docs of the libraries you use (React Router, TanStack Query, Redux Toolkit, Zustand, Testing Library, Next.js). Check the date on any tutorial: anything using class components, Create React App, `ReactDOM.render`, `forwardRef` for new code, or `useFormState` predates the React 19 era this book teaches.

16.21 Glossary (A–Z)

Each term links to the section that explains it.

A

- **AbortController** — Browser object whose `signal` is passed to `fetch`; calling `abort()` cancels the request — the usual cleanup for fetches in Effects. §5.7
- **Accessibility (a11y)** — Making a UI usable by everyone, including people using screen readers, only a keyboard, zoom or voice control; "a11y" counts the 11 letters between a and y. §14.10
- **Accessibility tree** — The simplified version of the page the browser gives to assistive technology: each element's role, name and state. §15.14
- **Accessible description** — Extra text a screen reader reads after an element's name, usually linked with `aria-describedby` (e.g. a field's error). §14.10
- **Accessible name** — What assistive technology announces for an element — a button's text, an input's label, `aria-label` / `aria-labelledby`, an image's `alt`; what `getByRole(role, { name })` matches. §12.3
- **`act()`** — Test helper that makes React finish every pending render and Effect before the next line runs; RTL wraps its helpers in it, so you call it only for direct updates (e.g. `renderHook` calls, fake timers). §12.2
- **Action** — In React 19, a function run inside a Transition — passed to `<form action>`, `formAction`, or `startTransition(async …)`; React tracks its pending state. §4.4
- **Action (React Router)** — A route's function for writes; `<Form method="post">` calls it, then the router re-runs loaders (revalidation). §9.6
- **Action (reducer)** — A plain object describing what happened, e.g. `{ type: 'added', text }`, sent to a reducer with `dispatch`. §6.2
- **Action creator** — A function returning an action object; Redux Toolkit's `createSlice` generates them (`added('p1')` → `{ type: 'cart/added', payload: 'p1' }`). §6.7
- **`<Activity>`** — React 19.2 component that hides a subtree while keeping its state; one of the real remounts StrictMode's double run prepares you for. §5.4
- **Alt text** — The `alt` attribute: a text alternative read instead of an image (empty `alt=""` for decorative images). §14.10
- **`any`** — TypeScript type meaning "don't check this"; it switches checking off for everything it touches — prefer `unknown`. §11.1

- **API** — Application Programming Interface: the functions, props or URLs a library, component or server offers to the code that uses it. §11.2
- **App Router** — Next.js's routing system in the `app/` folder (since Next.js 13), built on Server Components, streaming and Server Functions. §13.9
- **ARIA** — Accessible Rich Internet Applications: HTML attributes (`role`, `aria-*`) that tell assistive technology what an element is and its state; use only when native HTML can't say it. §14.10
- **`aria-activedescendant`** — Attribute on a focused input naming the id of the highlighted option, so focus stays in the input while screen readers announce the option (combobox pattern). §15.11
- **`aria-describedby`** — Links an element to text that describes it (e.g. a field's error message), making that text its accessible description. §4.3
- **`aria-invalid`** — Attribute set to `true` on a field whose value failed validation, so screen readers announce it. §4.3
- **`aria-live` / live region** — An element (`role="status"` or `role="alert"`) whose text changes are announced; it must already exist before the text changes. §14.10
- **`as const`** — TypeScript suffix that keeps literal values and makes arrays readonly tuples, e.g. a Hook returning `[on, toggle] as const`. §11.7
- **Assertion** — A check in a test that fails it if a condition is false, e.g. `assert.equal(a, b)` or `expect(el).toBeDisabled()`. §12.2
- **Assistive technology** — Software or hardware that helps disabled people use computers — screen readers, voice control, switch devices. §12.3
- **Astro** — Content-site framework that ships static HTML with small interactive "islands" (which can be React components). §13.11
- **async / await** — JavaScript syntax for promises: an `async` function returns a promise, and `await` pauses it until a promise settles. §12.5
- **Auth guard** — A wrapper (component, HOC, loader or middleware) that shows its content only to signed-in or allowed users. §14.6
- **Authentication / authorisation** — Authentication checks *who* the user is (sign-in); authorisation checks *what* they may do. Server Functions must do both. §13.8
- **Automatic batching** — Since React 18, all state updates from one event — including in timeouts, promises and native listeners — are combined into one re-render. §3.4
- **Automatic JSX runtime** — The React 17+ compile mode that imports `jsx` from `react/jsx-runtime` for you, so files don't need `import React`. §1.8
- **axe** — Automated accessibility checker (axe-core) used as a browser extension or in tests (`vitest-axe`, `jest-axe`, Playwright). §14.10
- **axios** — A popular HTTP client library, an alternative to `fetch`. §12.6

B

- **Babel** — JavaScript compiler that translates JSX, TypeScript and modern syntax into plain JavaScript; the React Compiler is a Babel plugin. §12.9
- **Bail-out** — When a setter receives a value `Object.is`-equal to the current state, React skips re-rendering that component and its children. §3.2
- **Barrel file** — An `index.js` that re-exports a module's contents; one per feature is fine, giant ones slow tools and hurt tree-shaking. §14.11
- **`baseDuration`** — `<Profiler>` value: estimated time for a full re-render of the subtree — compare with `actualDuration` to see what memoization saves. §10.3
- **BEM** — Block__Element--Modifier CSS naming convention that avoids clashes in global stylesheets. §8.2
- **Big-O notation** — Shorthand for how work grows with input size: O(n) grows in step with n, O(n^3) with its cube. §1.3
- **Blur** — The event fired when an element loses focus (the opposite of `focus`). §15.10
- **Boilerplate** — Repetitive set-up code that must be written but adds little meaning. §6.7
- **Brittle test** — A test that breaks on harmless changes (renamed state, new class name) even though the feature still works. §12.10

- **`browser()`** — React 19.3 `react-dom` function for `use(browser())`: suspends only on the server, so the nearest `<Suspense>` fallback is sent and the client renders the real content — no hydration mismatch. §13.5
- **Build (build time)** — The step before publishing that bundles, compiles and optimises your code; "build time" means it happens then, not when a user requests a page. §1.5
- **Bundle / chunk** — The JavaScript file(s) a bundler produces for the browser; a chunk is one piece of a split bundle, loaded on demand. §10.8
- **Bundle analyser** — A tool that draws a map of what is inside your bundle (e.g. `rollup-plugin-visualizer`) so you can see which libraries take space. §10.8
- **Bundler** — Tool (Vite, which bundles with Rolldown since Vite 8; esbuild; webpack; Turbopack) that combines modules into files for the browser and splits chunks for `import()`. §10.8
- **Business logic** — The rules of the product itself (prices, permissions, validation) as opposed to UI code. §14.13

C

- **Cache** — A stored copy of a result kept so it can be reused instead of fetched or computed again. §6.9
- **Cache Components** — Next.js 16 opt-in model: everything is dynamic unless cached with `'use cache'`; the build prerenders a static shell (PPR). §13.9
- **Callback** — A function you pass to other code to be called later (`onClick`, `renderItem`, `.then(fn)`). §11.2
- **camelCase** — Writing compound names with capitals inside (`onClick`, `className`); JSX attributes and style properties use it. §1.8
- **Canary (release)** — React's pre-release channel where new APIs appear before they are stable. §4.5
- **Capture phase** — The first half of DOM event propagation, from the root down to the target; React handlers ending in `Capture` (`onClickCapture`) run then. §3.1
- **Cascade (CSS)** — CSS's rules for deciding which declaration wins when several match: origin, layer, specificity, then source order. §8.1
- **Cascade layers (`@layer`)** — A CSS feature for deciding which group of rules wins regardless of specificity. §8.1
- **CDN** — Content Delivery Network: caching servers around the world that serve files from a location near the user. §13.1
- **Children (`children` prop)** — Whatever is nested between a component's tags; render it with `{children}` to build wrappers. §2.3
- **CI** — Continuous integration: a server that automatically runs type checks, linters and tests on every push or pull request. §11.1
- **Class component** — Legacy component style (`class X extends Component` with `render()`); still required for error boundaries. §2.1
- **Clean Architecture** — A layering style where business rules don't depend on the UI, database or frameworks; VIPER applies it to iOS screens. §14.13
- **Cleanup function** — The function an Effect's setup returns; it undoes the setup and runs before the next setup and on unmount. §5.3
- **Client Component** — A component whose code ships to the browser (marked by a `'use client'` entry point); may use state, Effects and events. §13.7
- **Client-side routing** — Changing screens without a page load: the router intercepts links, updates the URL with the History API, and renders the matching route. §9.1
- **`clientLoader`** — React Router framework-mode export that loads a route's data in the browser instead of on the server. §13.10
- **Closure** — A function that remembers the variables of the render that created it — why handlers see a state snapshot. §3.3
- **`clsx`** — Tiny helper that builds a `className` string from strings, conditions and objects, dropping falsy values. §8.5
- **Code splitting** — Splitting the bundle into chunks loaded on demand, e.g. `lazy(() => import('./Chart'))`. §10.8

- **Codebase** — All the source code of a project. §14.1
- **Codemod** — An automated script that rewrites code for you, e.g. to migrate from `forwardRef` to `ref` as a prop. §5.10
- **Colocation** — Keeping state, files and tests next to where they are used, so changes stay local and re-renders small. §14.11
- **Combobox** — A text input paired with a popup list of options (autocomplete); has its own ARIA keyboard pattern. §15.11
- **Commit phase** — The step where React applies DOM changes, sets refs and runs layout effects; runs once per committed update. §10.1
- **Compiler** — A program that reads code and checks and/or translates it (e.g. `tsc`, Babel, the React Compiler). §11.1
- **Component** — A capitalised JavaScript function that takes props and returns JSX (or `null`). §1.7
- **Component instance** — One rendered use of a component at a position in the tree, with its own state. §3.10
- **Component library** — A package of ready-made components (styled kits like MUI, headless ones like Radix, or copy-in ones like shadcn/ui). §8.9
- **Component tree** — The hierarchy of components React renders; visible in React DevTools, not in the DOM. §1.7
- `ComponentProps<'button'>` — TypeScript helper giving every prop a native element (or `typeof MyComponent`) accepts, including `ref`. §11.3
- **Composite widget** — One control made of several focusable items (tabs, toolbar, radio group, menu); usually uses a roving tabindex. §15.14
- **Composition** — Building UIs by nesting components and passing JSX (`children`, slots) instead of inheritance or many boolean props. §2.4
- **Compound components** — A family like `Tabs` / `Tabs.Tab` / `Tabs.Panel` that shares state through a private context. §14.4
- **Concurrent rendering** — React 18+ ability to prepare UI in the background, pause it for urgent work, and discard stale work. §7.10
- **Container / presentational** — Splitting "where data comes from" (container, Hook or Server Component) from "how it looks" (props-only view). §14.2
- **Context** — `createContext` + `<Ctx value>` + `useContext` / `use`: passes a value to any depth without prop drilling; a transport, not a store. §6.3
- **Contextual typing** — TypeScript inferring a value's type from where it is used, e.g. an inline `onChange={e => …}` handler. §11.4
- **Controlled component / controlled input** — A component or form field whose value comes from props/state: `value={state}` + `onChange` (or `value` / `onValueChange` in design systems). §3.9
- **Cookie** — A small named value the browser stores and sends to the server with every request to that site. §13.5
- **Coverage** — Which lines of code ran during the tests; a guide to untested code, not a goal in itself. §12.9
- **CPU throttling** — A DevTools setting that slows the computer down on purpose to mimic a slow phone while you measure performance. §10.3
- **Crawler** — A search-engine or link-preview bot that downloads pages to read them; it may not run your JavaScript. §13.3
- **Create React App (CRA)** — The old official starter tool (2016); deprecated for new apps in February 2025 in favour of frameworks or Vite. §1.5
- `createRoot` — `react-dom/client` API that creates a React root for a DOM element; `root.render(<App />)` starts rendering. §1.6
- `createSlice` — Redux Toolkit function that generates a reducer and action creators; its reducers use Immer. §6.7
- **Cross-cutting concern** — A need that touches many unrelated parts of an app — auth, logging, analytics. §14.6
- **CSR / SPA** — Client-side rendering in a single-page application: one near-empty HTML page; JavaScript renders every screen and handles navigation in the browser. §1.4

- **CSS custom property** — A CSS variable (`--accent: #36c` , read with `var(--accent)`); the usual home for design tokens and themes. §8.8
- **CSS Modules** — `*.module.css` files whose class names are renamed per file, so `styles.card` can't clash. §8.4
- **CSS-in-JS** — Writing styles in JavaScript; runtime libraries (styled-components, Emotion) inject styles while rendering, zero-runtime ones extract CSS at build. §8.7
- **Cursor (pagination)** — A marker for "the last item I saw" used to request the next page (`?after=<id>`); stable when items are inserted. §15.12
- **Custom Hook** — A function named `useSomething` that calls other Hooks; shares logic, never state. §7.3

D

- **`dangerouslySetInnerHTML`** — The only way to insert a raw HTML string in React; bypasses escaping, so sanitise the HTML first. §1.8
- **Dark mode** — A light-on-dark colour theme, usually driven by CSS custom properties and the user's OS setting. §8.8
- **Data binding** — Connecting UI to data so one updates when the other changes; one-way (state → view, React) or two-way (both directions). §14.13
- **Data router** — React Router set-up (`createBrowserRouter` + `<RouterProvider>`) that enables loaders, actions and fetchers. §9.2
- **Debounce** — Wait until input has been quiet for N ms before acting (e.g. searching). §7.5
- **Declarative UI** — Describing *what* the screen should look like for the current state and letting React work out the DOM changes — the opposite of imperative step-by-step DOM code. §1.2
- **Decorator pattern** — Wrapping an object or function to add behaviour without changing it; HOCs are decorators for components. §14.6
- **Deduplication** — Merging identical simultaneous requests into one (TanStack Query, router loaders). §6.9
- **`defaultValue`** — Starting value of an uncontrolled input; the DOM owns the value afterwards. §4.2
- **Deno / Bun** — JavaScript runtimes that are alternatives to Node.js. §13.2
- **Dependency (package)** — A library your project relies on, listed in `package.json` and installed into `node_modules` . §1.5
- **Dependency array** — The second argument of `useEffect` / `useMemo` / `useCallback` : every reactive value the callback reads; compared with `Object.is` . §5.5
- **Dependency injection** — Passing the things code relies on (a `search` function, an API client) in as parameters, so tests can hand in fakes. §14.3
- **Deploy** — Publishing a new version of an app to the servers or CDN that real users reach. §13.1
- **Deployment adapter** — A small package that makes a framework app run on a particular hosting platform. §13.10
- **Deprecated** — Still works but marked as outdated and likely to be removed; avoid in new code. §11.4
- **Derived state** — A value computed from props/state during render instead of being stored; never kept in `useState` . §3.8
- **Design pattern** — A named, reusable solution to a common design problem (decorator, observer, strategy…). §14.6
- **Design system** — Tokens + accessible primitives + usage docs shared across a product. §14.12
- **Design tokens** — Named design values (colours, spacing, radii) usually stored as CSS custom properties. §8.8
- **Destructuring** — JavaScript syntax that unpacks properties or array items into variables: `function Card({ title })` , `const [a, b] = arr` . §2.2
- **Deterministic** — Giving the same result every run — a property good tests need (no real network, time or randomness). §12.10
- **Dev server** — The local server you run while developing (`npm run dev`); serves your code with instant updates (HMR). §1.5
- **`<dialog>`** — Native HTML modal element; `showModal()` puts it in the top layer, makes the page inert, closes on Escape and restores focus. §14.8

- **Directive** — A string literal at the top of a file or function (`'use client'` , `'use server'`) that bundlers read. §13.8
- **Discriminated union** — A TypeScript union of object types sharing a literal field (e.g. `status`) that narrows to one member. §11.9
- **Dispatch** — Sending an action to a reducer: `dispatch({ type: 'added' })` ; stable identity. §6.2
- **Docker** — A tool that packages an app with everything it needs into a portable container that runs the same anywhere. §13.11
- **DOM** — Document Object Model: the browser's tree of objects representing the page, which React updates for you. §1.3
- **Double fetch** — When the client refetches data the server already used for SSR; frameworks serialize data into the page instead. §13.4
- **Dynamic segment** — Part of a route path that matches any value (`:id` in React Router, `[id]` in Next.js), read with `useParams()` / `params` . §9.3

E

- **E2E test** — End-to-end test: drives the whole app in a real browser (Playwright, Cypress); slow, so used for a few critical flows. §12.1
- **Edge case** — An unusual situation at the limits — empty data, errors, huge lists, very fast input — that interviewers probe. §15.1
- **Edge runtime** — A lightweight JavaScript server environment running in many locations close to users (e.g. Cloudflare Workers). §13.2
- **Effect** — Code in `useEffect` that runs after commit to synchronise a component with an external system. §5.2
- **Effect Event** — A function created by `useEffectEvent` : called from Effects, always sees the latest props/state, and is not a dependency. §5.5
- **Encapsulation** — Hiding a component's internals and exposing only a chosen interface (e.g. `useImperativeHandle` instead of the raw DOM node). §5.10
- **Endpoint** — One URL on a server that performs one operation (e.g. `POST /api/reviews`); every Server Function is a public one. §13.8
- **Entry point** — The file where a program or bundle starts (`main.jsx`), or a module where client code begins (`'use client'`). §1.6
- **Error boundary** — A class component with `getDerivedStateFromError` / `componentDidCatch` that shows a fallback when a descendant throws while rendering. §14.9
- **esbuild / Oxc / SWC** — Very fast compilers (written in Go or Rust) that bundlers use to strip TypeScript types and transform JSX; they don't type-check. §11.1
- **Escape hatch** — A lower-level tool for when the automatic way doesn't fit (refs, Effects, manual `useMemo` with the Compiler). §15.6
- **Escaping / sanitising** — Escaping turns `<` into text so it can't run (JSX does it for every string); sanitising strips dangerous parts from HTML you must insert raw. §1.8
- **ESLint** — The standard JavaScript linter; `eslint-plugin-react-hooks` and `eslint-plugin-jsx-a11y` add React rules. §7.2
- **ESM** — ES modules: JavaScript's standard `import` / `export` module format (as opposed to older CommonJS `require`). §13.10
- **Event bubbling** — An event travelling from the target element up through its ancestors; `stopPropagation()` ends it. React events bubble through the React tree, even out of portals. §3.1
- **Event delegation** — React attaches one listener per event type at the root container and dispatches to your handlers. §3.1
- **Event handler** — A function passed as an `on…` prop; the right place for side effects caused by a user action. §3.1
- **Event loop** — The JavaScript mechanism that runs one task at a time and then picks the next queued callback (timers, promises, events). §15.1
- **`exhaustive-deps`** — `eslint-plugin-react-hooks` rule that checks dependency arrays of Effects, `useMemo` and `useCallback` . §5.5

- **Exhaustiveness check** — Assigning a value to `never` in a `default` branch so an unhandled union case becomes a compile error. §11.5
- **Expo** — React Native framework for building iOS, Android and web apps with React. §13.11
- **Exponential backoff** — Retrying a failed request after waits that grow each time (1 s, 2 s, 4 s…); TanStack Query does this by default. §6.9
- **Express** — A popular Node.js web-server library. §13.2
- **External store** — Data that lives outside React and changes on its own (browser APIs, Zustand, Redux); read it with `useSyncExternalStore`. §7.9
- **External system** — Anything React doesn't control — network, timers, browser APIs, third-party widgets; what Effects synchronise with. §5.1

F

- **Fake timers** — Test replacements for `setTimeout` / `setInterval` that you advance by hand, so time-based code runs instantly. §12.7
- **Fallback** — What `<Suspense>` (a spinner, a skeleton) or an error boundary (an error message) shows instead of its children. §9.11
- **Falsy** — Values JavaScript treats as false: `false`, `0`, `''`, `null`, `undefined`, `NaN` — why `{0 && …}` renders 0. §2.7
- **Fast refresh** — React's HMR flavour: edits to a component appear instantly while its state is kept where possible. §1.5
- **Feature flag** — A switch that turns a feature on or off for some users without deploying new code. §14.6
- **`fetch`** — The browser's built-in function for HTTP requests; returns a promise of a `Response` and does **not** reject on 404/500 — check `res.ok`. §5.7
- **Fetcher** — React Router `useFetcher()`: submit or load without navigating (like buttons, inline edits). §9.6
- **Fiber** — React's internal record for each component instance, holding its Hook slots; also the name of the reconciler since React 16. §7.1
- **`findBy…` / `waitFor`** — Async Testing Library helpers: `findBy` retries a query until an element appears, `waitFor` retries any assertion (default 1000 ms). §12.5
- **`fireEvent`** — Testing Library helper that dispatches exactly one DOM event; prefer `user-event` for user actions. §12.2
- **Flaky test** — A test that sometimes passes and sometimes fails without any code change — usually from real timers, network or shared state. §12.5
- **Flamegraph** — Profiler chart where each bar is a component and its width shows how long it took to render. §10.3
- **`flushSync`** — `react-dom` function that commits a state update synchronously, opting out of batching; use sparingly. §3.4
- **Flux** — Facebook's 2014 one-way data-flow architecture (action → dispatcher → store → view) that inspired Redux. §14.13
- **Focus** — Which element currently receives keyboard input; moved with Tab or with `element.focus()`. §14.10
- **Focus management** — Moving keyboard focus in code when the UI changes — into a dialog, back to its trigger, to a new heading or the first invalid field. §14.10
- **Focus trap** — Keeping Tab and Shift+Tab cycling inside an open modal so keyboard users can't wander behind it. §15.13
- **`:focus-visible`** — CSS selector for showing a focus ring only when it helps (e.g. keyboard use); never remove outlines without it. §14.10
- **`<Form>` (React Router)** — Router form that serialises fields, calls the route `action`, then revalidates loaders. §9.6
- **Form Action** — React 19 `<form action={fn}>`: React prevents navigation, calls `fn(formData)` in a Transition, and resets uncontrolled fields after success. §4.4
- **`formAction`** — Prop on a submit button that overrides the form's action for that button. §4.4

- **`FormData`** — Browser object holding a form's name/value pairs; values are strings, unchecked boxes are missing, `getAll` for repeated names. §4.1
- **Formik** — An older, controlled-input form library; React Hook Form is the common modern choice. §4.9
- **`forwardRef`** — Legacy API to pass a ref to a function component; React 19 passes `ref` as a normal prop instead. §5.10
- **Fragment** — `<>…</>` or `<Fragment key>`: groups children without adding a DOM node. §1.8
- **Fragment ref** — React 19.3: a `ref` on `<Fragment>` gives an instance with `focus()`, `addEventListener`, `observeUsing` for its children. §5.10
- **Framework** — A library that also decides your project's structure and tooling — for React: Next.js, React Router framework mode, Expo. §1.5
- **Framework mode** — React Router's full-stack mode: route modules, SSR/prerendering, per-route code splitting, type generation. §9.7
- **Front end / back end** — The front end is the code that runs in the browser (the UI); the back end is the server side — APIs, databases, business logic. §15.5
- **Full-stack** — Covering both the server (back end) and the browser (front end) parts of an app. §13.11

G

- **`gcTime`** — defined in the Routing and TanStack Query sheet, §16.10.
- **Generic component** — A component with a type parameter, e.g. `List<T>`, whose `T` is inferred from props. §11.8
- **Generics (TypeScript)** — Types with a placeholder parameter (`<T>`) filled in where they're used, so one function or component works for many types and stays checked. §11.8
- **`getDerivedStateFromError`** — Static error-boundary method that returns state so the next render shows the fallback. §14.9
- **`getServerSnapshot`** — Third argument of `useSyncExternalStore`: the value used on the server and during hydration. §7.9
- **git / repository** — git is the version-control tool that records a project's history; a repository (repo) is the folder it tracks. §1.5
- **gzip** — A compression format servers use to shrink files in transit; bundle sizes are often quoted "gzipped". §1.1

H

- **Happy path** — The normal flow where everything goes right; build it first, then handle edge cases. §15.1
- **Headless browser** — A real browser (e.g. Chrome) run without a window, controlled by a script, for tests and screenshots. §13.4
- **Headless UI library** — Components that provide behaviour and accessibility but no styles (Radix, React Aria, Headless UI, Base UI). §8.9
- **Higher-order component (HOC)** — A function that takes a component and returns a wrapped one (`withAuth(Page)`); must be created at module level. §14.6
- **HMR** — Hot Module Replacement: the dev server swaps an edited module in the running page, keeping state. §1.5
- **Hook** — A function starting with `use` that lets a component use a React feature; called at the top level only. §7.1
- **Host element** — A built-in element such as `<div>` or `<button>` (lowercase in JSX), as opposed to your own components. §2.1
- **HTML** — HyperText Markup Language: the tags (`<button>`, `<p>`) that describe a page's structure; React produces it for you. HTML & CSS Ch 1
- **HTTP** — The request/response protocol browsers and servers use; each response has a status code (200 OK, 404 Not Found, 500 error). HTML & CSS Ch 1
- **HTTP headers** — Key–value information sent before a request's or response's body (content type, cookies, caching rules). §13.3

- **HTTP methods (GET / POST)** — The verb of an HTTP request: GET reads data, POST submits data (forms, Server Function calls). §4.1
- **Hydration / `hydrateRoot`** — React in the browser adopting server-rendered HTML: `hydrateRoot` (`react-dom/client`) renders the same tree, reuses the existing DOM instead of creating it, and attaches handlers. §13.4
- **Hydration mismatch** — The client's first render differs from the server HTML; React client-renders that tree and reports a recoverable error. §13.5

I

- **iframe** — An HTML element that embeds another page inside the current one. §14.8
- **Immer** — Library that lets you "mutate" a draft and produces a new immutable state with structural sharing; used by Redux Toolkit. §3.6
- **Immutability / mutation** — Mutating changes a value in place; React state must be immutable — create new objects/arrays so `Object.is` sees the change (a mutated object is the same reference, so the change is hidden). §3.6
- **Imperative code** — Code that gives step-by-step commands ("create this node, set this text") instead of describing the result. §1.2
- **Implementation details** — How code works inside (state names, Hook calls, class names), as opposed to what users see; good tests ignore them. §12.1
- **Import cycle** — Files importing each other in a loop (A → B → A); a sign of a misplaced shared piece. §14.11
- **Impression** — An analytics record that something was shown to the user — one of the few things legitimately logged from an Effect. §5.6
- **In flight** — A request that has been sent but not yet answered. §15.4
- **Index route** — A child route with `index` that renders at the parent's exact URL. §9.4
- **`index.html` / `main.jsx`** — In a Vite app, the HTML page with the root `<div>` and the script that calls `createRoot(...).render(<App />)`. §1.6
- **Inert** — State (the `inert` attribute, or a modal `<dialog>`) in which part of the page can't be clicked, focused or read by screen readers. §14.8
- **Infinite scroll** — Loading the next page automatically as the user nears the bottom, usually with an `IntersectionObserver` sentinel; `useInfiniteQuery` manages the pages. §15.12
- **Integration test** — A test of several units working together — in React, usually a component tree rendered with RTL. §12.1
- **`IntersectionObserver`** — Browser API that reports when an element enters or leaves a scroll area; used for infinite scroll and lazy-loading. §15.12
- **Invalidation** — Marking cached queries stale so they refetch: `queryClient.invalidateQueries({ queryKey })`. §9.10
- **`isFetching`** — defined in the Routing and TanStack Query sheet, §16.10.
- **Island** — A small interactive component inside an otherwise static HTML page (Astro's model). §13.11
- **`isPending`** — TanStack Query: no data yet. `useTransition` / `useActionState`: the transition or Action is still running. §9.8
- **ISR** — defined in the rendering and directives sheet, §16.15.

J

- **`javascript:` URL** — A link whose `href` runs code; React 19 blocks it as a security precaution. §1.8
- **Jest / Vitest** — JavaScript test runners that find test files, run them and report results; Vitest is the natural choice for Vite projects. §12.9
- **jest-dom** — `@testing-library/jest-dom`: readable DOM matchers such as `toBeInTheDocument()` and `toBeDisabled()`. §12.2
- **Jotai / MobX / Valtio / Recoil** — Other state libraries: Jotai (small independent "atoms"), MobX and Valtio (observable/proxy objects you mutate), Recoil (atoms; no longer maintained). §6.10
- **jsdom** — A JavaScript implementation of the DOM used to run component tests in Node; no layout or painting. §12.9

- **JSON** — JavaScript Object Notation: the text format for sending data (`{"id": 1}`); `JSON.stringify` and `JSON.parse` convert to and from it. §5.7
- **JSX** — HTML-like syntax in JavaScript that compiles to `jsx()` / `createElement` calls returning element objects. §1.8

K

- **Key** — A string/number that identifies an element among its siblings; also resets a component when it changes. §2.8
- `keyof` — TypeScript operator giving the union of an object type's property names, e.g. for a table's column keys. §11.8

L

- **Landmark** — A page region with a role (`<header>` , `<nav>` , `<main>` , `<footer>`) that screen-reader users can jump between. §14.10
- **Layout (route)** — A parent route that renders shared UI and an `<Outlet />` for the matching child. §9.4
- **Layout shift** — Content jumping around as images, fonts or data load; measured by the CLS Core Web Vital. §13.9
- `lazy` / **lazy-loading** — Lazy-loading loads something (code, an image) only when about to be needed; `lazy(() => import('./X'))` is a component whose code loads on first render — needs a default export and a `<Suspense>` above it. §10.8
- **Lazy initialiser** — A function passed to `useState` / `useReducer` that React calls only on the first render to compute the starting state. §3.2
- **LCP / FCP** — Largest / First Contentful Paint: when the main content / anything at all first appears on screen — loading-speed metrics. §13.1
- **Legacy** — Old but kept for compatibility (class components, `renderToString` , `forwardRef`); avoid in new code. §13.2
- **Lifecycle methods** — Class-component methods React calls at set moments (`componentDidMount` , `componentDidCatch` …); Hooks replaced them except for error boundaries. §14.9
- **Lifting state up** — Moving shared state to the closest common parent and passing it down with a callback. §3.7
- **Lighthouse** — Chrome's built-in audit that scores performance, accessibility, SEO and best practices. §14.10
- **Linter** — A tool that reads code without running it and flags likely bugs and bad patterns (ESLint). §7.2
- **Listbox** — An ARIA role for a list of selectable options, as in a combobox's popup. §15.11
- **Literal type** — A TypeScript type that is one exact value (`'info'` , `42`); unions of them describe allowed variants. §11.2
- **Loader** — React Router data-mode function that loads a route's data before it renders; read with `useLoaderData()` . §9.5
- **Local mutation** — Mutating values created during the same render (a local array) — allowed inside a pure component. §2.5
- **Locale** — Language and region settings (`en-US` , `de-DE`) that decide date, number and currency formats. §13.5
- `localStorage` / `sessionStorage` — Browser key-value string storage; `localStorage` survives reloads, `sessionStorage` is cleared when the tab closes. Not available on the server. §7.4
- **Lockfile** — `package-lock.json` (or the pnpm/yarn equivalent): records exact installed versions so every machine gets the same ones; commit it. §1.5
- **LTS** — Long-Term Support: a Node.js release line that gets fixes for years; the recommended version to install. §1.5

M

- **Main thread** — The single browser thread that runs JavaScript, layout and painting; long work there makes the page unresponsive. §7.4
- **Matcher** — An assertion helper such as `toBeDisabled()` or `toHaveTextContent()` (jest-dom). §12.2

- **Media query** — CSS rule that applies only under conditions such as screen width (`@media (max-width: 600px)`); not possible in the `style` prop. §8.3
- `memo` — Wraps a component so it skips re-rendering when its parent re-renders with shallow-equal props. §10.4
- **Memoization** — Caching a function's result for given inputs so repeated calls reuse it; React does it with `memo` , `useMemo` , `useCallback` and the React Compiler. §10.5
- `MemoryRouter` — React Router router that keeps history in memory instead of the address bar — used in tests. §12.8
- **Middleware (React Router)** — Functions that run before a route's loaders/actions and can wrap them with `await next()` ; on by default in v8. §9.7
- **Middleware (Redux)** — Code between `dispatch` and the reducer (thunks, logging). §6.7
- **Minification** — Shrinking code by removing spaces, comments and long names; part of a production build. §13.5
- **Mock** — A controllable fake that replaces a real dependency (network, timer, module) in a test and records how it was called. §12.6
- **Modal** — A dialog that sits on top of the page and blocks the rest until closed; needs a focus trap, Escape and focus restore. §14.8
- **Module** — One JavaScript file with its own scope that shares values through `export` and `import` . §13.7
- **Module graph** — The web of files importing one another; `'use client'` marks where the client part of it starts. §13.8
- **Monorepo** — One repository holding several apps or packages, with tools to build and enforce boundaries between them. §14.11
- **Mount / unmount / remount** — A component instance being created and inserted into the tree / removed from it (running every cleanup); a remount replaces it with a fresh instance, losing state — caused by a new `key` or component type. §5.2
- **MPA** — Multi-page application: every navigation loads a new HTML page from the server. §1.4
- **MSW** — Mock Service Worker: intercepts HTTP requests at the network level for tests and development. §12.6
- **Mutation (TanStack Query)** — A server write run with `useMutation` ; not cached by key; followed by invalidation or a cache update. §9.10
- **MVC** — Model–View–Controller: a controller handles input and updates the model, and the view observes the model. §14.13
- **MVI** — Model–View–Intent (Android/Kotlin): MVVM with one immutable state object and reducer-style updates — Redux on mobile. §14.13
- **MVP** — Model–View–Presenter: a presenter does the work and pushes display data into a passive view through an interface. §14.13
- **MVVM** — Model–View–ViewModel: the view binds to observable state on a ViewModel, which never references the view; in React a custom Hook plays that role. §14.13

N

- **Narrowing** — TypeScript shrinking a broad type after a check — after `if (user)` or `switch (action.type)` it knows the specific type. §11.5
- `never` — TypeScript type with no possible values; used for exhaustiveness checks and `prop?: never` to forbid a prop. §11.5
- **Next.js** — Vercel's full-stack React framework; its App Router is the most complete Server Components implementation. §13.9
- **Node.js** — The program that runs JavaScript outside a browser — dev servers, build tools, tests and SSR run on it. §1.5
- `node_modules` — The folder where npm installs dependencies; never commit it (commit the lockfile instead). §1.5
- **Non-null assertion (`!`)** — TypeScript `value!` claims a value isn't `null` / `undefined` without checking; narrow instead. §11.10

- **Normalised state** — Storing each item once by ID and referring to it by ID elsewhere, like database tables, to avoid duplicated data. §15.18
- `noValidate` — Form attribute that turns off the browser's validation bubbles while keeping the `validity` API. §4.3
- **npm** — Node's package manager: installs libraries listed in `package.json` (alternatives: pnpm, yarn, bun). §1.5
- `npx` — Runs a package's command (e.g. `npx create-next-app`) without installing it globally first. §13.9

O

- `Object.is` — The comparison React uses for state, dependencies and `memo` props: same value for primitives, same reference for objects. §3.6
- **Observer pattern** — Objects subscribe to a subject and are notified when it changes — the idea behind stores, `useSyncExternalStore` and MVC views. §14.13
- **Offset pagination** — Asking for "skip N, take M" items; allows jumping to page 17 but shifts when items are inserted (compare cursor pagination). §15.16
- **OOP** — Object-oriented programming: organising code as objects (classes) that combine data and behaviour. §14.6
- **Optimistic update** — Showing the expected result before the server confirms it, rolling back on failure (`useOptimistic`, `onMutate`). §4.7
- **Optional chaining (`?.`)** — JavaScript operator that stops and returns `undefined` if the value before it is `null` / `undefined`: `onSelect?.(id)`, `user?.name`. §11.2
- **Origin** — Protocol + domain + port (`https://shop.example`); browser storage and many security rules are per origin. §7.4
- **ORM** — Object-relational mapper: a library that maps database rows to objects in code. §13.7
- **OTP** — One-time password: a short code sent by SMS or email; `autoComplete="one-time-code"` lets phones fill it. §15.18
- **Outlet** — `<Outlet />`: where a layout route renders its matched child route. §9.4
- **Overscan** — Extra rows rendered above and below the visible window of a virtualized list to avoid blank gaps. §10.9

P

- `package.json` — The project file listing its name, scripts (`npm run dev`) and dependencies. §1.6
- **Pagination** — Showing a long list one page at a time (page numbers, or a cursor for "next"); can be client-side or server-side. §15.16
- **Paint** — The browser drawing pixels on screen after layout; `useLayoutEffect` runs before it, `useEffect` usually after. §5.11
- **Partial prerendering (PPR)** — defined in the rendering and directives sheet, §16.15.
- **Path alias** — A short import name mapped to a folder (`@/features/cart` instead of `../../../features/cart`). §14.11
- **Payload** — The data carried by a request, response or action (e.g. the JSON body of a POST). §12.10
- **Pending UI** — What the user sees while something loads: old content + indicator, skeletons, or disabled buttons. §9.12
- **Performance panel** — Chrome DevTools tab that records everything the browser did over time; React 19.2+ adds React tracks to it. §10.3
- **Polling** — Asking the server again every few seconds to see if data changed. §12.5
- **Polymorphic component** — A component that can render as a different element (an `as="a"` prop, or Radix's `asChild`). §14.12
- **Popover** — A small floating panel anchored to an element (menus, tooltips); the native Popover API (`popover` attribute) is one way to build it. §14.8
- **Portal / `createPortal`** — `react-dom` API rendering children into another DOM node while keeping them in the same React tree, so context and events still follow the React tree. §14.8
- **Position (in the tree)** — Where a component sits among its parent's children; React keys state to type + position + key. §3.10

- **Prefetch** — Loading code or data before the user asks for it (on hover, or when a link scrolls into view). §13.9
- **Preflight** — Tailwind's CSS reset, which removes default styles such as heading sizes. §8.6
- **Prerender** — Rendering HTML ahead of time; `react-dom/static` `prerender*` waits for all Suspense data. §13.6
- **`preventDefault()`** — Event method that stops the browser's built-in action — a form submit reloading the page, a link navigating, Home scrolling. §3.1
- **Primitive (value)** — A simple value — string, number, boolean, `null`, `undefined`, bigint, symbol — compared by value, unlike objects compared by reference. §5.5
- **Production build** — The optimised version of an app real users get: minified, without development checks; always measure performance on it. §10.3
- **Profiler** — React DevTools tab (and `<Profiler onRender>` component) that records which components rendered, why, and how long. §10.3
- **Progressive enhancement** — The basic feature works with plain HTML; JavaScript makes it nicer (forms with Server Functions work before hydration). §4.8
- **Promise** — An object standing for a result that arrives later; it resolves with a value or rejects with an error. §9.11
- **Prop drilling** — Passing a prop through components that don't use it, only to reach a deep child; fixed by composition or context. §2.4
- **Props** — The read-only object of inputs a parent passes to a component. §2.2
- **`PropsWithChildren<P>`** — TypeScript helper that adds `children?: ReactNode` to a props type. §11.3
- **Provider** — The component that supplies a context value (`<Ctx value={…}>`) to everything inside it; consumers read the nearest one. §6.3
- **Proxy** — A server in the middle that forwards (and may change) requests and responses; also Next.js 16's `proxy.js`, code that runs before routes. §13.5
- **Pseudo-class** — A CSS selector for a state, such as `:hover` or `:focus-visible`; not available in the `style` prop. §8.3
- **Pure function / pure component** — Output depends only on inputs and nothing outside changes; a pure component returns the same JSX for the same props, state and context (reducers, validators and `getPageRange` are pure functions). §2.5

Q

- **Query function** — TanStack Query `queryFn`: async function that returns data or throws (check `res.ok`). §6.9
- **Query key** — Array that identifies cached data, e.g. `['product', id]`; every input of the query function belongs in it. §9.9
- **`QueryClient`** — defined in the Routing and TanStack Query sheet, §16.10.
- **`queryOptions`** — defined in the Routing and TanStack Query sheet, §16.10.

R

- **Race condition (fetch)** — A slower, older response arriving last and overwriting newer data; fixed with an ignore flag or `AbortController` in cleanup. §5.7
- **Radix UI / React Aria** — Headless component libraries (Radix by WorkOS, React Aria by Adobe) that supply keyboard, focus and ARIA behaviour without styles. §14.12
- **Rate limit** — A cap on how many requests a client may send per time window; debounce and caching help stay under it. §15.11
- **React Compiler** — Build-time Babel plugin (1.0 since October 2025) that inserts memoization automatically. §10.7
- **React DevTools** — Browser extension to inspect the component tree, props, state and profile renders. §10.3
- **React element** — A plain immutable object (`{ type, props, key }`) describing a piece of UI; what JSX creates. §1.3
- **React Hook Form** — Form library that manages big forms with uncontrolled inputs and refs. §4.9

- **React Native** — React for real iOS and Android apps, rendering native views instead of DOM elements. §13.11
- **React Redux** — The bindings (`<Provider>` , `useSelector` , `useDispatch`) that connect a Redux store to React components. §6.7
- **React Router** — The most used React router; v8 has declarative, data and framework modes and ships as one `react-router` package. §9.2
- **React Testing Library (RTL)** — Helpers to render components into jsdom, query them like a user (by role, label, text) and assert on the DOM. §12.1
- **Reactive value** — Anything declared in the component body that can change between renders: props, state, and values computed from them. §5.2
- **`ReactNode`** — TypeScript type for anything React can render (elements, strings, numbers, `null` , arrays); the type for `children` . §11.3
- **Read-your-own-writes** — Guarantee that after you save something, your next read shows it (Next.js `updateTag`). §13.9
- **Reconciliation** — Comparing the new element tree with the previous one to find the minimal DOM changes. §1.3
- **Recursive component** — A component that renders itself for each nested level (file trees, comment threads). §15.18
- **Redirect** — Sending the user to another URL (`redirect('/list')` in a loader/action, `redirect()` in Next.js). §9.6
- **Reducer** — A pure function `(state, action) => nextState` . §6.2
- **Redux** — A state-management library with one central store changed by dispatching actions to reducers; write it with Redux Toolkit. §6.7
- **Redux Toolkit (RTK)** — The official way to write Redux: `createSlice` , `configureStore` , `createAsyncThunk` , RTK Query. §6.7
- **Ref** — A `{ current }` object from `useRef` that persists across renders; changing it doesn't re-render. §5.8
- **Ref callback** — A function passed as `ref` ; called with the node on attach; in React 19 may return a cleanup. §5.9
- **Refactor** — Restructuring code without changing what it does; good tests keep passing through it. §12.1
- **Reference (identity)** — For objects, *which* object it is in memory; two equal-looking objects are different references, so `Object.is` says they differ. §3.6
- **Regular expression (regex)** — A pattern for matching text, e.g. `/save/i` (case-insensitive "save"). §12.3
- **Remix** — A React framework that merged into React Router; React Router framework mode is its successor. §13.10
- **Render (phase)** — React calling component functions to compute the next element tree; must be pure; may repeat or be discarded. §10.1
- **Render prop** — A prop whose value is a function returning JSX, called by the component to decide what to render. §14.5
- **Renderer** — The package that turns React's element tree into a real output: `react-dom` for browsers, React Native for mobile. §1.1
- **`renderHook`** — RTL helper that runs a Hook inside a hidden test component; read `result.current` . §12.7
- **Rendering strategy** — Where and when a page's HTML is produced: CSR, SSR, SSG, ISR, streaming, PPR — chosen per route. §13.1
- **`renderToPipeableStream`** — defined in the rendering and directives sheet, §16.15.
- **`renderToString`** — defined in the rendering and directives sheet, §16.15.
- **Reserved word** — A word the language keeps for itself (`class` , `for`), so JSX uses `className` and `htmlFor` instead. §1.8
- **Reset keys** — Values that, when changed, clear an error boundary's error state. §14.9
- **`ResizeObserver`** / **`matchMedia`** — Browser APIs that report an element's size changes / whether a CSS media query matches; missing in jsdom. §12.9

- **`revalidatePath`** / **`revalidateTag`** — Next.js functions that mark cached server-rendered data stale after a mutation so the next request shows fresh data. §13.9
- **Revalidation** — Re-running loaders (React Router) or refetching/regenerating cached data so the UI shows fresh data. §9.5
- **Role (ARIA)** — What kind of thing an element is to assistive technology — `button`, `link`, `dialog`, `tab` …; the first thing `getByRole` matches. §12.3
- **Route** — A URL pattern plus the screen (and data) shown for it. §9.1
- **Route module** — In React Router framework mode, one file per route exporting `loader`, `action`, `meta`, `ErrorBoundary` and the component. §13.10
- **Router** — The library that maps URLs to screens and handles navigation (React Router, Next.js routing). §9.1
- **Roving tabindex** — Only the active item in a group has `tabIndex=0`; arrow keys move focus within the group. §14.4
- **RSC payload** — The serialized output of Server Components (rendered elements + client references + props) sent to the browser. §13.7
- **RTK Query** — Redux Toolkit's data-fetching and caching layer, similar in purpose to TanStack Query. §6.7
- **Rules of Hooks** — Call Hooks only at the top level of components/custom Hooks, and only from React functions. §7.2
- **Rules of React** — Components and Hooks are pure, props/state are immutable, Hooks follow their rules — assumptions the React Compiler relies on. §10.7
- **Runtime** — Two meanings: *when* the program is running (as opposed to compile/build time), and the environment that runs it (Node.js, a browser, an edge runtime). §11.7

S

- **Sandbox (code)** — An online editor that runs your code in the browser, often used in interviews. §15.1
- **Sass** — A CSS preprocessor that added variables and nesting before CSS had them; still common, no longer necessary. §8.1
- **Scheduler** — React's internal part that decides which pending work to run first (urgent updates before transitions). §10.3
- **Schema** — A description of the expected data shape that a library such as Zod checks real data against at runtime. §4.9
- **Scope (CSS)** — Limiting which elements a style rule can affect; plain CSS is global, CSS Modules and CSS-in-JS scope automatically. §8.1
- **Screen reader** — Software that reads the page aloud and lets blind or low-vision users navigate it by headings, landmarks and controls. §14.10
- **Search params** — The `?key=value` part of a URL; read/write with `useSearchParams()` for shareable UI state. §9.3
- **Secrets** — Values users must never see — API keys, database passwords; they may live only in server code. §13.7
- **Selective hydration** — With streaming, React hydrates boundaries independently and prioritises one the user interacts with. §13.4
- **Selector** — A function that reads part of a store (`state => state.cart.total`); components re-render only when its result changes. §6.7
- **Semantic HTML** — Using the element that means what you show (`<button>`, `<nav>`, `<h1>`), which brings roles, focus and keyboard support for free. §14.10
- **Semver** — Semantic versioning: MAJOR.MINOR.PATCH, where only a new major version may contain breaking changes. §13.7
- **Sentinel** — An invisible element placed after the last list item; when it scrolls into view, the next page loads. §15.12
- **SEO** — Search engine optimisation: making pages rank and preview well in search results — a main reason to render HTML on the server. §13.11

- **Serialization** — Turning data into text (such as JSON) so it can be sent over the network or embedded in a page; functions and class instances can't be serialized. §13.7
- **Server Component** — A component that runs only on the server, can be `async`, and ships no JavaScript; can't create or read context, but since 19.3 may render a provider imported from a `'use client'` module. §13.7
- **Server Function / Server Action** — An async function marked `'use server'` that runs on the server but can be called from the client; a public endpoint. "Server Action" is the older (and Next.js) name, used for one passed as a form action or called in a transition. §13.8
- **Server state** — A client-side cache of data owned by a server; needs fetching, caching, invalidation — a TanStack Query job. §6.1
- `server-only` — Package you import at the top of a module so the build fails if Client Component code imports it. §13.7
- **Serverless** — Hosting where your code runs on demand on the provider's machines, with no server for you to manage. §13.11
- **Service Worker** — A background browser script that sits between the page and the network; MSW uses one to fake responses in the browser. §12.6
- **shadcn/ui** — Copy-in component collection: the source is copied into your repo (built on headless primitives + Tailwind) and you own it. §14.12
- **Shallow comparison** — Comparing each top-level property with `Object.is`, as `memo` does with props. §10.4
- **Shell (streaming)** — Everything outside pending Suspense boundaries, sent first with fallbacks in place. §13.3
- **Side effect** — Anything code does besides computing its return value: DOM changes, requests, timers, logging. §5.1
- **Single source of truth** — Each piece of data is owned (stored) in exactly one place; everything else reads it or derives from it. §3.7
- **Singleton** — Exactly one shared instance of something for the whole app (e.g. a Zustand store created at module level). §6.8
- **Skeleton** — Grey placeholder shapes in the final layout shown during a first load; avoids layout shift. §9.12
- **Slot** — A named prop that receives JSX (`header={…}`), next to `children` . §2.4
- **Snapshot (state)** — The fixed state values of one render; setters request a new render but never change the variable you hold. §3.3
- **Snapshot test** — A test that saves rendered output to a file and fails when it changes; brittle when large — prefer explicit assertions. §12.10
- **Specificity (CSS)** — The weight of a selector (ids > classes > elements) that decides which rule wins when several match. §8.1
- **Spread syntax (`...`)** — Copies all properties or items into a new object/array (`{ ...state, done: true }`) or onto JSX (`{...rest}`). §3.6
- **SSG** — defined in the rendering and directives sheet, §16.15.
- **SSR** — defined in the rendering and directives sheet, §16.15.
- **Stacking context** — A CSS layering group (created by `z-index` , `transform` and similar); `z-index` only compares elements inside the same group — why modals use portals. §14.8
- **Stale closure** — A function that keeps reading values from an old render (missing dependency, or a timer set up once). §3.3
- **Stale-while-revalidate** — Serve cached data now and refresh it in the background (TanStack Query, ISR, HTTP caches). §6.9
- `staleTime` — defined in the Routing and TanStack Query sheet, §16.10.
- **State** — Data a component remembers between renders that changes over time; setting it schedules a re-render. §3.2
- **State machine** — A model with a fixed set of states and allowed transitions (idle → loading → success/error); XState is a library for it. §6.6
- **State updater** — A function passed to a setter, `setN(n => n + 1)`, computed from the queued value. §3.5

- **Static hosting** — Serving pre-built files (HTML, JS, CSS) with no code running per request — enough for SPAs and SSG. §1.4
- **Status code (HTTP)** — The number at the start of every HTTP response: 2xx success, 3xx redirect, 4xx client error (404), 5xx server error. §13.3
- **Store** — A central object that holds state outside components and notifies subscribers when it changes (Redux, Zustand). §6.6
- **Storybook** — A workshop app that renders each component on its own in all its states, for development, docs and visual tests. §14.12
- **Stream (Node / Web)** — Data delivered piece by piece instead of all at once; Node has its own stream type, browsers and edge runtimes use Web Streams. §13.2
- **Streaming SSR** — defined in the rendering and directives sheet, §16.15.
- **Strict mode (TypeScript)** — `"strict": true` in `tsconfig.json`, turning on null checks and the other strict checks; always use it (not the same as React's `<StrictMode>`). §11.1
- **`StrictMode`** — Development-only checks you opt into by wrapping the app (Vite and most templates do): renders twice, runs Effects setup→cleanup→setup on mount, re-runs ref callbacks. §5.4
- **Structural sharing** — Reusing unchanged parts of an old object in a new version (Immer, TanStack Query). §3.6
- **Styled component kits (MUI, Chakra UI, Mantine, Ant Design)** — Full component libraries with their own look and theming system. §8.9
- **Subscription** — A registration to be told when something changes, undone by unsubscribing (in an Effect's cleanup). §5.3
- **Subtree** — A component and everything rendered below it. §3.10
- **`suppressHydrationWarning`** — Silences a hydration mismatch warning for one element's own text/attributes; keeps the server value. §13.5
- **Suspense** — `<Suspense fallback>` shows a placeholder while something inside (lazy code, `use(promise)`, suspense-enabled data) isn't ready. §9.11
- **Synchronous** — Happening immediately, in the same step, before the next line of code runs (the opposite of asynchronous). §3.4
- **Synthetic event** — React's cross-browser wrapper around a native event, with the same interface; `e.nativeEvent` gives the original. §3.1

T

- **Tabs (ARIA pattern)** — `tablist` > `tab` + `tabpanel` roles with `aria-selected` / `aria-controls`, arrow keys and a roving tabindex. §15.14
- **Tagged template literal** — A template string passed to a function (`styled.button`…``), used by runtime CSS-in-JS libraries. §8.7
- **Tailwind CSS** — Utility-first CSS framework; generates only the classes it finds in your source; v4 configures with `@theme`. §8.6
- **`tailwind-merge`** — Helper that merges Tailwind class strings so the later conflicting class wins (`p-2` + `p-4` → `p-4`). §8.6
- **TanStack Query** — Server-state library: caching, de-duplication, background refetch, retries, mutations and invalidation. §6.9
- **TanStack Router** — Type-safe router from the TanStack family; the base of TanStack Start. §13.11
- **TanStack Start** — Full-stack React framework built on TanStack Router (type-safe routes, server functions, streaming); v1 release candidate, RSC experimental. §13.11
- **Tearing** — Different parts of the UI showing different versions of the same external data in one render; prevented by `useSyncExternalStore`. §7.9
- **Ternary** — The conditional expression `cond ? a : b`, used in JSX for either/or rendering. §2.7
- **Test double** — Any stand-in for a real dependency in a test — mock, stub, fake or spy. §12.6
- **Test suite** — A named group of related tests (`describe` block) or all of a project's tests. §12.2
- **Theming** — Switching a whole app's look (light/dark, brands) by changing token values in one place. §8.8
- **Third-party library** — Code written by someone else that you install as a dependency. §12.10

- **Throttle** — Run a function at most once per time interval while events keep coming (scroll, resize) — compare debounce, which waits for a pause. §15.11
- **Thunk** — A function dispatched to a Redux store for async logic; `createAsyncThunk` dispatches pending/fulfilled/rejected. §6.7
- **Toast** — A brief pop-up message that disappears on its own; announced through a live region and often rendered in a portal. §14.8
- **Tooltip** — A small hint box shown on hover or focus, positioned with a layout effect and rendered in a portal. §14.8
- **Top layer** — Browser layer above all page content where modal `<dialog>`s and popovers render, so no `z-index` or `overflow` can clip them. §14.8
- **Transition** — A non-urgent state update (`startTransition`) React can render in the background and interrupt; keeps old UI during suspense. §7.10
- **Tree-shaking** — Removing unused exports from a bundle at build time. §14.11
- **Trust boundary** — A place where data from outside your code enters (network, storage, URL); validate it there. §11.7
- **`tsc`** — The TypeScript compiler; `tsc --noEmit` type-checks a project without writing files — run it in CI. §11.1
- **Tuple** — A fixed-length array whose positions have their own types, e.g. `[boolean, () => void]` from `useToggle` . §11.7
- **Turbopack / webpack** — Bundlers: webpack is the long-standing standard; Turbopack is its Rust successor and Next.js 16's default. §13.9
- **Type assertion (`as`)** — Telling TypeScript "trust me, this is type T" without any check; not validation — use sparingly. §11.10
- **Type inference** — TypeScript working out a type automatically from the value, e.g. `useState(0)` → `number` . §11.5
- **TypeScript** — JavaScript plus static types checked before the code runs; files with JSX use the `.tsx` extension. §11.1

U

- **Uncontrolled input** — A form field whose value lives in the DOM (`defaultValue`), read with `FormData` or a ref. §4.2
- **Unidirectional data flow** — Data moves one way: state → view → action → new state (Flux, Redux, React's props-down/events-up). §14.13
- **Union type** — A TypeScript type that is one of several options: `'info' | 'error'` , `User | null` . §11.2
- **Unit test** — A test of one small piece in isolation, such as a reducer or formatter; fastest kind of test. §12.1
- **`unknown`** — TypeScript's safe "some value" type: you must narrow it (`typeof` , `instanceof` , a schema) before using it. §11.1
- **URL** — A web address (`https://shop.example/products/3?sort=price`); its path picks the route and its search params can hold shareable state. §9.1
- **`use`** — defined in the Hook table, §16.3.
- **`'use cache'`** — defined in the rendering and directives sheet, §16.15.
- **`'use client'`** — defined in the rendering and directives sheet, §16.15.
- **`'use no memo'`** — defined in the rendering and directives sheet, §16.15.
- **`'use server'`** — defined in the rendering and directives sheet, §16.15.
- **`useActionState`** — defined in the Hook table, §16.3.
- **`useCallback`** — defined in the Hook table, §16.3.
- **`useContext`** — defined in the Hook table, §16.3.
- **`useDeferredValue`** — defined in the Hook table, §16.3.
- **`useEffect`** — defined in the Hook table, §16.3.
- **`useEffectEvent`** — defined in the Hook table, §16.3.
- **`useFormStatus`** — defined in the Hook table, §16.3.
- **`useId`** — defined in the Hook table, §16.3.

- `useImperativeHandle` — defined in the Hook table, §16.3.
- `useInsertionEffect` — defined in the Hook table, §16.3.
- `useLayoutEffect` — defined in the Hook table, §16.3.
- `useLoaderData` — defined in the Routing and TanStack Query sheet, §16.10.
- `useMemo` — defined in the Hook table, §16.3.
- `useMutation` — defined in the Routing and TanStack Query sheet, §16.10.
- `useNavigate` — defined in the Routing and TanStack Query sheet, §16.10.
- `useNavigation` — defined in the Routing and TanStack Query sheet, §16.10.
- `useOptimistic` — defined in the Hook table, §16.3.
- `useParams` — defined in the Routing and TanStack Query sheet, §16.10.
- `useQuery` — defined in the Routing and TanStack Query sheet, §16.10.
- **user-event** — `@testing-library/user-event`: simulates full user interactions (focus, keys, pointer) — async, `await` each call. §12.4
- `useReducer` — defined in the Hook table, §16.3.
- `useRef` — defined in the Hook table, §16.3.
- `useSearchParams` — defined in the Routing and TanStack Query sheet, §16.10.
- `useState` — defined in the Hook table, §16.3.
- `useSuspenseQuery` — TanStack Query Hook that suspends instead of returning a pending state; `data` is always defined. §9.11
- `useSyncExternalStore` — defined in the Hook table, §16.3.
- `useTransition` — defined in the Hook table, §16.3.
- **Utility-first CSS** — Styling by composing single-purpose classes in markup (Tailwind). §8.6

V

- **Variant** — A named visual option of a component (`variant="primary" | "danger"`), chosen from a small closed set. §14.12
- **Vendor prefix** — Browser-specific property prefixes such as `-webkit-`; in React's `style` object written as `WebkitLineClamp`. §8.3
- **Vercel** — The company that makes Next.js and a hosting platform for it. §13.9
- **Viewport** — The visible area of the browser window (or of a scroll box). §10.9
- **VIPER** — View–Interactor–Presenter–Entity–Router: an iOS screen architecture that splits use-case logic, formatting and navigation. §14.13
- **Virtual DOM** — Informal name for React's in-memory element tree that is diffed against the previous one. §1.3
- **Virtualization (windowing)** — Rendering only the visible rows of a long list plus overscan inside a full-height spacer; libraries: TanStack Virtual, react-window. §10.9
- **Visual-regression test** — Comparing screenshots before and after a change to catch unintended visual differences. §14.12
- `visually-hidden` — A standard CSS class that hides an element from sight but keeps it available to screen readers and keyboard focus. §15.15
- **Vite** — Build tool and dev server with native-ES-module dev serving and HMR; common for React SPAs. Vite 8 strips types/JSX with Oxc (earlier versions: esbuild). §1.5

W

- **WAI-ARIA Authoring Practices** — The W3C guide describing how accessible widgets (tabs, combobox, dialog, menu) should behave with the keyboard and ARIA. §14.4
- **Waterfall** — Requests that start one after another because each waits for the previous render (fetch-in-Effect chains). §9.5
- **WCAG** — Web Content Accessibility Guidelines: the standard accessibility checklist (e.g. text contrast ≥ 4.5:1); current version 2.2. §14.10
- **WebSocket** — A connection kept open so the server can push messages to the browser; opened in an Effect and closed in its cleanup. §5.6
- **Widget** — A self-contained interactive piece of UI (tabs, autocomplete, date picker). §15.1

- `within` — Testing Library helper that limits queries to one element's subtree. §12.3
- `wrapper` **(test render option)** — RTL `render` / `renderHook` option that wraps the UI in providers (router, store, query client). §12.8

X

- **XSS** — Cross-site scripting: an attack where text supplied by one user runs as code in another user's browser; JSX escapes text to prevent it, `dangerouslySetInnerHTML` bypasses that. §1.8

Z

- `z-index` — CSS property that sets stacking order between positioned elements within the same stacking context. §14.8
- **Zero-runtime CSS-in-JS** — Libraries (vanilla-extract, StyleX, Panda) that let you write styles in TypeScript but extract plain CSS at build time. §8.7
- **Zod** — Schema library: describe data once, `parse` / `safeParse` it at runtime and infer TypeScript types. §4.9
- **Zustand** — Small store library: `create((set, get) => …)` returns a Hook read with selectors; no provider. §6.8

Chapter 16 TL;DR

Sheet	The one line to remember
JSX	One root, closed tags, camelCase attributes, `{expressions}`, `style` objects; `0 &&` renders 0; objects can't be children
Components	Capitalised, top-level, pure; props read-only; composition over configuration; stable keys
Hooks	Top level, same order; `use` is the exception; 19 added `use`, `useActionState`, `useFormStatus`, `useOptimistic`; 19.2 added `useEffectEvent`
State	Snapshot per render; updaters for queued changes; batched everywhere; never mutate; position + key decide identity
Lifecycle	Trigger → render (pure) → commit → layout effects → paint → Effects; children's Effects before parents'
Effects	Only for external systems; cleanup mirrors setup; StrictMode double run in dev; derive, key or handle instead where possible
Refs	Mutable box, no re-render, not during render; `ref` is a prop in 19
Forms	Uncontrolled + form Action + `useActionState` by default; controlled when every keystroke matters; validate on the server
State homes	Local → lifted → context (rare changes) → Zustand/RTK (busy); server data → TanStack Query; shareable → URL
Data & routing	Loaders start fetching early; the query cache avoids refetching; transitions keep old UI
Performance	Measure, fix structure, then memoize (or the Compiler), split code, virtualize
Server	SSR/SSG/ISR/streaming/PPR per route; RSC ships no JS; `'use client'` marks the boundary, `'use server'` marks Server Functions
Testing	Roles and names, `user-event`, `findBy`, mock the network, fresh providers per test
Errors	Read the message, find it in §16.18, follow the link

This is the end of the React Handbook. Next: build something (the plan in §16.20) — and when you forget a detail, come back to these sheets. Then continue with the LLD Handbook's front-end machine-coding case, the HTML & CSS Handbook, or go back to the book's front page or Chapter 1 for a second pass.

Chapter 17 — Interview Question Bank

How to use this bank

This chapter collects the React questions that come up again and again in real interviews: the ones repeated across the big public GitHub question collections, popular blog and community question lists, and the machine-coding prompts reported by candidates (the sources are listed at the end). The questions are worded the way interviewers usually say them, so the wording itself becomes familiar.

- ⋆ marks a question that appears in many of those sources — learn these first.
- Every question has a **short answer you can say out loud** in 20–60 seconds. Where this book teaches the topic in depth, a → **More** link takes you to the full explanation (with runnable code and real output). Questions the book does not cover elsewhere still get a complete short answer here, so nothing is missing.
- Chapter 15 already has 53 interview questions with 30-second answers and 8 machine-coding builds with tests. This bank **links to those** instead of repeating them in full, and adds every common question they don't include. ("Ch 15 Q20" means question 20 in Chapter 15.)
- Everything is written for **React 19** (the book uses 19.3), React Router 8, Redux Toolkit 2, Zustand 5, TanStack Query 5 and Next.js 16. Answers that mention older APIs say so.
- **How to drill:** cover the answer, say yours out loud (a full answer: *what it is → why → an example → a trade-off*), then uncover and compare. Mark the ones you stumbled on and repeat them the next day. For the coding tasks in the last groups, write the code on a blank file with a timer, then compare with the linked build.

Terms are defined the first time they appear. A few that appear everywhere:

- **Component** — a JavaScript function that returns a description of UI (JSX). **Props** — the inputs a parent passes to a component. **State** — data a component remembers between renders and that, when changed, makes React re-render.
- **Render** — React calling your component function to get the new UI description. **Commit** — React applying the differences to the real DOM (the browser's tree of page elements).
- **Hook** — a function whose name starts with `use` that lets a component use a React feature (state, effects, context …).

---

1. React fundamentals and JSX

Q1. ⋆ What is React, and what are its main features? React is a JavaScript **library** for building user interfaces out of **components**. Its main features: **declarative** UI (you describe what the screen should look like for the current data, and React works out the DOM changes), **component-based** reuse, **one-way data flow** (data goes down through props), a **virtual DOM / reconciliation** step that updates only what changed, Hooks for state and side effects, and the ability to render on the server (SSR, Server Components) as well as in the browser. It is "just the view layer": routing, data fetching and build tooling come from frameworks or libraries. → *More:* §1.1 What React is · §1.2 Declarative UI

Q2. ⋆ Is React a library or a framework? What are its advantages and limitations? React itself is a library — it renders UI and manages component state, but leaves routing, data loading and the build to you (or to a framework such as Next.js or React Router's framework mode). Advantages: a small, stable core idea (UI = f(state)), huge ecosystem, reusable components, same skills on web, server and mobile (React Native). Limitations: many decisions to make (router, state library, styling), JSX and build tools to learn, client-only apps send a lot of JavaScript and are poor for SEO unless you add SSR, and performance tuning (re-renders) needs understanding. → *More:* §1.1 What React is

Q3. What is declarative vs imperative UI programming? Why does React choose declarative? Imperative code lists the steps: "find the button, add a class, change the text, remove the spinner". **Declarative** code describes the result: "when `loading` is true, show a spinner; otherwise show the list". React is declarative: you return the UI for the current state, and React computes the DOM operations. This

removes a whole class of bugs where the DOM and the data get out of sync, and makes components easy to read and test. → *More:* §1.2 Declarative UI

Q4. ⋆ What is JSX? Do browsers understand it? **JSX** is a syntax extension to JavaScript that looks like HTML and describes UI: `<h1 className="t">Hi {name}</h1>`. Browsers do **not** understand it; a compiler (Babel, esbuild, SWC, TypeScript) turns each tag into a function call — with the modern "automatic runtime" that is `jsxs('h1', { className: 't', children: ['Hi ', name] })` (`jsx` for a single child), imported from `react/jsx-runtime`. The result is a plain object called a **React element**. JSX is optional but almost everyone uses it. → *More:* §1.8 JSX

Q5. What are the rules of JSX? (1) Return **one root** element — wrap siblings in a `<div>` or a Fragment `<>…</>`. (2) **Close every tag**, including `<img />` and `<br />`. (3) Use **camelCase** attribute names: `className`, `htmlFor`, `onClick`, `tabIndex` (but `aria-*` and `data-*` keep their dashes). (4) Curly braces `{}` hold any JavaScript **expression** (not statements like `if` or `for`). (5) `style` takes an object: `style={{ fontSize: 14 }}`. (6) Component names start with a **capital letter**. → *More:* §1.8 JSX · §16.1 JSX rules

Q6. Why must JSX return a single root element? What is a Fragment and why is it better than an extra `<div>`? A component returns one value, and JSX compiles to one function call, so two sibling tags would be two values. A **Fragment** (`<>…</>` or `<Fragment>…</Fragment>`) groups children without adding a DOM node. That keeps the DOM smaller and avoids breaking layouts that depend on direct children (CSS grid/flex, `<table>` rows, `<ul>` items). Use the long form `<Fragment key={id}>` when you need a key in a list. → *More:* §1.8 JSX · §2.8 Rendering lists and keys

Q7. Why does React use `className` instead of `class`, and `htmlFor` instead of `for`? JSX is JavaScript, and `class` and `for` are reserved words in JavaScript. React's props follow the DOM **property** names (`element.className`, `label.htmlFor`), not the HTML attribute names. (On a built-in element, writing `class` triggers a development warning; on custom elements — web components — React 19 passes `class` through as an attribute.) → *More:* §1.8 JSX

Q8. ⋆ What is the difference between a React element and a React component? (And a React node?) A **component** is the function (or class) you write — a recipe. An **element** is the plain, immutable object a component call produces from JSX, like `{ type: 'button', props: { children: 'Save' } }` — a description of what to show, not the DOM node itself. A **React node** is anything React can render: an element, a string, a number, an array of nodes, `null`, `undefined` or a boolean (the last three render nothing). → *More:* §1.3 Rendering, the virtual DOM and reconciliation (brief) · §2.1 Function components

Q9. How do you write comments, loops and conditions in JSX? Comments go inside braces: `{/* note */}`. Loops use `array.map()` returning elements with a `key`: `{items.map(i => <li key={i.id}>{i.name}</li>)}`. Conditions use expressions: the ternary `cond ? <A/> : <B/>`, `cond && <A/>`, or an early `return` / a variable computed with `if` above the JSX. Statements (`if`, `for`) can't appear directly inside `{}` because braces need an expression. → *More:* §2.7 Conditional rendering · §2.8 Rendering lists and keys

Q10. How does JSX protect against XSS (cross-site scripting) attacks? What is `dangerouslySetInnerHTML`? **XSS** is an attack where untrusted text is injected into a page as HTML/script and executed. React **escapes** every value you put in `{}` — it is inserted as text, so `<script>` shows as characters. `dangerouslySetInnerHTML={{ __html: html }}` bypasses this and inserts raw HTML; the long name is a warning. Use it only with HTML you trust or have **sanitised** (cleaned with a library like DOMPurify). Also watch `href={userUrl}` — recent React versions block `javascript:` URLs there, but validate user-supplied URLs anyway. → *More:* §1.8 JSX

Q11. What is the "new JSX transform"? Do you still need `import React from 'react'`? Since React 17, compilers use the **automatic runtime**: they insert `import { jsx } from 'react/jsx-runtime'` themselves, so files with JSX no longer need `import React`. It also gives slightly smaller output. You still import Hooks and APIs you use (`import { useState } from 'react'`). → *More:* §1.8 JSX

Q12. What is the difference between `react` and `react-dom`? Why are they separate packages? `react` holds the component model: elements, Hooks, Context, `memo`, `lazy`. `react-dom` is a **renderer** — it connects that model to the browser DOM (`react-dom/client` with `createRoot` / `hydrateRoot`, `createPortal`, `flushSync`, `useFormStatus`) and to HTML on the server (`react-dom/server`, `react-dom/static`). Other renderers (React Native, react-three-fiber, test renderers) reuse `react` with a different target. → *More:* §1.5 Setting up a React project · §13.2 The server rendering APIs

Q13. How do you mount a React app in React 19? What happened to `ReactDOM.render`? `createRoot(document.getElementById('root')).render(<App />)` from `react-dom/client`; for server-rendered HTML use `hydrateRoot(container, <App />)`. `ReactDOM.render` and `ReactDOM.hydrate` were deprecated in React 18 and **removed in React 19**, along with `unmountComponentAtNode` (use `root.unmount()`) and `findDOMNode` (use refs). → *More:* §1.7 Your first component · §13.4 Hydration end to end

Q14. What is `<StrictMode>` and what does it do? `<StrictMode>` is a development-only wrapper that helps find bugs. Inside it, React renders components **twice**, runs Effect **setup → cleanup → setup** once more on mount, runs ref callbacks twice, and warns about deprecated APIs. Pure components and symmetric cleanups aren't affected; impure ones show their bugs immediately. It does nothing in production. → *More:* §5.4 StrictMode's double run (development only) · Ch 15 Q19

Q15. What changed in React 18? (Common "what's new" question.) React 18 introduced **concurrent rendering** (React can prepare an update in the background and interrupt it for a more urgent one), the new `createRoot` API, **automatic batching** of state updates everywhere (timeouts, promises, native handlers), **transitions** (`useTransition`, `startTransition`) and `useDeferredValue`, **Suspense on the server** with streaming SSR and selective hydration, and new Hooks `useId`, `useSyncExternalStore` and `useInsertionEffect`. → *More:* §3.4 Batching · §7.10 `useTransition` — non-urgent updates · §13.3 Streaming SSR with `<Suspense>`

Q16. ⋆ What's new in React 19? Actions (async functions in transitions, with pending state handled for you), `<form action={fn}>`, the Hooks **`useActionState`**, **`useFormStatus`** and **`useOptimistic`**, the **`use()`** API for reading promises and context, **Server Components and Server Functions** as stable features, **`ref` as a normal prop** (no `forwardRef`), ref callback **cleanup** functions, `<Context value>` as a provider, built-in `<title>` / `<meta>` / `<link>` hoisting, stylesheet and script support, resource preloading (`preload`, `preinit`), better hydration error messages, and full custom-element support. 19.2 added `<Activity>` and `useEffectEvent`. Removed: `propTypes` and function `defaultProps`, string refs, legacy context, `ReactDOM.render`. → *More:* §4.4 Form Actions — `<form action={fn}>` · §5.10 Passing refs to your own components · §9.11 Suspense and `use()` · §13.7 React Server Components

Q17. What is Create React App's status? How would you start a React project today? Create React App was **deprecated** (February 2025) — it is unmaintained. react.dev recommends starting with a **framework** (Next.js, React Router framework mode, Expo for mobile) or, for a client-only app, a **build tool** like **Vite** (`npm create vite@latest`). Frameworks bring routing, data loading and server rendering; Vite gives a fast dev server and bundling only. → *More:* §1.5 Setting up a React project

Q18. Should you still learn class components? How do lifecycle methods map to Hooks? New code uses function components, but you will meet classes in old code — and error boundaries still need one. Rough mapping: `constructor` / `this.state` → `useState`; `componentDidMount` + `componentDidUpdate` + `componentWillUnmount` → one `useEffect` with setup, dependencies and cleanup (think in "synchronise with X", not lifecycle events); `shouldComponentUpdate` / `PureComponent` → `memo`; `getDerivedStateFromProps` → compute during render or reset with `key`; `componentDidCatch` → still a class (error boundary). → *More:* §5.2 `useEffect` basics · §14.9 Error boundaries

---

2. Components, props and composition

Q19. ⋆ What is the difference between state and props? Props are inputs passed **from the parent**; the component must treat them as read-only. **State** is data the component **owns** and changes with a setter, which triggers a re-render. A value that is state in a parent is often a prop in the child. Rule of thumb: if it can be computed from props or other state, it is neither — compute it during render. → *More:* §2.2 Props · §3.2 State: a component's memory — `useState`

Q20. ⋆ Why are props read-only? What happens if a child mutates a prop? React's model is that a component is a **pure function of its props and state**: same inputs, same output. Props belong to the parent; if a child changes them, the parent doesn't know, nothing re-renders, and the UI shows stale or inconsistent data (and other components sharing that object see surprise changes). To change something, the parent passes a callback (`onChange`) and updates its own state. → *More:* §2.2 Props · §2.5 Pure components

Q21. What is the `children` prop? Whatever you put between a component's opening and closing tags arrives as `props.children`: `<Card><h2>Title</h2></Card>`. It lets you build wrappers (cards, layouts,

modals) that don't need to know their content. `children` can be any React node — text, elements, arrays, or even a function (the render-prop pattern). → *More:* §2.3 The `children` prop

Q22. ⋆ What is prop drilling, and how do you avoid it? **Prop drilling** is passing a prop through several layers of components that don't use it, just to reach a deep child. It makes code noisy and refactors painful. Fixes, in order: **composition** (pass the deep component itself as `children` or a slot prop so the middle layers don't see the data), **Context** for values many components need (theme, current user, locale), or a **store** (Zustand, Redux) for frequently changing shared state. → *More:* §2.4 Composition · §6.3 Context — pass data deeply without props

Q23. How do you give props default values in React 19? Use JavaScript **default parameters** in destructuring: `function Button({ size = 'md', children })`. `Component.defaultProps` for function components was **removed in React 19** (it still works for classes). Defaults apply only when the prop is `undefined`, not when it is `null`. → *More:* §2.2 Props

Q24. How do you validate props? What happened to PropTypes? React 19 **removed** the built-in `propTypes` checks — they are silently ignored. Use **TypeScript** (checked at build time, and in your editor) for prop types; validate **runtime data** (API responses, form input) with a schema library such as **Zod**. → *More:* §11.2 Typing props

Q25. What is a pure component? What is `PureComponent`? A **pure** component returns the same JSX for the same props, state and context, and doesn't change anything outside itself while rendering (no mutating variables created before the render, no fetching, no DOM writes). React relies on this to render twice, pause or skip renders safely. `PureComponent` is a legacy **class** base that skipped re-rendering when props and state were shallowly equal — the function equivalent is `memo`. → *More:* §2.5 Pure components · §10.4 `memo` — skipping a child's re-render

Q26. What is composition, and why does React prefer it over inheritance? **Composition** means building components by combining others — passing JSX as `children` or as named "slot" props (`<Layout sidebar={<Nav/>}>`). React has no need for component inheritance: anything a base class would share can be a component you wrap, a prop, or a custom Hook. Composition keeps components decoupled and avoids deep, fragile class hierarchies. → *More:* §2.4 Composition · §14.1 Composition first

Q27. ⋆ How do you conditionally render content? Which patterns are there? Early `return null` (render nothing), `if` statements assigning JSX to a variable, the ternary `a ? <X/> : <Y/>`, and `cond && <X/>`. Watch the `&&` trap: `{count && <Badge/>}` renders `0` when `count` is 0, because `0` is a renderable number — write `{count > 0 && <Badge/>}`. For many cases, use an object map: `const views = { loading: <Spinner/>, error: <Err/> }; return views[status];`. → *More:* §2.7 Conditional rendering · Ch 15 Q7

Q28. How do you pass data from a child to a parent? Between siblings? Child → parent: the parent passes a **callback** prop, and the child calls it with data (`onSelect(item)`). Siblings: **lift the state up** to their closest common parent, which passes the value down to one and the setter (or a handler) down to the other. For distant components, use Context or a store. → *More:* §3.7 Lifting state up

Q29. What does "lifting state up" mean? When two components need the same changing data, you move that state to their **closest common parent** and pass it down as props, with callbacks to change it. There is then a single **source of truth** (one place that owns the data), so the two can't disagree. → *More:* §3.7 Lifting state up

Q30. What is one-way (unidirectional) data flow, and why is it useful? Data flows **down** the tree through props; changes flow **up** only as function calls (events) that update state at its owner, which then re-renders downwards. Because every piece of state has one owner and one way to change, you can trace any value to where it came from — easier debugging than two-way binding, where a change can come from anywhere. → *More:* §3.7 Lifting state up · §2.2 Props

Q31. Why must component names start with a capital letter? JSX uses the case of the tag to decide what to create: lowercase (`<div>`, `<my-widget>`) means a built-in DOM element or custom element given as a **string**; capitalised (`<MyWidget>`) means a **variable** holding a component. `<myWidget/>` would try to create an unknown HTML tag. → *More:* §1.7 Your first component

Q32. ⋆ Why should you never define a component inside another component? A component defined inside another's body is a **new function on every render**. React compares component types by identity, so it sees a different type each time, **unmounts the old subtree and mounts a fresh one** — all state inside is lost (inputs lose focus and text) and it's slow. Define components at the top level of the module and

pass data via props. → *More:* §3.10 State is tied to a position in the tree · §10.10 Common performance bugs

Q33. What is the spread-props pattern (`{...props}`), and what's its risk? `<input {...rest} />` forwards all remaining props, handy for wrapper components that extend native elements. Risks: you can pass unknown props to DOM elements (React warns about invalid attributes), accidentally override props you set earlier (order matters: later wins), and make the component's API unclear. Destructure what you use and spread only the rest, and type it with `ComponentProps<'input'>`. → *More:* §2.2 Props · §11.3 Typing children and extending native elements

Q34. What is a "controlled component" in general (not only inputs)? A component is **controlled** when its important state is passed in by the parent as a prop together with a change callback (`value` + `onChange` , `open` + `onOpenChange`); it is **uncontrolled** when it keeps that state itself (maybe with a `defaultValue`). Controlled gives the parent full power (validate, sync, reset); uncontrolled is simpler to use. Good reusable components often support both. → *More:* §4.2 Controlled vs uncontrolled inputs · §14.4 Compound components

---

3. State and events

Q35. ⋆ What is `useState` , and why does it return an array? `const [value, setValue] = useState(initial)` gives a component a piece of memory that survives re-renders, plus a function to change it and schedule a re-render. It returns an **array** (a pair) so you can name both parts freely with array destructuring — `[count, setCount]` , `[user, setUser]` — which an object return wouldn't allow as neatly. → *More:* §3.2 State: a component's memory — `useState`

Q36. ⋆ Is `setState` synchronous or asynchronous? Why doesn't the value change right after I call the setter? The setter **schedules** an update; it doesn't change the variable you are holding. Your `count` is a **snapshot** — a constant for this render. React re-runs the component later (after the event handler finishes) and that new render sees the new value. So `setCount(5); console.log(count)` logs the old value. If you need the next value, compute it in a variable first, or read it on the next render / in an Effect. → *More:* §3.3 State as a snapshot · Ch 15 Q1

Q37. ⋆ What is the functional (updater) form of `setState` , and when should you use it? `setCount(c => c + 1)` passes a function that receives the **latest pending value** and returns the next one. React queues updaters and applies them in order, so three calls add 3, whereas `setCount(count + 1)` three times adds only 1 (all read the same snapshot). Use it whenever the next state depends on the previous state — especially in async code, intervals and multiple updates in one event. → *More:* §3.5 Updater functions — queueing updates · Ch 15 Q8

Q38. ⋆ What is batching? Can you opt out? Batching means React groups several state updates made during the same event into **one** re-render. Since React 18 this is **automatic everywhere** — event handlers, timeouts, promises, native listeners. To force an immediate DOM update (rare, e.g. to scroll to an element you just added), wrap the update in `flushSync(() => setX(...))` from `react-dom` . → *More:* §3.4 Batching · Ch 15 Q9

Q39. What does `useState(() => compute())` (lazy initial state) do? Passing a **function** makes React call it **only on the first render** to get the initial value. Use it when the initial value is expensive — reading `localStorage` , parsing a big object. `useState(compute())` (calling it) would run the expensive work on every render and throw the result away. → *More:* §3.2 State: a component's memory — `useState` · §7.4 `useLocalStorage` — state that survives a reload

Q40. ⋆ How do you update an object or array in state? Why not mutate it? Create a **new** object/array: `setUser({ ...user, name })` , `setItems([...items, item])` , `items.filter(...)` to remove, `items.map(...)` to change one, `toSorted()` / `toReversed()` instead of `sort` / `reverse` . React compares old and new state with `Object.is` (reference equality); if you mutate and pass the same reference, React thinks **nothing changed** and skips the render. Mutation also breaks `memo` , Effects dependencies, undo/redo and DevTools history. → *More:* §3.6 Updating objects and arrays immutably · Ch 15 Q10

Q41. How do you update nested state? What is Immer? Copy every level on the path to the change: `setForm({ ...form, address: { ...form.address, city } })` . If this gets deep, **flatten** the state (store items by ID) or use **Immer**, a library that lets you write "mutating" code on a draft (`draft.address.city =`

`city`) and produces a new immutable object for you (`useImmer`, and Redux Toolkit's `createSlice` uses it internally). → *More:* §3.6 Updating objects and arrays immutably · §6.7 Redux Toolkit

Q42. Which array methods are safe for state, and which mutate? Mutating (avoid on state): `push`, `pop`, `shift`, `unshift`, `splice`, `sort`, `reverse`, assignment `arr[i] = x`. Non-mutating (use): spread `[...a, x]`, `concat`, `filter`, `map`, `slice`, and ES2023's `toSorted`, `toReversed`, `toSpliced`, `with(i, x)`. If you must use `sort`, copy first: `[...a].sort()`. → *More:* §3.6 Updating objects and arrays immutably

Q43. What is derived state, and why is copying props into state a bug? Derived values can be calculated from props or state (a filtered list, a full name, a total). Store the minimal state and compute the rest during render. Copying a prop into state (`useState(props.value)`) only uses the prop's **first** value — later changes are ignored, so the UI gets out of sync. If you really need to reset internal state when a prop changes, give the component a `key`. → *More:* §3.8 Derived state — choosing the minimal state · Ch 15 Q11

Q44. ⋆ How are React events different from DOM events? What is a SyntheticEvent? React handlers are camelCase props (`onClick`) that take a **function**, not a string. React passes a **SyntheticEvent** — a wrapper with the same interface across browsers (`e.target`, `e.preventDefault()`, `e.stopPropagation()`), the native one at `e.nativeEvent`. React attaches listeners at the **root container** and dispatches them itself (event delegation). You can't `return false` to prevent default — call `e.preventDefault()`. Since React 17 events aren't pooled, so reading `e` asynchronously is fine. → *More:* §3.1 Responding to events · Ch 15 Q15

Q45. `onClick={handleClick}` vs `onClick={handleClick()}` vs `onClick={() => handleClick(id)}`? The first passes the function — correct. The second **calls** it during render and passes its return value (often `undefined`); if it sets state, you get an infinite loop ("Too many re-renders"). The third passes a new arrow function that calls the handler with an argument when clicked — the normal way to pass arguments. → *More:* §3.1 Responding to events

Q46. What are capture-phase events in React? DOM events travel **down** from the root to the target (capture phase), then **up** back to the root (bubble phase). React handlers run in the bubble phase by default; add `Capture` to the name — `onClickCapture` — to run during capture, which fires before children's handlers (useful for analytics or intercepting). → *More:* §3.1 Responding to events

Q47. How do you reset a component's state? Change its `key`: `<Profile key={userId} />`. A different key tells React it's a different component, so it unmounts the old one and mounts a fresh one with initial state. Alternatives: lift the state up and reset it in the parent, or call the setters with initial values (for a "Reset" button). → *More:* §3.10 State is tied to a position in the tree · Ch 15 Q5

Q48. Where should state live? How do you decide the minimal state? Put state in the **lowest common parent** of the components that use it. Keep it **minimal**: don't store what you can compute, don't duplicate the same data in two places (store an ID, not a copy of the selected object), and don't use state for values that never change or don't affect rendering (use a constant or a ref). Server data belongs in a data-fetching cache, URL-shareable state in the URL. → *More:* §3.8 Derived state — choosing the minimal state · §6.1 Kinds of state — and where each belongs · Ch 15 Q12

Q49. If I set state to the same value, does the component re-render? React compares with `Object.is`. If the new value is the same, React **bails out** — it won't re-render the children or run Effects. (It may still call the component function once before deciding, so don't rely on the body not running.) This is another reason mutation fails: same reference = "no change". → *More:* §10.2 When components re-render

Q50. What does "state is tied to a position in the tree" mean? React keeps a component's state attached to **where** it sits in the rendered tree, not to the JSX variable. Render the same component type at the same position and state is kept; change the type at that position (or remove it) and the state is destroyed. That's why `{isA ? <Counter/> : <Counter/>}` keeps the count (same position, same type) unless you give them different keys. → *More:* §3.10 State is tied to a position in the tree

---

4. Rendering, reconciliation, keys and the virtual DOM

Q51. ⋆ What is the virtual DOM, and how does it work? The "virtual DOM" is React's in-memory description of the UI — the tree of elements your components return. When state changes, React renders a new tree, **compares** it with the previous one (reconciliation), and applies only the differences to the

real DOM. DOM work is the slow part, so doing only the minimal set of changes, batched, is efficient — and you get to write UI as if you re-rendered everything. React's own docs now talk about "render and commit" rather than "virtual DOM", but interviewers still use the term. → *More:* §1.3 Rendering, the virtual DOM and reconciliation (brief) · Ch 15 Q3

Q52. ⋆ What is reconciliation, and what is the diffing algorithm's main idea? Reconciliation is how React decides what changed between two renders. A general tree-diff costs $O(n^3)$; React uses an $O(n)$ **heuristic** with two assumptions: (1) elements of a **different type** produce different trees — React destroys the old subtree and builds a new one; (2) among siblings, **keys** identify which child is which across renders. Same type at the same position → React keeps the DOM node/component and updates only changed props. → *More:* §1.3 Rendering, the virtual DOM and reconciliation (brief) · §2.8 Rendering lists and keys

Q53. Virtual DOM vs real DOM vs Shadow DOM? The **real DOM** is the browser's live tree of page elements. The **virtual DOM** is React's JavaScript description of it, used to compute changes. The **Shadow DOM** is an unrelated browser feature for **web components**: it attaches a hidden, encapsulated DOM subtree to an element so its styles and markup don't leak in or out. React can render custom elements that use Shadow DOM. → *More:* §1.3 Rendering, the virtual DOM and reconciliation (brief)

Q54. Is the virtual DOM "faster than the DOM"? Not by itself — the diff is extra work on top of the DOM updates. Its value is that it lets you write simple declarative code ("re-render everything") while React keeps actual DOM writes small and batched. Frameworks without a virtual DOM (Svelte, Solid) use compile-time or fine-grained reactivity and can be faster in benchmarks. For React apps, slowness usually comes from too many re-renders or too much DOM, not from the diff itself. → *More:* §1.3 Rendering, the virtual DOM and reconciliation (brief) · §10.1 The render cycle: trigger → render → commit

Q55. ⋆ What is React Fiber? Fiber is the reconciler (the core algorithm) React has used since v16. Each component instance has a "fiber" — a unit of work in a linked tree — so React can **split rendering into small chunks, pause, prioritise, and resume** instead of doing one long blocking pass. That is what makes concurrent features possible (transitions, Suspense, `useDeferredValue`, selective hydration). Fibers also store each component's Hook state. → *More:* §7.1 What a Hook really is · §10.1 The render cycle: trigger → render → commit

Q56. ⋆ Why do lists need keys? Why is the array index a bad key? Keys tell React which item is which between renders, so it can keep the right DOM nodes and state when items are added, removed or reordered. With the **index** as key, inserting at the top or sorting shifts every key: React reuses the wrong components, so input text, checkbox state and focus jump to the wrong rows, and it updates every row. Use a stable, unique ID from the data (`item.id`); generate IDs when items are created, never during render (`Math.random()` as a key remounts everything every time). → *More:* §2.8 Rendering lists and keys · Ch 15 Q4

Q57. When is it safe to use the index as a key? When the list is **static**: it is never reordered, filtered or inserted into (other than appending at the end), and items have no state or uncontrolled inputs. Example: a fixed list of labels. If any of these might change, use real IDs. → *More:* §2.8 Rendering lists and keys

Q58. Do keys need to be globally unique? Can you read `props.key`? Keys must be unique only among **siblings** in the same list; two different lists can reuse keys. `key` (like `ref` in older versions) is consumed by React and **not passed** to the component as a prop — pass the ID again under another name if the child needs it. → *More:* §2.8 Rendering lists and keys

Q59. Can you use keys outside lists? Yes — a `key` on any element forces React to treat it as a different component when the key changes. `<ChatRoom key={roomId} />` remounts the room (fresh state, fresh Effects) when the room changes. This is the clean way to reset state on an identity change. → *More:* §3.10 State is tied to a position in the tree · Ch 15 Q5

Q60. ⋆ When does a component re-render? When (1) its **state** changes, (2) its **parent re-renders** (by default all children re-render, whether props changed or not), (3) a **context** it reads changes, or (4) an external store it subscribes to (via `useSyncExternalStore` / a store hook) changes. Props changing is not a separate trigger — props change only because the parent re-rendered. `memo` can skip case (2) when props are equal. → *More:* §10.2 When components re-render · Ch 15 Q6

Q61. ⋆ What are the render and commit phases? Which must be pure? **Render**: React calls components to compute the new UI — must be **pure** (no side effects), because React may call it twice, pause it, or throw it away. **Commit**: React applies the DOM changes, sets refs, runs `useLayoutEffect`, the browser paints, then `useEffect` runs. Side effects go in event handlers (first choice) or Effects. → *More:* §10.1 The render cycle: trigger → render → commit · Ch 15 Q2

Q62. Does re-rendering mean the DOM is updated? No. A **re-render** only means React called your component again. If the resulting JSX is the same, the commit does nothing to the DOM. Re-renders cost CPU time, but they are usually cheap; they become a problem when components are expensive or there are very many. → *More:* §10.1 The render cycle: trigger → render → commit

Q63. What happens when an element changes type, e.g. `<div>` to `<section>`, or `<A/>` to `<B/>` at the same position? React treats it as a different tree: it **unmounts** the old element and everything beneath it (running cleanups, destroying state and DOM nodes) and mounts the new one from scratch. This is why wrapping a subtree conditionally (`{wrap ? <div><Form/></div> : <Form/>}`) resets the form's state. → *More:* §3.10 State is tied to a position in the tree · §1.3 Rendering, the virtual DOM and reconciliation (brief)

Q64. What is concurrent rendering? In concurrent rendering React can work on a render **without blocking** the main thread: it can start rendering a low-priority update, pause to handle a click or keystroke, and then continue or restart. You opt in per update with transitions (`startTransition`, `useTransition`, `useDeferredValue`) or Suspense. Urgent updates (typing) stay responsive while big updates (filtering a long list) happen in the background. → *More:* §7.10 `useTransition` — non-urgent updates · §7.11 `useDeferredValue` — let part of the UI lag behind

---

5. `useEffect`, its pitfalls, and refs

Q65. ⋆ What is `useEffect`, and what is it for? `useEffect(setup, deps)` runs code **after React has committed** the render to the screen, to **synchronise the component with something outside React**: a subscription, a WebSocket, a timer, a non-React widget, `document.title`, or (in simple apps) a fetch. The setup may return a **cleanup** function that undoes it. It is not the place for logic that responds to a user action (that belongs in the event handler) or for computing values from state (do that during render). → *More:* §5.2 `useEffect` basics · Ch 15 Q16

Q66. ⋆ What do no dependency array, `[]`, and `[a, b]` mean? No array: the Effect runs after **every** render. `[]`: only after the **first** mount (and its cleanup on unmount). `[a, b]`: after mount and whenever `a` or `b` changed since the last render (compared with `Object.is`). The array is not a "choose when to run" switch — it must list **every reactive value** (props, state, and values computed from them) the Effect reads; the `react-hooks/exhaustive-deps` lint rule checks that. → *More:* §5.5 Dependencies in depth · Ch 15 Q17

Q67. ⋆ When does the cleanup function run? Before the Effect runs again (with the **old** values, before the next setup), and once when the component **unmounts**. In development with StrictMode it also runs once right after the first mount (setup → cleanup → setup) to prove the cleanup is correct. Typical cleanups: `clearInterval`, `removeEventListener`, `unsubscribe`, `controller.abort()`, `socket.close()`. → *More:* §5.3 Cleanup — undo what setup did · Ch 15 Q18

Q68. ⋆ Why does my Effect run twice in development? Because of `<StrictMode>`: in development, React mounts, unmounts and remounts each component once to check your cleanup is symmetrical. It only happens in development. The fix is never to "run it once" with a flag — write a cleanup that undoes the setup (abort the fetch, unsubscribe, clear the timer), so the double run is invisible. → *More:* §5.4 StrictMode's double run (development only) · Ch 15 Q19

Q69. ⋆ How do you avoid infinite loops with `useEffect`? A loop happens when an Effect sets state that changes one of its own dependencies (or has no dependency array and sets state each time). Fixes: don't put an object/array/function created during render in the deps (move it inside the Effect, memoize it, or depend on its primitive fields); use the **updater form** (`setCount(c => c + 1)`) so `count` isn't a dependency; and ask whether you need the Effect at all — derived data should be computed during render. → *More:* §5.5 Dependencies in depth · §5.6 You might not need an Effect

Q70. ⋆ Why does putting an object or function in the dependency array make the Effect run every render? Dependencies are compared by **reference** (`Object.is`). `const options = { roomId }` or `function onMessage() {}` created in the component body is a **new** object every render, so React sees "changed" each time. Create it **inside** the Effect, move it outside the component if it doesn't use props/state, depend on primitives (`options.roomId`), or memoize it with `useMemo` / `useCallback` (the React Compiler does the last automatically). → *More:* §5.5 Dependencies in depth

Q71. Can the Effect callback be `async`? What happens if you return a Promise from `useEffect`? No. React expects the Effect to return nothing or a cleanup **function**; an `async` function returns a Promise, which React warns about and can't use as cleanup. Define an async function inside and call it, with an `ignore` flag or `AbortController` for cleanup:

```
useEffect(() => {
  const ac = new AbortController();
  (async () => {
    const res = await fetch(`/api/users/${id}`, { signal: ac.signal });
    setUser(await res.json());
  })().catch((e) => { if (e.name !== 'AbortError') setError(e); });
  return () => ac.abort();
}, [id]);
```

→ *More:* §5.7 Fetching data in Effects and race conditions

Q72. ⋆ What is a race condition when fetching in an Effect, and how do you prevent it? If `id` changes from 1 to 2 quickly, the request for 1 might answer **after** the request for 2, and the stale result overwrites the correct one. Prevent it in the cleanup: set a local `ignore = true` flag (and skip `setState` if ignored) or `abort()` the previous request with `AbortController`. Libraries like TanStack Query handle this for you. → *More:* §5.7 Fetching data in Effects and race conditions · Ch 15 Q20

Q73. ⋆ What is a stale closure in a Hook, and how do you fix it? A **closure** is a function that remembers the variables from the render where it was created. If an Effect, interval or callback is created once (`[]` deps) but reads `count`, it keeps seeing the **first** render's `count` forever — it's "stale". Fixes: list the value in the dependencies, use the **updater** form (`setCount(c => c + 1)`), keep the latest value in a **ref**, or use `useEffectEvent` for non-reactive logic called from an Effect.

```
useEffect(() => {
  const id = setInterval(() => setCount(c => c + 1), 1000); // not count + 1
  return () => clearInterval(id);
}, []);
```

→ *More:* §5.5 Dependencies in depth · §7.7 Hooks that take callbacks — `useEffectEvent`

Q74. ⋆ "You might not need an Effect" — what are the common unnecessary Effects? (1) Computing derived data (`useEffect(() => setFullName(first + ' ' + last))`) — compute during render. (2) Caching expensive results — `useMemo`. (3) Resetting state when a prop changes — use a `key`. (4) Reacting to a user event (submit, buy, notify) — do it in the **event handler**. (5) Chains of Effects setting state that triggers other Effects — compute the next state in one handler. (6) Notifying the parent of a change — call the parent's callback in the same handler. Effects are for syncing with **external** systems. → *More:* §5.6 You might not need an Effect · Ch 15 Q16

Q75. In which order do Effects run in a parent and its children? Rendering goes top-down (parent body, then child body), but Effects run **bottom-up**: children's Effects run before the parent's, so by the time a parent's Effect runs, its whole subtree is in the DOM and has already run its own Effects. On updates, all relevant **cleanups** run before any new setups. `useLayoutEffect` functions all run before `useEffect` ones. → *More:* §5.3 Cleanup — undo what setup did · Ch 15 Q18

Q76. Can you have multiple `useEffect`s in one component? Should you? Yes, and you should split by **purpose**: one Effect per external system ("sync with the chat connection", "sync with analytics"). Each then has its own correct dependencies and cleanup. One Effect doing unrelated things re-runs all of them whenever any dependency changes. → *More:* §5.2 `useEffect` basics

Q77. What is the difference between `useEffect` with `[]` and `componentDidMount`? Both run after the first mount, but `useEffect` runs **after the browser paints** (componentDidMount ran before paint — closer to `useLayoutEffect`), it captures props/state of that render (closures), and in StrictMode development it

runs setup → cleanup → setup. More importantly, the mental model differs: an Effect describes how to **start and stop syncing**, not a lifecycle moment. → *More:* §5.2 `useEffect` basics · §5.11 `useLayoutEffect` — before the browser paints

Q78. ⋆ `useEffect` vs `useLayoutEffect`? `useEffect` runs **after** the browser paints — non-blocking, right for most side effects. `useLayoutEffect` runs synchronously **after the DOM is updated but before paint**, so you can **measure** layout (an element's size or position) and set state that adjusts it without a visible flicker (tooltips, popovers, scroll restoration). It blocks painting, so use it only when needed. On the server it does nothing (React warns in old versions); in Server-rendered apps measure on the client only. → *More:* §5.11 `useLayoutEffect` — before the browser paints · Ch 15 Q21

Q79. What is `useInsertionEffect`? A Hook that runs during the commit, **before any layout Effects fire**, intended **only for CSS-in-JS library authors** to inject `<style>` tags before any layout is read. Application code shouldn't need it. → *More:* §8.7 CSS-in-JS

Q80. ⋆ What is `useRef`, and what are its use cases? `useRef(initial)` returns a mutable object `{ current }` that **persists across renders**; changing `.current` does **not** re-render. Use cases: (1) a reference to a **DOM node** (`<input ref={inputRef}>` to focus, scroll, measure); (2) storing values that don't affect the UI — interval IDs, `AbortController`s, the previous value, "is mounted" flags, a third-party widget instance. Don't read or write `ref.current` during render (except lazy initialisation) — it makes the component impure. → *More:* §5.8 `useRef` for values · §5.9 Refs to DOM nodes

Q81. ⋆ `useRef` vs `useState`? Both persist across renders. Updating **state** triggers a re-render and the new value is seen on the next render; updating a **ref** is immediate, mutable, and does **not** re-render. Use state for anything shown on screen; use a ref for things only your event handlers/Effects need (timer IDs, DOM nodes, latest values for callbacks). → *More:* §5.8 `useRef` for values · Ch 15 Q22

Q82. ⋆ How do you focus an input on mount? Use a ref and an Effect (or the `autoFocus` attribute for the simple case):

```
function Search() {
  const inputRef = useRef(null);
  useEffect(() => { inputRef.current.focus(); }, []);
  return <input ref={inputRef} />;
}
```

→ *More:* §5.9 Refs to DOM nodes

Q83. ⋆ How do you get the previous value of a prop or state (`usePrevious`)? The classic answer stores the value in a ref after each commit:

```
function usePrevious(value) {
  const ref = useRef();
  useEffect(() => { ref.current = value; }, [value]);
  return ref.current; // value from the previous render
}
```

Note it reads a ref during render, which React's docs discourage. The React-recommended alternative keeps the previous value in **state** and updates it during render when it differs (`if (value !== prev) { setPrev(value); … }`). → *More:* §5.8 `useRef` for values

Q84. ⋆ How do you pass a ref to your own component in React 19? What was `forwardRef`? In React 19, `ref` is an **ordinary prop** for function components: `function Input({ ref, ...props }) { return <input ref={ref} {...props} /> }`. Before 19 you had to wrap the component in `forwardRef((props, ref) => …)`; it still works but is planned for deprecation. Class components still don't receive `ref` as a prop. → *More:* §5.10 Passing refs to your own components · Ch 15 Q23

Q85. What is `useImperativeHandle`, and when should you use it? It lets a component **customise what a parent's ref receives** — instead of the DOM node, expose a small API: `useImperativeHandle(ref, () => ({ focus() {…}, scrollToTop() {…} }), [])`. Use it rarely, for imperative actions that can't be expressed as props (focus, play/pause a video, open a native dialog). In React 19 it works with `ref` taken as a prop — no `forwardRef` needed. → *More:* §5.10 Passing refs to your own components

Q86. What are callback refs, and what changed about them in React 19? Instead of a ref object you can pass a **function**: `<div ref={node => …}>`. React calls it with the DOM node when attached. React 19 lets

the callback **return a cleanup function**, called when the node is detached (instead of the old behaviour of calling the callback with `null`). Useful for measuring nodes in a list or attaching observers per element. → *More:* §5.9 Refs to DOM nodes

Q87. Can you read a ref during rendering? You shouldn't. `ref.current` is not tracked by React, so a render that reads it may show stale data and breaks purity (and the React Compiler will skip such components). DOM refs are also `null` during the first render because the node doesn't exist yet. Read and write refs in **event handlers and Effects**. → *More:* §5.8 `useRef` for values

---

6. Other Hooks, Rules of Hooks and custom Hooks

Q88. ⋆ What are Hooks, and why were they introduced? Hooks are functions (`useState`, `useEffect`, `useContext` …) that let function components use state and other React features. Introduced in React 16.8 (2019) because classes made logic reuse hard (HOCs and render props produced "wrapper hell"), spread related logic across lifecycle methods, and `this` confused people. Hooks let you group logic by concern and share it through **custom Hooks**. → *More:* §7.1 What a Hook really is

Q89. ⋆ What are the Rules of Hooks, and why do they exist? (1) Call Hooks only at the **top level** of a component or custom Hook — not in conditions, loops, nested functions, after an early `return`, or in `try/catch`. (2) Call them only from **React function components or custom Hooks**, not from ordinary functions or classes. React identifies each Hook's state by its **call order** in the component; a Hook called conditionally shifts the order and every Hook after it reads the wrong state. Enforced by `eslint-plugin-react-hooks`. (`use()` is the exception: it may be called conditionally.) → *More:* §7.2 The Rules of Hooks · Ch 15 Q25

Q90. What happens if you call `useState` conditionally? On a render where the condition flips, the number or order of Hook calls changes. React matches state by position, so later Hooks get the wrong values, and React throws "Rendered more/fewer hooks than during the previous render". Move the condition **inside** the Hook usage (or into an Effect body) instead. → *More:* §7.2 The Rules of Hooks

Q91. How does `useState` work internally (high level)? Each component instance has a **fiber** that stores a linked list of its Hooks in call order. On the first render `useState` creates a slot with the initial value and an update queue; the setter adds an update to that queue and schedules a render. On the next render, the n-th Hook call reads the n-th slot, applies the queued updates in order, and returns the result. That's why call order must be stable. → *More:* §7.1 What a Hook really is

Q92. ⋆ What is a custom Hook? Do two components using the same Hook share state? A **custom Hook** is a function whose name starts with `use` and which calls other Hooks, used to share **stateful logic** (not state) between components: `useFetch`, `useDebounce`, `useLocalStorage`, `useOnlineStatus`. Each call gets its **own independent state** — two components calling `useCounter()` have separate counters. To share the data itself, lift state up, use Context or a store. → *More:* §7.3 Custom Hooks — reusing stateful logic · Ch 15 Q26

Q93. Why must a custom Hook's name start with `use`? It's how React's lint rules and the React Compiler know the function may call Hooks, so they can check the Rules of Hooks inside and at the call site. A function that doesn't call any Hook shouldn't be named `use…` — make it a plain helper. → *More:* §7.3 Custom Hooks — reusing stateful logic

Q94. ⋆ What is `useMemo`, and when should you use it? `useMemo(() => compute(a, b), [a, b])` **caches the result** of a calculation between renders and recomputes only when a dependency changes. Use it for (1) genuinely **expensive** calculations (sorting/filtering thousands of items — measure first), and (2) keeping a **stable object/array reference** passed to a `memo` child or used as an Effect dependency. Don't wrap cheap computations — it adds overhead and noise. It's a performance hint, not a guarantee (React may drop the cache). → *More:* §10.6 `useMemo` for expensive calculations · §10.5 `useCallback` and `useMemo` for stable props

Q95. ⋆ What is `useCallback`, and why is it used? `useCallback(fn, deps)` returns the **same function object** between renders until a dependency changes. Functions are re-created on every render, so without it a function prop breaks `memo` on a child, or retriggers an Effect that depends on it. It doesn't make the function faster. `useCallback(fn, deps)` equals `useMemo(() => fn, deps)`. → *More:* §10.5 `useCallback` and `useMemo` for stable props

Q96. ⋆ `useMemo` vs `useCallback` — what's the difference? `useMemo` caches the **value returned** by calling a function; `useCallback` caches the **function itself**. Use `useMemo` for computed data and objects, `useCallback` for event handlers passed to memoized children or used in Effect deps. Both only matter when someone compares references (`memo`, dependency arrays). → *More:* §10.5 `useCallback` and `useMemo` for stable props · Ch 15 Q34

Q97. Does `useMemo` prevent a child from re-rendering? Not on its own. A child re-renders when its parent re-renders, regardless of props — unless the child is wrapped in `memo`. `useMemo` / `useCallback` only keep props referentially stable so `memo`'s shallow comparison can succeed. (Exception: memoizing the JSX element itself, `useMemo(() => <Child/>, [...])`, does let React skip it.) → *More:* §10.4 `memo` — skipping a child's re-render · §10.5 `useCallback` and `useMemo` for stable props

Q98. ⋆ What is `useReducer`, and when would you choose it over `useState`? `const [state, dispatch] = useReducer(reducer, initialState)` keeps state that you change by **dispatching actions** (`dispatch({ type: 'added', text })`) to a pure **reducer** function `(state, action) => newState`. Choose it when several values change together, the next state depends on complex rules, many event handlers update the same state, or you want update logic testable outside the component. `useState` is simpler for independent values. → *More:* §6.2 `useReducer` — consolidate update logic · Ch 15 Q27

Q99. What rules should a reducer follow? It must be **pure**: no side effects (no fetching, no timers, no logging to servers), no mutation of `state` or `action` — return a new object — and the same input must give the same output (no `Math.random()` / `Date.now()` inside; pass such values in the action). Handle unknown actions by throwing or returning state. React may call reducers twice in StrictMode to check this. → *More:* §6.2 `useReducer` — consolidate update logic

Q100. Is `dispatch` asynchronous? Can you dispatch several actions in a row? Like a state setter, `dispatch` **queues** the action and schedules a re-render; the `state` variable in the current render doesn't change. Several dispatches in one event are batched into one render, and each action is applied to the result of the previous one. `dispatch` has a stable identity, so it's safe to pass down or omit from Effect deps. → *More:* §6.2 `useReducer` — consolidate update logic

Q101. ⋆ What is Context, and how does `useContext` work? Context passes a value deeply without props. `const ThemeCtx = createContext('light')` creates it; `<ThemeCtx value="dark">…</ThemeCtx>` provides it (React 19 — earlier `<ThemeCtx.Provider value>`); `useContext(ThemeCtx)` (or `use(ThemeCtx)`) reads the value from the **nearest provider above**. When the provided value changes, every component that reads it re-renders. Typical uses: theme, current user, locale, a store or service instance. → *More:* §6.3 Context — pass data deeply without props

Q102. What does a component get from `useContext` when there's no provider? What's the default value for? It gets the **default value** passed to `createContext(default)`. The default is useful for tests, for optional contexts, and as documentation of the shape. Many teams use `createContext(null)` and a custom Hook that throws a clear error if no provider is present (`if (!ctx) throw new Error('useCart must be used inside <CartProvider>')`). → *More:* §6.3 Context — pass data deeply without props · §11.7 Typing custom Hooks and Context

Q103. ⋆ What is the common pitfall when passing an object as a Context value? `<Ctx value={{ user, setUser }}>` creates a **new object every render** of the provider, so every consumer re-renders even when `user` didn't change. Fix: memoize the value with `useMemo`, **split** contexts (state vs dispatch/actions, or rarely vs often changing data), and keep providers low in the tree. Context has no selector — a consumer re-renders on any change of the value, and `memo` on the consumer doesn't stop it. → *More:* §6.4 Context performance caveats · Ch 15 Q29

Q104. ⋆ Is Context a state-management tool? Context vs Redux? Context is a **transport** (dependency injection): it delivers a value; `useState` / `useReducer` in the provider holds the state. It's fine for low-frequency global values. Redux (or Zustand) is a **store** with selective subscriptions (components re-render only when their selected slice changes), middleware, DevTools time-travel and conventions for large teams. Use Context for theme/user/locale; a store for frequently updated, widely shared client state. → *More:* §6.3 Context — pass data deeply without props · §6.7 Redux Toolkit · Ch 15 Q28

Q105. What is `useId`? `useId()` generates a **unique, stable ID** per component instance that is the **same on the server and the client**, so it doesn't cause hydration mismatches. Use it to link labels and ARIA attributes: `<label htmlFor={id}>` + `<input id={id}>`, `aria-describedby`. Don't use it for list keys. → *More:* §7.8 `useId` — IDs for accessibility

Q106. ⋆ What are `useTransition` and `useDeferredValue`, and how do they differ? Both mark work as **non-urgent** so typing and clicks stay responsive. `useTransition` returns `[isPending, startTransition]`; you wrap the **state update** yourself (`startTransition(() => setTab(next))`) — use it when you own the setter. `useDeferredValue(value)` gives a **lagging copy** of a value you receive (often a prop); React renders the urgent UI with the old copy first, then re-renders with the new one in the background. Neither debounces network requests. → *More:* §7.10 `useTransition` — non-urgent updates · §7.11 `useDeferredValue` — let part of the UI lag behind · Ch 15 Q41

Q107. What is `useSyncExternalStore`? A Hook for subscribing to data that lives **outside React** (a store, browser APIs like `navigator.onLine` or `matchMedia`, a third-party library) in a way that's safe with concurrent rendering (no "tearing", where different components show different versions). Signature: `useSyncExternalStore(subscribe, getSnapshot, getServerSnapshot?)`. Store libraries (Redux, Zustand) use it internally. → *More:* §7.9 `useSyncExternalStore` — subscribing to outside data · Ch 15 Q32

Q108. ⋆ What is `useEffectEvent` (React 19.2)? It creates an **Effect Event**: a function you call from inside an Effect that always sees the **latest** props and state but is **not reactive** — it doesn't need to be (and must not be) in the dependency array. Use it for the "read but don't re-sync on" part of an Effect: e.g. reconnect only when `roomId` changes, but show a notification using the current `theme`. Call it only from inside Effects (or other Effect Events) — never during render, from an `onClick`, or by passing it to other components or Hooks. Its identity deliberately **changes every render**, so it is not a stable callback and must never appear in a dependency array. → *More:* §7.7 Hooks that take callbacks — `useEffectEvent` · Ch 15 Q24

Q109. What is `useDebugValue`? A Hook for custom-Hook authors: `useDebugValue(isOnline ? 'Online' : 'Offline')` shows a label next to the Hook in React DevTools. It has no effect on behaviour; use it in shared library Hooks. → *More:* §16.3 Every Hook in one table

Q110. Do Hooks replace HOCs and render props? Mostly. Custom Hooks share stateful logic without adding wrapper components or changing the tree. HOCs and render props are still used when you need to wrap rendering (e.g. a boundary, a layout, a library API built on them), and error boundaries are still classes. In new code, prefer a custom Hook first. → *More:* §14.7 Choosing a reuse pattern · Ch 15 Q50

Q111. What are best practices for Hooks? Follow the Rules of Hooks with the lint plugin on; keep Effects for external syncing only; list every dependency honestly (fix the code, not the array); one Effect per concern with a cleanup; prefer derived values and event handlers to state + Effects; extract custom Hooks for repeated logic and name them by purpose (`useOnlineStatus`, not `useMount`); avoid premature `useMemo` / `useCallback` (and let the React Compiler handle memoization where it's enabled). → *More:* §7.2 The Rules of Hooks · §5.6 You might not need an Effect

---

7. React 19 features

Q112. ⋆ What is an Action in React 19? An **Action** is a function — usually **async** — that you run inside a **transition**, typically to submit or mutate data. React tracks it for you: `isPending` is true while it runs, errors go to the nearest error boundary, and optimistic updates are rolled back when it finishes. One caveat: inside a hand-written `startTransition(async () => …)`, state updates made **after an `await`** are not part of the transition — wrap them in another `startTransition` (a known limitation until JavaScript gets async context). You get Actions by passing a function to `startTransition`, to `<form action>`, `<button formAction>`, or `useActionState`. → *More:* §4.4 Form Actions — `<form action={fn}>` · Ch 15 Q14

Q113. ⋆ How do forms work with `<form action={fn}>`? Pass a function to the `action` prop; on submit React calls it with a `FormData` object (the form's fields by `name`), inside a transition, and **resets** uncontrolled fields after a successful action. No `e.preventDefault()`, no `onSubmit`, no controlled inputs needed:

```
async function save(formData) { await api.addTodo(formData.get('title')); }
<form action={save}><input name="title" /><button>Add</button></form>
```

If the function is a **Server Function**, the form even works before JavaScript loads (progressive enhancement). → *More:* §4.4 Form Actions — `<form action={fn}>`

Q114. ⋆ What does `useActionState` do? `const [state, formAction, isPending] = useActionState(action, initialState)` wraps an action so its **return value becomes state**. Your action receives `(previousState, formData)` ; whatever it returns (an error message, the saved item, field errors) becomes the new `state` . Pass `formAction` to `<form action>` . It replaced the canary-only `useFormState` from `react-dom` . Ideal for showing validation/server errors after a submit.

```
const [error, submit, pending] = useActionState(async (prev, fd) => {
  const res = await signup(fd.get('email'));
  return res.ok ? null : res.message;
}, null);
```

→ *More:* §4.5 `useActionState` — the result of the last submission

Q115. ⋆ What does `useFormStatus` do, and what's its main gotcha? `const { pending, data, method, action } = useFormStatus()` (from `react-dom`) tells a component the status of the `<form>` **it is rendered inside** — perfect for a reusable `<SubmitButton>` that disables itself while submitting. Gotcha: it only reads a **parent** form, so it returns `pending: false` if called in the same component that renders the `<form>` ; put it in a child component. → *More:* §4.6 `useFormStatus` — status of the parent form

Q116. ⋆ What does `useOptimistic` do? `const [optimisticItems, addOptimistic] = useOptimistic(items, (current, newItem) => [...current, newItem])` shows a **temporary** version of the state while an Action is running — e.g. the new comment appears instantly with a "sending…" style. When the Action ends, React drops the optimistic value and shows the real state again (so on failure it rolls back automatically). Call `addOptimistic` inside an Action/transition. → *More:* §4.7 `useOptimistic` — instant feedback, automatic rollback

Q117. ⋆ What is the `use()` API? How is it different from other Hooks? `use(resource)` reads a **promise** or a **context**. With a promise, the component **suspends** (the nearest `<Suspense>` fallback shows) until it resolves, then gets the value; a rejection goes to the nearest error boundary. Unlike Hooks, `use` **can be called conditionally** and in loops (but not inside `try/catch`). The promise must be **stable** — created in a Server Component, a loader or a cache — not created fresh during a Client Component's render: a new promise every render suspends again each time (React warns about an "uncached promise"). → *More:* §9.11 Suspense and `use()` · Ch 15 Q40

Q118. `use(Context)` vs `useContext(Context)` ? They return the same value. `use` is more flexible because it can be called **after an early return or inside an** `if` : `if (!show) return null; const theme = use(ThemeCtx);` . `useContext` must follow the Rules of Hooks. → *More:* §6.3 Context — pass data deeply without props · §9.11 Suspense and `use()`

Q119. ⋆ What are React Server Components (RSC)? Server Components run **only on the server** (at build time or per request) and send their rendered result — not their code — to the browser. They can be `async` , read databases and files directly, keep secrets on the server, and add **zero JavaScript** to the client bundle. They can't use state, Effects, event handlers or browser APIs. **Client Components** (marked by `'use client'` at the top of a module) are the interactive parts and are still server-rendered to HTML then hydrated. They need a framework/bundler that supports them (Next.js App Router, React Router's RSC support). → *More:* §13.7 React Server Components · Ch 15 Q45

Q120. ⋆ When do you choose a Server Component vs a Client Component? Server Component (the default in Next.js App Router): fetching data, accessing backend resources or secrets, rendering large static content, heavy dependencies (markdown parsers, date libraries) you don't want to ship. Client Component: anything interactive — state, Effects, event handlers, browser APIs (`localStorage` , `window`), custom Hooks using these, and most third-party UI libraries. Push `'use client'` down to the **leaves** (a `LikeButton` , not the whole page). → *More:* §13.7 React Server Components · §13.8 Directives: `'use client'` and `'use server'`

Q121. ⋆ What are Server Functions (Server Actions)? What does `'use server'` mean? A **Server Function** is an async function that runs on the server but can be **called from the client** — React sends a request under the hood. Mark it with `'use server'` (at the top of the function body or of a module). Used as a form `action` or called from an event handler/transition, it performs mutations (save to DB, then revalidate). `'use server'` does **not** mark Server Components — those need no directive. Treat every Server Function as a public API endpoint: validate input and check authorisation. → *More:* §4.8 Server

Functions and progressive enhancement (overview) · §13.8 Directives: `'use client'` and `'use server'` · Ch 15 Q46

Q122. Can a Server Component import a Client Component, and vice versa? A Server Component can import and render a Client Component, passing **serialisable** props (plain data, JSX, promises, Server Functions — not ordinary functions or class instances). A Client Component **cannot import** a Server Component (the import would become client code), but it can **receive one as `children` or another prop** that was rendered by a Server Component parent. → *More:* §13.7 React Server Components

Q123. ⋆ What is the React Compiler? A **build-time** tool (a Babel plugin, stable since v1.0 in October 2025) that analyses components and Hooks and **automatically memoizes** values, callbacks and JSX — like perfectly placed `useMemo`, `useCallback` and `memo`. It requires code that follows the **Rules of React** (pure render, no mutation of props/state, Rules of Hooks) and skips components it can't prove safe. With it, manual memoization is mostly unnecessary; it doesn't fix slow algorithms, large DOMs or big bundles. Opt a component out with `'use no memo'`. → *More:* §10.7 React Compiler · Ch 15 Q35

Q124. What other React 19 changes might an interviewer ask about? `<Context value>` as provider; **ref cleanup** functions; `useDeferredValue(value, initialValue)`; rendering `<title>`, `<meta>`, `<link>` anywhere (hoisted to `<head>`); stylesheets with `precedence`; `async` scripts deduplicated; **resource APIs** `preload`, `preinit`, `prefetchDNS`, `preconnect` from `react-dom`; clearer **hydration-mismatch** errors with a diff; errors no longer re-thrown, with new root options `onCaughtError` / `onUncaughtError` / `onRecoverableError`; full support for **custom elements**. 19.2 added `<Activity>` (hide UI while keeping its state), `useEffectEvent`, `cacheSignal` and Performance Tracks in Chrome DevTools. → *More:* §5.10 Passing refs to your own components · §14.9 Error boundaries · §13.5 Hydration mismatches

Q125. What was removed or deprecated in React 19 (migration questions)? Removed: `ReactDOM.render` / `hydrate`, `unmountComponentAtNode`, `findDOMNode`, `propTypes` checks, `defaultProps` on function components, legacy context (`contextTypes`), string refs, module-pattern factories, `react-test-renderer/shallow`; `act` moved to `react`. Deprecated: `forwardRef` (planned), `element.ref` (ref is a prop now), `react-test-renderer`. TypeScript: `useRef` requires an argument, refs are mutable, `JSX` namespace moved to `React.JSX`. A codemod (`npx codemod react/19/migration-recipe`) automates much of it. → *More:* §11.1 Setup and how type-checking fits in · §16.18 Common errors and fixes

Q126. What is `<Activity>`? A React 19.2 component: `<Activity mode={visible ? 'visible' : 'hidden'}>`. Hidden content keeps its **state** (and DOM), but React hides it with `display: none`, cleans up its Effects (they run again when it becomes visible), and re-renders it only at low priority. Use it for tabs or panels you want to switch back to instantly, or to pre-render the likely next screen. → *More:* §5.4 StrictMode's double run (development only)

Q127. What are transitions and why do Actions use them? A **transition** marks a state update as non-urgent (`startTransition(() => …)`): React can interrupt it, keeps showing the current UI instead of a Suspense fallback, and exposes `isPending`. Since React 19 `startTransition` accepts **async** functions, which is what an Action is — so pending state, error handling and optimistic updates are all tied to one transition. → *More:* §7.10 `useTransition` — non-urgent updates · §4.4 Form Actions — `<form action={fn}>`

---

8. State management: Context, Redux, Zustand and server state

Q128. ⋆ How do you decide between local state, Context and an external state manager? Start **local** (`useState` in the component, lifted only as high as needed). Use the **URL** for state that should survive refresh or be shareable (filters, page, tab). Use a **server-state library** (TanStack Query, RTK Query, framework loaders) for data from APIs. Use **Context** for rarely changing global values (theme, user, locale). Reach for a **store** (Zustand, Redux Toolkit) when lots of client state is shared and updated often, or you need devtools/middleware and team conventions. → *More:* §6.10 Choosing — comparison and decision flow · Ch 15 Q30

Q129. ⋆ What is Redux, and what are its core principles? **Redux** is a predictable state container: (1) a **single source of truth** — the whole app state in one store; (2) **state is read-only** — the only way to change it is to dispatch an **action** (a plain object `{ type, payload }`); (3) changes are made by **pure reducers** `(state, action) => newState`. This makes every change traceable (Redux DevTools, time-travel

debugging) and testable. Today you write Redux with **Redux Toolkit** and the `react-redux` Hooks `useSelector` and `useDispatch`. → *More:* §6.7 Redux Toolkit

Q130. ⋆ Why is Redux Toolkit (RTK) recommended over "plain" Redux? RTK is the official way to write Redux. It removes boilerplate: `configureStore` (sensible defaults, DevTools, thunk middleware, mutation checks), `createSlice` (action creators and reducer in one place, with **Immer** so you can write "mutating" updates), `createAsyncThunk` for async logic, `createSelector` for memoized selectors, and **RTK Query** for data fetching and caching. Hand-written action types, switch statements and spread-heavy reducers are no longer needed. → *More:* §6.7 Redux Toolkit

Q131. What happens when you dispatch an action in Redux? `dispatch(action)` passes the action through the **middleware** chain (thunks, logging …), then the store calls the root **reducer** with the current state and the action; the reducer returns the new state. The store notifies subscribers; each component's `useSelector` re-runs its selector and the component re-renders **only if the selected value changed** (compared by reference). → *More:* §6.7 Redux Toolkit

Q132. What is Redux middleware? What are thunks? Middleware wraps `dispatch` to add behaviour between dispatching and the reducer: logging, crash reporting, async logic. Its signature is `store => next => action => { … next(action) … }`. A **thunk** is a function you dispatch instead of an object: `dispatch(async (dispatch, getState) => { … })`, letting you run async work and dispatch real actions later. `createAsyncThunk` generates `pending` / `fulfilled` / `rejected` actions for you. Redux-Saga (generators) and Redux-Observable (RxJS) are heavier alternatives. → *More:* §6.7 Redux Toolkit

Q133. What are selectors and `createSelector`? How do you prevent unnecessary re-renders with Redux? A **selector** is a function that reads part of the state: `state => state.cart.items`. `useSelector` re-renders when the selector's result changes **by reference**, so selectors that build new arrays/objects (`state.items.filter(...)`) re-render every time. Use `createSelector` (Reselect) to **memoize** derived data — it recomputes only when its inputs change. Select the smallest piece you need, and use `shallowEqual` when returning objects. → *More:* §6.7 Redux Toolkit

Q134. What is normalised state, and why does it help? Normalised state stores each entity **once**, in a lookup by ID — `{ ids: [1, 2], entities: { 1: {...}, 2: {...} } }` — and refers to it by ID elsewhere, like database tables. Updates touch one place, there are no duplicate copies to get out of sync, and lookups are O(1). RTK's `createEntityAdapter` generates this structure and its reducers. → *More:* §6.7 Redux Toolkit

Q135. Should all state go into Redux? No. Keep **local UI state** (an open dropdown, a form's draft input) in components, **server data** in a server-state cache (RTK Query / TanStack Query), and URL state in the URL. Put in Redux only state that many unrelated parts of the app read and update, or that needs time-travel/debugging. Over-centralising makes every keystroke a global action. → *More:* §6.1 Kinds of state — and where each belongs · §6.6 When to reach for a library

Q136. ⋆ What is Zustand? How does it compare with Redux and Context? Zustand is a tiny store library: `const useCart = create((set) => ({ items: [], add: (i) => set((s) => ({ items: [...s.items, i] })) }))`, then `useCart((s) => s.items)` in any component — **no provider**, no actions/reducers ceremony. Components re-render only when their **selected** value changes (unlike Context). Compared with Redux it has less structure and tooling but far less code. Gotcha (v5): a selector returning a new object each time causes an infinite loop — wrap it in `useShallow`. → *More:* §6.8 Zustand

Q137. ⋆ What is the difference between client state and server state? What is TanStack Query (React Query)? Client state is owned by the browser (UI toggles, form drafts) — synchronous and always up to date. **Server state** lives on a server: you only have a **cached copy** that can go stale, needs loading/error states, deduplication, refetching and invalidation. **TanStack Query** manages server state: `useQuery({ queryKey: ['todos', page], queryFn })` gives `data`, `isPending`, `error`, caching by key, background refetch, retries; `useMutation` + `invalidateQueries` refresh data after writes. → *More:* §6.9 TanStack Query — server state · §9.8 TanStack Query basics · Ch 15 Q31

Q138. What is Flux? Flux is the architecture Facebook proposed alongside React: data flows one way — **actions** go through a **dispatcher** to **stores**, which update and notify **views**, which create new actions. Redux simplified it to a single store and pure reducers. It's mainly asked as history/background. → *More:* §6.7 Redux Toolkit

Q139. MobX, Jotai, Recoil — what are they? MobX: observable objects with automatic dependency tracking — mutate state directly and observing components update (OOP/reactive style). **Jotai**: **atoms** — small

independent pieces of state composed bottom-up, with fine-grained re-renders. **Recoil**: Meta's atom library, now **archived** (unmaintained) — don't choose it for new projects. The trade-off across all: less boilerplate vs. Redux's explicitness and tooling. → *More:* §6.10 Choosing — comparison and decision flow

Q140. How do you keep a user logged in across page refreshes with Context? React state is lost on reload, so persist the **source of truth** outside React: an **HTTP-only cookie** session (safest — JavaScript can't read it, so XSS can't steal it), and on app start call `/api/me` to restore the user into Context (show a loading state meanwhile). Avoid storing long-lived tokens in `localStorage`, because any XSS can read it. Server-rendering frameworks can read the cookie on the server and render the logged-in page directly. → *More:* §6.3 Context — pass data deeply without props · §9.7 Middleware, lazy routes and framework mode

---

9. Forms, routing and data fetching

Q141. ⋆ Controlled vs uncontrolled inputs — what's the difference and when to use each? Controlled: React state holds the value — `<input value={name} onChange={e => setName(e.target.value)} />` — so you can validate, format or disable on every keystroke. **Uncontrolled**: the DOM holds the value; you read it on submit via `FormData` or a ref, with `defaultValue` for the initial value. React 19 form Actions make uncontrolled forms the simpler default; use controlled when the UI must react to each change (live validation, dependent fields). → *More:* §4.2 Controlled vs uncontrolled inputs · Ch 15 Q13

Q142. ⋆ Why do I get "A component is changing an uncontrolled input to be controlled"? The `value` prop went from `undefined` (uncontrolled) to a string (controlled), usually because state started as `undefined` or a field was missing on an object. Initialise with an empty string (`useState('')`, `value={user.name ?? ''}`) and keep the input controlled for its whole life. → *More:* §4.2 Controlled vs uncontrolled inputs · §16.18 Common errors and fixes

Q143. How do you handle multiple inputs with one change handler? Give each input a `name` and update by key:

```
const [form, setForm] = useState({ email: '', password: '' });
const onChange = (e) => setForm({ ...form, [e.target.name]: e.target.value });
<input name="email" value={form.email} onChange={onChange} />
```

For checkboxes use `e.target.checked`. With form Actions you may not need state at all — read `formData.get('email')`. → *More:* §4.2 Controlled vs uncontrolled inputs

Q144. ⋆ How do you validate forms in React? Layers: (1) **native HTML** attributes (`required`, `type="email"`, `minLength`, `pattern`) — free and accessible; (2) **custom checks** on submit (and on blur for touched fields), storing an `errors` object and rendering messages linked with `aria-describedby` and `aria-invalid`; (3) a **schema** (Zod) shared by client and server; (4) the **server always re-validates**. Libraries: React Hook Form + Zod resolver for large forms. Show errors after the user leaves a field or submits, not on the first keystroke. → *More:* §4.3 Validation · §4.9 Form libraries — React Hook Form and Zod (overview)

Q145. What are React Hook Form and Formik? Why use a form library? They manage values, touched/dirty flags, validation and submission for big forms. **React Hook Form** uses uncontrolled inputs registered with refs (`register('email')`), so typing doesn't re-render the whole form — fast and small, with Zod/Yup resolvers. **Formik** is controlled and older, re-renders more, and is less actively maintained. For simple forms, React 19 Actions + native validation are often enough. → *More:* §4.9 Form libraries — React Hook Form and Zod (overview)

Q146. How do you handle file uploads in React? Use an uncontrolled `<input type="file">` (its value can't be set by script). Read `e.target.files` or `formData.get('file')`, and send with `fetch` and a `FormData` body (don't set `Content-Type` manually — the browser adds the multipart boundary). For progress, use `XMLHttpRequest`'s `upload.onprogress` (fetch has no upload progress); preview images with `URL.createObjectURL(file)` and revoke it in cleanup. → *More:* §4.1 How a form submission works

Q147. ⋆ What is client-side routing, and how does React Router work? In a **single-page app (SPA)** the browser loads one HTML page; a router reads the URL and renders the matching components, and `<Link>` changes the URL with the **History API** (`pushState`) instead of loading a new page. React Router v8 (one

`react-router` package) has three modes: **declarative** (`<BrowserRouter>`, `<Routes>`, `<Route>`), **data** (`createBrowserRouter` + `<RouterProvider>` with loaders/actions) and **framework** (route modules, SSR). → *More:* §9.1 Client-side routing · §9.2 React Router v8 and its three modes

Q148. How do you read URL params and query strings, and navigate programmatically? `const { id } = useParams()` for `/users/:id`; `const [searchParams, setSearchParams] = useSearchParams()` for `?q=…`; `const navigate = useNavigate(); navigate('/home', { replace: true })` after a successful action; `<Navigate to="/login" replace />` to redirect during render. `replace` swaps the current history entry instead of adding one (so Back doesn't return to the form). → *More:* §9.3 Routes, links and URL parameters (declarative mode)

Q149. How do you implement nested routes, layouts and a 404 page? Nest `<Route>`s (or route objects with `children`); the parent renders shared UI (header, sidebar) and an **`<Outlet />`** where the matching child goes. An `index` route renders at the parent's exact path. A catch-all `path="*"` route renders the Not Found page. In data mode, `errorElement` / `ErrorBoundary` handles loader errors and unknown routes. → *More:* §9.4 Nested routes and layouts

Q150. ⋆ How do you implement protected (private) routes? Declarative mode: a wrapper that checks auth and either renders `<Outlet/>`/children or `<Navigate to="/login" replace state={{ from: location }} />`. Data/framework mode: **middleware** or a loader that throws `redirect('/login?next=…')` before the page loads (no flash of protected content). Remember client-side guards are **UX only** — the API must still reject unauthorised requests.

```
function RequireAuth() {
  const { user } = useAuth(); const location = useLocation();
  return user ? <Outlet /> : <Navigate to="/login" replace state={{ from: location }} />;
}
```

→ *More:* §9.7 Middleware, lazy routes and framework mode

Q151. What are loaders and actions in React Router? A **loader** is a function attached to a route that fetches its data **before** the route renders (in parallel for nested routes), read with `useLoaderData()` — no fetch-in-Effect waterfalls. An **action** handles a route's mutations (`<Form method="post">`), after which loaders re-run automatically so the page shows fresh data. `useNavigation()` exposes pending state; `useFetcher()` does mutations without navigating. → *More:* §9.5 Loaders — data before render (data mode) · §9.6 Actions, `<Form>` and fetchers

Q152. ⋆ How do you fetch data in React? Compare the options. (1) **`useEffect` + `fetch`** — fine for learning, but you must handle loading, errors, races, caching and no SSR yourself. (2) A **data library** — TanStack Query, SWR, RTK Query — caching, dedupe, retries, refetch. (3) **Router loaders** — data starts loading with navigation, in parallel. (4) **Server Components** / framework data APIs — fetch on the server, no client JS. (5) **`use()` + Suspense** with a cached promise. In production prefer 2–4. → *More:* §5.7 Fetching data in Effects and race conditions · §9.8 TanStack Query basics · §9.5 Loaders — data before render (data mode)

Q153. ⋆ What are common pitfalls when fetching data in React? Race conditions (stale response wins) — no cleanup; **waterfalls** (child fetches only after parent's data arrives) — hoist or parallelise with loaders; no loading/error/empty states; fetching in an Effect with an object dependency — infinite loop; double requests in StrictMode dev with no abort; no caching so every mount refetches; setting state after unmount; and leaking secrets by calling third-party APIs with keys from the browser. → *More:* §5.7 Fetching data in Effects and race conditions · §9.12 Loading and error states that users like

Q154. What is Suspense, and how is it used for data fetching? `<Suspense fallback={<Spinner/>}>` shows the fallback while any component inside **suspends** — waiting for lazy code (`lazy`), a promise read with `use()`, or a Suspense-enabled library (TanStack Query's `useSuspenseQuery`, framework loaders). It lets you declare loading states by **placement** instead of `if (loading)` in every component, and enables streaming SSR. Effects or `useEffect` fetches don't trigger Suspense. → *More:* §9.11 Suspense and `use()` · Ch 15 Q39

Q155. How do you cancel a request or debounce API calls in a search box? Debounce the **value** (`useDebounce(query, 300)`) so the fetch runs only after the user pauses, and **abort** the previous request in the Effect's cleanup with `AbortController`. Also ignore empty queries and cache results per query.

TanStack Query does the cancel/caching part when you put the query in the key. → *More:* §7.5 `useDebounce` — wait until the user stops · Ch 15 Q20

Q156. What are optimistic updates? How do you roll back on error? Update the UI **before** the server confirms (a like count goes up instantly), then reconcile with the real response. In React 19 use `useOptimistic` inside an Action — rollback is automatic when the Action ends. With TanStack Query: in `onMutate` snapshot and update the cache, in `onError` restore the snapshot, in `onSettled` invalidate. Use only when success is very likely. → *More:* §4.7 `useOptimistic` — instant feedback, automatic rollback · §9.10 Mutations, invalidation and optimistic updates

Q157. How do you handle authentication tokens and API calls (interceptors, refresh)? Centralise requests in one API module (a `fetch` wrapper or axios instance) that attaches credentials and handles `401`: try a **refresh** once, retry the original request, otherwise log out. Prefer **HTTP-only, Secure, SameSite cookies** over tokens in `localStorage`. Keep auth state in Context/store and guard routes. → *More:* §9.7 Middleware, lazy routes and framework mode

---

10. Performance

Q158. ⋆ How do you optimise a React application's performance? Measure first (React DevTools Profiler, Performance panel, Lighthouse/Web Vitals). Then: fix unnecessary re-renders (move state down, lift content up as `children`, split contexts, `memo` + stable props — or enable the **React Compiler**); make expensive work cheaper (`useMemo`, better algorithms, `useDeferredValue`/transitions); render less DOM (**virtualise** long lists, paginate); ship less JS (**code splitting** with `lazy`, smaller dependencies, Server Components); optimise loading (SSR/streaming, images with sizes and lazy loading, caching with a data library). → *More:* §10.11 A performance checklist · Ch 15 Q36

Q159. ⋆ What is `React.memo`, and when doesn't it help? `memo(Component)` makes React **skip re-rendering** the component when its props are **shallowly equal** (each prop compared with `Object.is`) to last time. It doesn't help when a prop is a new object/array/function every render (fix with `useMemo`/`useCallback`), when `children` JSX is passed (new elements each render), when the component's own state or a context it reads changes, or when the component is cheap anyway. A custom comparison can be passed as the second argument. → *More:* §10.4 `memo` — skipping a child's re-render · Ch 15 Q33

Q160. How do you prevent unnecessary re-renders without `memo`? Two composition tricks: **move state down** — put the state into a small component so only it re-renders (e.g. an input's state in `<SearchBox>` instead of the page); and **lift content up** — pass expensive subtrees as `children` to the component that owns the changing state, so they are created by the parent and keep the same element reference. Also keep context values stable and avoid creating components inside components. → *More:* §10.2 When components re-render · §10.10 Common performance bugs

Q161. ⋆ What is code splitting? How do `lazy` and `Suspense` work? Code splitting breaks the bundle into chunks loaded on demand, so the first page downloads less JavaScript. `const Settings = lazy(() => import('./Settings'))` creates a component whose code is fetched (via dynamic `import()`) the first time it renders; wrap it in `<Suspense fallback={…}>` to show something meanwhile. Split by **route** first, then by heavy, rarely used widgets (charts, editors, modals). → *More:* §10.8 Code splitting with `lazy` and `<Suspense>` · Ch 15 Q37

Q162. Does `lazy` support named exports? No — `lazy` expects a module with a **default export**. For a named export, map it: `lazy(() => import('./Charts').then(m => ({ default: m.LineChart })))`, or create a small re-export file. → *More:* §10.8 Code splitting with `lazy` and `<Suspense>`

Q163. ⋆ How do you render a list of 10,000 rows efficiently? What is virtualization (windowing)? Virtualization renders only the rows **visible** in the scroll area (plus a small buffer), positioning them inside a container sized for the full list, and swaps rows as you scroll. DOM stays small (~30 rows) no matter how long the list. Libraries: **TanStack Virtual**, react-window, react-virtuoso. Alternatives: pagination, "load more", or CSS `content-visibility: auto`. Give rows stable keys and fixed or measured heights. → *More:* §10.9 List virtualization · Ch 15 Q38

Q164. ⋆ What are debouncing and throttling? Where do you use them in React? Debounce: run only after events **stop** for N ms — search-as-you-type, autosave, resize end. **Throttle**: run **at most once** every N ms while events keep coming — scroll position, mouse move, infinite-scroll checks. In React, debounce a

value with `useDebounce`, or keep a debounced function stable in a ref/`useMemo` (a new debounced function each render resets its timer), and clear timers in cleanup. → *More:* §7.5 `useDebounce` — wait until the user stops

Q165. What does the React DevTools Profiler show? How do you find why a component re-rendered? The **Profiler** records commits and shows, per component, how long it took to render and (with "Record why each component rendered" on) **why** — props changed, state, hooks, parent rendered. Flame and ranked charts show the most expensive components. "Highlight updates when components render" visualises re-renders live. Programmatically, `<Profiler id onRender>` reports timings. → *More:* §10.3 Measure first: DevTools Profiler and `<Profiler>`

Q166. ⋆ What are common React performance bugs? Components defined inside components (remount every render); index keys on dynamic lists; new object/array/function props breaking `memo`; one giant context with frequently changing value; storing derived data in state + Effects (extra renders); fetching waterfalls; expensive work in render without memoization; huge lists without virtualization; importing whole libraries (lodash, moment) instead of what you use; memory leaks from missing cleanup. → *More:* §10.10 Common performance bugs

Q167. How do you optimise images, bundles and initial load (Core Web Vitals)? Core Web Vitals are Google's user-experience metrics: LCP (largest content shown), INP (responsiveness to input), CLS (layout shift). Improve them with SSR/SSG or streaming for faster first content, responsive images with width/height set (no layout shift) and `loading="lazy"` below the fold, `preload` of critical resources, code splitting and tree-shaking, analysing bundles (e.g. `rollup-plugin-visualizer`), transitions for heavy updates (INP), and caching/CDNs. → *More:* §10.11 A performance checklist · §13.1 Rendering strategies: CSR, SSR, SSG, ISR, streaming, PPR

Q168. What is tree-shaking? Tree-shaking is the bundler removing code that is never imported/used, based on static ES module `import` / `export` analysis. It works best with ESM libraries and named imports (`import { debounce } from 'lodash-es'`). CommonJS modules and side-effectful files are hard to shake. → *More:* §10.8 Code splitting with `lazy` and `<Suspense>`

Q169. When should you NOT use `memo` / `useMemo` / `useCallback`? When the component or calculation is cheap, when props change on almost every render anyway, when nothing compares the reference, or when the React Compiler already handles it. Memoization costs memory and comparison time and makes code harder to read; add it where the Profiler shows a real problem. → *More:* §10.4 `memo` — skipping a child's re-render · §10.7 React Compiler

---

11. Error boundaries, portals, patterns and accessibility

Q170. ⋆ What are error boundaries, and what don't they catch? An **error boundary** is a component that catches errors thrown while **rendering** its children (and in their lifecycle methods/constructors) and shows a fallback UI instead of a blank screen. They do **not** catch errors in **event handlers**, **async code** (`setTimeout`, promise callbacks outside of React's render), **server-side rendering**, or errors in the boundary itself. Handle those with `try/catch` and state — or rethrow into React via `setState(() => { throw err })`; Action errors in transitions do reach boundaries. → *More:* §14.9 Error boundaries · Ch 15 Q48

Q171. ⋆ How do you write an error boundary? Can it be a function component? It must be a **class** implementing `static getDerivedStateFromError(error)` (return state that shows the fallback) and/or `componentDidCatch(error, info)` (log it — `info.componentStack` shows where). There's no Hook equivalent; most apps use the **`react-error-boundary`** package (`<ErrorBoundary FallbackComponent={…} onReset={…}>`), which also offers `useErrorBoundary` to show async errors.

```
class Boundary extends React.Component {
  state = { error: null };
  static getDerivedStateFromError(error) { return { error }; }
  componentDidCatch(error, info) { logError(error, info.componentStack); }
  render() { return this.state.error ? this.props.fallback : this.props.children; }
}
```

→ *More:* §14.9 Error boundaries

Q172. Where should you place error boundaries? At several levels: one at the **top** (a "something went wrong" page so the app never shows a white screen), one per **route**, and around independent **widgets** (a chart, a comments panel) so one failure doesn't take down the page. Give users a way to **retry** (reset the boundary, e.g. with a `key` or `resetKeys`). → *More:* §14.9 Error boundaries

Q173. ⋆ What is a portal? How do events behave inside one? `createPortal(children, domNode)` renders children into a **different DOM node** (often `document.body`) while keeping them in the same place in the **React tree**. Used for modals, tooltips, dropdowns and toasts that must escape `overflow: hidden` or `z-index` stacking. Events from inside the portal **bubble through the React tree** (to React ancestors), not the DOM tree, and context still works. → *More:* §14.8 Portals · Ch 15 Q49

Q174. ⋆ What are Higher-Order Components (HOCs)? A **HOC** is a function that takes a component and returns a new component with extra behaviour: `const withAuth = (Comp) => (props) => useAuth().user ? <Comp {...props}/> : <Login/>`. Historic examples: `connect` from react-redux, `withRouter`. Drawbacks: wrapper nesting, prop-name collisions, hidden props and harder typing. Custom Hooks replace most of them today. → *More:* §14.6 Higher-order components (HOCs)

Q175. What are render props? A component takes a **function as a prop** (often `children`) and calls it to decide what to render, passing its internal state: `<MouseTracker>{({ x, y }) => <Dot x={x} y={y}/>}</MouseTracker>`. It shares logic while letting the caller control the UI. Mostly replaced by custom Hooks, still used by some libraries (e.g. list virtualizers, form libraries). → *More:* §14.5 Render props

Q176. ⋆ What are compound components? A set of components that work together and share implicit state through **Context**, giving a flexible, readable API: `<Tabs><Tabs.List><Tabs.Tab>…</Tabs.Tab></Tabs.List><Tabs.Panel>…</Tabs.Panel></Tabs>`. The parent owns state; children read it via context. Used by design systems (Radix, Headless UI, Reach) for tabs, menus, accordions, selects. → *More:* §14.4 Compound components

Q177. What is the container/presentational pattern? Is it still relevant? Container components handle data and logic (fetching, state); **presentational** components only receive props and render UI. It separates concerns and makes UI easy to test and reuse in Storybook. Today the "container" part is usually a **custom Hook** (or a Server Component / loader) rather than a wrapper component, but the idea — keep UI components dumb — still applies. → *More:* §14.2 Container / presentational

Q178. What are some React anti-patterns? Mutating state or props; copying props into state; Effects for derived data or event logic; index keys on dynamic lists; components defined inside components; giant components and "God" contexts; prop drilling through many layers; `useEffect` chains; disabling the exhaustive-deps lint rule; overusing `memo` everywhere without measuring; `dangerouslySetInnerHTML` with unsanitised input; deeply nested state that's hard to update. → *More:* §10.10 Common performance bugs · §5.6 You might not need an Effect

Q179. ⋆ How do you make React components accessible (a11y)? Use **semantic HTML** first (`<button>`, `<a href>`, `<label>`, `<nav>`, headings in order) — it brings keyboard support and screen-reader roles for free. Label every input (`<label htmlFor>` with `useId`), give images `alt`, keep visible **focus** and logical tab order, manage focus in modals (trap, restore), use ARIA only to fill gaps (`aria-expanded`, `aria-live` for updates, `aria-invalid`), ensure colour contrast, and test with keyboard, a screen reader, `eslint-plugin-jsx-a11y` and axe. Testing Library's role queries push you toward accessible markup. → *More:* §14.10 Accessibility in React · Ch 15 Q52

Q180. How do you manage focus in a modal dialog? On open, move focus into the dialog (the first focusable element or the dialog itself); **trap** Tab/Shift+Tab inside; close on Escape; on close, **return focus** to the element that opened it. Mark the rest of the page `inert` (or use the native `<dialog>` element with `showModal()`, which does most of this). Add `role="dialog"`, `aria-modal="true"` and `aria-labelledby`. → *More:* §15.13 Build 4 — Modal dialog (portal, focus trap, Escape)

Q181. How do you announce dynamic updates to screen readers? Use a **live region**: an element with `aria-live="polite"` (or `role="status"`) that exists in the DOM before the update; when its text changes, screen readers read it out. Use `role="alert"` (assertive) for errors. Typical uses: "3 results found", "Saved", toast messages. → *More:* §14.10 Accessibility in React

Q182. How do you structure a large React project? Group by **feature** (`features/cart/` with its components, hooks, API calls, tests) rather than by file type (`components/`, `hooks/`), plus a `shared/` or `ui/` folder for design-system components and an `app/` folder for routing and providers. Keep a public `index`

per feature, avoid deep cross-feature imports, colocate tests and styles, and let the framework's file routing define `app/` or `routes/`. → *More:* §14.11 Folder structure

Q183. What are the ways to style React components, and how do you choose? Plain CSS / Sass with naming conventions, **CSS Modules** (locally scoped class names), **utility CSS** (Tailwind), **CSS-in-JS** (styled-components, Emotion — runtime cost and friction with Server Components; zero-runtime options like vanilla-extract), inline `style` (dynamic values only), and component libraries (MUI, Chakra, shadcn/ui on Radix). Choose by team skills, Server Component compatibility, theming needs and performance. → *More:* §8.1 The styling landscape · §8.10 Choosing an approach

Q184. How do you implement a dark/light theme toggle? Store the choice in state (initialised from `localStorage` or `prefers-color-scheme`), set a `data-theme` attribute or class on `<html>`, and define colours as **CSS custom properties** per theme so no component re-renders for colours. Persist on change. With SSR, set the attribute in a tiny inline script before paint to avoid a flash of the wrong theme. → *More:* §8.8 Theming and dark mode

Q185. How do you localise (internationalise) a React app? Use a library — **react-i18next**, **FormatJS/react-intl**, or Lingui — that loads translation files per language and provides `t('key')` or `<FormattedMessage>`; format dates, numbers and currencies with the built-in `Intl` APIs; handle plurals and right-to-left languages (`dir="rtl"`); keep the locale in the URL or a cookie so SSR renders the right language; lazy-load translation bundles. → *More:* §6.3 Context — pass data deeply without props

---

12. SSR, SSG, hydration and Next.js

Q186. ⋆ CSR vs SSR vs SSG vs ISR — what's the difference? CSR (client-side rendering): the server sends an almost empty HTML page plus JavaScript, which builds the UI in the browser — simple hosting, slow first paint, weak SEO. **SSR** (server-side rendering): the server renders HTML **per request**, then the browser hydrates it — fast first content and fresh data, costs server work. **SSG** (static site generation): HTML rendered **at build time** and served from a CDN — fastest, but data is as old as the last build. **ISR** (incremental static regeneration, Next.js): static pages **re-generated in the background** after a time or on demand. Streaming and partial prerendering mix these per part of a page. → *More:* §13.1 Rendering strategies: CSR, SSR, SSG, ISR, streaming, PPR · Ch 15 Q43

Q187. ⋆ What is hydration? Hydration is React, in the browser, taking over HTML that was rendered on the server: it renders the same components, matches them to the existing DOM nodes instead of creating new ones, and **attaches event handlers and state** so the page becomes interactive. Use `hydrateRoot(container, <App/>)`. Until hydration finishes, the page is visible but clicks don't do anything. → *More:* §13.4 Hydration end to end · Ch 15 Q44

Q188. ⋆ What causes a hydration mismatch, and how do you fix it? A mismatch happens when the client's first render produces different HTML than the server's: using `Date.now()`, `Math.random()`, `window` / `localStorage`, the user's locale or time zone during render, invalid HTML nesting (`<div>` inside `<p>`), or browser extensions changing the DOM. Fixes: render the same output on both sides and switch to client-only values **after** hydration (in an Effect, or `useSyncExternalStore` with a server snapshot), use `useId` for IDs, and `suppressHydrationWarning` only for unavoidable one-element differences such as a timestamp. → *More:* §13.5 Hydration mismatches

Q189. What is streaming SSR and selective hydration? With **streaming** (`renderToPipeableStream` in Node, `renderToReadableStream` for Web streams), the server sends HTML in chunks: the shell first, and each `<Suspense>` boundary's content when its data is ready (with the fallback shown meanwhile). **Selective hydration** lets React hydrate parts independently and prioritise the part the user clicks. Result: fast first paint without waiting for the slowest data. `renderToString` can't wait for data or stream. → *More:* §13.3 Streaming SSR with `<Suspense>` · §13.2 The server rendering APIs

Q190. What is partial prerendering (PPR)? A page is split into a **static shell** prerendered at build time (served instantly from a CDN) and **dynamic holes** inside `<Suspense>` boundaries that are rendered and streamed per request. You get static speed for most of the page and fresh, personalised data where needed. React provides the primitives (`prerender`, `resume`); Next.js 16 enables it with Cache Components. → *More:* §13.6 Static generation and partial prerendering

Q191. ⋆ What is Next.js, and why use it? **Next.js** is a React framework: file-based routing, server rendering (SSR, SSG, ISR, streaming, PPR), React Server Components and Server Functions, API route handlers, image/font optimisation, bundling (Turbopack) and deployment integrations. You get SEO-friendly, fast-loading pages and full-stack features without wiring them yourself. React's docs recommend starting new apps with a framework like it. → *More:* §13.9 Next.js App Router overview

Q192. ⋆ Next.js App Router vs Pages Router? The **Pages Router** (`pages/`) uses `getServerSideProps` / `getStaticProps` for data and treats every component as a client component. The **App Router** (`app/` , since Next.js 13) is built on **Server Components** by default, nested **layouts**, `loading.js` / `error.js` files (Suspense and error boundaries per segment), streaming and Server Functions; you fetch with plain `async` / `await` in components. New features go to the App Router; Pages is still supported. → *More:* §13.9 Next.js App Router overview

Q193. What are the special files in the Next.js App Router? In each route folder: `page.js` (the route's UI, makes it reachable), `layout.js` (shared UI that **persists** across child navigations), `loading.js` (Suspense fallback), `error.js` (error boundary — a client component), `not-found.js` , `route.js` (an API handler for GET/POST…), `template.js` (like layout but remounts). Dynamic segments use folders like `[id]` ; `params` and `searchParams` are **Promises** since Next.js 15 (`await params`), and Next.js 16 removed the old synchronous access. At the project root, `proxy.ts` (renamed from `middleware.ts` in Next.js 16) intercepts requests before routing. → *More:* §13.9 Next.js App Router overview

Q194. How do caching and revalidation work in Next.js 16? By default (without Cache Components), pages are static unless they read dynamic data; `export const revalidate = 60` or `fetch(url, { next: { revalidate: 60 } })` gives ISR. With **Cache Components** (`cacheComponents: true`), everything is dynamic unless you mark it with the `'use cache'` directive (with `cacheLife` / `cacheTag`); dynamic parts must sit in `<Suspense>` . After a mutation, call `revalidatePath` , `revalidateTag(tag, 'max')` (Next.js 16 requires a `cacheLife` profile as the second argument; it serves stale data while refetching in the background) or, inside a Server Action, `updateTag(tag)` for read-your-own-writes; `refresh()` re-renders only uncached data. → *More:* §13.9 Next.js App Router overview

Q195. ⋆ What goes wrong if you put `'use client'` at the top of the root layout? `'use client'` marks a **boundary**: that module and everything it imports become Client Components. At the root, the whole app ships as client JavaScript, so you lose Server Components' benefits — server data fetching in components, secrets on the server, zero-JS rendering. Keep layouts as Server Components and put `'use client'` on small interactive leaves; providers go in a small client `Providers` component that wraps `children` . → *More:* §13.8 Directives: `'use client'` and `'use server'` · Ch 15 Q47

Q196. How do you do SEO in a React app? Render real HTML on the server (SSR/SSG) so crawlers and link previews see content; set a unique `<title>` , meta description and Open Graph tags per page (React 19 hoists `<title>` / `<meta>` rendered anywhere; Next.js has the `metadata` export / `generateMetadata`); use semantic headings and links (`<a href>`), clean URLs, a sitemap and `robots.txt` , and good Core Web Vitals. → *More:* §13.1 Rendering strategies: CSR, SSR, SSG, ISR, streaming, PPR · §13.9 Next.js App Router overview

Q197. What is `window is not defined` during SSR, and how do you avoid it? On the server there is no `window` , `document` or `localStorage` ; code that touches them during render or at module top level crashes SSR. Access browser APIs inside **Effects or event handlers** (they run only on the client), guard with `typeof window !== 'undefined'` for module code, or load a browser-only component lazily with SSR disabled (`next/dynamic` with `ssr: false` in a Client Component). → *More:* §13.5 Hydration mismatches

Q198. When would you choose a plain SPA (Vite) over Next.js? For apps behind a login where SEO doesn't matter (admin panels, dashboards, internal tools), apps embedded in another site, or teams that want static hosting with no server. Choose a framework when you need SEO, fast first load for public pages, server data access or Server Components. React Router's framework mode is the middle ground (SPA, SSR or prerender per route). → *More:* §13.11 Choosing a framework

---

13. Testing and TypeScript

Q199. ⋆ How do you test React applications? Unit/component tests with **Vitest** or **Jest** plus **React Testing Library** (render components, interact like a user, assert on what's visible); **mock network** with MSW

(Mock Service Worker) rather than mocking `fetch` per test; **custom Hooks** with `renderHook`; **end-to-end tests** of real user flows in a real browser with **Playwright** or Cypress; plus TypeScript, ESLint and visual tests (Storybook) as cheap safety nets. Aim for most value from integration-style component tests. → *More:* §12.1 What to test, and the Testing Library philosophy · §12.10 Good tests, bad tests, and the bigger picture

Q200. ⋆ What is the philosophy of React Testing Library? Why not Enzyme? "The more your tests resemble the way your software is used, the more confidence they can give you." Query by what users see — **role, label, text** — not by class names or component internals; interact with `user-event`; assert visible results. Tests then survive refactors. **Enzyme** tested implementation details (state, instance methods, shallow rendering) and has no React 18/19 support. → *More:* §12.1 What to test, and the Testing Library philosophy · Ch 15 Q51

Q201. ⋆ What is the priority of queries, and what's the difference between `getBy`, `queryBy` and `findBy`? Priority: `getByRole` (with `name`) → `getByLabelText` → `getByPlaceholderText` → `getByText` → `getByDisplayValue` → `getByAltText` / `getByTitle` → `getByTestId` as last resort. `getBy*` **throws** if not found (use for things that must be there); `queryBy*` returns `null` (use to assert absence); `findBy*` returns a **Promise** that retries until found or timeout (use for things that appear asynchronously). `*AllBy*` variants return arrays. → *More:* §12.3 Queries and their priority

Q202. How do you test async behaviour (data loading)? Mock the network (MSW or a stubbed `fetch`), render, then `await screen.findByText('Alice')` or `await waitFor(() => expect(...))` for the result. Assert the loading state first if relevant, and the error state with a failing mock. Don't use arbitrary `setTimeout` sleeps; for timers use fake timers (`vi.useFakeTimers()`) and advance them. → *More:* §12.5 Async tests · §12.6 Mocking `fetch` (and when to use MSW)

Q203. `fireEvent` vs `user-event`? `fireEvent.click(el)` dispatches a single DOM event. `user-event` (`const user = userEvent.setup(); await user.type(input, 'hi')`) **simulates real interactions** — focus, keydown, keypress, input, keyup for each character, pointer events, disabled checks — so it catches more bugs. Prefer `user-event`; its methods are async. → *More:* §12.4 `user-event` — interacting like a user

Q204. What is `act()`, and why do I see "not wrapped in act(...)" warnings? `act()` makes sure all state updates, Effects and re-renders caused by some code are **finished** before you assert. Testing Library's `render`, `fireEvent`, `user-event` and `findBy` / `waitFor` already wrap in `act`. The warning usually means an update happened **after** your test finished looking — e.g. a fetch resolved later; fix it by awaiting the result (`findBy…`) rather than wrapping things manually. In React 19 `act` is imported from `react`. → *More:* §12.5 Async tests

Q205. How do you test a custom Hook? Components that need providers or a router? Hooks: `renderHook(() => useCounter())` from `@testing-library/react`, then `act(() => result.current.increment())` and check `result.current`. Or test through a small component that uses it. Providers/routers: pass a `wrapper` (`render(ui, { wrapper: Providers })`) or write a **custom render** that wraps with `QueryClientProvider`, store and `MemoryRouter` / `createMemoryRouter`. → *More:* §12.7 Testing custom Hooks · §12.8 Providers, routers and custom render

Q206. Are snapshot tests a good idea? Snapshots save the rendered output and fail when it changes. They catch accidental changes cheaply but are often **approved blindly**, break on any markup tweak, and don't tell you what behaviour matters. Use small, focused snapshots (e.g. a serialised data structure) sparingly; prefer explicit assertions about what the user sees. → *More:* §12.10 Good tests, bad tests, and the bigger picture

Q207. What should you test, and what not? Test **behaviour** that users and other code depend on: rendering for given props, interactions, form validation, loading/error/empty states, accessibility of key flows. Don't test implementation details (state variable names, which Hook is used, CSS classes), third-party libraries, or trivial markup. Write a test for every bug you fix. → *More:* §12.10 Good tests, bad tests, and the bigger picture

Q208. ⋆ How do you type component props, children and events in TypeScript? Props with a `type` / `interface` and destructuring: `function Button({ label, onClick }: { label: string; onClick: () => void })`. Children: `children: React.ReactNode`. Events: `React.ChangeEvent<HTMLInputElement>`, `React.MouseEvent<HTMLButtonElement>`, `React.FormEvent<HTMLFormElement>`; or let inference work by writing the handler inline. Extend native props with `React.ComponentProps<'button'>`. Avoid `React.FC` for most

components (it used to add implicit `children`). → *More:* §11.2 Typing props · §11.4 Typing events · Ch 15 Q53

Q209. How do you type `useState`, `useRef`, `useReducer` and Context? `useState<User | null>(null)` when the initial value doesn't show the full type. `useRef<HTMLInputElement>(null)` for DOM refs (React 19 types require an argument); `useRef<number | null>(null)` for timer IDs. Reducers: type state and a **discriminated union** of actions (`{ type: 'added'; text: string } | { type: 'removed'; id: number }`), and TypeScript narrows in each `case`. Context: `createContext<CartCtx | null>(null)` plus a custom Hook that throws if null, so consumers get a non-null type. → *More:* §11.5 Typing state and reducers · §11.6 Typing refs · §11.7 Typing custom Hooks and Context

Q210. What are generic components, and when do you need them? A **generic component** has a type parameter so its props relate to each other: a `List<T>` takes `items: T[]` and `renderItem: (item: T) => ReactNode`, and TypeScript checks `renderItem` receives the right item type. Write it as `function List<T>({ items, renderItem }: ListProps<T>)` (in `.tsx`, arrow functions need `<T,>`). Used for tables, selects, autocompletes. → *More:* §11.8 Generic components

Q211. How do you model props where some combinations are invalid? Use **discriminated unions**: `type Props = { variant: 'link'; href: string } | { variant: 'button'; onClick: () => void }`. TypeScript then rejects `variant="link"` without `href`, and narrows inside the component when you check `props.variant`. Better than many optional props that may or may not go together. → *More:* §11.9 Discriminated unions for props

Q212. `interface` vs `type` for props? What are the benefits of TypeScript with React? Both work for props; `interface` can be extended/merged and gives slightly clearer errors, `type` supports unions and mapped types — pick one convention. Benefits of TypeScript: catches wrong/missing props and typos at compile time, autocompletion and safe refactoring across a codebase, self-documenting component APIs, and typed API responses and context. It replaces PropTypes. → *More:* §11.1 Setup and how type-checking fits in · §11.2 Typing props

---

14. "What will this code print?" — output and behaviour puzzles

Q213. ⋆ What does the button show after one click?

```
const [n, setN] = useState(0);
<button onClick={() => { setN(n + 1); setN(n + 1); setN(n + 1); }}>{n}</button>
```

1. All three calls read the same snapshot `n = 0` and set 1. With `setN(x => x + 1)` three times it would show **3**. `setN(n + 5); setN(x => x + 1)` gives **6** (replace with 5, then add 1). → *More:* §3.5 Updater functions — queueing updates · Ch 15 Q8

Q214. ⋆ What is logged?

```
function handleClick() { setCount(count + 1); console.log(count); }
```

The **old** value (e.g. `0` on the first click). The setter schedules a re-render; the local `count` is a constant for this render. → *More:* §3.3 State as a snapshot

Q215. What does this render when `items` is empty: `{items.length && <List items={items} />}`? A literal `0`. `0 && …` evaluates to `0`, and React renders numbers. `false`, `null`, `undefined` and `true` render nothing; `0` and `NaN` render as text. Use `items.length > 0 && …` or a ternary. → *More:* §2.7 Conditional rendering · Ch 15 Q7

Q216. ⋆ What is logged, and in what order?

```
function Child() { console.log('child render'); useEffect(() => console.log('child effect')); return null; }
function Parent() { console.log('parent render'); useEffect(() => console.log('parent effect')); return <Child />;
}
```

`parent render`, `child render`, `child effect`, `parent effect`. Rendering is top-down; Effects run **bottom-up** after commit. (Under StrictMode in development each component body runs twice — `parent render` ×2, `child render` ×2 — and then the Effects run, are cleaned up and run again: `child effect`, `parent effect`, `child effect`, `parent effect`.) → *More:* §5.3 Cleanup — undo what setup did · Ch 15 Q18

Q217. What does this counter show after 5 seconds?

```
const [count, setCount] = useState(0);
useEffect(() => {
  const id = setInterval(() => setCount(count + 1), 1000);
  return () => clearInterval(id);
}, []);
```

1, forever. The interval callback closed over the first render's `count = 0`, so it keeps setting 1 (a stale closure). Fix: `setCount(c => c + 1)`. (And with no cleanup, StrictMode in development would start two intervals.) → *More:* §5.5 Dependencies in depth

Q218. ⋆ Why does this component re-render endlessly?

```
function Profile({ id }) {
  const [user, setUser] = useState(null);
  const options = { id };
  useEffect(() => { fetchUser(options).then(setUser); }, [options]);
}
```

`options` is a **new object** every render; the Effect runs, sets state, re-renders, creates a new `options`, runs again … Depend on `id` and build the object inside the Effect. → *More:* §5.5 Dependencies in depth

Q219. What happens here?

```
function Counter() {
  const [n, setN] = useState(0);
  setN(n + 1);
  return <p>{n}</p>;
}
```

React throws **"Too many re-renders. React limits the number of renders to prevent an infinite loop."** Setting state unconditionally during render schedules another render, forever. The same happens with `onClick={setN(n + 1)}` (called during render instead of passed). → *More:* §3.1 Responding to events · §16.18 Common errors and fixes

Q220. Does this list update when you click "Add"?

```
const [items, setItems] = useState([]);
const add = () => { items.push('x'); setItems(items); };
```

No. `items` was mutated and the **same array reference** passed back; `Object.is(old, new)` is true, so React skips the render. Use `setItems([...items, 'x'])`. → *More:* §3.6 Updating objects and arrays immutably

Q221. ⋆ A user types in the first row's input, then you add an item at the top of a list keyed by index. What happens? The typed text now appears in the **new** first row instead of staying with its original item. With index keys, the old row 0's component (and its uncontrolled input state) is reused for whatever is now at index 0. Stable IDs as keys fix it. → *More:* §2.8 Rendering lists and keys · Ch 15 Q4

Q222. What does this log in development, and why?

```
function App() { console.log('render'); return null; }
createRoot(root).render(<StrictMode><App /></StrictMode>);
```

`render` logged **twice** in development (React DevTools dims the second log), once in production. StrictMode double-invokes component bodies to expose impure rendering. → *More:* §5.4 StrictMode's double run (development only)

Q223. In what order are these logged on mount: `useEffect`, `useLayoutEffect`, `render`? `render` → `layout effect` → (browser paints) → `effect`. Layout effects run synchronously after DOM mutation and before paint; passive Effects run after paint. → *More:* §5.11 `useLayoutEffect` — before the browser paints · §16.5 The render and commit lifecycle

Q224. How many renders happen after clicking this button (React 18+)?

```
onClick={() => { setTimeout(() => { setA(1); setB(2); setC(3); }, 0); }}
```

One re-render. Since React 18, automatic batching groups updates inside timeouts, promises and native handlers too (in React 17 it would have been three). → *More:* §3.4 Batching

Q225. What happens to the input text when `isEditing` toggles?

```
return isEditing ? <input key="a" /> : <input key="b" />;
```

The text is **lost** — different keys mean different components, so React unmounts one input and mounts a fresh one. Without the keys (same type, same position) the DOM input and its text would be **kept**. → *More:* §3.10 State is tied to a position in the tree

Q226. ⋆ Does `Child` re-render when `Parent`'s state changes?

```
const Child = memo(function Child({ onClick }) { … });
function Parent() { const [n, setN] = useState(0); return <Child onClick={() => setN(n + 1)} />; }
```

Yes, every time: the arrow function is a new reference each render, so `memo`'s shallow comparison fails. Wrap the handler in `useCallback` (with `setN(x => x + 1)` so deps can be `[]`), or rely on the React Compiler. → *More:* §10.4 `memo` — skipping a child's re-render · §10.5 `useCallback` and `useMemo` for stable props

Q227. What is rendered by `{[1, 2, 3]}`, `{true}`, `{null}`, `{undefined}`, `{{ a: 1 }}`? `123` (arrays render each item — React warns about missing keys only for arrays of elements); nothing for `true`, `null` and `undefined`; and `{{ a: 1 }}` **throws** "Objects are not valid as a React child" — render a field (`obj.a`) or `JSON.stringify(obj)`. → *More:* §1.8 JSX · §16.18 Common errors and fixes

Q228. A ref counter is incremented on click. Does the screen update?

```
const clicks = useRef(0);
<button onClick={() => { clicks.current++; }}>Clicked {clicks.current}</button>
```

No — changing a ref doesn't trigger a render, so the text stays at 0 until something else re-renders the component (then it jumps to the current number). Use state for anything displayed. → *More:* §5.8 `useRef` for values

---

15. Machine coding and scenario questions

These are the prompts most often given in 45–90-minute "machine coding" rounds. The answer gives the **core state and key idea** you should say before coding; the linked build in Chapter 15 has the full code and tests. Approach for every one: clarify requirements → sketch component tree and state → build the happy path → add edge cases (empty, loading, error, keyboard, a11y) → mention what you'd do with more time.

Q229. ⋆ Build a todo app (add, toggle, delete, edit, filter all/active/completed). State: `todos: { id, text, done }[]` and `filter`. Visible todos are **derived** (`todos.filter(...)`), not stored. Add with a generated `id` (`crypto.randomUUID()`), toggle/edit with `map`, delete with `filter` — all immutable; key rows by `id`. Use a `<form>` so Enter submits, trim/ignore empty text, and a reducer once actions multiply. Extras: persist to `localStorage`, "clear completed", item count. → *More:* §15.10 Build 1 — Todo list with filters · §3.11 Putting it together — a to-do app

Q230. ⋆ Build an autocomplete / typeahead search. State: `query`, `results`, `status`, `activeIndex`, `open`. Debounce the query (≈300 ms), fetch with `AbortController` and abort the previous request in cleanup (no stale results), cache results by query, handle empty/loading/error/no-results. Keyboard: ArrowUp/Down move `activeIndex`, Enter selects, Escape closes; ARIA `combobox` / `listbox` / `option` with `aria-activedescendant`; highlight the matched text; close on outside click. → *More:* §15.11 Build 2 — Autocomplete (debounce, abort, keyboard) · §7.5 `useDebounce` — wait until the user stops

Q231. ⋆ Implement infinite scroll. State: `items`, `page` / `cursor`, `hasMore`, `loading`, `error`. Put a **sentinel** `<div>` after the list and watch it with an `IntersectionObserver` in an Effect (disconnect in

cleanup); when it becomes visible and not already loading and `hasMore`, fetch the next page and append. Guard against duplicate requests (a ref flag), show a loader/error with retry, and virtualise if the list gets huge. TanStack Query's `useInfiniteQuery` handles pages/cursors.

```
useEffect(() => {
  const io = new IntersectionObserver(([e]) => { if (e.isIntersecting) loadMore(); });
  io.observe(sentinelRef.current);
  return () => io.disconnect();
}, [loadMore]);
```

→ *More:* §15.12 Build 3 — Infinite scroll

Q232. ⋆ Build pagination. State: `page` (ideally in the URL search params), `pageSize`; `totalPages = Math.ceil(total / pageSize)`. Render Prev/Next (disabled at the ends), page numbers with ellipses for many pages (first, last, current ± 1), `aria-current="page"`. Client-side: slice the array; server-side: fetch `?page=&limit=` and keep the previous page visible while loading (`placeholderData: keepPreviousData` in TanStack Query). Clamp page when filters reduce the total. → *More:* §15.16 Build 7 — Pagination · §9.9 Query keys, pagination and dependent queries

Q233. ⋆ Build a modal dialog. Props: `open`, `onClose`, `title`, `children`. Render through a **portal** to `document.body`; overlay click and **Escape** close it; **focus** moves inside on open, Tab is trapped, focus returns to the trigger on close; body scroll locked; `role="dialog"`, `aria-modal`, `aria-labelledby`. Or use the native `<dialog>` with `showModal()` and a ref. → *More:* §15.13 Build 4 — Modal dialog (portal, focus trap, Escape) · §14.8 Portals

Q234. ⋆ Build tabs. State: `activeIndex` (or ID). Buttons with `role="tab"`, `aria-selected`, `aria-controls`; panels with `role="tabpanel"`; arrow keys move between tabs (wrap around), Home/End jump; only the active tab is in the Tab order (`tabIndex` 0 vs −1). A compound-component API (`<Tabs.List>`, `<Tabs.Panel>`) via Context is a nice touch. Decide whether inactive panels unmount (lose state) or stay hidden (`<Activity>` or `hidden`). → *More:* §15.14 Build 5 — Accessible tabs · §14.4 Compound components

Q235. ⋆ Build an accordion. State: `openIds` — a `Set` (multiple open) or a single ID (one at a time; make it a prop option). Each header is a `<button aria-expanded={open} aria-controls={panelId}>` inside a heading; panel has `id` and `role="region"`. Toggle by creating a **new** Set. Simplest accessible version: native `<details>` / `<summary>`. Animate height with CSS `grid-template-rows` or `interpolate-size`.

```
const toggle = (id) => setOpen(prev => {
  const next = new Set(prev); next.has(id) ? next.delete(id) : next.add(id); return next;
});
```

→ *More:* §15.18 Other common prompts — which techniques they need

Q236. ⋆ Build a star rating component. Props: `value`, `onChange`, `max = 5`; state: `hover` (preview). Render `max` stars; filled if `i < (hover ?? value)`; click sets value, `mouseleave` clears hover; support keyboard (it's a radio group: `role="radiogroup"` with `role="radio"` / `aria-checked`, or real `<input type="radio">`s with labels), optional half stars and read-only mode. → *More:* §15.15 Build 6 — Star rating

Q237. ⋆ Build nested comments (Reddit-style thread). Data: either a tree (`{ id, text, replies: [] }`) or — better for updates — a **flat map by ID** with `parentId`, building children lists. Render with a **recursive** `<Comment>` that renders its replies (keyed by ID), indented. Features: reply (insert under parent), edit, delete (soft-delete if it has replies), collapse/expand, votes, sort. Updates on a tree need a recursive immutable update; on a flat map they're O(1).

```
function Comment({ id, byId }) {
  const c = byId[id];
  return <li>{c.text}<ul>{c.childIds.map(cid => <Comment key={cid} id={cid} byId={byId} />)}</ul></li>;
}
```

→ *More:* §15.18 Other common prompts — which techniques they need

Q238. ⋆ Build a file explorer (folder tree). Data: a tree of `{ id, name, isFolder, children }`. Recursive `<Node>` component; state `expanded: Set<id>` (folders toggle on click), plus add file/folder, rename and

delete via immutable recursive updates (or a normalised map). Show icons, sort folders first, keyboard navigation with ARIA `tree` / `treeitem` / `aria-expanded`. Talk about lazy-loading children for huge trees.

```
function insert(node, parentId, item) {
  if (node.id === parentId) return { ...node, children: [...node.children, item] };
  if (!node.children) return node; // a file: nothing to search
  return { ...node, children: node.children.map(c => insert(c, parentId, item)) };
}
```

→ *More:* §15.18 Other common prompts — which techniques they need

Q239. ⋆ Build a countdown timer / stopwatch. State: `status` (idle/running/paused), `endAt` or `startedAt` timestamp, `remaining`. Start a `setInterval` in an Effect **only while running**, clear it in cleanup; compute remaining from `Date.now()` each tick instead of decrementing (intervals drift and pause in background tabs). Start/pause/resume/reset; stop at zero and fire `onComplete` once; format `mm:ss`. Keep the interval ID in a ref if you start it from handlers.

```
useEffect(() => {
  if (status !== 'running') return;
  const id = setInterval(() => setNow(Date.now()), 250);
  return () => clearInterval(id);
}, [status]);
const remaining = Math.max(0, endAt - now);
```

→ *More:* §15.18 Other common prompts — which techniques they need · §5.3 Cleanup — undo what setup did

Q240. ⋆ Build a form with validation (sign-up form, multi-step wizard). State: `values`, `errors`, `touched`, `status`. Validate per field on blur and all on submit with a pure `validate(values)` (or a Zod schema); disable submit while submitting; show errors next to fields with `aria-invalid` and `aria-describedby`; focus the first invalid field. Multi-step: one `values` object across steps, validate the current step before Next, keep Back working, review step before submit. React 19 alternative: `<form action>` + `useActionState` returning field errors. → *More:* §15.17 Build 8 — Multi-step form wizard with validation · §4.3 Validation

Q241. ⋆ Write a `useFetch` custom Hook. Return `{ data, error, loading }` (a status enum is cleaner); refetch when the URL changes; abort the old request in cleanup to avoid races; handle non-2xx responses as errors.

```
function useFetch(url) {
  const [state, setState] = useState({ status: 'loading' });
  useEffect(() => {
    const ac = new AbortController(); setState({ status: 'loading' });
    fetch(url, { signal: ac.signal })
      .then(r => { if (!r.ok) throw new Error(r.status); return r.json(); })
      .then(data => setState({ status: 'success', data }))
      .catch(error => { if (error.name !== 'AbortError') setState({ status: 'error', error }); });
    return () => ac.abort();
  }, [url]);
  return state;
}
```

Mention: in production use TanStack Query (caching, retries, dedupe). → *More:* §7.6 `useFetch` — loading, error, success, and no races

Q242. ⋆ Write a `useDebounce` Hook. Return a copy of the value that updates only after it stops changing for `delay` ms; each change clears the previous timer.

```
function useDebounce(value, delay = 300) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const t = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(t);
  }, [value, delay]);
  return debounced;
}
```

Variant: `useDebouncedCallback(fn, delay)` keeps the timer in a ref and the latest `fn` in a ref. → *More:* §7.5 `useDebounce` — wait until the user stops

Q243. Write a `useThrottle` Hook. Allow the value through at most once per `interval`; schedule a trailing update so the final value isn't lost.

```
function useThrottle(value, ms = 200) {
  const [v, setV] = useState(value); const last = useRef(0);
  useEffect(() => {
    const wait = Math.max(0, last.current + ms - Date.now());
    const t = setTimeout(() => { last.current = Date.now(); setV(value); }, wait);
    return () => clearTimeout(t);
  }, [value, ms]);
  return v;
}
```

→ *More:* §7.5 `useDebounce` — wait until the user stops

Q244. Write `useLocalStorage`. Like `useState`, but the initial value is read lazily from `localStorage` (with `try/catch` and `JSON.parse`) and every change is written back. Handle SSR (no `window`), quota/JSON errors, and optionally sync across tabs with the `storage` event (or `useSyncExternalStore`).

```
function useLocalStorage(key, initial) {
  const [v, setV] = useState(() => {
    try { const s = localStorage.getItem(key); return s ? JSON.parse(s) : initial; } catch { return initial; }
  });
  useEffect(() => { try { localStorage.setItem(key, JSON.stringify(v)); } catch {} }, [key, v]);
  return [v, setV];
}
```

→ *More:* §7.4 `useLocalStorage` — state that survives a reload

Q245. Write `useOnClickOutside` (close a dropdown when clicking elsewhere). Listen on `document` for `pointerdown`; if the target isn't inside `ref.current`, call the handler. Keep the latest handler in a ref (or `useEffectEvent`) so the listener isn't re-attached each render; remove it in cleanup. Also close on Escape and on focus leaving.

```
function useOnClickOutside(ref, handler) {
  const onOutside = useEffectEvent(handler);
  useEffect(() => {
    const listener = (e) => { if (ref.current && !ref.current.contains(e.target)) onOutside(e); };
    document.addEventListener('pointerdown', listener);
    return () => document.removeEventListener('pointerdown', listener);
  }, [ref]);
}
```

→ *More:* §7.7 Hooks that take callbacks — `useEffectEvent`

Q246. Write `useInterval` (Dan Abramov's pattern) and `useWindowSize` / `useMediaQuery`. `useInterval(callback, delay)`: keep the latest callback in a ref (or `useEffectEvent`), start `setInterval` in an Effect keyed on `delay`, pause when `delay` is `null`. `useWindowSize` / `useMediaQuery`: subscribe to `resize` / `matchMedia` change events — best with `useSyncExternalStore(subscribe, () => window.matchMedia(q).matches, () => false)` so it's tear-free and SSR-safe. `getSnapshot` must return a primitive or a cached value — returning a fresh `{ width, height }` object each call causes an infinite re-render loop, so for `useWindowSize` read `window.innerWidth` and `window.innerHeight` with two separate subscriptions (or cache the object). → *More:* §7.9 `useSyncExternalStore` — subscribing to outside data · §7.7 Hooks that take callbacks — `useEffectEvent`

Q247. Build a toast/notification system. A `ToastProvider` holds a queue `{ id, message, type }[]` and exposes `toast.show()` via Context (or a store so it's callable outside React). Render toasts through a **portal**; each auto-dismisses with a timer cleared on unmount and paused on hover; limit how many show; announce with `role="status"` / `aria-live`; let users close them. → *More:* §15.18 Other common prompts — which techniques they need · §14.8 Portals

Q248. Build an image carousel/slider. State: `index`. Next/prev with wrap-around (`(i + 1) % n`), dots to jump, optional autoplay via `setInterval` in an Effect that pauses on hover/focus and when the tab is

hidden; preload the next image; keyboard arrows; `aria-roledescription="carousel"` and a live region for slide changes; swipe with pointer events on touch devices. → *More:* §15.18 Other common prompts — which techniques they need

Q249. Build an OTP (one-time password) input. State: an array of N digits; an array of refs (`useRef([])`). On input: keep the last digit, move focus to the next box; Backspace on an empty box moves back; paste spreads digits across boxes; only digits allowed (`inputMode="numeric"`); `autoComplete="one-time-code"` on the first box; call `onComplete` when all are filled. → *More:* §15.18 Other common prompts — which techniques they need

Q250. Build a progress bar. Props: `value` 0–100 (clamped). Render an outer track and an inner bar with `width: ${value}%` (or `transform: scaleX`) and a CSS transition; accessible with the native `<progress>` or `role="progressbar"` with `aria-valuenow/min/max`. Variants asked: animate from 0 to 100 over N seconds, or N bars filling one after another (a queue in state, each starting when the previous ends). → *More:* §15.18 Other common prompts — which techniques they need

Q251. Build a data table with sorting, filtering and search. State: `sort { key, dir }`, `filters`, `query`, `page`; rows derived with `filter` → `sort` (copy before sorting) → `slice`, memoized with `useMemo` for big data; debounce the search; keep state in the URL; clickable headers with `aria-sort`; virtualise or paginate large data; for server data, send these as query params instead. → *More:* §15.18 Other common prompts — which techniques they need · §10.6 `useMemo` for expensive calculations

Q252. Build tic-tac-toe (or a memory/match game). State: `squares` (array of 9), `xIsNext` (or derive from move count), optional `history` for time travel. `winner` is **derived** with a `calculateWinner` check over the 8 lines — don't store it. Ignore clicks on filled squares or after a win; show status (winner / draw / next player); reset button. Extension: N×N board, undo. → *More:* §3.8 Derived state — choosing the minimal state

Q253. Build a traffic light, or a "boxes turn green then reset in click order" grid. Traffic light: state `color`; an Effect sets a `setTimeout` for the current colour's duration from a config map (`{ red: 4000, green: 3000, yellow: 500 }`) and moves to `next`; clear the timer in cleanup. Click-order grid: store the **order** of clicked boxes in an array; when all are active, deactivate them one by one with timers in that order (store timer IDs in a ref to clean up on unmount). → *More:* §5.3 Cleanup — undo what setup did

Q254. Build a chips/tags input, a multi-select, or a drag-and-drop list (kanban). Chips: state `chips: string[]` and `input`; Enter adds a trimmed unique chip, Backspace on empty input removes the last, × button removes one. Multi-select: autocomplete + chips. Drag-and-drop: state as columns → ordered IDs (normalised), move with immutable splice logic; use `@dnd-kit` or native HTML5 drag events, plus a keyboard alternative. → *More:* §15.18 Other common prompts — which techniques they need

Q255. Build a "shopping cart" / checkout page. State: cart lines `{ productId, qty }` (normalised; product details looked up), derived subtotal/tax/total, quantity +/− with min 1 and stock max, remove, persist to `localStorage`, coupon validation. Shared across pages via Context + reducer or Zustand. Checkout form with validation; disable submit while paying; optimistic quantity changes with rollback. → *More:* §6.11 Putting it together — a bookshop

Q256. ⋆ Scenario: the page gets slower the longer it stays open. How do you investigate? Suspect a **memory leak**: listeners, intervals, subscriptions or observers not cleaned up; caches growing forever; detached DOM nodes kept by refs. Use the browser's Memory panel (heap snapshots before/after an action, compare), the Performance panel, and React DevTools Profiler for growing render times. Fix with Effect cleanups, bounded caches, and virtualization for growing lists. → *More:* §5.3 Cleanup — undo what setup did · §10.10 Common performance bugs

Q257. ⋆ Scenario: typing in a search box is laggy because it filters a large list. How do you fix it? Measure first. Then: keep the input state local and **defer** the expensive part — `useDeferredValue(query)` for the list, or `startTransition` around the filter update — so keystrokes stay urgent; memoize the filtered result (`useMemo`) and `memo` the list; **virtualise** the list; debounce if the filter hits the network; move heavy work to a Web Worker if it's CPU-bound. → *More:* §7.11 `useDeferredValue` — let part of the UI lag behind · §10.9 List virtualization

Q258. Scenario: a component works but shows stale data after an update on another screen. Why? Typical causes: data copied into local state from props (doesn't follow later changes); a client cache not invalidated after a mutation (call `invalidateQueries`, revalidate the route, or `revalidatePath` / `updateTag`

in Next.js); a `memo`/selector returning a cached object because state was mutated in place; or two separate copies of the same data in different stores. Keep one source of truth and invalidate after writes. → *More:* §9.10 Mutations, invalidation and optimistic updates · §3.8 Derived state — choosing the minimal state

Q259. ⋆ Scenario: how would you design a large-scale React app (senior question)? Talk through: rendering strategy per route (SSR/SSG/SPA, Server Components); routing and code splitting by route; state layers (server state cache, URL state, local state, a small global store); a design system with accessible primitives; feature-based folders with clear boundaries; type safety end-to-end (TypeScript, schema validation); testing pyramid (component tests + E2E on critical flows); performance budgets and Web Vitals monitoring; error boundaries + logging (Sentry); feature flags and CI/CD. → *More:* §14.11 Folder structure · §14.12 Design systems · §13.11 Choosing a framework

Q260. Scenario: how would you migrate a large app from React 18 to 19 (or from classes to Hooks)? Upgrade to React 18.3 first and fix its deprecation warnings; run the React 19 codemods (`ReactDOM.render` → `createRoot`, string refs, `propTypes`, `act` import); update `@types/react` and fix type errors (`useRef` argument, `JSX` namespace); update libraries that rely on removed internals; run the test suite and StrictMode in development. For classes → Hooks, migrate **incrementally**, leaf components first, behind tests — both styles work together, and error boundaries stay as classes. → *More:* §11.1 Setup and how type-checking fits in

Q261. How do you debug a React application? React DevTools (Components tab: inspect props/state/hooks, find which component rendered an element, "why did this render"; Profiler tab), browser DevTools (breakpoints, `debugger`, Network, Performance), StrictMode and lint warnings, error boundaries with `componentStack` logging, React 19's improved error messages (hydration diffs), and reproducing the bug in a failing test first. → *More:* §10.3 Measure first: DevTools Profiler and `<Profiler>` · §16.18 Common errors and fixes

Q262. What is the difference between React and Angular (or Vue)? React is a UI library with JSX, one-way data flow and a large, unopinionated ecosystem (you pick router, state, forms). **Angular** is a full framework (TypeScript, dependency injection, RxJS, its own router, forms and HTTP client, CLI, templates with two-way binding, Signals for reactivity) — more structure, steeper learning curve. **Vue** uses HTML-like templates and fine-grained reactive state, with an official router and store; somewhere between the two. Choose by team experience and how much convention you want. → *More:* §1.1 What React is

---

Sources of the question lists

Question wording and frequency were gathered from these lists and posts (titles only; all answers above are written for this book):

- sudheerj — *reactjs-interview-questions* (GitHub, "List of top 500 ReactJS Interview Questions & Answers", including its Hooks, React 18/19 and Redux sections) — github.com/sudheerj/reactjs-interview-questions
- GreatFrontEnd — *top-reactjs-interview-questions* (GitHub) and blog posts "50 Essential React.js Interview Questions by Ex-Interviewers", "30 Essential React Hooks Interview Questions You Must Know", "30 Basic to Advanced React Interview Questions with Solutions", "Frontend LLD Interview Guide: React machine coding questions" — greatfrontend.com
- h5bp — *Front-end-Developer-Interview-Questions* (GitHub)
- Harendra-jagariya — *frontend-machine-coding-interview-questions-reactjs* (GitHub: counter, countdown, accordion, todo, infinite scroll, pagination, autocomplete, star rating, carousel, toast, tabs, modal, OTP input, progress bar, multi-step form, folder tree, nested comments)
- NamasteDev — "Top machine coding questions for frontend interviews" (parts 1 and 2)
- Peerlist — "80 frontend interview questions asked in most interviews"
- techinterview.org — "Build a Comments Thread: Nested Replies, Pagination, Optimistic Updates", "Machine-Coding: Build an Infinite Scroll List with Virtualization"
- Hyring — "Top 62 React Interview Questions and Answers"; Coursesity — "React Interview Questions: 50 Most Asked"

- react.dev — the React 19, React 19.2 and React Compiler v1.0 release posts (used to check version-specific facts, not as a question list)